

Прозрач ное програм мное обеспе чение

Безопасность цепочек поставок ПО

Команда ветеранов в области информационной безопасности предлагает экспертную трактовку безопасности цепочек поставок программного обеспечения.

Прозрач ное програм мное обеспе чение

Безопасность цепочек поставок ПО

Крис Хьюз
Тони Тернер





Software Transparency

Supply Chain Security in an Era of a
Software-Driven Society

Chris Hughes
Tony Turner

WILEY

Прозрачное программное обеспечение

Безопасность цепочек
поставок ПО

Крис Хьюз
Тони Тернер



Санкт-Петербург • Москва • Минск

2025

ББК 32.973.2-018-07
УДК 004.415
Х98

Хьюз Крис, Тернер Тони

Х98 Прозрачное программное обеспечение: безопасность цепочек поставок ПО. — СПб.: Питер, 2025. — 352 с.: ил. — (Серия «Для профессионалов»).

ISBN 978-5-4461-2182-3

Изучите влияние на кибербезопасность взаимосвязанных цепочек поставки ПО!

Могут ли цепочки поставок ПО быть безопасными? Команда преданных своему делу профессионалов предлагает по-новому взглянуть на безопасность вашей организации. Вы познакомитесь со множеством тем: от истории движения за прозрачность ПО до подходов к спецификации и подтверждению достоверности в быстро меняющемся ландшафте программного обеспечения.

Авторы знакомят нас с основными векторами атак, такими как мобильные и социальные сети, банковские системы и системы розничной торговли и даже критически важные инфраструктуры, а также с системами защиты, на которые мы все полагаемся. Вы узнаете, как защититься от угроз, и познакомитесь с практическими рекомендациями как для потребителей, так и для поставщиков ПО.

«Прозрачное программное обеспечение» — ценный источник знаний для специалистов по кибербезопасности и безопасности приложений, а также для профессионалов, работающих в области безопасности промышленных систем управления, облачной безопасности, мобильной безопасности, DevOps и DevSecOps..

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ББК 32.973.2-018-07
УДК 004.415

Права на издание получены по соглашению с John Wiley & Sons, Inc. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. В книге возможны упоминания организаций, деятельность которых запрещена на территории Российской Федерации, таких как Meta Platforms Inc., Facebook, Instagram и др. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

ISBN 978-1394158485 англ.
ISBN 978-5-4461-2182-3

© 2023 by John Wiley & Sons, Inc. All rights reserved
© Перевод на русский язык ООО «Прогресс книга», 2024
© Издание на русском языке, оформление ООО «Прогресс книга», 2024
© Серия «Для профессионалов», 2024

Краткое содержание

1	История угроз для цепочек поставок ПО.....	30
2	Существующие подходы — традиционное управление рисками поставщиков.....	58
3	Базы данных уязвимостей и методы оценки	77
4	Появление концепции SBOM.....	112
5	Сложности в обеспечении прозрачности ПО	143
6	Облако и контейнеризация	156
7	Актуальные рекомендации для коммерческих разработок	185
8	Актуальные рекомендации для государственных разработок	234
9	Прозрачность ПО в операционных технологиях	281
10	Практическое руководство для поставщиков	293
11	Практическое руководство для потребителей	312
12	Перспективы повышения прозрачности ПО	329

Оглавление

Об авторах.....	14
О научном редакторе	16
Благодарности.....	17
Предисловие	18
Введение	22
Почему это важно?	24
Структура издания	26
Для кого эта книга будет полезна.....	28
Пояснения.....	29
Связь с авторами.....	29
От издательства.....	29
1 История угроз для цепочек поставок ПО.....	30
Мотивы злоумышленников.....	30
Модели угроз	32
Методы моделирования угроз	33
Моделирование угроз.....	38
Нашумевший случай 1: SolarWinds.....	46
Нашумевший случай 2: Log4j.....	50
Нашумевший случай 3: Kaseya	54
Какие выводы можно сделать из этих случаев	56
Резюме	57

2	Существующие подходы — традиционное управление рисками поставщиков.....	58
	Анализ.....	58
	Проверка SDL.....	61
	Модели зрелости безопасности приложений.....	62
	Администрирование	64
	Проектирование	65
	Реализация	65
	Верификация	66
	Операции	67
	Гарантии безопасности ПО	67
	Статическое тестирование безопасности приложений	68
	Динамическое тестирование безопасности приложений.....	70
	Интерактивное тестирование безопасности приложений	72
	Тестирование безопасности мобильных приложений	73
	Анализ композиции программного обеспечения	73
	Хеширование и подпись кода	74
	Резюме	76
3	Базы данных уязвимостей и методы оценки	77
	Типичные уязвимости и слабые места.....	77
	Национальная база данных уязвимостей.....	80
	Форматы идентификации ПО.....	82
	Процесс создания и присвоения SWID	84
	PURL.....	85
	Sonatype OSS Index.....	87
	База данных уязвимостей открытого ПО	88
	Global Security Database.....	89
	Общая система оценки уязвимостей	91
	Базовые метрики	93
	Временные метрики	95
	Метрики среды.....	96
	Шкала оценки CVSS.....	97

Система прогнозирования эксплойтов	99
Модель EPSS	99
Критика EPSS	101
Подход CISA	103
Стандарт CSAF	103
Стандарт VEX	105
Методология SSCV	106
Движение вперед	111
Резюме	111
4 Появление концепции SBOM	112
SBOM в стандартах: провалы и успехи	112
NTIA: пропаганда потребности в SBOM	113
Инициативы индустрии: научно-исследовательские центры	118
Форматы SBOM	120
Обмен информацией об уязвимостях (VEX) и их раскрытие	125
На сцену выходит VEX	126
VEX: добавление контекста и ясности	126
VEX и VDR	127
Дальнейшие шаги	131
Использование SBOM с другими методами аттестации	132
Достоверность источника	132
Управление зависимостями и их верификация	134
Sigstore	136
Подписание коммитов	139
Критика и недочеты SBOM	139
Резюме	142
5 Сложности в обеспечении прозрачности ПО	143
Прошивки и встраиваемое ПО	143
Прошивки Linux	143
Прошивки для операционных систем реального времени	144
Встраиваемые системы	144
SBOM для устройств	145

Открытое ПО и проприетарный код	145
Пользовательское ПО	150
Устаревшее ПО	151
Безопасный транспорт	153
Резюме	155
6 Облако и контейнеризация	156
Модель общей ответственности	157
Разделение обязанностей в модели общей безопасности	157
Ответственность в рамках модели общей безопасности	158
Четыре столпа облачной безопасности	161
Контейнеры	164
Kubernetes	170
Бессерверная модель	177
SaaS BOM и сложность API	178
CycloneDX SaaS BOM	179
Инструменты и обсуждения	181
Применение в DevOps и DevSecOps	182
Резюме	184
7 Актуальные рекомендации для коммерческих разработок	185
Уровни безопасности в цепочках поставок компонентов программного обеспечения	185
Граф для понимания состава артефактов	189
Руководство по обеспечению безопасности цепочек поставок CIS	192
Исходный код	194
Пайплайны сборки	195
Зависимости	197
Артефакты	197
Развертывание	198
Лучшие практики CNCF по защите цепочек поставок	199
Защита исходного кода	202
Защита материалов	204
Защита пайплайнов сборки	205

Защита артефактов.....	207
Защита развертываний.....	208
Эталонная архитектура защищенной фабрики ПО от CNCF.....	208
Эталонная архитектура защищенной фабрики ПО.....	209
Ключевые компоненты	210
Компоненты управления.....	211
Компоненты распространения.....	211
Переменные и функциональность	212
Подводя итоги	212
Фреймворк Secure Supply Chain Consumption от Microsoft.....	213
Практики S2C2F	215
Руководство по реализации S2C2F.....	218
OWASP Software Component Verification Standard.....	220
Уровни SCVS	221
Инвентаризация.....	222
Перечень программных компонентов ПО и их зависимостей (SBOM).....	223
Среда сборки.....	224
Управление пакетами.....	225
Анализ компонентов	226
Происхождение и провенанс.....	227
Политика использования открытых компонентов	228
OpenSSF Scorecard	228
Система оценки безопасности проектов с открытым исходным кодом	229
Как организации могут использовать Scorecard	231
Предстоящий путь	232
Резюме	233
8 Актуальные рекомендации для государственных разработок	234
Практики по защите цепочек поставок ПО для систем и организаций	234
Критическое ПО	236
Меры по обеспечению безопасности критического ПО.....	238

Верификация ПО.....	242
Моделирование угроз.....	243
Автоматизированное тестирование.....	245
Статический анализ и динамическое тестирование.....	245
Анализ на наличие в коде жестко прописанных секретов.....	245
Запуск кода с проверками и защитой, обеспечиваемыми языком.....	246
Тестирование по принципу «черного ящика».....	246
Тестовые случаи в отношении кода.....	247
Тестирование в хронологическом порядке.....	247
Фаззинг.....	247
Сканирование веб-приложений.....	247
Проверка компонентов ПО.....	248
Фреймворк SSDF.....	248
Подробности SSDF.....	250
NSA: серия руководств по защите цепочек поставок ПО.....	255
Руководство по безопасности для разработчиков.....	257
Приложения к руководству NSA.....	268
Рекомендуемые практики для поставщиков.....	270
Подготовка организации.....	270
Защита ПО.....	271
Создание надежно защищенного ПО.....	272
Реагирование на уязвимости.....	274
Рекомендации для потребителей.....	275
Резюме.....	280
9 Прозрачность ПО в операционных технологиях.....	281
Кинетический эффект ПО.....	282
Риски устаревшего ПО.....	284
Релейная логика и заданные значения в системах управления.....	286
Поверхность атаки ICS.....	288
Умные электросети.....	290
Резюме.....	292

10	Практическое руководство для поставщиков	293
	Раскрытие информации об уязвимостях и реагирование на них.....	293
	PSIRT	295
	Делиться или не делиться?	302
	Лицензирование и личная ответственность за код.....	304
	Подразделения по управлению OSS	305
	Согласованность среди команд по разработке продуктов	308
	Ручной труд или автоматизация.....	309
	Резюме	311
11	Практическое руководство для потребителей	312
	Мыслить в ширину и глубину	312
	Действительно ли мне нужен SBOM?	313
	Что мне с ним делать?.....	317
	Получение и управление SBOM в больших масштабах	318
	Уменьшение шума	321
	Когда применить патч нельзя	323
	Подготовка	324
	Идентификация.....	324
	Анализ.....	325
	Создание виртуального патча	325
	Реализация и тестирование	327
	Восстановление и дальнейшее сопровождение.....	327
	Перспективное мышление	327
	Резюме	328
12	Перспективы повышения прозрачности ПО.....	329
	Современные инициативы, законы и требования	329
	Влияние правительственных цепочек поставок на рынки.....	336
	Рост числа атак на цепочки поставок	338
	Растущая взаимосвязанность IoT-устройств.....	341
	Что дальше?	344

Посвящается моим детям: Каролине, Кельвину, Келли и Клейтону.
Моей жене Кэтлин. Их любовь и поддержка мотивируют меня быть
лучше. Я также хочу поблагодарить мою маму Доун и моего дедушку
Билла — они научили меня трудолюбию.

Крис Хьюз

Работа над книгой стала важным периодом в моей жизни, на
протяжении которого моя семья, жена Бекки и дети Алекс и Гэвин,
терпеливо поддерживали меня. Я искренне благодарен им за то, что
они всегда были рядом, напоминая мне о самом важном в жизни.

Тони Тернер

Об авторах



Крис Хьюз — сооснователь и начальник управления информационной безопасностью компании Aquia. За его плечами уже около 20 лет опыта в сфере информационных технологий и кибербезопасности, включая военную службу в ВВС США, гражданскую службу в ВМС США и Администрации общих служб США/Федеральной программе управления рисками, а также работу консультантом в частном секторе. Кроме того,

он преподает магистерскую программу по кибербезопасности в Капитолийском технологическом университете (Capitol Technology University) и Университете Мэриленда (University of Maryland Global Campus). Крис также состоит в рабочих группах Cloud Security Alliances Incident Response и SaaS Security Working Group, параллельно занимая должность председателя в Cloud Security Alliance D.C. Вдобавок ко всему он ведет подкаст Resilient Cyber.

У Криса есть различные профессиональные сертификаты CISSP/CCSP (Certified Information System Security Professional/Cisco Certified Security Professional), выданные международной некоммерческой организацией (ISC), сертификаты безопасности AWS и Azure, а также сертификат ССАК (Certificate of Cloud Auditing Knowledge) от Cloud Security Alliance. При этом у него степень бакалавра информационных систем, степень магистра по кибербезопасности и деловому администрированию. Крис регулярно консультирует руководителей из различных индустрий по вопросам IT и кибербезопасности, помогая их организациям трансформировать свое цифровое пространство с сохранением акцента на безопасности. Крис написал множество статей и заметок на тему защиты цепочек поставок программного обеспечения (ПО), а также выступал на различных мероприятиях, включая DevSecOps Days, проводимое Университетом Карнеги — Меллона в Вашингтоне, округ Колумбия.



Тони Тернер — основатель и генеральный директор компании Opswright, специализирующейся на проектировании систем безопасности для критической инфраструктуры. За 25 лет работы в сфере кибербезопасности Тони успел потрудиться в качестве инженера внутренней безопасности, разработчика, системного администратора, консультанта по безопасности, архитектора ПО и управляющего профессиональными

сервисами для поставщиков систем безопасности. В последнее время он занимает должность вице-президента отдела исследований и разработки в команде Fortress Labs компании Fortress Information Security, где возглавил исследование и разработку плана по обеспечению прозрачности ПО и управлению уязвимостями. В 2011 году Тони основал (и до сих пор руководит) проект по обеспечению безопасности открытых веб-приложений в Орландо, сделав упор на повышение прозрачности. При этом он является ведущим проекта по документированию брандмауэров веб-приложений (WAFEC) и соорганизатором конференции Security BSides в Орландо. Он состоит во многих торговых организациях и промышленных группах, включая MITRE, ISA, OWASP, CISA, GlobalPlatform и другие, в основном сосредоточенных на темах прозрачности ПО и безопасности программных продуктов в критических инфраструктурах.

Тони получил степень бакалавра в Университете Ходжеса в Нейплсе, штат Флорида, а также имеет несколько профессиональных сертификатов, включая CISSP, CISA, шесть сертификатов от SANS/GIAC, OPSE, CSTP и CSWAE, а также множество сертификатов от компаний-вендоров вроде F5, Imperva и др.

Кроме того, Тони участвовал в составлении сертификационных экзаменов GIAC и F5 ASM по брандмауэрам веб-приложений и является автором курса по защите цепочек поставок продуктов для SANS, вышедшего в 2023 году. Проживает в городе Индиалантик, штат Флорида, со своей женой, двумя сыновьями, Алексом и Гэвином, пятью кошками, тремя ящерицами, двумя морскими свинками и целой армией белок, которые обитают на заднем дворе и каждое утро требуют, как им кажется, заслуженных подношений.

О научном редакторе

Стивен Спрингетт руководит командами по разработке продуктов около 25 лет, из которых более 14 лет посвятил сфере безопасности цепочек поставок. Сейчас Стив занимает должность директора отдела разработки систем безопасности в ServiceNow, где отвечает за построение архитектуры, моделирование угроз и адаптацию новых разработчиков. Стив с удовольствием помогает организациям выявлять и снижать риски, связанные с использованием сторонних и открытых компонентов. Он выступает в поддержку открытого ПО, а также ведет проект OWASP Dependency-Track, является соавтором OWASP SCVS и председателем рабочей группы по стандарту CycloneDX Core, обеспечивающему продвинутые возможности для отслеживания списка зависимостей компонентов ПО в цепочках поставок с целью снижения киберрисков. Стив имеет ряд сертификатов в области безопасности, включая CSSLP и CCSK. Он проживает на северном побережье Чикаго со своей женой Верой и дочерью Арианой.

Благодарности

Мы хотим поблагодарить всех профессионалов нашей отрасли, с которыми нам довелось сотрудничать на протяжении многих лет.

К ним относятся такие новаторы в области обеспечения безопасности цепочек поставок, как Аллан Фридман, Джошуа Корман, Роберт Вуд, Вирджиния Райт, Джейсон Вайс и, конечно же, Стив Спрингетт, который, помимо того что является лидером в этой области, выступил научным редактором книги и предоставил объективную критику. Кроме того, мы выражаем признательность многим руководителям предприятий государственного и частного секторов, которые участвовали в рабочих группах и содействовали повышению безопасности цепочек поставок посредством OWASP, Linux Foundation, OpenSSF, Министерства обороны США, Национального управления США (National Telecommunications and Information Administration, NTIA), Агентства по кибербезопасности и защите инфраструктуры (Cybersecurity and Infrastructure Security Administration, CISA), Института программной инженерии Карнеги — Меллона (Software Engineering Institute, SEI) и пр. Благодаря совместному опыту и сотрудничеству мы смогли реализовать нечто очень полезное для всей индустрии. Программное обеспечение затрагивает практически все сферы нашей жизни, и мы вместе стоим на страже его безопасности.

Предисловие

Многие из нас запомнили декабрь 2021 года как период, когда мы, сутулясь над своими ноутбуками в гостях у родственников, боролись с кризисом Log4j. Эта уязвимость в открытом фреймворке ведения журналов Java, разработанном Apache Software Foundation, была признана серьезной как независимыми экспертами, так и официальными организациями и стандартами. При этом ее устранение нельзя отнести к разряду сложнейших, будь то с помощью патчей или посредством других встраиваемых средств противодействия. Тем не менее самая трудная задача для организаций, подверженных этой уязвимости, заключается в ее поиске! Она оказалась запрятана в цепочках поставок бесчисленных современных приложений. Ни производители, ни пользователи ПО просто не располагали адекватной схемой, которая позволила бы понять, где искать.

По идее вся сложность с обеспечением безопасности должна заключаться в определении уязвимостей и обнаружении злоумышленников, пытающихся проникнуть в наши цепочки поставок. Но по факту, как мы в итоге поняли, она состоит в необходимости значительной доработки создаваемого нами ПО.

Идея отслеживания поступающих в ПО компонентов не нова. Академики говорят об этом с 1990-х годов. Более обширно эту тему начали обсуждать в 2000-х. Невозможность отчитаться за используемое открытое ПО привела несколько крупных компаний к серьезным юридическим проблемам. Политика сбора и использования данных поставщиков стала неотъемлемой частью революции в сфере качества услуг в тяжелой промышленности, начавшейся еще в конце 1940-х годов. Тот период был ознаменован новаторскими идеями Эдварда Деминга и «революцией Тойоты», что послужило толчком к появлению DevSecOps и разворачиванию современной революции ПО.

Действительно, весьма странно, что наши цепочки поставок не отличаются повышенной прозрачностью. Я часто привожу в пример кексы Twinkie (впервые это сравнение сделали Одра Хатч и Джошуа Корман), когда поднимаю вопрос о структуре ПО: почему мы больше знаем о составе начинки не поддающегося разложению снека, чем о содержимом ПО, которое

работает в наших компаниях, правительственных учреждениях и критической инфраструктуре?

Погружаясь в тему правильной реализации концепции SBOM (Software Bill Of Materials, перечень программных компонентов ПО и их зависимостей), желательно подумать о том, почему мы не озадачились этим изначально, каковы наши успехи на этом пути и в чем заключается ценность прозрачности. Есть ряд субъективных причин, по которым многие организации не хотели предоставлять эту информацию, включая нежелание подвергаться вышеупомянутым рискам, связанным с лицензированием открытого ПО. Честно говоря, многие компании не хотели признавать масштаб своего технического долга или тратить на налаживание простой внутренней инфраструктуры и процессов для отслеживания данных об их зависимостях. Более того, нужно признать, что начать внедрять SBOM было непросто — для этого потребовалось объединить технический опыт с пониманием сложной экосистемы ПО и объективных стимулов. Однако за последние годы был достигнут значительный прогресс в этой области.

Первым делом нам понадобилось сформировать единое видение концепции SBOM. Некоторые эксперты имели собственные представления об этой идее — их обсуждали и уточняли в период с 2018 по 2020 год в ходе открытого международного процесса, организованного NTIA. В итоге сообщество утвердило фундаментальные принципы SBOM и определило основные сценарии использования данной концепции. Затем нам потребовались машиночитаемые средства для распространения этой информации по цепочке поставок. К счастью, некоторые представители IT-индустрии опередили нас, и мы смогли адаптировать под эти задачи разработанный при The Linux Foundation формат обмена данными Software Package Data Exchange (SPDX) и формат CycloneDX, реализованный в рамках проекта Open Worldwide Application Security Project (OWASP).

Следующим шагом стало создание инструментов для генерации и использования полученных машиночитаемых данных. Удалось достичь значительного прогресса по части реализации SBOM в целом. Экосистема стала наполняться новыми инструментами. Сегодня мы видим все больше точечных решений, ориентированных на конкретный сектор или технологию, поскольку потребности в области систем управления производственными процессами (Industrial Control Systems, ICS) и операционных технологий (Operational Technology, OT) несколько отличаются от традиционного корпоративного ПО, которое, в свою очередь, имеет уникальные особенности и модели интеграции в сравнении со сферой облачных вычислений. Инструменты

генерации перешли на стадию зрелости параллельно с появлением новых решений, помогающих компаниям эффективно использовать собранные данные в операционных и стратегических целях. (Это было частью моей работы, сначала в NTIA, а сейчас в CISA — я проводил встречи с руководителями стартапов и новаторами в области открытого ПО, чтобы найти способы решения реальных задач на рынке программного обеспечения.)

Естественно, третьим компонентом этой триады, наряду с технологией и рынками, является правительство. В последние годы со стороны руководства США также наблюдается повышенный интерес к данной сфере. Оно сделало акцент на обеспечении безопасности цепочек поставок и внедряет новые политики, как и наши партнеры из других стран. Концепция SBOM переместилась на главную сцену индустрии информационной безопасности в мае 2021 года, когда был издан Указ о повышении уровня национальной кибербезопасности. Этот указ, помимо прочего, утвердил, что поставщики ПО для Правительства США должны предоставлять покупателю списки зависимостей предоставляемого ПО (то есть SBOM). Данный закон согласуется с законами об обороте медицинских устройств в США, утвержденными Управлением по санитарному надзору за качеством пищевых продуктов и медикаментов (Food and Drug Administration, FDA), деятельностью различных регуляторов страны и предложениями аналогичных законов в других государствах.

Если вы решили прочесть эту книгу, то наверняка придерживаетесь мнения о важности прозрачности цепочек поставок ПО. Тем не менее другие люди в вашей организации или смежных областях индустрии могут считать иначе, и тогда вам придется вести разъяснительную работу, отстаивать бюджет, а также убеждать поставщиков ПО делиться соответствующими данными. Однако SB в SBOM — это не Silver Bullet (серебряная пуля. — *Примеч. пер.*). SBOM — это слой данных. Мы по-прежнему создаем инструменты и процессы для превращения этих данных в знания и в конечном счете в действия. Подобно тому как система идентификации общеизвестных уязвимостей информационной безопасности (Common Vulnerabilities and Exposures, CVE) сегодня является неотъемлемой частью глобальной экосистемы уязвимостей, так и SBOM станет обязательным компонентом для обеспечения безопасности и повышения качества ПО.

Прозрачность в цепочках поставок ПО важна на протяжении всего жизненного цикла продуктов. Для тех из нас, кто занят разработкой ПО, SBOM представляет мощный инструмент, позволяющий лучше понять реализуемые процессы и избежать создания небезопасных по своей структуре решений или поставки продуктов, содержащих известные риски. Если смотреть

с позиции покупателя ПО, то логично спросить: «А зачем обращаться к поставщику, который сам до конца не понимает, что продает? Зачем внедрять ПО, которое уже устарело?» Если рассуждать с позиции пользователя, то как при отсутствии SBOM можно реагировать на новые угрозы или планировать системы, опирающиеся на ПО, жизненный цикл которого близится к завершению? Естественно, многие из нас воплощают в себе все три описанные роли. Очевидно, что по мере дальнейшего внедрения концепции SBOM мы будем находить все новые варианты использования этих данных.

Движение за повышение прозрачности ПО внесло огромный вклад в индустрию, включая развитие концепции SBOM, создание инструментов и обсуждение важнейших пограничных случаев. Авторы книги, Хьюз и Тернер, проделали огромную работу, отразив все эти достижения, а также некоторые собственные и объяснив все это в подробностях, плавно перейдя от концепции SBOM к практике SBOM. И хотя я ожидаю, что эту книгу возьмут на вооружение специалисты и организации по всему миру, во мне также присутствует парадоксальное желание, чтобы критическая потребность в данном руководстве оказалась недолгой. По мере того как все больше людей начинают создавать и управлять ПО, используя те принципы интероперабельной автоматизации, которые авторы так уместно предвидят, концепция SBOM утратит свою новизну, став неотъемлемой частью создания, распространения и использования ПО. И только тогда мы сможем переключиться на следующую задачу.

Аллан Фридман

Аллан Фридман — старший советник и стратег в Агентстве по кибербезопасности и защите инфраструктуры. Он координирует глобальную деятельность сообщества по реализации и распространению концепции SBOM. До этого он занимал должность директора по инициативам в сфере кибербезопасности в NTIA, руководил развитием передовых идей, связанных с раскрытием уязвимостей, а также внедрением SBOM и прочих концепций безопасности. До работы в федеральном правительстве Фридман более десяти лет занимался исследованиями в области информационной безопасности и технологической политики на кафедре компьютерных наук в Гарвардском университете, Брукингском институте, а также школе инженерии и прикладных наук при Университете Джорджа Вашингтона. Он является соавтором популярной статьи *Cybersecurity and Cyberwar: What Everyone Needs to Know*, получил образование в сфере компьютерных наук, окончив Суортморский колледж, и имеет докторскую степень, полученную в Гарвардском университете.

Введение

Мы живем во времена, когда программное обеспечение затрагивает все аспекты нашего общества. Оно применяется повсюду, начиная с критических инфраструктур и цифровой коммерции и заканчивая национальной безопасностью. Согласно данным Всемирного экономического форума (ВЭФ), к концу 2022 года 60 % внутреннего валового продукта (ВВП) во всем мире было привязано к цифровым системам (<https://clck.ru/3CKKQR>).

Тем не менее тот же отчет о перспективах ВВП показывает, что только 45 % людей доверяют технологии, лежащей в основе современных экономик и общества. Отчасти это можно связать с длинной историей громких взломов систем безопасности и давней проблемой с прозрачностью цепочек поставок ПО.

Атаки на цепочки поставок далеко не новое явление, и шаткое доверие к коду корнями уходит еще во времена публикации известной работы Кена Томпсона, вышедшей в 1984 году под названием *Reflections on Trusting Trust*. В ней Томпсон писал, что нельзя доверять коду, который писали не вы. И хотя идея, что код, получаемый из внешних источников, может оказаться ненадежным или откровенно вредоносным, не нова, активно она начала подтверждаться лишь в последние годы, когда все чаще стали происходить атаки на цепочки поставок. Злоумышленники осознали, что вместо нападения на одну сущность, можно скомпрометировать широко используемое ПО (проприетарное или открытое), добившись каскадного эффекта по всей экосистеме потребителей.

По мере учащения подобных инцидентов организации начали больше внимания уделять изучению проблем цепочек поставок ПО и связанных с этим сложностей, а также активнее реализовывать меры безопасности для противодействия соответствующим рискам. Фонд Cloud Native Computing Foundation (CNCF) составил каталог (<https://clck.ru/3CKKRV>) уязвимостей цепочек поставок, обнаруженных с начала 2003 года. В нем перечислены угрозы различного характера, включая взлом инструментов разработчика, небрежность инженеров, недобросовестных мейнтейнеров и даже цепочки атак (когда для реализации атаки используется серия брешей в ПО).

Еще до взлома SolarWinds и выявления уязвимости Log4j Офис директора национальной разведки США (Office of the Director of National Intelligence, ODNI) в 2017 году опубликовал отчет (<https://clck.ru/3CKKSE>), назвав тот год переломным в контексте атак на цепочки поставок ПО. В этом документе описано несколько серьезных атак 2017 года, затронувших организации, поддерживающие Правительство США, а также крупных бизнесменов. Причем некоторые из этих атак были организованы государственными субъектами. В работе ODNI отмечается, что многим каналам разработки и распространения ПО не хватает должной защиты. В ней также говорится, что хакеры начали взламывать более высокоуровневое ПО, поскольку компании стали внимательнее относиться к своей кибербезопасности. Злоумышленнику зачастую оказывается удобнее скомпрометировать вышестоящую систему и атаковать широкий круг нижестоящих потребителей вместо того, чтобы нацеливаться на одну отдельную организацию. Одни атаки на цепочки поставок могут быть бессистемными и нацеленными на любых потребителей, в то время как в процессе других злоумышленники ищут конкретных разработчиков вышестоящего ПО, отталкиваясь от известных им нижестоящих потребителей.

Никто не отрицает, что цифровые системы предоставили нам беспрецедентную эффективность, продуктивность и технологические инновации. Тем не менее эти же системы ПО привели к появлению различных уровней общесистемных рисков, которые еще только предстоит до конца понять. Считается, что комплексные взаимные зависимости повышают системные риски. А текущее состояние экосистемы ПО и современные цифровые решения явно не назовешь простыми. Злоумышленники также осознают потенциал атак на цепочки поставок. Согласно прогнозу Гартнера, лидера индустрии в сфере технологических исследований, к 2025 году жертвами атак на цепочки поставок станут около 45 % организаций. Однако эта оценка может быть занижена. Например, компания Sonatype в 2022 году опубликовала отчет (<https://clck.ru/3CKKSm>), в котором говорится, что за предыдущие три года число атак на цепочки поставок увеличилось на 742 %.

В свете роста количества атак наблюдается повышенное внимание к вопросам безопасности со стороны правительства и компаний частного сектора. Белый дом издал Указ 14 028 «О повышении уровня национальной кибербезопасности» (<https://clck.ru/3CKKX8>). В этом документе есть два интересующих нас пункта: один посвящен усилению безопасности цепочек поставок ПО,

а другой указывает на повсеместный недостаток прозрачности в отношении программного обеспечения, используемого организациями, включая федеральное правительство.

В книге мы подробно разберем актуальные случаи атак на цепочки поставок, действия, предпринимаемые индустрией в их отношении, возникающие решения, а также поговорим о значительных сложностях, связанных с реализацией необходимой защиты.

Почему это важно?

Прежде чем мы начнем разбирать специфику угроз для цепочек поставок ПО, а также вытекающие из этого схемы и руководства, следует прояснить, почему это критически важный вопрос, который затрагивает все аспекты современного общества. Как уже говорилось, цифровые платформы при текущих тенденциях вскоре будут влиять на добрую половину ВВП всей глобальной экономики. В основе этих платформ и систем лежит ПО, значительная часть которого представлена в виде компонентов с открытым исходным кодом (Open Source Software, OSS). Такие программы широко распространены в современной экосистеме ПО. Недавнее исследование (<https://clck.ru/3CKKW9>) показало, что 97 % текущих кодовых баз содержат OSS. Причем больше половины из них сами являются OSS.

Сегодня открытое ПО интегрировано в ряд критических инфраструктур и систем. Более 90 % кодовой базы в таких отраслях, как транспорт, финансовые услуги и промышленное производство, содержат OSS. Аналогичные тенденции наблюдаются также в области телекоммуникации и здравоохранения. Министерство обороны США выпустило памятку *Software Development and Open Source Software* (<https://clck.ru/3CKKXp>). Эта памятка как часть их более обширной стратегии по модернизации сферы ПО (<https://clck.ru/3CKKYU>) описывает OSS как «основу мира, управляемого программным обеспечением, критически важную для ускоренной и надежной поставки ПО, а также выступающую ключевым элементом в стратегии по модернизации». Согласно данной стратегии, «возможность безопасно и быстро поставлять надежное ПО является конкурентным преимуществом, которое будет определять исход будущих конфликтов».

Как подчеркивает Министерство обороны, использование ПО не только критически важно для обеспечения национальной безопасности, но и оказывает

решающее влияние на развитие общества в целом. На слушаниях Комитета по науке, космосу и технологиям Палаты представителей Конгресса США (U.S. House of Representatives Committee on Science, Space and Technology) в рамках доклада *Securing the Digital Commons: Improving the Health of the Open Source Software Ecosystem* (<https://clck.ru/3CKKZK>) несколько выбранных представителей вместе с экспертами из сферы кибербезопасности указали на значимость обеспечения защищенности экосистемы открытого ПО. Конгрессмен Билл Фостер назвал OSS «скрытой рабочей силой цифровой экосистемы». Конгрессвумен Хейли Стивенс заявила, что «динамичная экосистема открытого ПО является двигателем конкурентоспособности и развития США». На саммите по безопасности программного обеспечения с открытым исходным кодом, проведенном Белым домом в 2022 году, было подчеркнуто, что большинство основных программных пакетов содержат OSS, включая ПО, используемое сообществом национальной безопасности (<https://clck.ru/3CKKa7>).

Спикеры также утверждали, что присвоение OSS статуса критической инфраструктуры (наравне с автомагистралями, электросетями, водоочисткой и другими фундаментальными аспектами нашего общества [<https://clck.ru/3CKKbD>]) приведет к привлечению дополнительных ресурсов, улучшит межотраслевую коммуникацию, повысит осведомленность общества и позволит наладить с ним диалог.

Несмотря на повсеместность OSS и его значимость для национальной безопасности, коммерческой индустрии и социума, можно смело заявить, что вопросам его безопасности уделяется недостаточно внимания. В статье *Open-Source Security: How Digital Infrastructure Is Built on a House of Cards* (<https://clck.ru/3CKKc7>) исследовательница Чинмайя Шарма (Chinmayi Sharma) делает вывод, что OSS и связанные с ним уязвимости присутствуют во всех секторах критической инфраструктуры, представляя общесистемную угрозу для общества ввиду нехватки ресурсов и мотивации.

Недостаточное внимание также уделяется реальным злоумышленникам. Так, некоторые исследования показывают, что с 2021 по 2022 год количество атак на цепочки поставок возросло на 650 %. И это не единичный случай: с 2020 по 2021 год прирост составил 400 % по сравнению с предыдущим отчетным периодом. Столь резкий подъем в том числе отражается в каталоге фонда облачно-ориентированных вычислений, содержащем список уязвимостей цепочек поставок.

Об учащении атак также говорится и в исследовании под названием *Breaking Trust: Shades of Crisis Across an Insecure Software Supply Chain* (<https://clck.ru/3CKKd7>) Атлантического совета при НАТО. Эта работа демонстрирует резкое увеличение числа нападений на цепочки поставок ПО, в которых среди основных векторов атаки, помимо OSS, присутствуют сторонние приложения. В отчете задокументировано более 100 атак за последнее десятилетие с использованием различных векторов, таких как перехват обновлений, вредоносные зависимости, взлом платформ разработки ПО и учетных записей. Это указывает на последовательный рост числа атак на цепочки поставок и разнообразие векторов атаки, доступных хакерам для поражения нижестоящего ПО, из-за сложности современной экосистемы. Подобные атаки не только влияют на миллионы пользователей, но также быстро становятся стандартными методами борьбы между государствами на мировой цифровой арене. Применение этих атак со стороны других государств также было подчеркнуто ODNI в публикации 2017 года. Целью становятся не только компоненты OSS, но и поставщики управляемых услуг (Managed Service Providers, MSP), облачные провайдеры (Cloud Service Providers, CSP) и несколько других сущностей, играющих различные роли в современной экосистеме ПО.

Злоумышленники осознали эффективность нападения на компоненты цепочек поставок или самих поставщиков, что позволяет воздействовать на большое количество конечных пользователей. Такой подход оказывается намного продуктивнее, чем атаки на отдельные цели.

В книге мы разберем все темы, начиная с истории возникновения угроз для цепочек поставок ПО и крупнейших инцидентов и заканчивая вытекающими из этого руководствами, техническими возможностями и лучшими практиками, которые можно применить для решения возникающих проблем. Вдобавок ко всему мы поговорим о том, что нас ждет в будущем.

Структура издания

В книге раскрываются темы, связанные с актуальными дискуссиями и сложностями в сфере прозрачности и обеспечения безопасности цепочек поставок ПО. Здесь подробно описываются история возникновения угроз для цепочек поставок, существующие подходы противодействия и появление инновационных инструментов, технологий и процессов, направленных на устранение этой экспоненциально растущей проблемы. Мы рассмотрим

влияние данных угроз на различные аспекты общественной жизни и практические руководства для защиты от них. В книге будут описаны соответствующие законы и их влияние, а также даны прогнозы для индустрии и общества на ближайшее будущее.

Глава 1. История угроз для цепочек поставок ПО. В этой главе раскрываются основные темы, такие как мотивы злоумышленников и анатомия атаки на цепочку поставок, подкрепленные яркими примерами из жизни.

Глава 2. Существующие подходы — традиционное управление рисками поставщиков. Здесь рассматриваются существующие подходы к обеспечению безопасности ПО, такие как управление рисками, модели безопасности приложений и методы вроде хеширования и цифровой подписи.

Глава 3. Базы данных уязвимостей и методы оценки. Здесь мы поговорим о существующих и появляющихся базах данных уязвимостей, а также разберем распространенные методы, используемые для оценки и установления приоритетов уязвимостей в программном обеспечении и приложениях.

Глава 4. Появление концепции SBOM. В этой главе рассказывается об истории развития концепции SBOM, ее неудачах и достижениях, а также о государственных и промышленных организациях, которые способствовали ее внедрению.

Глава 5. Сложности в обеспечении прозрачности ПО. Эта глава акцентирует внимание на проблемах, связанных с обеспечением прозрачности ПО. Здесь речь пойдет об отличиях между открытым и проприетарным кодами, а также о прошивках и встраиваемом ПО.

Глава 6. Облако и контейнеризация. В этой главе описываются эволюция сферы информационных технологий, включая развитие облачных структур и контейнеризации, а также сложности, связанные с обеспечением прозрачности в сфере предоставления ПО как услуги (Software as a Service, SaaS). Здесь также будут описаны усилия, направленные на развитие направления DevSecOps.

Глава 7. Актуальные рекомендации для коммерческих разработок. В этой главе разбираются уже имеющиеся и только возникающие руководства по обеспечению прозрачности ПО и защиты цепочек его поставки со стороны государственных и коммерческих организаций.

Глава 8. Актуальные рекомендации для государственных разработок.

В этой главе освещаются имеющиеся и создаваемые руководства по обеспечению прозрачности ПО и безопасности цепочек его поставок со стороны правительства.

Глава 9. Прозрачность ПО в операционных технологиях. Эта глава посвящена некоторым уникальным аспектам в сфере прозрачности ПО и операционным технологиям, а также их влиянию на более широкую сферу обеспечения безопасности цепочек поставок.

Глава 10. Практическое руководство для поставщиков. Здесь вы познакомитесь с практическим руководством для поставщиков ПО, которое поможет более эффективно внедрять лучшие практики и вносить вклад в повышение безопасности цепочек поставок.

Глава 11. Практическое руководство для потребителей. В этой главе поясняются ситуации, в которых применение концепции SBOM действительно актуально, рассказывается о том, как правильно ее реализовать и как повысить уровень организации по части управления рисками для цепочек поставок ПО.

Глава 12. Перспективы повышения прозрачности ПО. Здесь мы обсудим перспективы обеспечения прозрачности ПО, включая появляющиеся законы и их влияние на более широкие рынки. Помимо этого, мы поговорим о новых технологиях и попытаемся представить обозримое будущее индустрии и общества в целом.

Для кого эта книга будет полезна

Эта книга особенно пригодится профессионалам в сфере технологий и кибербезопасности, например директорам по информационной безопасности (Chief Information Security Officer, CISO), техническим директорам (Chief Technology Officer, CTO), ведущим инженерам в сфере технологий и безопасности, а также энтузиастам из сферы разработки открытого ПО. Кроме того, она может заинтересовать специалистов по закупкам, в задачи которых входят поиск и приобретение безопасного программного обеспечения, а также специалистов по аудиту, желающих понять новейшие руководства и требования в сфере безопасности цепочек поставок. Нелишней книга окажется для исследователей и директивных органов, заинтересованных в освоении новейших практик и изучении угроз для программного обеспечения.

Пояснения

Небольшие общие заметки, заслуживающие отдельного внимания или имеющие значение, выходящее за границы непосредственного контекста, будут приводиться в виде выносок.

Связь с авторами

С Крисом Хьюзом можно связаться по электронной почте chris.matthew.hughes@gmail.com, а также на LinkedIn <http://linkedin.com/in/resilientcyber> или через платформу X по нику @ResilientCyber.

С Тони Тернером можно связаться по почте tony@opswright.com, на LinkedIn <http://linkedin.com/in/tonyturnercissp> или через X по нику @tonylturner.

От издательства

Ваши замечания, предложения, вопросы отправляйте по адресу comp@piter.com (издательство «Питер», компьютерная редакция).

Мы будем рады узнать ваше мнение!

На веб-сайте издательства www.piter.com вы найдете подробную информацию о наших книгах.

1

История угроз для цепочек поставок ПО

<https://t.me/portalToIT>

В этой главе раскрываются такие темы, как мотивы злоумышленников, анатомия атак на цепочку поставок ПО и некоторые яркие примеры таких атак. Начнем с обсуждения мотивов, которые подталкивают хакеров совершать нападения именно на цепочки поставок.

Мотивы злоумышленников

Традиционные виды защиты более уязвимы для атак на цепочки поставок, что очень привлекает хакеров. Компании вкладывают значительные средства в создание межсетевых экранов, предотвращение взломов и управление доступом. Все эти виды защиты применяются против атак в стиле «передачи» (push), непосредственно нацеленных на инфраструктуру компании. Что же касается атак на цепочки поставок, то они действуют по схеме «получения» (pull), когда легитимные пользователи информационной технологии запрашивают уже зараженные программные обновления, непреднамеренно подвергая опасности всю компанию. Поскольку запрос в этом случае происходит от доверенного пользователя, находящегося внутри корпоративной среды, или поступает от иной доверенной сущности, уже одобренной системой управления рисками, получаемые обновления также считаются доверенными. По факту компании компрометируют сами себя.

При изучении средств для защиты от атак становится ясно, что одного уровня недостаточно. По аналогии с тем, как сетевые администраторы осознали потребность мониторинга исходящего трафика или реализации

контроля на стороне хоста, нам также крайне важно углублять защиту, учитывая факторы, выходящие за внешний периметр. Из-за особенностей облачной и мобильной инфраструктур, а также инфраструктуры социальных сетей и современных приложений понятие внешнего периметра необходимо расширить. Теперь его следует рассматривать как дополнительный слой между зонами доверия независимо от того, находятся они на границе вашей сети или выступают логическим барьером внутри вашего приложения либо механизма контроля доступа. Таким образом, при использовании средств контроля и моделировании угроз необходимо учитывать всю поверхность атаки, анализируя каждую точку взаимодействия и доверительную связь.

Атаки на цепочки поставок также выступают в роли своеобразных усилителей, то есть способны оказать воздействие на множество других субъектов, связанных с этой целью через цепочку поставок, тем самым умножая ущерб и разрушения, вызванные атакой. Выявив ключевые зависимости или широко используемое ПО, атакующий может попытаться внедрить вредоносный код в любую среду, которая этот код использует. Подобные случаи происходят регулярно. Принцип работы похож на атаку типа «водопой», когда злоумышленник взламывает сайт, широко используемый целевой группой, например форум, посвященный обсуждению программируемых логических контроллеров, которым пользуются интеграторы промышленных систем управления. Если атакующему удастся скомпрометировать каждого посещающего форум пользователя, он теоретически может получить доступ к любой сущности критической инфраструктуры, на которую этот интегратор работает. Аналогичным образом любое ПО, используемое этими скомпрометированными интеграторами, может внести нежелательную или вредоносную функциональность в среду, где установлено это ПО, даже если человек к тому времени уже будет занят другим проектом.

Все это означает, что атаки на цепочки поставок являются для хакеров крайне привлекательными. В кибершпионаже присутствует экономическая составляющая, которая сильно выигрывает от повторно используемых эксплойтов и низких базовых затрат при реализации атак на цепочки поставок. Кроме того, многие недавние атаки включали использование вирусов-вымогателей, внедряемых через цепочки поставок. И это не только упрощает взлом, но также несет прямую финансовую выгоду для хакера и повышает уровень беспокойства среди организаций, в чей бизнес произошло вмешательство.

Модели угроз

О моделировании угроз часто говорят, но, как показывает опыт, мало какие организации этим реально занимаются. В индустрии принято обсуждать угрозы в весьма обобщенной форме, не разбирая даже контекста ПО или систем, необходимого для правильного их моделирования. Тем не менее в основе этих действий лежат определение поверхности атаки — точек взаимодействия — и изучение возможных проблем.

Для максимальной ясности определим несколько ключевых терминов.

Угроза. Негативное событие, ведущее к нежелательному результату. Угрозы могут быть по своей природе как естественными и незлонамеренными, так и откровенно вредоносными. В качестве примеров можно привести невозможность платежной системы вашего бизнеса регистрировать записи о транзакциях или разрушение дата-центра ураганом.

Источник угрозы. Сущность, ответственная за возникновение угрозы. Примерами являются хакеры, внутренние злоумышленники, враждебно настроенный партнер по бизнесу, скомпрометированный консультант и даже метеосводка. Было отмечено, что источник угрозы нередко определяется ошибочно, в результате чего может выбираться неподходящая модель защиты. Например, если вы считаете, что угроза исходит из страны, известной склонностью к распространению вирусов-вымогателей, но по факту это оказывается страна, ориентированная на шпионаж, как это повлияет на вашу схему защиты? Имеет ли значение фактическая страна-источник угрозы или же это просто создает излишнюю предвзятость?

Реализация угрозы. Действие, предпринимаемое источником угрозы, в конечном счете ведущее к ее возникновению. Например, источник угрозы, скажем хактивист, может подкупить вашего системного администратора, чтобы тот перенастроил платежную систему. Это создаст угрозу, которая в итоге повлияет на финансы компании.

Модель угрозы. Процесс документирования систем и угроз таким образом, чтобы можно было моделировать определенные виды решений, связанные с управлением рисками для системы из-за угрозы. Моделирование угроз преследует различные задачи, включая общее управление киберрисками, проектирование и анализ систем, а также модели обмена информацией.

Методы моделирования угроз

Существует несколько методологий моделирования угроз. Первой разберем методику под названием STRIDE.

STRIDE

STRIDE — это очень распространенная техника, позволяющая определить, что может пойти не так. Ее название представляет собой акроним, каждая буква которого означает категорию угроз.

Spoofing (спуфинг): выдача себя за пользователя или компонент системы с целью получить соответствующий ему доступ.

Tampering (подмена): изменение системы или данных, в результате чего они становятся менее полезными для своих пользователей.

Repudiation (отрицание): правдоподобное отрицание действий, предпринятых от лица конкретного пользователя или процесса.

Information Disclosure (разглашение информации): утечка конфиденциальной информации.

Denial of Service (отказ в обслуживании): приведение системы в состояние недоступности для конечных пользователей.

Elevation of Privilege (повышение привилегий): предоставление пользователю или процессу дополнительного уровня доступа к системе без авторизации.

Большинство атак на цепочки поставок являются результатом работы с системой, и традиционные модели вроде STRIDE спроектированы для противодействия прямым атакам на систему. В этом смысле они противоположны опосредованному влиянию, возникающему, когда администратор запрашивает очередное обновление приложения, которое до этого проблем не вызывало, но на этот раз оказывается вредоносным.

STRIDE-LM

Исследователи из Lockheed Martin расширили методику STRIDE, добавив в нее седьмое измерение, известное как *горизонтальное перемещение*. Несмотря на то что STRIDE хорошо подходит для проектирования систем,

эта техника не отвечает требованиям сетевой защиты. При этом STRIDE-LM предоставляет механизм, с помощью которого защитники реализуют схемы контроля, позволяющие более эффективно защищать систему уже после ее компрометации.

При оценке моделей угроз на предмет их возможного использования в атаках на цепочки поставок нужно задаваться вопросом: «Для чего модель проектировалась и как она вписывается в рассматриваемый сценарий?» Например, многие атаки на цепочки поставок реализуются через этапы обслуживания ПО посредством зараженных обновлений или же опираются на доверительные отношения, за счет чего обходят системы контроля, предназначенные для обнаружения первичной точки входа в программу. Также из-за единой точки отказа, характерной для цепочек поставок ПО, нижестоящее влияние таких действий нельзя эффективно учесть в оригинальной методологии STRIDE. По мере изучения примеров на протяжении книги вы начнете понимать, как использование горизонтального перемещения обеспечивает дополнительный контекст для моделирования и делает технику STRIDE-LM более пригодной для противодействия атакам на цепочки поставок.

Методика оценки рисков проекта OWASP

Методика проекта OWASP применяется в качестве модели количественной оценки конкретных рисков, используя критерии технической угрозы и влияния на бизнес. В основе данной методики лежит простая формула: *влияние × вероятность*. Самое же интересное в том, как создаются эти две составляющие. Ниже описывается, из чего конкретно строится эта формула, но здесь нужно учесть, что на практике иногда множители изменяются или же к наиболее значимым из них применяются веса. Например, в критической инфраструктуре мы видим, что многие сущности желают добавить к оценке влияния на бизнес пятый фактор «безопасности» и зачастую наделяют этот критерий бóльшим весом.

Вероятность = AVG (источник угрозы + уязвимость),

где *источник угрозы* = навык, мотивация, возможность и размер; а *уязвимость* = легкость обнаружения, удобство эксплуатации, осведомленность и обнаружение вторжения.

Влияние = AVG (технические последствия + бизнес-последствия),

где *технические последствия* = утрата конфиденциальности, целостности, доступности и отслеживаемости; а *бизнес-последствия* = финансовый ущерб, ущерб для репутации, несоответствие требованиям и конфиденциальность.

Показатель рисков = вероятность × влияние.

OWASP пригоняется для оценки обнаруженного риска и определения приоритета реагирования на него, но эта метрика не так полезна для моделирования угроз с целью отслеживания неизвестных рисков. Внедрение OWASP в ваши процессы может иметь смысл *после* применения других техник моделирования угроз. По нашему опыту данный метод намного превосходит такие механизмы приоритизации, как общая система оценки уязвимостей (Common Vulnerability Scoring System, CVSS), но для эффективного применения требует наличия контекста.

DREAD

Техника DREAD — это наследие Microsoft. Мы редко встречали ее применение в современном цифровом мире. На сегодняшний день она считается «мертвой» методологией. Ее название можно разбить также, как STRIDE.

Damage (ущерб): насколько серьезной будет атака?

Reproducibility (воспроизводимость): насколько легко будет воспроизвести атаку?

Exploitability (эксплуатируемость): насколько сложно будет запустить атаку?

Affected users (потенциальные пострадавшие): какое количество людей окажется подвержено атаке?

Discoverability (обнаруживаемость): насколько легко обнаружить угрозу?

Использование деревьев атак

Деревья атак (<https://click.ru/3CKMaw>) позволяют наглядно прорисовать путь от момента нежелательных последствий (чтобы понять, как это произошло и каковы наиболее вероятные векторы атаки), а также создать ранжированный список методов контроля, которые нужно реализовать для предотвращения возникших последствий. На рис. 1.1 показан самый «дешевый»

путь, которым злоумышленник может открыть сейф. В данном случае будет разумным сделать упор на системы физического контроля, чтобы предотвратить доступ к сейфу в принципе, или же купить более дорогой сейф, устойчивый к подобным атакам. И хотя внутренние угрозы являются допустимым вектором атаки, из экономических соображений атакующий вряд ли пойдет подобным путем.

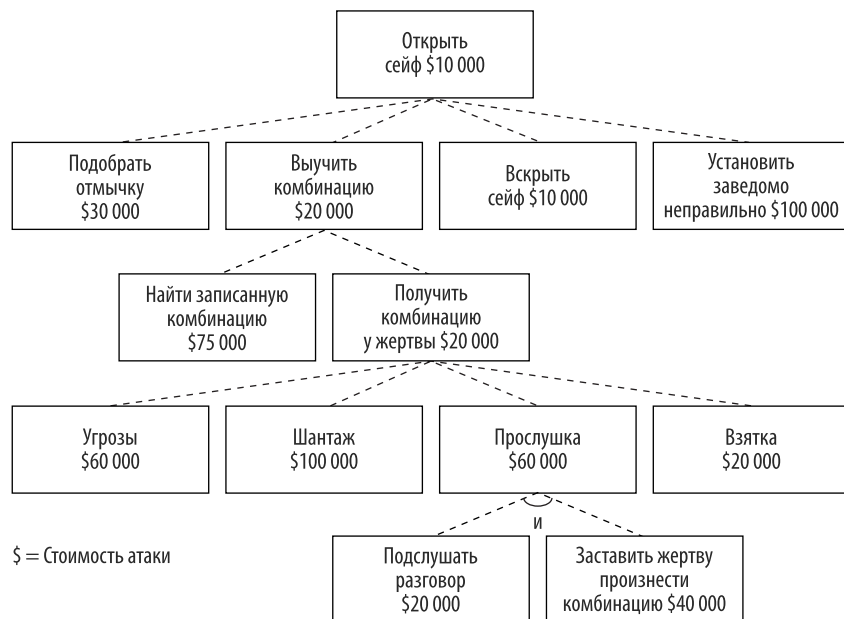


Рис. 1.1

Это хороший подход для изучения принципов атак на цепочки поставок и осознания того, насколько легко их реализовывать в сравнении с более традиционными атаками. В случае последних половина успеха заключается в получении физического доступа, необходимого для обхода системы контроля доступа. В случае же атаки на цепочку поставок в действиях, необходимых для взлома системы, участвует доверенная сущность.

В таком сценарии приходится преодолеть гораздо меньше преград. Разберем пример атаки, построенной по принципу тайпсквоттинга (рис. 1.2). Тайпсквоттинг — это метод мошенничества, когда злоумышленники выкладывают на GitHub зараженные пакеты с именами, похожими на имена настоящих, маскируя тем самым вредоносный компонент под вид легитимной библиотеки. Имейте в виду — это неполный пример.

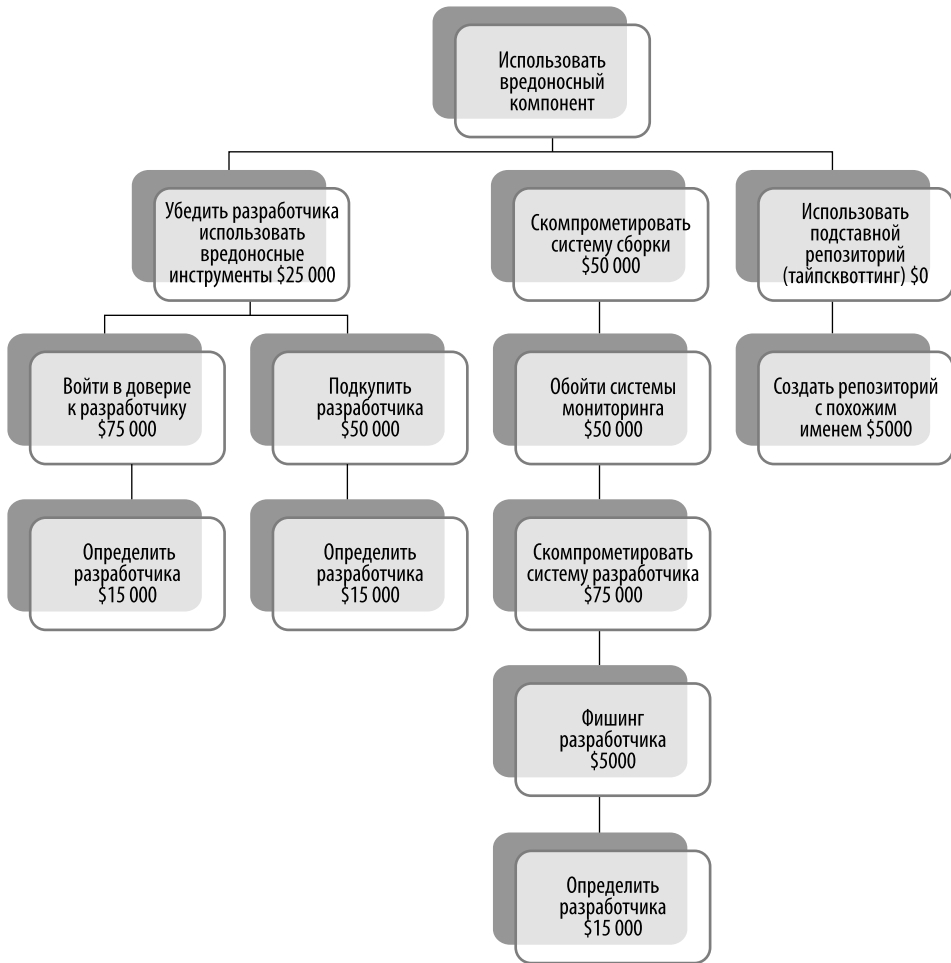


Рис. 1.2

Для сравнения подумайте, какие шаги потребуются, чтобы реализовать атаку. Исследователи из Lockheed Martin разработали методику, названную ими Cyber Kill Chain (<https://clck.ru/3CKMbZ>), которая описывает этапы атаки. В этом процессе есть много нюансов, но на рис. 1.3 представлены только основные шаги.



Рис. 1.3

Что, если злоумышленник может пропустить многие из этих шагов и перейти сразу к получению контроля? Для него окажется крайне заманчивым такой вариант, который позволит уменьшить затраты и сложность выполнения атаки, что вполне реально за счет анализа и понимания доступных путей.

Моделирование угроз

В этом разделе мы опишем типичный процесс моделирования угроз. Для начала вам нужно выбрать систему или приложение, которое вы создаете либо обновляете. Определите ее компоненты и то, каким образом они взаимодействуют друг с другом. Так вы начнете закладывать основу для определения поверхности атаки системы. Это может быть, к примеру, крайне востребованный API или HTTP-сервис. Возможно, в ней присутствуют сторонние зависимости вроде промежуточного ПО или сервера базы данных (БД), с которым необходимо взаимодействовать для выполнения логики приложения, или службы аутентификации, использующие интеграцию. Как бы то ни было, но для начала определения поверхности атаки вам потребуется фундаментальное понимание архитектуры.

В стандарте для проверки безопасности приложений OWASP ASVS (Application Security Verification Standard, <https://clck.ru/3CKMdh>) это понимание архитектуры определяется в качестве основного требования, включая обязательное моделирование угроз во время проектирования и внесения изменений.

Стандарт для проверки программных компонентов OWASP SCVS (Software Component Verification Standard, <https://clck.ru/3CKMgN>) расширяет данную концепцию, внося в нее список сторонних зависимостей, о чем мы поговорим позднее. Тем не менее становится ясно, что для понимания угроз и способов защиты от них необходимо уверенно ориентироваться в защищаемой системе.

Для документирования системы можно использовать различные инструменты. Один из типичных — это разработанный Microsoft инструмент моделирования угроз. Долгое время он являлся единственным эффективным решением, но индустрия не стоит на месте. Этот инструмент позволяет архитекторам ПО выявлять потенциальные угрозы на ранних стадиях процесса разработки еще до того, как их устранение станет затратным. Моделирование угроз в нем реализуется с помощью модели STRIDE.

На рис. 1.4 представлен простой контейнер приложения, разделяющий контексты интернет-доступа к открытому веб-сервису и локального доступа вплоть до уровня файловой системы с помощью границ доверия. Подобное разделение системы позволяет вам определить, как потенциальный злоумышленник может взаимодействовать с ней, и ответить на вопросы, касающиеся второй части этого процесса. Существуют и другие бесплатные инструменты, например *pytm* от OWASP, позволяющий автоматизировать моделирование угроз с помощью определений из общего перечня и классификации шаблонов атак (Common Attack Pattern Enumeration and Classification, CAPEC), а также *Threat Dragon*, который больше похож на инструмент Microsoft. Помимо этого, есть очень надежные коммерческие решения, такие как *Threat Modeler* и *IriusRisk*, построенные с учетом вышеописанных принципов стандарта ASVS.

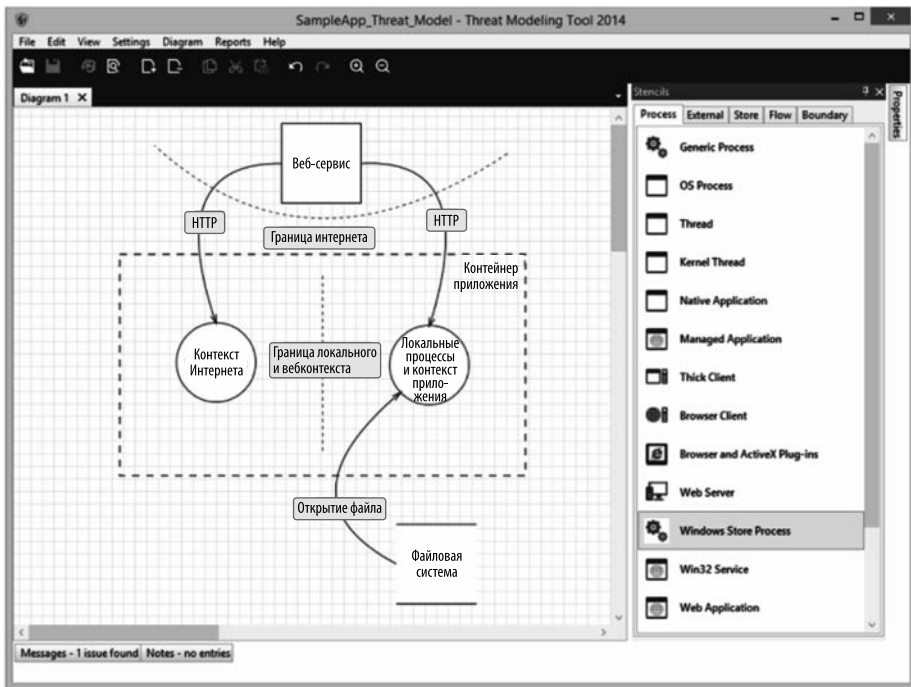


Рис. 1.4

Следующим шагом в моделировании угроз идет определение возможных проблем безопасности. Ими может оказаться как простое условие возникновения ошибки или пользователь, которого не научили правильно работать

с приложением, так и более значительные факторы вроде хакера, замышляющего взлом системы. Мы сосредоточимся на сценариях целенаправленного киберсаботажа.

Чтобы проиллюстрировать потенциальный сценарий угрозы атаки на цепочку поставок, возьмем рис. 1.4, где злоумышленник внедрил вредоносные записи журнала в веб-службу. В свою очередь, эта служба при локальном открытии средством просмотра журналов выполняет вредоносный JS-код в браузере пользователя. Если эту службу потребляют многие пользователи, то одна такая атака затронет их всех. В итоге возникает сценарий, когда злоумышленник, скомпрометировав одну веб-службу, нарушил работу множества организаций и в итоге получил плацдарм для дальнейших атак этих организаций либо их нижестоящих потребителей, доверяющих им как элементам своих цепочек поставок. Подобная атака путем манипуляций с данными может оказать колоссальное каскадное влияние.

После выявления возможных проблем необходимо определить, какие системы управления можно реализовать для минимизации угроз. Этот процесс наверняка будет включать в себя расстановку приоритетов, поскольку невозможно исключить все потенциальные угрозы. Однако акцентирование внимания на наиболее значимых сценариях, имеющих высокую вероятность возникновения, поможет в обсуждении необходимых систем управления доступом.

Например, в описанной выше ситуации дополнительную защиту от подобной атаки, возможно, обеспечит проверка вносимых в журнал записей или экранирование символов. Поскольку существует множество способов, которыми атакующие могут обойти традиционные системы защиты, например защиту от межсайтового скриптинга (Cross-site Scripting, XSS), будет нелишним рассмотреть аналогичные техники, описанные в OWASP XSS Prevention Cheat Sheet (памятка по противодействию XSS). Эта памятка содержит список техник для предотвращения или ослабления последствий XSS-атак (<https://clck.ru/3CKMhC>).

Заключительная стадия моделирования угрозы обычно состоит в оценке степени готовности модели для применения и того, насколько четко она описывает границы доверия. Во-первых, проанализируйте, охватили ли вы все вредоносные действия, прописанные в используемой вами модели, такой как STRIDE, и все ли потоки данных проанализировали. Наконец, определите, для всех ли выявленных угроз вы выработали план действий, даже

если было установлено, что необходимые меры противодействия находятся вне вашего контроля и не могут быть применены. Завершать цикл нужно проверкой модели и повторять эту процедуру всякий раз, когда в систему вносятся значительные архитектурные изменения.

Кроме того, мы считаем, что существенную роль в контексте угроз для цепочек поставок играют следующие принципы.

- **Провенанс (Provenance).** Провенанс (источник) — это описание того, откуда возник артефакт. Этот принцип часто путают с происхождением (следующий пункт). Провенанс в случае ПО может описывать то, откуда конкретный компонент взялся или кто внес код в конкретную библиотеку. В организациях редко отслеживают внесение изменений на уровне отдельных авторов, но в сценариях, требующих особого внимания к безопасности, это может оказаться оправданным. На собственном опыте мы убедились, что почти в каждую открытую библиотеку вносят вклад участники из потенциально враждебных стран. Поэтому для организаций, которые решают прибегнуть к столь детальному отслеживанию, можно порекомендовать дополнительные системы противодействия или более глубокий анализ для определения, не вызывают ли эти авторы подозрений.
- **Происхождение (Pedigree).** Принцип «отслеживания» не нов. В действительности он является фундаментальным в криминалистике и тесно связан с применением «цепочки ответственности», то есть журнала, в который вносится каждый, кто прикасается к артефакту с момента его создания. Отслеживание указывает на перемещения артефакта, а цепочка ответственности — на тех людей или организации, которые к нему прикасались. По аналогичному принципу была разработана концепция безопасной доставки оборудования, позволяющая отслеживать перемещение материальных активов по всему миру. Она включает использование уникальных идентификаторов Lot ID и Global Trade Identifiers семейства стандартов GS1 и таких средств защиты, как физически неклонлируемые функции (Physical Unclonable Function, PUF). Поскольку мы в первую очередь описываем цифровой артефакт в его отношении к прозрачности ПО, для выполнения этих требований подходит использование криптографических журналов и журналов прозрачности. Происхождение в отношении прозрачности ПО может описывать историю программного пакета или патча безопасности для этого пакета и является принципом, который можно задокументировать в некоторых форматах перечней компонентов

ПО. Кроме того, концепция происхождения также позволяет отслеживать изменение исходной точки, например, в случае перехода репозитория GitHub от одного мейнтейнера к другому или отказа поставщика от выпуска определенной линии продукции и ее перепродажи сторонней организации. Все эти принципы будут подробно рассмотрены в книге.

Многие фреймворки моделирования угроз, такие как DREAD и STRIDE, сегодня используются для различных задач. Их зачастую применяют для описания схем безопасности и анализа. Фреймворк Consequence-Driven Cyber-Informed Engineering (CCE) организации Idaho National Labs (INL) ориентирован на управление угрозами, связанными с конкретными последствиями для бизнеса, а не обязательно с каждой точкой поверхности атаки. Еще есть такие фреймворки, как STIX и PRE-ATT&CK, предназначенные для обмена информацией и прочих подготовительных этапов.

Фреймворки CAPEC и MITRE ATT&CK используют конкретную технологию или ориентированы на определенный контекст. К фреймворкам, созданным под задачи конкретных организаций, относится NIPRNet/SIPRNet Cyber Security Architecture Review (NSCSAR). https://t.me/it_boooks/2

Аналогичным образом в масштабном моделировании угроз основной акцент делается на паттерны атаки. Причина, по которой организации моделируют угрозы ошибочно, заключается в том, что эту задачу сложно реализовать в больших масштабах. В цикле разработки безопасного программного обеспечения Microsoft (Security Development Lifecycle, SDL) моделирование угроз числится в качестве четвертого по счету действия из двенадцати — сразу же после определения требований безопасности и ключевых показателей производительности (Key Performance Indicators, KPI) и перед проектированием. И такое расположение этой задачи вполне разумно при проектировании новой системы, но как следует моделировать угрозу для сотен или тысяч приложений в нашей среде? Как правило, в таких случаях мы ищем легкие пути. Именно поэтому использование паттернов атак столь популярно, особенно если мы можем сократить их количество на основе выявленных злоумышленников или целевых жертв этих злоумышленников.

CAPEC является открытым каталогом с описанием и классификацией типов атак на компьютерные системы и сети. Он предоставляется корпорацией MITRE и изначально создан Министерством внутренней безопасности США. В нем есть запись для *цепочек* поставок (<https://clck.ru/3CKMip>), которая помогает защитникам понять принципы построения этих атак, чтобы более эффективно выбирать меры противодействия:

Паттерны атак в этой категории ориентированы на вмешательство в жизненный цикл цепочки поставок посредством манипуляций с аппаратным или программным обеспечением системы либо сервисами в целях шпионажа, кражи важных данных или технологий, а также нарушения работы критически важных операций либо инфраструктур. Операции цепочек поставок обычно являются многонациональными, то есть их компоненты, сборка и доставка реализуются разными странами, предлагая атакующему множество точек для вмешательства.

К этому семейству паттернов атак также относятся:

- раскопки (excavation);
- атаки на целостность ПО;
- изменение во время производства;
- манипуляции во время распространения;
- атаки на целостность информации об аппаратном обеспечении.

Помимо этого, полезными могут оказаться сопутствующие материалы из MITRE ATT&CK, такие как запись, посвященная *компрометированию цепочек поставок* (<https://clck.ru/3CKMjb>), где также отражены три дополнительные техники вместе с методами противодействия, которые пригодятся защитникам:

- компрометация программных зависимостей и инструментов разработки;
- компрометация цепочки поставок ПО;
- компрометация цепочки поставок аппаратного обеспечения.

К сожалению, на данном этапе они предоставляют довольно базовый уровень защиты цепочек поставок, и разумнее будет совместить этот подход с более специализированными решениями вроде CNCF, которые могут обеспечить намного бóльшую специфичность, необходимую для защиты от угроз. CNCF ведет каталог угроз для цепочек поставок, в котором также присутствует индекс (<https://clck.ru/3CKMkT>), определяющий несколько типов атак. Индекс включает в себя:

- инструменты разработки;
- халатность;
- выпускающую инфраструктуру;

- исходный код;
- доверие и подписи;
- недобросовестного мейнтейнера;
- цепную атаку.

Стоит отметить, что несколько видов атак связаны с существующими фреймворками, такими как АТТ&СК корпорации MITRE, в котором есть разделы, посвященные угрозам для цепочек поставок, включая программные зависимости, инструменты разработки и саму цепочку поставок.

Нападения на инструменты разработки представляют собой взлом программ, используемых при разработке ПО. Это может быть конечное устройство разработчика, наборы разработки и цепочки инструментов. MITRE рекомендует использовать механизмы проверки целостности, такие как валидация и сканирование скачиваемых программ на предмет вредоносных признаков. Если злоумышленник может скомпрометировать инструменты разработки, то он сумеет внести потенциально вредоносный код в самом начале цикла разработки ПО (Software Development Life Cycle, SDLC) и скомпрометировать все последующие процессы разработки вместе с потребителями.

Халатность — это когда человек пренебрегает рекомендуемыми практиками. Причем встречается она довольно часто, учитывая, что существует огромное множество передовых методов обеспечения безопасности и наша цифровая экосистема непрерывно усложняется. Такой простой нюанс, как пренебрежение проверкой имени зависимости, может подвергнуть организацию серьезной угрозе. Злоумышленники все чаще используют атаки, именуемые *тайпсквоттингом*, которые, как уже говорилось, ориентированы на невнимательность потребителя к деталям имен скачиваемых зависимостей. Атакующие обычно нацеливаются на популярный фреймворк или библиотеку, вносят свой вредоносный код под именем, похожим на имя оригинального пакета и затем ожидают, пока ничего не подозревающая жертва скачает и использует этот пакет в своем приложении.

Выпускающая инфраструктура становится все более важной в свете того, что для доставки программных компонентов организации зачастую используют конвейеры и платформы непрерывной интеграции/поставки (Continuous Integration/Continuous Delivery, CI/CD). Одной из техник противодействия атакам на эту инфраструктуру является использование цифровой подписи исполняемых файлов, которая позволяет обеспечить сохранение

целостности опубликованной программы. Тем не менее, как вы увидите в первом примере, компрометация самой инфраструктуры CI/CD может позволить атакующим легитимно подписывать программные компоненты, которые будут восприниматься потребителям как доверенные. Подобный метод атаки может привести к разрушительным последствиям. Он дополнительно подчеркивает, почему выпускающая инфраструктура должна быть защищена так же, как производственная среда, и что эти среды необходимо согласовывать с помощью такого фреймворка, как Supply Chain Levels for Software Artifacts (SLSA), о котором мы поговорим в последующих главах.

Атаки на исходный код включают взлом репозитория исходного кода, будь то напрямую через разработчика или через его учетные данные.

В 2022 году у компании Verizon вышел отчет под названием *Data Breach Investigations Report* (<https://clck.ru/3CKMmA>), в котором отражено, что несанкционированный доступ к учетным данным фигурировал более чем в 80 % эпизодов взломов данных. Сюда относятся случаи, влияющие на исходный код или репозитории исходного кода. Злоумышленники, нацеленные на исходный код и репозитории, зачастую пытаются внести в них уязвимости либо бэкдоры, чтобы позднее воспользоваться ими или воздействовать на нижестоящих потребителей.

Проверка целостности является критическим элементом принципа доверия в цепочке поставок ПО. Как правило, ее реализуют с помощью цифровых подписей и аттестаций. Подписывание кода обеспечивает для нижестоящих потребителей уровень достоверности в отношении провенанса кода и его целостности.

Таким образом, компрометация процесса подписывания может способствовать реализации атак на цепочки поставок. В связи с этим ключевую роль продолжают играть фундаментальные принципы безопасности вроде многоуровневой защиты. *Многоуровневая (эшелонированная) защита* — это давняя техника кибербезопасности, подразумевающая использование многослойной защиты таким образом, чтобы никакая уязвимость или слабость не вела к полномасштабной компрометации системы либо организации. В таком случае атакующим для достижения своих целей вместо эксплуатации одной слабости приходится преодолевать уже несколько слоев защиты и мер безопасности. Иногда одной только цифровой подписи недостаточно. Подобный вид атаки находится в списке индекса CNCF и указан в разделе, посвященном уязвимостям цепочек поставок. Потенциальные угрозы включают такие действия, как кража закрытых ключей, злоупотребление доверием,

нарушения при выдаче сертификатов и не только, о чем говорится в работе Национального института стандартов и технологий (National Institute of Standards and Technology, NIST) под названием *Security Considerations for Code Signing* (<https://cick.ru/ЗСКМnY>). К техникам противодействия в этом случае относятся такие приемы, как установка доверенных пользователей, разделение ролей, использование сложной криптографии и защита подписей ключей.

Но с учетом сложности цепочки поставок ПО и того факта, что мейнтейнеры зачастую трудятся на добровольных началах, не все угрозы исходят извне проекта. Вектор атаки, связанный с недобросовестным мейнтейнером, также находится в списке CNCF. Это может быть умышленное внедрение вредоносного ПО или уязвимостей как по решению самого мейнтейнера, так и в результате взлома его учетной записи. На подобные атаки могут пойти, например, хактивисты или злоумышленники, выдающие себя за мейнтейнеров, которые используют полученные разрешения и доступ для реализации своих преступных замыслов.

В завершении отметим, что атаки и уязвимости возникают не только по отдельности. Злоумышленники могут реализовывать свои коварные планы, выстраивая цепочку из нескольких уязвимостей и векторов атаки. Если рассматривать некоторые из описанных выше видов атак, гипотетический сценарий цепной атаки может выглядеть следующим образом. Преступник выступает в роли заинтересованного исследователя либо мейнтейнера исключительно с целью получить соответствующий уровень доступа, внедрить зараженный код, эксплуатировать механизм подписания кода и так далее. Объединение нескольких векторов атаки способно оказать разрушительное влияние и обычно называется цепной атакой. Этот вид атак активно обсуждается специалистами вроде доктора Никки Робинсон. Постепенно появляются новые методологии и фреймворки, способные обезопасить цепочки поставок ПО и позволяющие смоделировать потенциальные угрозы. К примерам таких фреймворков относится вышеупомянутый *SLSA*, подробнее о котором мы поговорим в последующих главах.

Нашумевший случай 1: SolarWinds

Ни одно обсуждение современных инцидентов в сфере безопасности цепочек поставок ПО не будет полноценным без упоминания SolarWinds. SolarWinds — это один из крупнейших провайдеров инструментов цифрового менеджмента на рынке. В 2019 году компания подверглась атакам, которые

впоследствии были приписаны российской службе внешней разведки. На момент нападения SolarWinds обслуживала 300 000 клиентов, включая множество государственных служб США и большую часть компаний из Fortune 500. По приблизительным оценкам, 18 000 клиентов из 300 000, включая правительственные организации и частных пользователей, получили зараженное обновление ПО. Заявлено, что после этого злоумышленники выбрали определенный список из скомпрометированных клиентов, которые являлись для них особо ценными мишенями.

И хотя реализация первых стадий атаки остается не совсем понятной, на данный момент существует множество отчетов, которые проливают свет как на высокую техническую сложность атаки, так и на ее последствия. В одном из таких отчетов, составленном Счетной палатой США, взлом SolarWinds был назван «одной из наиболее широкомасштабных и сложных хакерских кампаний, совершенных в отношении федерального правительства и частного сектора, за всю историю» (<https://clck.ru/3CKMpm>).

Счетная палата США также выпустила отчет, представленный на рис. 1.5 (<https://clck.ru/3CKMra>), чтобы показать высокоуровневую картину развертывания атаки на SolarWinds.



Рис. 1.5

Компания SolarWinds стала хрестоматийным примером атаки на цепочку поставок ПО, поскольку атакующих интересовала не сама платформа, а скорее ее клиенты и нижестоящие потребители. До этой атаки SolarWinds гордо публиковала на своем сайте широкий список высокопоставленных клиентов, который впоследствии удалила.

Атака на SolarWinds изначально была выявлена одним из ее клиентов, компанией FireEye, которая как раз специализируется на кибербезопасности. Сообщается, что сотруднику FireEye поступило подозрительное предложение сбросить настройки многофакторной аутентификации, о чем он сообщил команде. В итоге компания занялась расследованием инцидента. Технический директор Mandiant, дочерней компании FireEye, ответственной за реагирование на инциденты, заявил, что организации пришлось перебрать 50 000 строк кода, чтобы обнаружить наличие бэкдора в ПО SolarWinds Orion. Как только FireEye определили, что взломанное программное обеспечение принадлежит SolarWinds, они уведомили об этом и поставщика, и правоохранительные органы.

И хотя в разных источниках детали атаки описываются по-разному, есть фундаментальные факты, которые были указаны в ранее упомянутом отчете Счетной палаты США. Процесс развертывания атаки во времени можно рассмотреть с позиции как действий подверженного ей частного сектора, так и реагирования государственных структур, когда об этом узнали правоохранительные органы и к делу подключилось ФБР совместно с CISA.

Сообщается, что первое проникновение в системы SolarWinds произошло в сентябре 2019 года. Тем не менее на конференции RSA Conference, прошедшей в 2021 году (<https://clck.ru/3CKMtb>), генеральный директор SolarWinds Судхакар Рамакришна сообщил, что первую разведывательную активность удалось проследить до января 2019 года, когда хакеры внедрили тестовый код в среду SolarWinds с целью убедиться, смогут ли они действительно взломать системы и реализовать задуманные действия.

Злоумышленники скомпрометировали процесс разработки и сборки ПО SolarWinds с помощью вредоносной программы Sunspot. Затем, ориентировочно в феврале 2020 года, они внедрили в продукт Orion бэкдор, позднее названный Sunburst. В марте того же года был выпущен пакет исправлений, который скачали 18 000 клиентов SolarWinds. В июне 2020-го злоумышленники удалили вредоносное ПО, которое ранее внедрили в машины сборки SolarWinds. И только в декабре 2020 года компанию уведомили

об обнаружении Sunburst. После этого SolarWinds вместе с Комиссией по ценным бумагам и биржам США (Securities and Exchange Commission, SEC) выпустили отчет по форме 8-K (<https://clck.ru/3CKMur>), который пролил свет на ситуацию в той степени, в которой она была ясна для компании на тот момент.

Затем 15 декабря 2020 года SolarWinds выпустили исправление для Orion. Получается интересный парадокс, если учесть, что их предыдущее обновление оказалось носителем вредоносной программы, которая и заразила клиентов. Далее 17 декабря 2020 года CISA объявило тревогу (AA20-352A; <https://clck.ru/3CKMvN>), предупредив о целевой кибератаке (Advanced Persistent Threat, APT) на правительственные учреждения, критическую инфраструктуру и частный сектор. В этом оповещении говорилось об обнаружении заражения пяти версий динамически подключаемой библиотеки (Dynamic Link Library, DLL). В нем также отмечались терпеливость хакеров, сложность атаки и общий высокий уровень ее подготовки. Дополнительная информация, связанная с Sunspot, была обнаружена в январе 2021 года.

Компания CrowdStrike, специализирующаяся на кибербезопасности, опубликовала подробный технический анализ атаки на SolarWinds (<https://clck.ru/3CKMww>). Эксперты выяснили, что злоумышленники использовали вредоносное ПО, известное как Sunspot, чтобы скомпрометировать процесс сборки программного обеспечения и внедрить бэкдор Sunburst в продукт SolarWinds Orion, предназначенный для комплексного управления ИТ-инфраструктурой. Sunspot отслеживал процессы, участвующие в компиляции Orion, и заменил один из файлов исходного кода кодом Sunburst. Затем этот бэкдор позволил создать обратную оболочку, через которую злоумышленники получили доступ к инфраструктуре скомпрометированных жертв, запустивших зараженную версию Orion.

О кибератаке на SolarWinds, задействованном в ней вредоносном ПО и бэкдорах можно сказать еще многое. Это была одна из крупнейших и сложнейших атак на цепочки поставок ПО. Некоторые государственные и коммерческие деятели назвали ее «звоночком» и подчеркнули необходимость в повышенной бдительности в сфере обеспечения безопасности цепочек поставок.

Отчасти причиной столь масштабного успеха атаки явилось то, что злоумышленники сделали ставку на давнюю практику в области кибербезопасности: обязательный выпуск патча для программного обеспечения сразу же после появления обновления от его поставщиков. В данном примере обновление

оказалось заражено, поэтому те, кто следовал этой стратегии, своевременно обновляя свое ПО, оказались уязвимы для атаки. Компрометация процесса сборки SolarWinds также позволила злоумышленникам передавать взломанное ПО как подписанное и легитимное.

С тех пор компания SolarWinds инвестирует значительные средства в усиление безопасности процессов сборки и реализацию дополнительных мер. Согласно некоторым отчетам и комментариям, например от начальника отдела информационной безопасности Тима Брауна, реализация инициативы «Безопасное проектирование» обойдется компании в 20 миллионов долларов ежегодно (<https://clck.ru/3CKMyD>).

Представители SolarWinds приняли участие в вебинаре под названием «Обеспечение безопасности среды разработки» (*Securing the Software Development Build Environment*; <https://clck.ru/3CKMzD>), где обсуждали свой подход «безопасного проектирования», включающий использование воссоздаваемых сборок для исключения несоответствий в двоичном коде. Сюда входит изменение используемой компанией схемы управления версиями и зависимостями, а также присутствие нескольких сред, которые злоумышленнику придется взломать, чтобы добраться до исходного кода организации. Воссоздаваемые сборки (<http://reproducible-builds.org>) упоминаются во фреймворке SLSA, в частности в SLSA Level 4 (<http://slsa.dev/spec/faq>). Мы рассмотрим эти виды сборок в последующих разделах, однако надо отметить, что они представляют собой зрелую реализацию, которую крупным поставщикам ПО следовало бы использовать изначально и которая, по словам начальника отдела по информационной безопасности в SolarWinds, не является дешевым или поверхностным решением.

Нашумевший случай 2: Log4j

Если кибератака на SolarWinds была нацелена на конкретного вендора, то в случае Log4j под угрозой было широко используемое OSS. Проблема Log4j привлекла внимание общественности 9 декабря 2021 года, когда специалисты по безопасности обнаружили уязвимость в популярной библиотеке Log4j. Log4j — это утилита для ведения журнала на основе Java, входящая в состав Apache Logging Services, проекта Apache Software Foundation. В то время эта библиотека в основном использовалась разработчиками для регистрации информации, связанной с отладкой. На момент инцидента Log4j, по приблизительным оценкам, присутствовала более чем в 100 миллионах сред и приложений.

На следующий день, 10 декабря 2021 года, уязвимость Log4j была внесена Институтом NIST в национальную базу данных уязвимостей (National Vulnerability Database, NVD), получив оценку 10.0 по шкале CVSS. Помимо этого, она была зарегистрирована в системе CVE под кодом CVE-2021-44228. Впоследствии были опубликованы и другие CVE, связанные с Log4j.

Вскоре после публикации уязвимости нулевого дня новозеландская группа реагирования на критические происшествия в компьютерных сетях предупредила, что она уже активно эксплуатируется в Интернете (<https://clck.ru/3CKN3h>). Следом агентство CISA выпустило срочную директиву (<https://clck.ru/3CKN4a>), требуя от государственных структур принять меры в отношении Apache Log4j. Затем 22 декабря 2021 года CISA, FBI и NSA (National Security Agency, Агентство национальной безопасности) совместно с международными партнерами опубликовали рекомендации (<https://clck.ru/3CKN53>) по противодействию этой уязвимости. Директор CISA Джен Истерли назвала уязвимость Log4j серьезной актуальной угрозой для организаций и правительств по всему миру. Аналогичные заявления делали и лидеры многих других государств. Схему развертывания инцидента с Log4j вы можете пронаблюдать на рис. 1.6 (<https://clck.ru/3CKN5a>). В отличие от SolarWinds, которая повлияла на определенное семейство продуктов конкретного поставщика, атака на Log4j оказалась гораздо масштабнее. Эта библиотека использовалась во всевозможном ПО, начиная с инструментов разработчика и облачных сервисов, заканчивая продуктами для обеспечения информационной безопасности. Как будет говориться в главе, посвященной сфере облачных услуг, крупнейшие облачные провайдеры, такие как Amazon Web Services (AWS), Microsoft Azure и Google Cloud, опубликовали руководства по противодействию уязвимости Log4j, поскольку на момент инцидента данная утилита использовалась во многих их сервисах. Естественно, это могло оказать каскадное влияние не только на клиентов, непосредственно использующих облачные сервисы, но и на другие организации, опирающиеся на облачную инфраструктуру.

Получив информацию от Счетной палаты США и других сторон, Экспертный совет по вопросам кибербезопасности (Cybersecurity Safety Review Board, CSRB) также обозначил Log4j в качестве приоритетного инцидента, подлежащего расследованию и составлению отчета (<https://clck.ru/3CKN6o>). Этот совет был создан по Указу Президента США (<https://clck.ru/3CKN83>) и рассматривался как аналог Национального совета по безопасности перевозок в том смысле, что его целью выступали мониторинг главных киберсобытий и составление конкретных рекомендаций для внесения доработок как в государственный, так и в частный сектор.

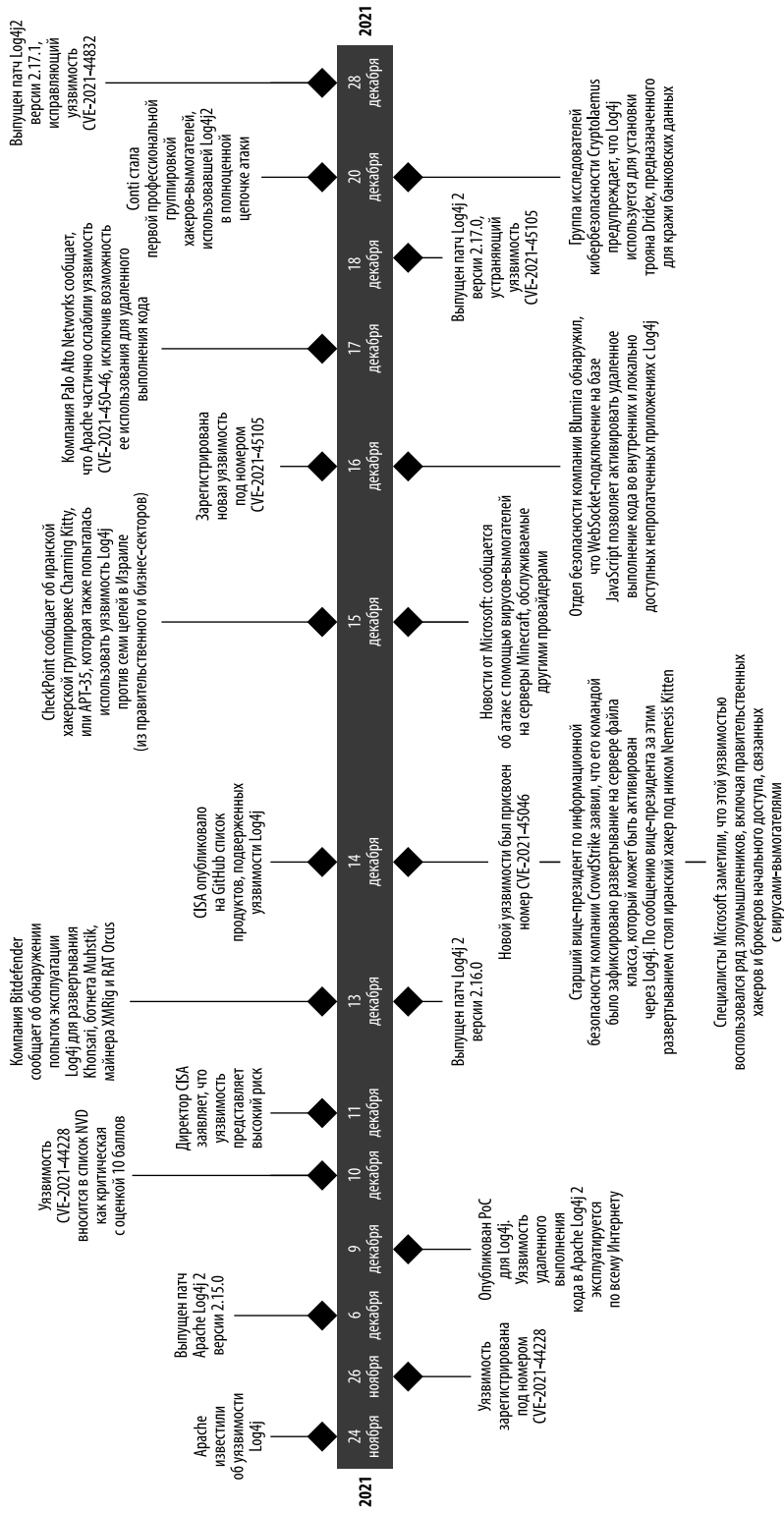


Рис. 1.6

Членами этого совета также выступают лидеры из государственного и частного секторов, имеющие различные уникальные знания и опыт. В их первом отчете, который, как уже упоминалось, был посвящен Log4j, говорится об острой необходимости в повышении прозрачности ПО, его инвентаризации и управлении с применением концепции SBOM как основы. Отчет призывает организации в целом использовать перечни компонентов для повышения точности инвентаризации IT-активов и приложений, а такие компании, как Office of Management and Budget (OMB), Office of the National Cyber Director и CISA, — обеспечить руководства для эффективного применения SBOM по мере взросления экосистемы. SBOM упоминается в отчете 18 раз с призывом как к внедрению этой концепции, так и к увеличению инвестирования в сферу повышения прозрачности ПО со стороны организаций государственного и частного секторов.

Отчет CSRB по Log4j получился обширным, и приведенные в нем рекомендации были разделены по четырем категориям: работа по устранению актуальных угроз в отношении Log4j, применение существующих лучших практик для повышения гигиены в сфере кибербезопасности, повышение качества экосистемы ПО и инвестиции в дальнейшее развитие. Авторы отчета признают, что организациям предстоит бороться с уязвимостями Log4j еще в течение ближайших лет. Они призывают компании продолжать сообщать обо всех регистрируемых способах эксплуатации этой уязвимости и больше инвестировать в информационную безопасность. Помимо этого, они рекомендуют вырабатывать точные схемы инвентаризации используемых приложений, в рамках чего и возникает актуальность внедрения концепции SBOM, а также контроля использования открытого ПО. Организации с надежной системой инвентаризации программных компонентов окажутся более подготовлены к реагированию на очередной инцидент Log4j, так как смогут своевременно обнаружить появление уязвимости.

В отчете говорится, что разработчикам открытого ПО необходимо участвовать в организуемых сообществом инициативах по повышению безопасности и выделять средства на обучение специалистов по части разработки безопасного ПО, что является ключевой рекомендацией в рамках предложенного OpenSSF Плана по повышению безопасности ПО с открытым исходным кодом (OSS Security Mobilization Plan), о котором мы поговорим позднее. В отчете также содержится призыв к совершенствованию инструментария SBOM, а также инвестированию в обслуживание OSS для критических сервисов. Наконец, авторы призывают вкладываться в развитие ключевых областей, таких как базовые требования к прозрачности

ПО для компаний, поставляющих его правительственным учреждениям, развитие системы уведомления об инцидентах кибербезопасности и продумывание мотивационных схем, подталкивающих к созданию безопасного ПО. Все эти рекомендации перекликаются с советами, высказанными другими ведущими организациями со стороны как государственного, так и частного секторов. К ним относятся NIST, Linux Foundation, OpenSSF и многие другие.

В связи с растущей угрозой, исходящей от уязвимых компонентов OSS, таких как Log4j, FBI и CISA, в конце ноября 2022 года опубликовали Памятку по кибербезопасности. В ней сообщается, что Федеральное агентство исполнительной власти США (Federal Civilian Executive Branch) было подвержено атаке, проспонсированной иранским правительством. Злоумышленники использовали уязвимость Log4Shell на непропатченном сервере VMWare Horizon, установив ПО для майнинга и применив метод горизонтального перемещения для захвата контроллера доменов, что позволило им скомпрометировать учетные данные и создать обратные прокси для сохранения присутствия в системе (<https://clck.ru/3CKN8v>).

Нашумевший случай 3: Kaseya

Еще одним инцидентом, о котором стоит упомянуть, стал случай с компанией Kaseya. Компания (<http://kaseya.com>) занимается управлением ИТ-активами и разработкой ПО для обеспечения безопасности, предоставляя помощь поставщикам управляемых услуг и ИТ-командам. Их решения предназначены для повышения уровня информационной безопасности, эффективности и доставки услуг.

Атака на Kaseya произошла в июле 2021 года и, по примерным оценкам, затронула 2000 организаций. Ответственность за нее приписали группе хакеров-вымогателей REvil, состоящей в основном из русскоговорящих участников. Отдельные представители этой группировки были связаны не только с атаками на Kasey, но и с другими нападениями на компании США, в том числе на правительственные учреждения.

Kaseya предлагает клиентам как локальные решения, так и SaaS-продукты. Команда по реагированию на инциденты 2 июля 2021 года обнаружила потенциальную проблему безопасности, связанную с их ПО для удаленного управления под названием Kaseya VSA. Изначальное расследование вызвало достаточную степень беспокойства, в результате чего компания посоветовала

клиентам, использующим ее ПО локально, отключить серверы VSA до последующего уведомления. При этом компания и сама остановила свои SaaS-сервисы (<https://clck.ru/3CKN9w>), сообщив клиентам, что отключение было критически необходимым, поскольку хакеры в ходе первичных атак удалили из Kaseya VSA возможность административного доступа.

Затем компания задействовала внешние структуры и обратилась к FBI и CISA для совместного реагирования на инцидент. Kaseya оперативно предоставила связанным с ней компаниям инструменты для обнаружения взлома. В рамках своего сотрудничества с Kaseya FBI и CISA также проинформировали потенциальных жертв атаки о необходимых действиях (<https://clck.ru/3CKNAQ>). Они давали рекомендации относительно использования предоставленного инструмента обнаружения взлома, усиления процессов аутентификации и минимизации сетевых взаимодействий с известными доверенными IP-адресами.

В то же время специалисты Kaseya тестировали патчи для устранения уязвимостей и реализовывали прочие меры безопасности. Кроме того, они опубликовали руководство для клиентов, подготавливая их к установке предстоящих патчей (<https://clck.ru/3CKNAt>).

В ходе этого процесса 8 июля 2021 года Белый дом сделал официальные заявления, приписав атаку России. Лидеры Правительства США неоднократно повторяли, что обязательно призовут нападавшую сторону к ответу. Когда в Kaseya усердно старались устранить возникшие у клиентов проблемы как в локальных системах, так и в облачной среде, в адрес компании начали поступать обвинения, что ее руководство неоднократно предупреждали об уязвимостях в их ПО (<https://clck.ru/3CKNBY>). Дополнительную интригу инциденту добавлял тот факт, что использованная уязвимость, CVE-2015-2862, впервые была раскрыта и зарегистрирована самой Kaseya.

И хотя изначально сообщалось, что атака затронула лишь 0,1 % из 40 000 клиентов организации, по словам ее генерального директора Фреда Вокколы, этот показатель колебался между 800 и 1500 компаниями малого и среднего размера, на которые напали через провайдеров, использовавших ПО Kaseya. Это нисходящее каскадное воздействие является прекрасным примером того, как сложность современных цепочек поставок и воздействие на всего один элемент ПО могут подвергнуть угрозе тысячи потребителей. Большинство из них ничего не знают о программном обеспечении, используемом их провайдером, из-за непрозрачности цепочек поставок, что особенно актуально в случае работы с провайдерами внешних сервисов, такими как MSP и CSP.

Нижестоящим клиентам известны лишь сервисы или ПО, потребляемые ими непосредственно от провайдеров.

Стоит также отметить, что комментарии представителей Белого дома о наказании виновных оказались непустыми — серия арестов действительно последовала. В ноябре 2021 года Министерство юстиции США объявило (<https://clck.ru/3CKNCg>), что МВД России арестовало 14 человек, связанных с группировкой REvil. В ходе ареста также было изъято 6,1 миллиона долларов, полученных, как предполагается, за счет различных вымогательских эксплойтов группировки. Руководство Минюста США также подчеркнуло, что они продолжают отслеживать хакеров, преследующих свои цели на территории Штатов.

Какие выводы можно сделать из этих случаев

Самый важный вывод заключается в том, что полагаться только на традиционные системы защиты вроде хеширования и подписей кода недостаточно, так как это может создать иллюзию безопасности. Наиболее успешные атаки осуществляются на предшествующие этим процессам стадии, а подписывание уже зараженного кода может причинить больше вреда, чем отсутствие подписи в принципе.

Кроме того, как и в случае традиционной защиты сети, стандартным средством обнаружения аномалий в системе может стать установка некоего базового «здорового» эталона для сравнения. Сюда относятся как создание множества артефактов перечней компонентов, так и метаданные об этих артефактах вроде хешей и цифровых подписей. И хотя подобные меры не гарантируют абсолютной защиты (что может противоречить предыдущему абзацу), данные техники вполне актуальны, только применять их нужно на ранних стадиях процесса и регулярно перепроверять получаемые эталоны.

Помимо этих статических артефактов, для обнаружения вредоносной функциональности полезно понимать потоки выполнения кода и сетевого поведения. Например, в случае SolarWinds именно изменения в поведении привели к обнаружению взлома, а не результаты проверки кода.

Разработка программного обеспечения — это многоступенчатый процесс со множеством лазеек для компрометации. И хотя для доступа к некой сущности и ресурсам были разработаны архитектуры с нулевым доверием, например контроль доступа к приложению, фундаментальные принципы

сохраняют свою актуальность в разработке безопасного ПО. Напомню основные положения архитектуры с нулевым доверием:

- пользователи и идентификация;
- устройство;
- сеть и окружение;
- рабочие нагрузки приложения;
- цепочка поставок ПО;
- данные.

Безопасное ПО требует комплексного подхода на протяжении всего жизненного цикла. И хотя концепция SBOM является одним из инструментов на вооружении, она не предотвратила атаку на SolarWinds. Не существует единого универсального средства безопасности, которое разом решило бы все обозначенные проблемы. Поэтому необходимо фундаментально изменить принципы разработки программного обеспечения и способы управления доверием на всех этапах и по всей цепочке поставок ПО. Поставщики отвечают за весь свой код, а не только за тот, который они пишут. И это означает, что проверка вышестоящих активов так же важна, как и проверка программного обеспечения, создаваемого их собственными командами.

Резюме

В этой главе мы рассмотрели историю возникновения угроз для цепочек поставок, а также актуальные мотивы злоумышленников. Мы разобрали потенциальную структуру атаки на цепочки поставок, попутно изучив основы моделирования угроз и роль, которую этот процесс может играть в предотвращении нападений. В завершение мы осветили несколько наиболее известных атак, которые повлияли на различные субъекты, такие как поставщики проприетарного ПО, компоненты OSS и провайдеры MSP.

2

<https://t.me/portalToIT>

Существующие подходы — традиционное управление рисками поставщиков

Традиционные подходы к управлению рисками поставщиков исторически заключаются в обращении к поставщику с просьбой ответить на вопросы о том, как он реализует конкретные меры безопасности, понимает ли он, что ему следует делать, задокументированы ли у него соответствующие процессы и делает ли он реально то, о чем заявляет. Для подтверждения достоверности полученных ответов поставщика просят предъявить дополнительное свидетельство, подтверждающее его действия. Давайте подробнее рассмотрим эффективность этого подхода.

Анализ

Начнем с того, что этому анализу недостает возможности масштабирования. Как может корпорация выполнять сотни, а то и тысячи анализов ежегодно для поддержания своей программы управления рисками? Наиболее распространенный метод — стандартное анкетирование для сравнения поставщиков друг с другом. Но какие в таком случае следует задавать вопросы и сколько? Всех ли поставщиков нужно оценивать одинаково? Как получить контекст, необходимый для постановки вопросов? Как эффективно обработать собранные данные? Все это определенно потребует немалых усилий и утомит

не только потребителей ПО, но и поставщиков, которым придется отвечать на запросы потребителей, особенно если это крупная организация с большой клиентской базой.

Как только вы определили процесс и схему его реализации, самостоятельно или с помощью внешних исполнителей, может возникнуть проблема с получением согласия о сотрудничестве от поставщика. Как правило, эффективнее всего проводить анализ непосредственно перед продлением сотрудничества или на основе условий контракта, которые поставщик должен соблюдать. Прохождение подобной аттестации обходится вендорам недешево, и они могут не проявлять к этому особого интереса. Например, если по условиям вашего контракта поставщик должен извещать вас об обнаруженных уязвимостях и оперативно их устранять, вы можете попросить его добровольно поучаствовать в процессах, которые могут быть невыгодны ему с финансовой стороны.

Наконец, что можно считать приемлемым подтверждением выполнения необходимых действий? Снимок экрана того единственного случая, когда компания такое действие выполнила? Это не более чем скриншот конкретной конфигурации или активности, никак не гарантирующий ее постоянство. Может, тогда журнал действий? Но откуда вам знать, что журнал не сфальсифицирован или что скриншот демонстрирует процесс, повторяющийся по всей программной среде? Хорошим вариантом будет свидетельство применяемой политики, но откуда вам знать, что она действующая? Подобные аттестации зачастую выполняются командами по управлению рисками без участия специалистов по безопасности, которые могли бы помочь оценить полученные результаты или даже понять, что означают практики, о применении которых поставщик сообщает. По этим причинам многие безопасники отказались от идеи традиционного анализа рисков как не имеющего особой ценности. Естественно, есть исключения, но во многих случаях традиционный метод анализа недостаточен для реализации полноценной системы управления рисками без дополнительных усилий. Кроме того, он создает сложности и на стороне потребителя. Если вы представляете предприятие, которое потребляет ПО и сервисы от сторонних поставщиков, то ваша команда располагает ограниченными ресурсами для проведения всех необходимых аттестаций, внедрения соответствующих процессов и взаимодействия с поставщиками в установленные временные рамки (например, месяца или года). По нашему опыту, команды зачастую добавляют огромное число программных продуктов, поставщиков и сервисов в обычную организацию, превышая возможности команды для полноценного анализа или управления ими. Это нередко приводит к возникновению в корпоративных средах сегмента

теневой ИТ-инфраструктуры, проще говоря, неуправляемого ПО. Помимо этого, существуют сложности и с традиционными процессами аттестаций из-за таких факторов, как субъективность экспертов и недостаток знаний в области анализируемых технологий или ПО. Это не означает, что профессиональный анализ не имеет ценности. Однако в сфере обеспечения прозрачности программного обеспечения, автоматизации, видимости и артефактов возникают различные инновации, которые позволяют повысить уверенность в безопасности конкретного ПО или поставщика и используемых им практик.

Как правило, традиционный процесс анализа поставщиков выполняется посредством самоаттестации или через стороннюю организацию. Каждый из этих способов имеет свои сложности и недостатки. Самоаттестация может подталкивать к субъективной оценке, естественно, в свою пользу, и предвзятости, особенно если речь идет о получении прибыли. Ярким примером является военно-промышленный комплекс (ВПК) при Министерстве обороны США, где поставщики должны пройти самоаттестацию на соответствие требованиям и директивам NIST 800-171.

Несколько отчетов раскрыли существенные пробелы в результатах самопроверок, которые оказались сильно завышены и расходились с показателями, полученными в ходе аттестации сторонней организацией. Подобный метод самостоятельной оценки также был поставлен под сомнение в свете взломов нескольких поставщиков федеральных агентств США, которые можно было предотвратить или, хотя бы, смягчить их последствия, если бы заявленные меры безопасности были реализованы и функционировали должным образом. Эти факты привели к созданию Министерством обороны Модели сертификации зрелости кибербезопасности (Cybersecurity Maturity Model Certification, СММС), в которой указано, что анализ поставщиков на соответствие требованиям должна проводить сторонняя организация. В качестве альтернативы самоаттестации доступно привлечение независимых компаний, которые выполняют изначальную и периодическую аттестацию субъекта на соответствие конкретной системе или набору требований. К примерам можно отнести Государственную программу США по управлению рисками и авторизацией (Federal Risk and Authorization Management Program, FedRAMP), которая помогает авторизовывать провайдеров облачных сервисов на предоставление услуг федеральному правительству. И хотя такой подход предоставляет более высокий уровень безопасности в сравнении с самоаттестацией, он является очень сложным, дорогостоящим и времязатратным.

Для сравнения: FedRAMP существует уже порядка десяти лет, и на момент написания книги в рамках этой программы было авторизовано менее

300 предложений облачных услуг, притом что на коммерческом рынке доступны десятки тысяч подобных предложений. Эта парадигма ведет к потенциально более высокой степени уверенности в безопасности за счет значительного уменьшения доступа к инновационным решениям и провайдерам. Причиной этого являются высокие затраты средств и времени, которые стратегия с привлечением независимых организаций накладывает на поставщиков и рынок, делая данный подход либо слишком дорогостоящим и обременительным для небольших компаний, либо неоправданным для других.

Традиционный процесс на основе анкетирования утомителен, долог и дорог. Хуже того, он связан с рисками, поскольку потребители получают гарантии в виде сиюминутных свидетельств, основанных на субъективном анализе и зачастую самоаттестации поставщика ПО или услуги, который стремится показать себя в выгодном свете ради заработка. Традиционная практика управления рисками обычно опирается на ручные «бумажные» процессы. С повышением прозрачности программного обеспечения наблюдается переход к аттестациям на базе свидетельств, которые позволяют заглянуть внутрь используемого ПО и связанных с ним компонентов для определения степени представляемых им рисков. Это уже оказывается более эффективным методом проверки реализуемых практик по обеспечению безопасности, чем вербальная коммуникация и анкетирование.

Стоит также отметить появление на рынке инновационных решений, особенно в сфере облачной и декларативной инфраструктуры и приложений. Они позволяют проводить анализ практически в реальном времени, используя API и средства автоматизации, предоставляющие более высокую степень уверенности в выполнении требований, одновременно избегая «бумажных» процессов, которые заполняли индустрию в течение десятилетий. Тем не менее мы также осознаем, что не все меры обеспечения безопасности можно автоматизировать и не все они являются техническими по своей природе. Кроме того, они всегда будут требовать профессионального понимания и анализа с участием человека.

Проверка SDL

Анализ цикла разработки программного обеспечения (SDL) можно применять и для изучения процессов поставщика. Помимо этого, используемой схемой может выступать традиционный анализ рисков, но обычно это более специализированные мероприятия, выполняемые квалифицированным специалистом по безопасности. Как правило, подобный анализ сочетается

с документируемым процессом и может передаваться посредством определенных практик SDL, таких как Microsoft SDL. Это позволяет эксперту наглядно представить процессы поставщика и определить наиболее подходящие для конкретного случая практики. Отталкиваясь от этой информации, он способен сделать вывод, понимает ли поставщик предъявляемые требования и реализует ли их.

Одна из распространенных проблем поставщиков — несогласованность в применении необходимых практик между командами. Например, крупный OEM-производитель создает отдельные юридические лица для каждой линейки продукции, что приводит к появлению 10–20 различных организаций с собственными процессами. Из-за юридических особенностей возникают несогласованности в отношении требований контракта по части обеспечения безопасности. Если проанализировать SDL одного из этих 20 субъектов, то теоретически можно перенести полученные результаты и на другие продукты этой компании. Но подобный подход окажется неточным и может вести к ошибкам, а также неправильным решениям в отношении управления рисками.

В этом состоит одно из преимуществ использования таких фреймворков, как SLSA, который будет описан позднее. Он использует схожее с SDL представление ландшафта угроз для определения вероятных слабых мест — точек взаимодействия — внешних участков доверия, которые необходимо проверять, так как они могут негативно сказаться на вашем цикле разработки.

Модели зрелости безопасности приложений

Как и анализ SDL, рассмотрение таких моделей, как Building Security in Maturity Model (BSIMM) или Software Assurance Maturity Model (SAMM), может оказаться очень эффективным способом оценки зрелости используемого поставщиком ПО. SDL, естественно, является частью этого подхода, но будет нелишним взглянуть на одну из следующих моделей, например OWASP SAMM, также известную в прежних версиях как OpenSAMM (<https://owasp.org/www-project-samm>).

BSIMM тоже довольно популярна, но это проприетарная модель, а мы предпочитаем оценивать решения с открытым исходным кодом. SAMM определяет сферы, схематично представленные на рис. 2.1.

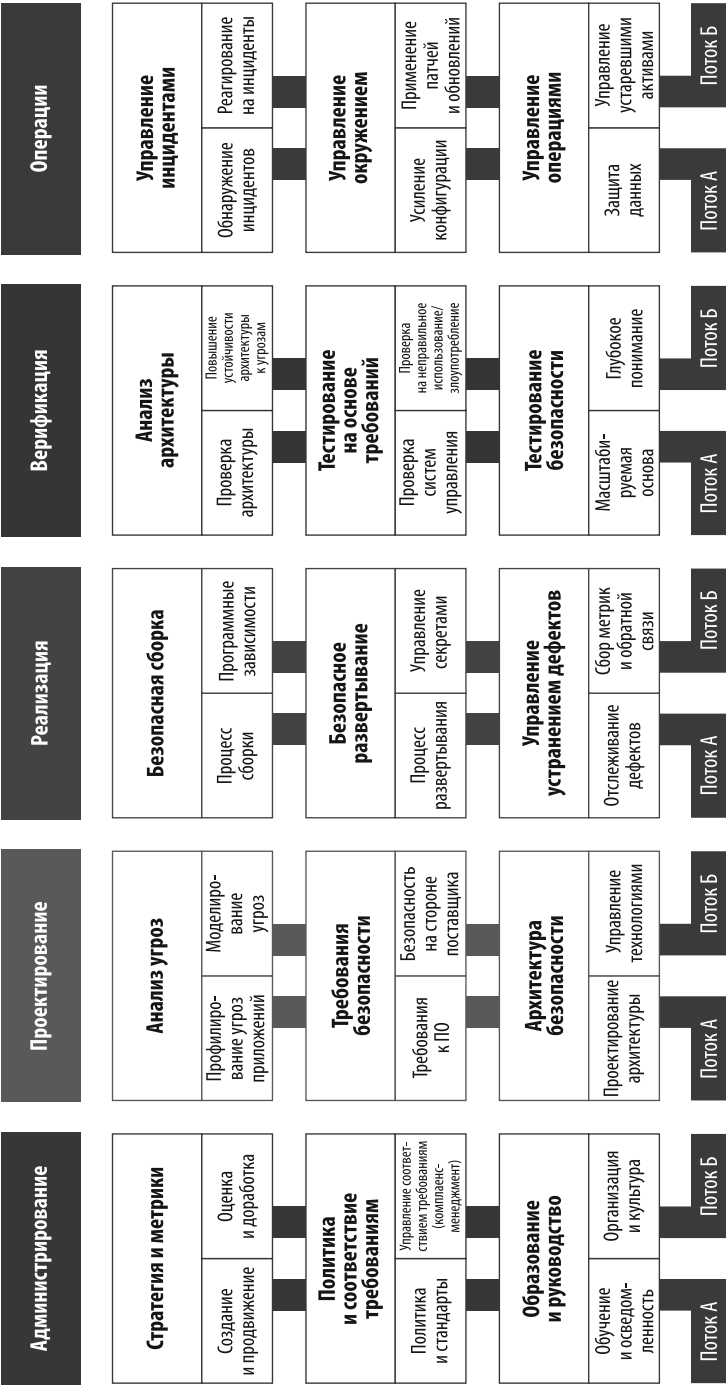


Рис. 2.1

Источник: Модель OWASP SAMM v2 (<https://owasp.org/www-project-samm>), CC BY 4.0

SAMM определяет пять бизнес-функций, расписанных на рис. 2.1: администрирование, проектирование, реализация, верификация и операции. В каждой из них присутствуют по три практики реализации безопасности, которые, в свою очередь, включают по два вида взаимодополняющих и основанных друг на друге мероприятия. Поскольку SAMM — открытая модель, она может использоваться для анализа организации как изнутри, так и сторонней компанией. Стоит отметить, что в руководстве созданного NIST фреймворка безопасной разработки программного обеспечения (Secure Software Development Framework, SSDF), о котором вы узнаете в главе 7 и следование которому требуется от поставщиков ПО для Правительства США, мы часто обращаемся к SAMM, упоминая эту модель в практиках SSDF и связанных с ними задачах.

Как и любые проекты OWASP, SAMM — это начинание сообщества, направленное на измеряемость, эффективность и гибкость. В отличие от BSIMM, SAMM предписывает использование конкретных действий и практик, которые организация может предпринять для повышения гарантии надежности своего ПО. SAMM представляет собой модель зрелости. Она охватывает уровни с 1 по 3, неявно начиная с нулевой точки. Однако это не означает, что все использующие ее организации должны достичь наивысшего уровня зрелости во всех практиках. Требования зрелости и цели зависят от ресурсов организации, условий соответствия и установленных задач. Далее мы разберем некоторые из бизнес-функций, связанных с практиками безопасности в рамках модели SAMM.

Администрирование

Данная бизнес-функция акцентируется на процессах и действиях организации, связанных с управлением разработкой ПО. К практикам здесь относятся «Стратегия и метрики», «Политика и соответствие требованиям», а также «Образование и руководство». Функция подразумевает создание и развитие стратегий и метрик с последующим их планомерным изменением и доработкой. Направление «Политика и соответствие требованиям» включает разработку политик и стандартов, а также управление процессом их внедрения и мониторинг выполнения по всей организации. Направление «Образование и руководство» состоит из двух ветвей: «Обучение и осведомленность», а также «Организация и культура». Первая акцентируется на повышении знаний различных заинтересованных сторон в области безопасности ПО, вторая ориентирована на развитие культуры безопасности в компании.

Проектирование

Это направление отвечает за процессы и деятельность организации в области создания и проектирования ПО. К соответствующим практикам безопасности относятся «Анализ угроз», «Требования безопасности» и «Архитектура безопасности». «Анализ угроз» делится на «Профилирование угроз приложений» и «Моделирование угроз». В рамках профилирования организация определяет, какие приложения представляют серьезные угрозы в случае компрометации, а в рамках моделирования помогает командам понять возможные проблемы и способы их решения. Направление «Требования безопасности» связано с проработкой требований к процессам создания и защиты ПО, а также требований к поставщикам, участвующим в разработке приложений, например внештатным программистам. Направление «Архитектура безопасности» ориентировано на различные компоненты и технологии, задействованные в проектировании архитектуры программного обеспечения компании. Сюда входит создание архитектуры, гарантирующей высокую степень безопасности, а также управление технологиями, учитывающими риски, связанные с используемыми в приложении технологиями, фреймворками, инструментами и интеграциями.

Реализация

Реализация связана с процессом создания и развертывания программных компонентов, а также может включать работу с дефектами. Здесь применяются следующие практики безопасности:

- «Безопасная сборка», основанная на последовательно повторяемых процессах сборки с учетом зависимостей;
- «Безопасное развертывание», повышающее безопасность процессов вывода программного обеспечения в рабочую среду;
- «Управление устранением дефектов», подразумевающее управление недочетами безопасности в развернутом ПО.

Практика «Безопасная сборка» подразделяется на такие потоки, как «Процесс сборки» и «Программные зависимости». Первый подразумевает развертывание предсказуемых, повторяемых операций, второй же ориентирован на внешние библиотеки и обеспечение их соответствия заявленным организацией требованиям безопасности и допустимым рискам. «Безопасное развертывание» сосредоточено на заключительных стадиях

доставки ПО в рабочую среду, а также на обеспечении его целостности и безопасности в ходе этого процесса. Данная практика делится на потоки «Процесс развертывания» и «Управление секретами». «Процесс развертывания» отвечает за обеспечение повторяемых и последовательных процессов развертывания для передачи программных артефактов в рабочую и тестовую среды.

«Управление секретами» связано с обеспечением правильной обработки чувствительной информации, такой как учетные данные, ключи API и прочие секреты, которые могут быть использованы злоумышленниками для взлома программных сред и систем, участвующих в процессе разработки ПО. Последней практикой безопасности в рамках этой бизнес-функции является «Управление устранением дефектов». Она отвечает за сбор, регистрацию и анализ дефектов ПО с целью принятия основанных на собранной информации решений. Данная практика подразделяется на два потока: «Отслеживание дефектов» и «Сбор метрик и обратной связи». Оба подразумевают обнаружение и последующее устранение дефектов, а также повышение безопасности в ходе этих процессов.

Верификация

Бизнес-функция «Верификации» отвечает за процессы и действия, связанные с проверкой и тестированием артефактов в ходе разработки ПО. К этой функции относятся такие практики безопасности, как «Анализ архитектуры», «Тестирование на основе требований» и «Тестирование безопасности». «Анализ архитектуры» подразумевает проверку безопасности и соответствия требованиям ПО и сопутствующей архитектуры. При этом «Тестирование безопасности» основывается на таких элементах, как пользовательские истории, с целью обнаружения и разрешения проблем безопасности посредством автоматизации.

«Анализ архитектуры» включает в себя два потока: «Проверку архитектуры» и «Повышение устойчивости архитектуры к угрозам», подразумевающие контроль выполнения задач по обеспечению безопасности и требований, а также ослабление или нейтрализацию выявленных угроз в существующей архитектуре. Потоки тестирования нацелены на обеспечение выполнения организацией таких действий, как проверка на неправильное использование/злоупотребление. Для этого применяется метод тестирования ПО, позволяющий вводить недопустимые или вредоносные входные значения для раскрытия программных дефектов или определения функциональности,

которую можно использовать для атаки на приложение. «Тестирование безопасности» несет в себе обширную базу из автоматизированных и ручных тестов компонентов, особо подверженных рискам.

Операции

Бизнес-функция «Операции» отвечает за обеспечение конфиденциальности, целостности и доступности (Confidentiality, Integrity and Availability, CIA) приложений, а также обслуживание связанных с ними данных в течение всего жизненного цикла, включая среды выполнения. В рамках этой операции задействуются такие практики обеспечения безопасности, как «Управление инцидентами», «Управление окружением» и «Управление операциями». Реализуемые внутри этих практик потоки включают различные области: обнаружение инцидентов и реагирование на них, усиление конфигурации и применение патчей. Наконец, «Управление операциями» обеспечивает реализацию защиты данных на протяжении всего цикла создания, обработки и хранения, а также наличие механизмов «Управления устаревшими активами» с целью прекращения развертывания устаревших сервисов и ПО. Это сокращает поверхность атаки и устраняет потенциально уязвимые компоненты из ее систем и приложений.

Задействуя модель SAMM, а также различные бизнес-функции, практики безопасности и потоки, организации смогут повысить уверенность в зрелости своих систем безопасности, что в том числе касается и потребителей их ПО.

Гарантии безопасности ПО

Традиционно практики обеспечения безопасности приложений фокусировались на принципах тестирования, которые мы здесь будем рассматривать как гарантии безопасности программного обеспечения. В этом направлении безопасность приложений выходит за пределы программных аспектов, таких как администрирование и процессы, и переходит в область технических рисков приложения. В данной сфере есть пять основных подходов, включающих несколько техник:

- статическое тестирование безопасности приложений (Static Application Security Testing, SAST);
- динамическое тестирование безопасности приложений (Dynamic Application Security Testing, DAST);

- интерактивное тестирование безопасности приложений (Interactive Application Security Testing, IAST);
- тестирование безопасности мобильных приложений (Mobile Application Security Testing, MAST);
- анализ композиции ПО (Software Composition Analysis, SCA).

Статическое тестирование безопасности приложений

Концепция рецензирования кода (code review, или код-ревью) заключается в статическом анализе исходного кода, используемого для рассматриваемого приложения или программного компонента. Зачастую исчерпывающее статическое тестирование безопасности приложения (SAST) выполняется в значимые моменты его жизненного цикла, такие как большой релиз или изменение архитектуры. При этом ревизия кода, проводимая на регулярной основе, ограничена проверкой нового или обновленного кода в базе.

SAST можно выполнять в ручном или автоматизированном режиме. При этом крупные проекты могут страдать от проблем масштабируемости, особенно в случае ручного ревью. Данный процесс также может вызывать недопонимание у тестировщика, поскольку некоторые фреймворки иногда повторно используют имена файлов в других путях приложения. И даже в случае ручного код-ревью нередко тестировщики используют расстановку приоритетов в поиске известных уязвимых функций или областей приложения с высоким уровнем доверия, таких как аутентификация и доступ, управление паролями, шифрование и др. Проблема здесь в том, что важные функции вроде проверки входных данных могут не всегда попадать в код-ревью, в результате чего их проверка сильно зависит от навыка тестировщика и времени, выделенного на тестирование.

Аналогичным образом автоматизированное тестирование помогает более полноценно проверить и обработать большие объемы за короткое время, но среди всех рассматриваемых нами методов SAST наиболее подвержен ложноположительным показателям. Даже лучшие инструменты SAST обычно выдают примерно на 5 % больше ложноположительных результатов. Это может показаться незначительным, если не учитывать, что в крупном приложении вскрываются тысячи багов. Тем не менее в этом отношении можно применять инновации, основанные на передовых технологиях вроде

машинного обучения. Они позволяют отталкиваться от прежних проверок и обеспечивают более точные результаты. (Ложноположительный результат — это когда на основе установленных правил обнаруживается баг, который по факту не представляет угрозы для системы. Обнаружение и обработка таких результатов ведут к трате времени командами, занятыми анализом выявленных проблем.) Это также относится к распространенной позиции критики концепции SBOM как средства обнаружения уязвимостей. Ложные результаты вносят путаницу, подрывают доверие к системе безопасности приложения и могут создавать условия, задерживающие выпуск ПО или перенаправляющие усилия по обеспечению безопасности приложения в неправильное русло без какой-либо пользы. Такая дополнительная когнитивная нагрузка способна снизить мотивацию команды и, как следствие, замедлить процесс поставки ПО.

Но, несмотря на описанные проблемы, SAST широко используется и остается ценным средством обнаружения дефектов в ПО, а также проверки его безопасности и качества на уровне исходного кода. Кроме того, это один из лучших способов помочь разработчикам понять, когда они пишут небезопасный код. Многие современные интегрированные среды разработки, такие как Eclipse, имеют плагины для инструментов SAST, которые предоставляют разработчикам мгновенную обратную связь, когда те используют небезопасные соглашения или иным образом прибегают к сомнительным решениям в коде. Помимо этого, на многих платформах, обучающих написанию безопасного кода, приводятся примеры небезопасных фрагментов программ, помогающие разработчикам лучше понять эту тему на практике.

Один из наиболее эффективных способов применения SAST в рамках программы по обеспечению прозрачности ПО подразумевает оценку и доработку SBOM, а также выявление областей использования крупных библиотек, где фактически задействуется лишь небольшая часть функциональности. Нередко бывает так, что разработчики импортируют библиотеку с сотней или более функций, но на практике в ПО реализовано только десять из них. Это создает сценарий, также уязвимый к ложноположительным показателям при тестировании, поскольку концепция SBOM предполагает выявление уязвимостей, которые фактически эксплуатировать невозможно. Если вы в силах отсеять ненужный код, это уменьшит лишний шум при проверках. Главное — не забудьте после внесения изменений переименовать библиотеку, иначе инструменты SBOM будут ошибочно выявлять баги на основе ее имени и версии.

Динамическое тестирование безопасности приложений

В то время как SAST анализирует приложение изнутри, через их исходный код, методика DAST нацелена на анализ извне посредством HTTP и прочих веб-протоколов. Важно отметить, что тестирование DAST обычно ограничивается веб-приложениями и зачастую не может быть использовано для десктопных, мобильных и прочих приложений, которые не прослушивают HTTP и не отвечают на соответствующие запросы. Данный класс тестирования ориентирован на взаимодействие непосредственно с выполняющимся приложением или его компонентом, таким как API или служба прослушивания. Естественной проблемой здесь является то, что DAST нельзя применить, пока приложение не окажется достаточно работоспособным для тестирования, в связи с чем эту методику используют гораздо позже, нежели SAST. Ключевое ее преимущество состоит в том, что у DAST гораздо ниже процент ложноположительных результатов, но при этом повышенное число ложноотрицательных. (Ложноотрицательный результат — это когда тест упускает объективную проблему безопасности, что также нежелательно.)

Как правило, реализуется эта методика посредством сканера, который настроен на программное взаимодействие с приложением, например, через заскриптованную эмуляцию интерфейса браузера вроде Selenium WebDriver. Сначала сканер просматривает веб-приложение, определяя все пути, кроме тех, что тестировщик исключит из области теста. Затем он проходит по всем встречающимся ссылкам, игнорируя веб-домены, находящиеся за пределами области тестирования. Этот механизм также зачастую называется *веб-краулером*, или *пауком*. Если в процессе требуется аутентификация, ее тоже можно настроить, как и обработку конкретных веб-форм приложения.

После завершения работы паука инструмент DAST создает список доступных путей для тестирования и использует предварительно настроенный набор тестов для поиска брешей, таких как возможность внедрения SQL, выполнения межсайтового скриптинга и прочих. Когда инструмент обнаруживает потенциальную уязвимость и свидетельства, указывающие на наличие проблемы, он возвращает эти результаты тестировщику в виде отчета. Автоматизированное DAST-тестирование обычно не требует особых навыков и участия, но при этом обеспечивает хорошую видимость работы

приложения. В некоторых случаях эта видимость может оказаться очень полезной для создания правил брандмауэра приложения и понимания его поверхности атаки.

Более сложные сценарии DAST обычно включают принцип перехватывающих прокси-серверов. В этом случае тестировщик настраивает в своем браузере прокси-сервер, который перехватывает его HTTP-запросы и ответы, задействуя их для вредоносного использования приложения. И хотя это чаще всего делается вручную, наличие карты сайта позволяет тестировщику определить область тестирования, и большинство таких инструментов обеспечивают определенный уровень автоматизации внутренне или посредством плагинов, например, используемых в семействе инструментов Burp Suite или Zed Attack Proxy.

Благодаря сочетанию преимуществ SAST и DAST были разработаны различные техники, позволяющие получить результаты гибридного статично-динамического тестирования. Принцип состоит в том, что сначала SAST наиболее полноценно анализирует ПО и выявляет потенциальные угрозы, после чего DAST производит исчерпывающее сканирование поверхности атаки приложения, в поисках уязвимостей, соответствующих результатам SAST. Это крайне эффективная методика для приоритизации уязвимостей с максимальной степенью уверенности. Однако даже после того, как был проведен статический и динамический анализ и выявлены потенциальные проблемы в ПО, все еще необходимо проверить оставшиеся результаты SAST.

Поскольку это применимо к теме обеспечения безопасности цепочек поставок, рекомендуется вернуться к моделям угроз для вашего приложения и подумать о том, как они могут быть использованы. Например, если приложение взаимодействует с внешним скриптом, возможно ли, что захват этого домена приведет к внедрению вредоносного кода в сессии пользователей? Перехватывающие прокси-серверы предоставляют возможность для манипулирования и перенаправления трафика, имитируя подобные сценарии. А что, если приложение выполняет обновление компонентов? Вариантов развития событий может быть множество. Предоставив вашим тестировщикам необходимые инструменты и позволив им проявить креативность, вы сможете лучше понять вероятные риски.

Отдельно в случае DAST стоит отметить потенциальную возможность этой методики выявлять сторонние зависимости на стороне клиента и их

известные уязвимости. Одним из примеров является веб-приложение, в которое библиотека включена с помощью тега *script*, а не через менеджер пакетов вроде `npm`. Большинство инструментов SBOM упустят подобный нюанс контекста, в то время как инструменты DAST смогут его обнаружить.

Интерактивное тестирование безопасности приложений

Интерактивное тестирование безопасности приложений (IAST) задействует инструменты приложения в среде его выполнения либо набор для разработки ПО, чтобы проследить потоки выполнения кода приложения и процесс перемещения по нему данных. Как заявляют поставщики IAST, такой способ позволяет получить более точное представление о работе приложения, чем другие методы тестирования. В некоторых случаях применение IAST заключается в простой установке программного агента на сервер, где размещается приложение, хотя это доступно не для всех языков и фреймворков.

Данная технология также лежит в основе нового вида защитных механизмов, известных как самозащита приложения в среде выполнения (Runtime Application Self-Protection, RASP). Этот механизм был разработан не так давно в качестве альтернативы или дополнения для брандмауэров веб-приложений (Web Application Firewalls, WAF). Поскольку RASP работает с процессами внутри приложения, его не удастся обмануть с помощью атак WAF или измененных сигнатур. Тем не менее, поскольку подобный механизм не видит оригинального веб-трафика, он не может противодействовать более шумным атакам до того, как они повлияют на приложение.

В действительности исследование, проведенное компанией Contrast Security, поставщиком инструментов IAST, показало, что значительная часть программного обеспечения, которое, как мы думаем, создает риски, в реальности никогда не используется нашим кодом. Зачастую открытые компоненты задействуют лишь 10–20 % своей функциональности. Кроме того, в контексте SaaSBOM (BOM, который охватывает сервисы), API, потоков данных и прочего IAST может быть одним из наиболее эффективных способов получения такой информации. Этот сегмент пока не поддерживается поставщиками инструментов SBOM, но будет интересно увидеть роль, которую сыграет IAST, когда рынок продукции, обеспечивающей прозрачность ПО, достигнет зрелости.

Тестирование безопасности мобильных приложений

В свете активного распространения мобильных устройств и связанных с ними приложений в индустрии возросла актуальность тестирования безопасности мобильных приложений (MAST). MAST подразумевает использование статического и динамического тестирования с целью обнаружения уязвимостей безопасности и недочетов, связанных с мобильными приложениями, платформами и устройствами, например, на Android и iOS. Этот вид тестирования помогает выявить опасную функциональность, излишние разрешения и уязвимости, которые могут эксплуатироваться на мобильных устройствах злоумышленниками. С развитием концепции SBOM перед нами открываются такие инновационные решения для мобильных устройств, как NowSecure, которое позволяет генерировать SBOM для выполняющихся приложений, включая библиотечные зависимости, зависимости уровня операционной системы и сервисы, с которыми приложение взаимодействует (<https://clck.ru/3CKNXF>).

Анализ композиции программного обеспечения

Еще один распространенный метод обеспечения гарантии безопасности приложения — анализ композиции программного обеспечения (SCA). Если коротко, SCA помогает выявить в базе кода открытое ПО и связанные с ним зависимости. Организации начали активно использовать компоненты OSS и библиотеки. SCA же помогает оценить безопасность, соответствие лицензиям и качество кода приложения. Некоторые инструменты SCA позволяют создавать SBOM, обеспечивая более подробное описание компонентов и зависимостей приложения. Как мы увидим в главе 10, понимание того, какие OSS-компоненты присутствуют в приложении или продукте, а также осведомленность о присутствующих уязвимостях и необходимых лицензиях имеют решающее значение. Специалисты в этой области рекомендуют использовать инструменты SCA на ранних стадиях SDLC для выявления и устранения уязвимостей в ходе разработки ПО, чтобы исключить их передачу конечным пользователям приложения. Их можно устранить и на более поздних этапах цикла разработки, но тогда данный процесс окажется более затратным и сложным. Подробнее об этом мы поговорим в главе 6 при обсуждении темы развития DevSecOps.

Хеширование и подпись кода

Применение криптографической верификации началось несколько десятилетий назад с использования алгоритмов MD5 и SHA-256 для создания односторонних хешей. Идея состоит в том, что ввиду необратимости такого шифрования им нельзя манипулировать, а поскольку в теории хеш должен быть уникален, его можно использовать для проверки целостности файла. Если атакующий изменит программное обеспечение и вычисленный вами для этого файла хеш в итоге не совпадет с хешем, предоставленным поставщиком, вы заметите подлог и избежите проблем. Но что, если злоумышленник также заменит хеш при замене ПО? Вычисляется ли он где-либо вне программы? Если нет, тогда будет сложно убедиться в подлинности хеша. Более того, инструменты вроде monomorph (<https://clck.ru/3CKNXx>) позволяют создавать так называемую *коллизию хешей*, когда два разных файла дают одинаковый хеш, лишая его уникальности. В связи с этим эксперты цифровой криминалистики долгое время не считали алгоритм MD5 безупречным. Более надежные алгоритмы вроде SHA-256 оказываются эффективнее, но тоже имеют проблемы с доверием.

Кроме того, механизм вычисления хеша в отдельности является не очень эффективным в операционном смысле. Только взгляните на следующий восьмиэтапный процесс.

1. Определение файла для скачивания.
2. Скачивание файла.
3. Определение места хранения хеша.
4. Скачивание хеш-файла.
5. Определение метода хеширования.
6. Скачивание и установка инструмента вычисления хеша, если он еще не установлен.
7. Прогонка файла через инструмент хеширования.
8. Сравнение вывода с хешем файла.

Справедливости ради стоит сказать, что есть ряд инструментов, которые упрощают этот процесс, но вам все равно потребуются оригинальные файлы. Некоторые поставщики начали выполнять валидацию хеша в рамках механизмов обновления их ПО, но в этом случае сложно понять, что конкретно они делают и насколько это эффективно.

Тем не менее хеширование пригождается не только для проверки целостности скачиваемого файла. Эта техника помогает обеспечить валидность элементов в перечне компонентов ПО, найти хеши компонентов в базах данных вредоносных программ и понять, соответствует ли наблюдаемый результат ожидаемому. Многие инструменты для реверс-инжиниринга даже позволяют создавать хеши блоков кода или выполнять хеширование на уровне функций, присваивая им идентификаторы FID (Function ID).

В то же время хеши порой оказываются крайне хрупкими. Например, два конкретных файла могут полностью совпадать и давать при этом разный хеш, в зависимости от того, как или когда хеш генерировался.

К типичным сценариям относится хеширование компонента в рамках сборки, а не в том виде, в каком он представлен в исходном коде. В результате оптимизаций кода, таких как удаление символов или специальных знаков, может получаться ПО, которое идентично по своей функциональности, но с другим хешем. Различия в хешах могут быть обусловлены даже тем, что они были получены в разных ОС, например Windows и Linux, в зависимости от способа обработки символов переноса строки при создании хеша.

В цифровой криминалистике появилась техника для определения сходства хешей на основе алгоритма вычисления расстояния. Хеш-функции вроде ssdeep и TlSH (<https://tlsh.org>) позволяют определить степень сходства двух хешей. Это может быть полезно при анализе ПО, когда нужно сравнить два похожих, но не совсем одинаковых компонента.

Аналогом хеширования является цифровая подпись кода. Это техника, в которой код или объекты файлов криптографически подписываются доверенной сущностью, подтверждающей, что объект был произведен именно ей. Осуществляется это с помощью криптографии с открытым ключом, где подписывающая сторона использует закрытый ключ, подтверждающий ее идентичность. Проверить эту подпись может любой, у кого есть открытый ключ.

Такой механизм можно считать очень надежным, пока вы не задумаетесь о том, что сертификат для подписи может получить кто угодно. И если проверяющий также не будет убеждаться в том, что сторона, подписавшая файл, является ожидаемой сущностью, это может вызвать ложное чувство безопасности. В контексте криптографии с открытым ключом, если доверенная сертифицирующая инстанция будет скомпрометирована, это делает подписанные сертификаты недействительными. Наконец, мы видели

несколько случаев, когда криптографические подписи были украдены из легитимного ПО и встроены во вредоносные файлы или когда сущности были взломаны, а их закрытые ключи похищены. Тем не менее подобный риск в некоторой степени можно уменьшить, если установить требование использовать службу доверенных временных меток (Time Stamp Authority).

Одной из перспективных техник является применение цепочек аттестаций с помощью цифровых подписей. Например, крупный SBOM может требовать участия нескольких стейкхолдеров, или же мы сами решим связать аттестацию сборки со SBOM. Подписывая эти аттестации, мы создаем механизмы для автоматизированной верификации и связывания нескольких аттестаций таким образом, что ни одна отдельная сущность не будет знать все аспекты SBOM.

Одной из последних инноваций в сфере использования подписей является проект Sigstore, о котором мы поговорим в последующих главах.

Резюме

В этой главе мы рассмотрели некоторые аспекты исторического контекста, связанные с традиционным анализом рисков поставщиков, проблемами внутренней и внешней аттестации, а также поговорили о моделях зрелости приложений. Кроме того, мы осветили распространенные инструменты и методики, обеспечивающие гарантии безопасности приложений, а также способы выявления и устранения уязвимостей. В следующей главе мы изучим подходы к оценке серьезности уязвимостей, их базы данных, а также последние тенденции в сфере улучшения традиционных методов приоритизации уязвимостей.

3

Базы данных уязвимостей и методы оценки

<https://t.me/portalToIT>

Одним из важнейших вопросов при обсуждении безопасности приложений являются категоризация и оценка уязвимостей. Это весомая часть стратегии по развитию концепции прозрачности ПО и, что более важно, безопасности приложений. Без понимания сути и степени серьезности уязвимостей организациям становится сложно расставлять приоритеты при их устранении. Разработчики программного обеспечения могут приоритезировать уязвимости по срочности устранения, снижая риски для своих клиентов, а также уведомляя их об уровне угрозы и возможности эксплуатации тех или иных уязвимостей в предоставляемых ими продуктах. Это позволяет осведомленным потребителям принимать осознанные решения в отношении целесообразности их применения. Текущую тему мы начнем со знакомства с наиболее распространенными терминами, связанными с уязвимостями ПО.

Типичные уязвимости и слабые места

Система идентификации уязвимостей (CVE) нацелена на выявление, определение и каталогизацию публично раскрытых киберугроз, влияющих на программное и аппаратное обеспечение. CVE подразумевает участие международных исследователей и компаний, которые помогают находить и публиковать различные уязвимости, описывая их в стандартизированном формате.

Истоки программы CVE и самого этого принципа уходят корнями в работу MITRE от января 1999 года под названием *Towards a Common Enumeration*

of Vulnerabilities за авторством Дэвида Манна и Стивена Кристи (<https://clck.ru/3CKNZD>). Эта работа в свое время заложила основу кибербезопасности, включая несколько баз данных уязвимостей, уникальных по своему формату, критериям и систематизации. Ее авторы объяснили, как это повлияло на интероперабельность и обмен информацией об уязвимостях. Среди помех для интероперабельности они назвали несогласованные методы именования, управление одной и той же информацией с разных ресурсов и сложность сопоставления содержимого баз данных.

С целью повышения интероперабельности между инструментами безопасности ПО была предложена система CVE. Она подразумевала ведение списка всех известных зависимостей, присваивание им уникальных имен и предоставление доступа без каких-либо ограничений.

В итоге более 20 лет эта концепция занимала центральное место в индустрии. В ее развитии участвовало более 200 организаций из более чем 35 стран, описав в общей сложности более 200 000 уязвимостей. Участвующим организациям присваивался статус CVE Numbering Authority (CNA; <https://clck.ru/3CKNaN>). Примерами CNA — компаний, уполномоченных на внесение в список CVE информации об уязвимостях, — являются поставщики ПО, открытые проекты, провайдеры услуг по устранению багов и исследователи. Любая из таких компаний может быть авторизована программой CVE. Организации могут запрашивать получение статуса CNA, но при этом должны отвечать определенным требованиям, таким как наличие политики раскрытия публичных уязвимостей, открытого ресурса для размещения информации и согласие с условиями использования CVE (<https://clck.ru/3CKNaq>).

В главе 1 мы обсудили Национальную базу данных уязвимостей (NVD), и хотя между NVD и CVE есть тесная связь, это два разных начинания. CVE родилась вместе с MITRE и сегодня является совместным проектом сообщества. NVD же была запущена институтом NIST. Обе программы спонсируются Министерством внутренней безопасности США и CISA и доступны для свободного использования. В то время как CVE проявляется в виде набора записей об уязвимостях, включая идентификационные номера, описания и публичные ссылки, NVD — это база данных, которая синхронизируется с CVE для сохранения их согласованности. NVD также обеспечивает возможности поиска по CVE на основе дополнительных метаданных и полей. CVE состоит из нескольких рабочих групп, ориентированных на отдельные сферы, включая автоматизацию, стратегическое планирование, координацию и качество. Эти рабочие группы трудятся над повышением

эффективности программы CVE в целом, увеличивая ее ценность для общества. Помимо рабочих групп, в CVE есть Программный совет, следящий за тем, чтобы осуществляемые процессы соответствовали потребностям глобальной кибербезопасности и сообщества (<https://clck.ru/3CKNbc>). Этот Совет контролирует программу, регулирует ее стратегическое направление и доносит его до сообщества.

Вот несколько фундаментальных терминов, относящихся к программе CVE:

- уязвимости;
- идентификатор CVE;
- область;
- список CVE;
- запись CVE.

CVE определяет уязвимости как слабые места в программном обеспечении, встроенном ПО или оборудовании, которые могут быть эксплуатированы и негативно повлиять на конфиденциальность, целостность и доступность компонентов этого ПО. Идентификаторы CVE (CVE ID) — это уникальные буквенно-цифровые имена, присваиваемые программой CVE конкретным уязвимостям. Они автоматизируют процесс присвоения и упрощают обсуждение уязвимостей разными сторонами. Области — это отдельные группы аппаратного или программного обеспечения, за которые отвечают определенные организации — участники программы CVE. Список CVE — это всеобъемлющий каталог записей, которые были выявлены в рамках программы CVE или о которых было сообщено в извне. Запись CVE представляет собой описательные данные об уязвимости, закрепленной за CVE ID, и предоставляется CNA. Эти записи могут резервироваться, публиковаться или отклоняться. Статус *«Зарезервировано»* — это изначальное состояние, когда запись резервируется CNA. *«Опубликовано»* — означает, что CNA заполнила все необходимые данные, чтобы запись стала полноценной. *«Отклонено»* — это когда CVE ID и запись перестают быть актуальными, но остаются в списке CVE для сопоставления с ними в будущем.

Записи CVE также проходят свой определенный жизненный цикл, состоящий из следующих стадий: обнаружение, оповещение, запрос, резервирование, отправка и публикация (рис. 3.1). Сначала отдельная организация обнаруживает новую уязвимость, сообщает о ней в программу CVE или ее участнику и запрашивает CVE ID, который резервируется на время, пока уведомленный участник программы не отправит необходимые детали.

Наконец, запись CVE публикуется для открытого скачивания и просмотра, что позволяет принимать меры по реагированию, если это необходимо.



Рис. 3.1

Источник: корпорация MITRE (www.cve.org/About/Process, последнее обращение 27.08.2024)

Программа CVE и список записей CVE с момента своего появления развивались непрерывно. Если посмотреть на количество публикуемых записей CVE (<https://clck.ru/3CKNeP>), мы увидим, что на момент написания книги за квартал их число увеличилось с нескольких сотен до 6000. Все дело в росте популярности программы, увеличении числа специалистов по безопасности, более активном участии со стороны разработчиков, а также в общем прогрессе в области кибербезопасности.

Национальная база данных уязвимостей

Сбор информации об уязвимостях позволяет вырабатывать эффективные решения для снижения рисков поставщиков и потребителей, а также повышает общую осведомленность об их присутствии в экосистеме. И хотя всего существует несколько отдельных баз данных уязвимостей, наиболее известным является NVD (<https://nvd.nist.gov>). NVD — это всеобъемлющая база данных, объединяющая в себе все имеющиеся в открытом доступе правительственные источники данных об уязвимостях и предоставляющая ссылки на связанные с индустрией ресурсы.

Официально NVD была сформирована в 2005 году (<https://nvd.nist.gov/general/brief-history>), хотя корнями уходит в 1999-й, когда был создан ее предшественник:

каталог интернет-атак (Internet Category of Attack Toolkit, ICAT). ICAT был разработан студентами Института SANS как база данных сценариев атак. ICAT столкнулся с проблемами финансирования, но его существование поддерживалось усилиями SANS и сотрудников NIST. В итоге, когда база данных разрослась до 10 000 уязвимостей, ее наконец проспонсировало Министерство внутренней безопасности США, трансформировав в NVD, которую мы знаем сегодня. Со временем в NVD начали использовать популярные методы сбора и оценки данных об уязвимостях, такие как CVSS и CPE (Common Platform Enumeration; <https://nvd.nist.gov/products/cpe>).

На момент написания книги NVD содержала более 200 000 записей об уязвимостях, и по мере появления все новых их видов это число продолжает расти. NVD используется профессионалами по всему миру, а также поставщиками, стремящимися учесть новейшие проблемы безопасности и их детали.

NVD облегчает этот процесс путем анализа CVE, опубликованных в Словаре CVE, благодаря которому сотрудники NVD формируют для уязвимостей важные метаданные, включая показатели CVSS, типы по общему каталогу уязвимостей (Common Weakness Enumeration, CWE) и заключения о возможностях применения, представляя все это в виде CPE. Стоит упомянуть, что сотрудники NVD не проводят тестирования уязвимостей и для формирования всех названных атрибутов используют информацию со стороны. По мере появления актуальных сведений NVD зачастую пересматривает полученные метаданные, такие как показатели CVSS и информацию CWE.

В NVD данные поступают из программы CVE. Сотрудники NVD тщательно анализируют новые CVE после их интеграции в базу данных, в том числе просматривая публично доступную в Интернете информацию. CVE присваивается один или более идентификаторов CWE, помогающих категоризировать конкретную уязвимость. Помимо этого, с помощью метрик CVSS ей также присваивается степень эксплуатируемости и возможного вреда. Вдобавок к этому посредством CPE создаются заключения о спектре применения той или иной уязвимости, позволяющие выявить подверженные ей конкретные версии программного или аппаратного обеспечения и системы. Это помогает организациям предпринять правильные действия в зависимости от того, влияет ли уязвимость на используемое ими программное или аппаратное обеспечение. После первичного анализа все собранные метаданные, такие как CWE, CVSS и CPE, в качестве меры контроля качества просматриваются старшим аналитиком и лишь затем публикуются на сайте NVD и связанных с ним информационных каналах.

NVD предлагает обширный набор таких каналов (<https://nvd.nist.gov/vuln/data-feeds>) и API (<https://nvd.nist.gov/developers/start-here>), через которые организации и отдельные люди могут получать данные об опубликованных уязвимостях. API позволяют заинтересованным сторонам программно получать такую информацию автоматизированным и масштабируемым образом, что гораздо удобнее ручного просмотра. NVD API регулярно обновляются и предлагают возможность поиска, включая и прочие удобства, такие как сопоставление данных. Они зачастую используются поставщиками средств кибербезопасности, желающими дополнить свои предложения актуальными сведениями.

Форматы идентификации ПО

Одним из фундаментальных аспектов обеспечения безопасности цепочек поставок и прозрачности ПО является потребность в эффективных способах определения программных компонентов, включая наличие достаточной информации для уникальной идентификации каждого из них. И это нетривиальная задача, учитывая, что рынок ПО представлен тысячами различных поставщиков, секторов и сообществ. Национальное управление телекоммуникациями и информацией США (NTIA) акцентирует внимание на этой проблеме в своей работе *Software Identification Challenges and Guidance* (<https://clck.ru/3CKNfw>), которая была написана при участии множества стейкхолдеров NTIA. В ней подчеркиваются важность оперативного функционального определения программных компонентов и необходимость оптимизации текущих систем идентификации. Это упростит некоторые из сложностей и поможет индустрии выработать единое решение. Стремясь развеять некоторую путаницу в отношении идентификации компонентов, NTIA рекомендует использовать существующие системы, но отмечает, что это не всегда возможно, в связи с чем приходится выбирать решения по принципу «лучшее из доступного».

Хотя многие поставщики и потребители ПО стремятся внедрить концепцию SBOM, в настоящее время нет надежного ресурса для идентификации компонентов. Поставщики ПО зачастую определяют и выявляют их на основе потребностей организации и соответствующих активностей. Причем эта проблема усугубляется тем, что идентификация компонентов не является статичным процессом и может меняться под влиянием таких факторов, как разветвление проектов, внедрение новых направлений, слияние компаний, и других.

Тем не менее существуют основные стандарты идентификации ПО, которые признаются в индустрии и NTIA, а именно: CPE, идентификационные SWID-теги (Software Identification Tags) и идентификатор URL пакета (Package URL, PURL).

Common Platform Enumeration

Common Platform Enumeration (CPE) — это структурированная схема именования, используемая для описания и определения классов приложений, операционных систем и аппаратных устройств среди корпоративных IT-ресурсов. Изначально выпущенная NIST в августе 2011 года (<https://clck.ru/3CKNgQ>), CPE претерпела серию доработок и на момент написания книги имеет версию 2.3 (<https://clck.ru/3CKNgm>). CPE возникла как ответ на потребность в стандартизации обращений к IT-продуктам и платформам, сохраняя при этом машиночитаемость схемы для автоматизации. NIST ведет словарь CPE (<https://clck.ru/3CKNh6>), который представляет собой утвержденный список имен CPE. Он доступен для публичного использования в формате XML.

В спецификации CPE 2.3 этот стандарт разбивается на стек отдельных спецификаций, представляющих:

- имя;
- сопоставление имен;
- словарь;
- язык описания применимости.

Имя является основой спецификации CPE и нужно для определения стандартизированных методов присваивания имен классам IT-продуктов. *Сопоставление имен*, в свою очередь, используется для выбора метода сравнения исходного имени CPE с целевым именем CPE, что позволяет определить, существуют ли общие связи между двумя сущностями. Например, инструменты управления IT-активами могут собирать информацию об установленном в системе ПО и выявлять среди него конкретные программы. *Словарь* определяет стандартизированный метод для создания и управления словарями CPE, являющимися хранилищами имен CPE и их метаданных. Последняя спецификация, *язык описания применимости*, определяет стандартизированный способ описания IT-платформ путем формирования комплексных логических выражений на основе имен CPE и ссылок. В качестве примера

такого описания можно привести операционную систему вроде Windows XP, в которой работает Microsoft Office 2007, и присутствует конкретный элемент конфигурации, такой как активный адаптер беспроводной связи.

Процесс создания и присвоения SWID

Идентификация программного обеспечения (Software Identification, SWID) — это формат структурированных метаданных, признанный Международной организацией по стандартизации и применяемый для описания программных продуктов (<https://clck.ru/3CKNks>). SWID использует элементы данных для идентификации ПО и его версий, а также организаций или отдельных лиц, участвовавших в создании и распространении продукта и составляющих его компонентов. При реализации некоторых процессов по управлению и обеспечению безопасности программного обеспечения SWID задействуется в таких мероприятиях, как анализ на наличие уязвимостей программных активов, отсутствие патчей, или в анализе управления конкретной конфигурацией. Этот формат также можно использовать при верификации целостности ПО или в оперативных мероприятиях по обеспечению безопасности.

NIST подробно описывает применение тегов SWID в своей работе *Guidelines for the Creation of Interoperable Software Identification (SWID) Tags* (<https://clck.ru/3CKNnf>).

NTIA также рассматривает SWID в своей публикации *Survey of Existing SBOM Formats and Standards* (<https://clck.ru/3CKNoQ>). Теги SWID используются различными стандартами, такими как Trusted Computing Group и Internet Engineering Task Force. Стандарт SWID помогает определять жизненный цикл программного продукта. Всего в нем используются четыре вида тегов:

- основной;
- патч;
- пакет;
- вспомогательный.

Основной тег нужен для идентификации и описания программных продуктов, установленных на устройстве, а тег для *патчей* представляет постепенные изменения, вносимые в эти продукты. Тегами для *пакетов* обозначается установочное ПО в состоянии перед установкой, например метаданные об установочных пакетах или программном обновлении. *Вспомогательный* тег

можно использовать для добавления дополнительной информации в любой тег SWID, на который идет ссылка, без его изменения. Функциональность тегов SWID представлена на рис. 3.2.

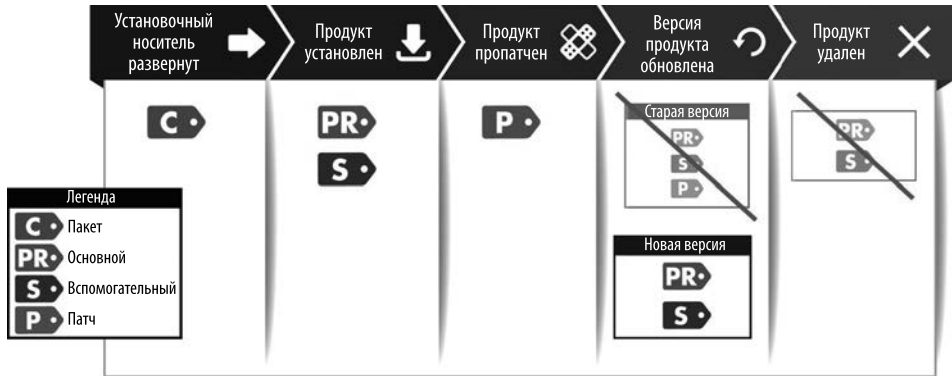


Рис. 3.2

Источник: Опрос по форматам NTIA SBOM, Министерство торговли США, публичная информация

Несмотря на то что мы рассмотрели формат SWID в контексте обсуждения SBOM, он здесь имеет не такое значение, как другие форматы SBOM вроде SPDX и CycloneDX. Причина в том, что он в основном ориентирован на идентификацию ПО и лишен некоторого контекста, предоставляемого другими форматами. Но при этом стоит отметить, что CycloneDX нативно поддерживает SWID для идентификации ПО.

PURL

Идентификатор URL пакета (Package URL, PURL; <https://clck.ru/3CKNpU>) помогает решить проблему, связанную с соглашениями и протоколами именования программных пакетов. Современный процесс разработки ПО включает использование и создание значительного числа программных пакетов вроде NPM и Rubygems. Тем не менее, когда дело доходит до их идентификации, локализации и инициализации, в каждом менеджере пакетов и экосистеме применяются разные соглашения именования и методы.

Для внесения ясности компания nexB (которая предлагает инструменты SCA, ScanCode и VulnerableCode, а также бесплатные решения для

сканирования на предмет наличия уязвимостей) разработала спецификацию PURL (<https://clck.ru/3CKNpz>). Целью проекта PURL является стандартизация методов идентификации и определения местоположения программных пакетов среди различных менеджеров пакетов, инструментов и более широкой экосистемы ПО. PURL состоит из шести элементов данных:

- схема;
- тип;
- пространство имен/имя;
- версия;
- квалификаторы;
- подкаталог.

Некоторые из этих элементов являются обязательными, а некоторые — нет. В итоге мы получаем URL-строки вроде `pkg:bitbucket/birkenfeld/pygments-main@244fd47e07d1014f0aed9c`. *Схема* — это схема URL с постоянным значением `pkg`. *Тип* — это необходимый элемент, указывающий тип пакета, например `maven` или `NPM`. *Пространство имен*, например `Maven` и `Docker`, является необязательным элементом. *Имя* — это необходимое поле, отражающее название пакета. *Версия* — это необязательный элемент. *Квалификаторы* предоставляют дополнительный контекст, такой как операционная система, архитектура или дистрибутив. *Подкаталог*, в свою очередь, может указывать дополнительные подкаталоги в пакете. Эти два последних элемента являются необязательными.

Изначально PURL демонстрировался на таких мероприятиях, как FOSDEM 2018. С момента своего появления он стал восприниматься такими проектами, как CycloneDX и SPDX, OSS Index и другими организациями по всему миру, как стандарт для URL пакетов.

Члены OWASP, Linux Foundation и Oracle, помимо прочих, также занялись вопросом внедрения PURL в NVD для повышения эффективности автоматизации и идентификации компонентов в сфере управления уязвимостями (<https://clck.ru/3CKNrX>). Команда SBOM Forum, возглавляемая Томом Алрихом, блогером (<http://tomalrichblog.blogspot.com>) и консультантом по вопросам защиты цепочек поставок, опубликовала документ под названием *A Proposal to Operationalize Component Identification for Vulnerability Management* (<https://clck.ru/3CKNss>). В нем сделан акцент на проблеме, которая заключается в том, что одни и те же открытые продукты или компоненты

в различных менеджерах пакетов и реестрах числятся под разными именами. Авторы пришли к выводу, что для более эффективного обнаружения и устранения уязвимостей организациям необходимо стандартизированное соглашение об именовании программных компонентов.

На момент написания книги в NVD для программных продуктов используются имена CPE. Однако соглашения об именовании CPE несут в себе ряд сложностей, препятствующих автоматизации процессов обеспечения безопасности ПО. CPE в NVD в первую очередь идентифицируют программные и аппаратные компоненты через строки поставщика, продукта и версии. Интеграция PURL в эту базу данных должна смягчить некоторые проблемы, связанные с использованием CPE, а также повысить эффективность идентификации ПО и упростить процесс внедрения концепции SBOM и ее автоматизации.

Sonatype OSS Index

По мере развития и внедрения ПО с открытым исходным кодом (OSS) в современные приложения и продукты возросла и потребность в понимании рисков, связанных с его компонентами. В качестве решения был создан каталог Sonatype OSS Index. Он содержит миллионы компонентов OSS, а также инструменты сканирования, позволяющие организациям выявлять присутствующие в этих компонентах уязвимости и связанные с ними риски. Кроме того, Sonatype OSS Index предоставляет рекомендации по устранению возникших проблем и, во многом аналогично NVD, имеет надежный API для сканирования и выполнения запросов. Одним из ключевых отличий между NVD и индексом OSS является применение PURL вместо CPE. NVD использует CPE для уникальной идентификации подверженных уязвимостям продуктов, а индекс OSS — спецификацию PURL для описания координат компонентов или пакетов.

Помимо поиска компонентов OSS и любой информации, связанной с их уязвимостями, индекс OSS поддерживает сканирование проектов на уязвимости в их OSS и интеграцию с современными цепочками инструментов CI/CD посредством REST (Representational State Transfer) API. В контексте стремления сместить вопросы обеспечения безопасности на более ранние этапы цикла разработки ПО эти возможности способствуют выявлению уязвимостей в компонентах OSS до развертывания кода в рабочей среде. OSS Index API объединяет в себе многие популярные инструменты

сканирования, такие как плагин Java Maven, Rust Cargo Pants, OWASP DependencyCheck, ossaudit в Python и другие. Стоит отметить, что, хотя OSS Index и является бесплатным, он не включает какой-либо отдельно курируемый анализ со стороны экспертов. Тем не менее Sonatype предлагает прочие сервисы для получения дополнительной информации, управления библиотеками и автоматизации.

База данных уязвимостей открытого ПО

В 2021 году подразделение Google Security запустило (<https://clck.ru/3CKNtc>) проект Open Source Vulnerabilities (OSV; <https://osv.dev>), нацеленный помочь разработчикам и потребителям открытого ПО путем улучшения системы приоритизации уязвимостей. Этот проект родился на основе предыдущих начинаний Google Security, известных под лозунгом «Знай, предотвращай, исправляй» (Know, Prevent, Fix; <https://clck.ru/3CKNu9>). Задачей OSV является предоставление данных о том, где были обнаружены и устранены уязвимости. Это позволяет потребителям ПО понимать, каким рискам они подвержены и как их снизить. OSV включает задачи по минимизации необходимой работы со стороны мейнтейнеров, связанной с публикацией данных об уязвимостях, а также по повышению точности запросов об уязвимостях, выполняемых нижестоящими потребителями. OSV автоматизирует процессы приоритизации уязвимостей для потребителей пакетов открытого ПО посредством надежного API и запросов данных. Эта БД также снижает издержки, связанные с усилиями мейнтейнеров по предоставлению точных списков подверженных уязвимостям версий ПО, способных повлиять на нижестоящих потребителей.

Команда Google Security публиковала статьи, демонстрирующие, как OSV связывает SBOM с уязвимостями благодаря способности OSV описывать уязвимости в формате, предназначенном для сопоставления с версиями пакетов открытого ПО и хешами коммитов. Помимо этого, OSV агрегирует информацию из различных источников, таких как GitHub Advisory Database (GHSA) и Global Security Database (GSD). Существует даже инструмент OSS, известный как *spdx-to-osv* (<https://github.com/spdx/spdx-to-osv>). Он создает файл JSON с описанием уязвимости на основе информации, указанной в SPDX.

OSV — это не просто БД, доступная по адресу OSV.dev, но также схема и формат OSV. Разработчики OSV указывают, что существует множество

баз данных уязвимостей, но для них нет единого стандартизированного формата (<https://ossf.github.io/osv-schema>).

Организации стремятся расширить охват баз данных об уязвимостях, однако неизбежно сталкиваются с тем фактом, что каждая из них имеет собственный уникальный формат и схему. Это затрудняет потребителям ПО интеграцию в свои системы нескольких баз данных, а разработчикам — публикацию в разные базы данных. OSV же предлагает универсальный формат JSON, с которым смогут работать все и который не только повысит интероперабельность, но и уменьшит хлопоты для потребителей ПО, его разработчиков и исследователей безопасности, желающих использовать несколько БД.

Global Security Database

Несмотря на широкое распространение программы CVE, многие указали на существующие в ней проблемы, связанные с применяемым подходом и невозможностью адаптироваться к быстро меняющейся технологической среде. С развитием индустрии облачных вычислений появились такие авторитетные лидеры, как Cloud Security Alliance (CSA, <https://cloudsecurityalliance.org>), предлагающие необходимые руководства и проводящие передовые исследования. Одним из значимых достижений CSA стало создание базы данных GSD, представляющей собой современный подход к актуальным вызовам. CSA называет CVE устаревшим решением, неспособным удовлетворить потребности современного сложного ИТ-ландшафта. Инициаторами проекта стали Джош Брессерс и Курт Сейфрид, обладающие обширным опытом в области идентификации и управления уязвимостями в компании Red Hat, а Курт даже был членом коллегии CVE.

В своей статье *Why We Created the Global Security Database* (<https://clck.ru/3CKPko>) Курт рассказывает о причинах создания GSD и объясняет, почему другие программы вроде CVE не оправдали возложенных на них надежд. Разработке CVE предшествовала программа под названием BugTraq IDs, которая требовала от заинтересованных сторон подписаться и ознакомиться с информацией об уязвимостях. Автор описывает развитие программы CVE, в рамках которой изначально обнаруживалось около тысячи уязвимостей в год. Максимальное число выявленных CVE достигло 8000 в 2011 году, после чего начало снижаться. В статье также говорится об увеличении числа CNA, о чем мы писали выше. Тем не менее, несмотря на рост количества CNA, по словам Курта, около 25 % из них как минимум год не проявляли никакой

активности. В 2017 году количество опубликованных и присвоенных CVE достигло своего пика, и, за исключением скачка в 2020 году, дальнейшая активность по их присваиванию оставалась на одном уровне (рис. 3.3).

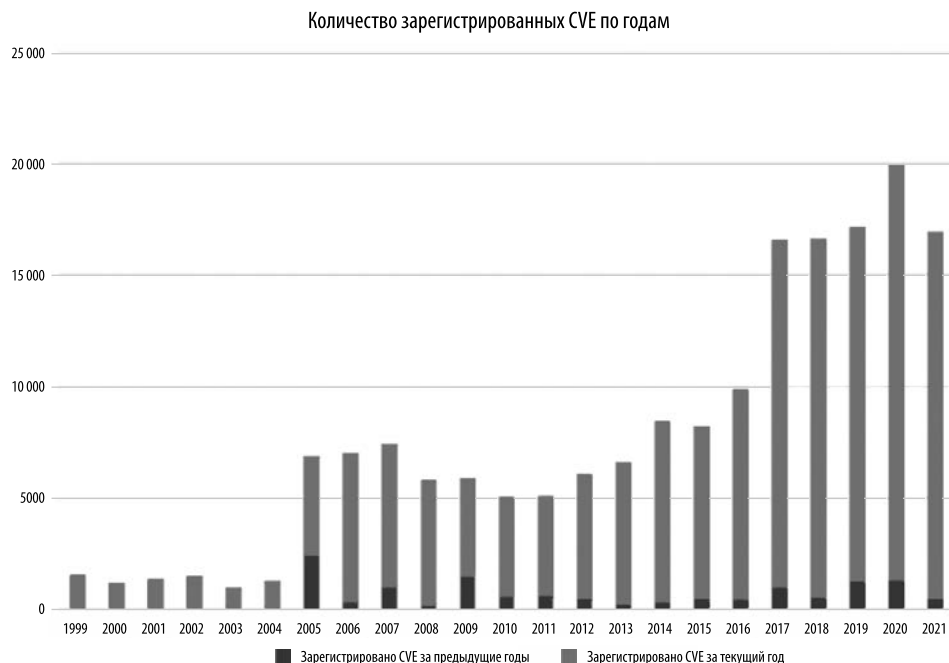


Рис. 3.3

Источник: Why We Started the Global Security Database (<https://clck.ru/3CKYmz>). Cloud Security Alliance, последнее обращение 27.08.2024

В статье говорится о прежних инициативах, в которых Джош и Курт принимали участие, например о проекте Distributed Weakness Filing (распределенная картотека уязвимостей) и о том, что в нем участвовало даже сообщество Linux. Проект Linux отказался работать с программой CVE, о чем подробно рассказал Грег Кроан-Хартман в своем выступлении Kernel Recipes 2019 — CVEs are dead, long live the CVE! (<https://clck.ru/3CKPmQ>). В этом видео он отметил, что программа CVE плохо сочетается с Linux из-за множества исправлений, которые применялись и передавались пользователям множеством способов. Грег объясняет, что средняя продолжительность «запроса на исправление» Linux CVE составляет 100 дней. Это говорит о недостатке внимания к данному вопросу, а в более широком смысле — о том, что проект Linux движется слишком быстро в сравнении с ригидным процессом CVE.

В заявлении GSD сказано о том, что сервисы «поглотили мир» и современное ПО в большинстве случаев поставляется *как услуга* (Software as a Service). С этим сложно поспорить, учитывая активное распространение облачных вычислений в виде всевозможных моделей, но в основном для приложений, предоставляемых в рамках парадигмы SaaS. Авторы говорят о том, что программе CVE недостает согласованного подхода, когда дело касается приложений SaaS.

Помимо прочих упомянутых авторами и руководителями GSD изменений, в работе отмечается безудержный рост пакетов, например Python (200 000) или NPM (1,3 миллиона). Также есть ощущение, что программе CVE недостает прозрачности, доступа сообщества, вовлеченности и данных, поскольку она также относится к поставщикам, использующим открытые пакеты. В качестве примера приводится случай с Log4j. Если вам интересно принять участие в открытом, управляемом сообществом проекте GSD, то подробнее о нем можете узнать на сайте (<https://globalsecuritydatabase.org>), а также в информационных каналах, таких как Slack, или через рассылки (<https://clck.ru/3CKPnM>).

Общая система оценки уязвимостей

В то время как программа CVE предоставляет способ идентификации и регистрации уязвимостей, а NVD дополняет ее и обобщает всю эту информацию в виде надежной базы данных, единая система оценки уязвимостей (CVSS) обеспечивает анализ серьезности уязвимостей и присваивает им соответствующий ранг. CVSS отражает технические характеристики уязвимостей программного и аппаратного обеспечения, а также прошивок. Она предлагает стандартизованный способ оценки серьезности уязвимостей, который не зависит от платформ или поставщиков и является полностью прозрачным в отношении характеристик и методологии, на основе которых проводится оценка.

CVSS была разработана на базе исследований Консультативного совета национальной инфраструктуры (National Infrastructure Advisory Council, NIAC) в начале 2000-х, а ее первая версия увидела свет в 2005 году. И хотя к созданию CVSS привели исследования NIAC, в 2005 году Совет выбрал Форум для координации действий при возникновении инцидентов и угроз кибербезопасности (Forum of Incident Response and Security Teams, FIRST) для руководства и управления инициативой CVSS в будущем. FIRST — это базирующаяся в США некоммерческая организация, которая помогает

командам реагирования на инциденты ресурсами, такими как CVSS. Несмотря на то что CVSS принадлежит и управляется FIRST, другие организации тоже могут использовать или внедрять эту систему при условии следования руководствам FIRST. CVSS претерпела несколько изменений: первая версия вышла в начале 2005 года, затем в 2007-м появилась CVSS v2, а в 2015-м — CVSS v3. На момент написания книги ее спецификация достигла v3.1. CVSS состоит из трех основных групп показателей, представленных на рис. 3.4.

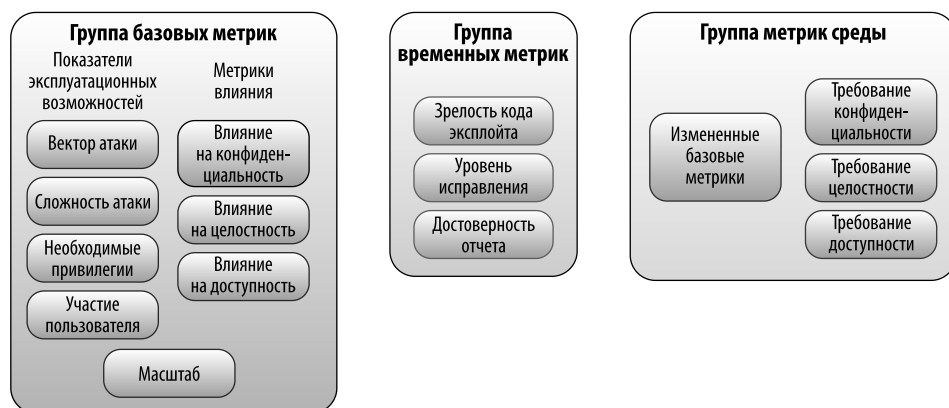


Рис. 3.4

Источник: Forum of Incident Response and Security Teams, CVSS v3.1 Specification Document (www.first.org/cvss/specification-document), последнее обращение 27.08.2024

Как было сказано, CVSS состоит из трех групп метрик: базовых, временных и метрик среды. Серьезность уязвимости на основе внутренних, неизменных характеристик предполагает возникновение худших сценариев, независимо от развернутой среды. Временные, напротив, корректируют базовый показатель с учетом факторов, которые изменяются во времени, включая фактическую доступность эксплойта. Метрики среды корректируют как базовую, так и временную оценку серьезности с учетом особенностей вычислительной среды, включая наличие мер по снижению рисков. Базовый показатель присваивается организацией — владельцем продукта или сторонней организацией, например исследователями безопасности. Именно эти метрики/показатели в итоге публикуются, так как они остаются неизменными после присваивания. Поскольку публикующая сторона не будет иметь полного представления о хостинге или операционной среде, а также о потенциальных средствах защиты, потребители CVSS могут сами

корректировать показатели с помощью временных и средовых факторов. Зрелые организации, которые вносят обоснованные корректировки, способны эффективнее приоритизировать мероприятия по устранению уязвимостей, исходя из более точных расчетов рисков, учитывающих их среду и доступные средства.

Далее мы рассмотрим каждую группу чуть подробнее. Базовые метрики включают внутренние, согласованные во времени характеристики и изменяются лишь в свете появления новой информации, независимо от среды, в которой группа существует. Сюда относятся метрики оценки эксплуатируемости уязвимостей и их возможного влияния. Эксплуатируемость включает в себя такие показатели, как вектор атаки, сложность и необходимые привилегии. Эти метрики строятся в отношении уязвимого компонента и отражают простоту его эксплуатации злоумышленником. По другую сторону эксплуатируемости находятся метрики влияния, включающие традиционную для кибербезопасности триаду: конфиденциальность, целостность, доступность (CIA). Они связаны с последствиями эксплуатации уязвимости и влиянием, которое может быть оказано на уязвимый компонент.

Базовые метрики

Базовые метрики строятся на основе различных аспектов, связанных с пониманием возможностей атакующего в отношении системы, а также ее механизмов защиты. На момент написания книги ведется разработка версии CVSS 4.0. Актуальной же является CVSS 3.1, и далее мы будем разбирать эту тему относительно нее. Как уже говорилось, к базовым метрикам относятся показатели эксплуатируемости и потенциального влияния на систему. Эксплуатируемость включает в себя такие факторы, как вектор атаки и ее сложность, необходимые привилегии, участие пользователя и область.

Вектор атаки отражает контекст доступных для атакующего методов эксплуатации. На эту метрику влияют факторы вроде физической доступности, так как в реальности число злоумышленников, находящихся в непосредственной близости к уязвимому компоненту, чаще будет меньше, чем число тех, кто имеет к нему логический доступ. Вектор атаки характеризуется следующими значениями: сетевой, смежный, локальный или физический. Сетевой вектор атаки может быть связан с локальной сетью, подразумевать удаленную эксплуатацию или даже относиться ко всему Интернету. Смежный же, напротив, будет связан с конкретной физической или логической областью, такой

как общий диапазон/сеть Bluetooth или присутствие в общей локальной подсети. Локальный вектор включает такие факторы, как доступ к системной клавиатуре или консоли, но может подразумевать и удаленный доступ по SSH. Помимо этого, он может потребовать участия пользователя, например в случае фишинговых атак и социальной инженерии. Наконец, физический аспект метрики показывает, требуется ли злоумышленнику непосредственный доступ к уязвимому компоненту для его эксплуатации.

Если вектор атаки основывается на доступных атакующему путях и методах, то сложность атаки определяется условиями, которые находятся вне контроля злоумышленника и могут включать такие нюансы, как наличие в уязвимом компоненте конкретной конфигурации. Сложность атаки может быть оценена как высокая или низкая. Низкая присваивается в случаях, когда злоумышленник способен эксплуатировать уязвимость неоднократно, а высокая — когда ему требуется предварительно собрать необходимую информацию, подготовить целевую среду или внедриться в сетевые пути, как при реализации атак типа on-path (например, Man in the Middle).

Необходимые привилегии описывают уровень привилегий, которыми должен обладать атакующий для успешной эксплуатации уязвимости. Ситуации, в которых привилегии не требуются, получают максимальный балл по этому показателю. Данная метрика может быть отражена тремя значениями:

- нет — доступ к системе или компонентам не требуется;
- низкий — необходим доступ на уровне простого пользователя;
- высокий — необходим административный уровень доступа к уязвимому компоненту.

Участие пользователя описывает, насколько важна роль обычного пользователя в процессе эксплуатации уязвимости: нужно ли пользователю что-то делать, чтобы уязвимость была успешно использована, или же она может быть использована полностью автоматически без его участия. Эта метрика проста и отражается с помощью одного из двух значений: «не требуется» или «требуется».

Метрика «масштаб» показывает, может ли взлом компонента оказать влияние за пределами его области безопасности, воздействовав на другие ресурсы и компоненты. Эта метрика требует понимания степени, в которой уязвимый компонент может влиять на другие ресурсы, находящиеся под его контролем, и, что более важно, какие компоненты изначально входят в его ведение. У метрики «масштаб» существуют два значения: «неизменяемый»

или «изменяемый». Они отражают, может ли эксплуатация повлиять на компоненты, управляемые другой инстанцией системы безопасности.

Внутри базовых метрик также присутствует метрика «влияние», представленная традиционной триадой CIA. CVSS рекомендует аналитикам сопоставлять свои оценки влияния с реальным воздействием, которое может оказать злоумышленник. Если вы знакомы с предоставляемым NIST руководством Risk Management Framework в отношении CIA, то заметите сходство. Эта метрика может быть представлена значениями «не влияет», «низкое влияние» или «высокое влияние». «Не влияет» означает, что влияние на CIA оказано не будет, «низкое влияние» — частичную утрату CIA, а «высокое влияние» — полную потерю CIA и наиболее разрушительное потенциальное воздействие.

Временные метрики

Как говорилось ранее, временные метрики могут изменяться во времени под воздействием различных факторов. Например, показатель CVSS обычно повышается при наличии действующего эксплойта или понижается в случае выхода патча. Помните, что эти факторы относятся к индустрии в целом и не являются специфичными для среды пользователя или конкретных мер противодействия, реализованных организацией. К временным метрикам относятся зрелость кода эксплойта, уровень исправления и достоверность отчета.

Зрелость кода эксплойта основывается на текущем состоянии применяемых в нем техник и доступности самого кода для эксплуатации. Зачастую это называется эксплуатацией «в реальных условиях», и ярким примером является каталог известных эксплуатируемых уязвимостей (<https://clck.ru/3CKPoG>), который регулярно обновляется. В этом списке уже числятся сотни уязвимостей, а также широкий спектр программных и аппаратных эксплойтов, которые могут коснуться многих организаций. Зрелость кода эксплойта описывается следующими показателями:

- не подтверждено;
- проверка концепции (Proof of Concept, PoC);
- функциональный;
- высокий уровень;
- не определено.

Эти степени представляют все возможные состояния кода, начиная с момента, когда он недоступен или существует в теории, и заканчивая полностью функциональным и автономным, не требующим ручной активации эксплойтом.

Уровень исправления помогает определять приоритеты по устранению уязвимостей с учетом возможных решений, таких как обходные пути посредством доступных официальных исправлений. Отражается эта метрика следующими значениями:

- недоступно;
- обходной путь;
- временное исправление;
- официальное исправление.

В одних случаях уязвимость может быть устранена сразу, в других сначала используется обходной путь, пока официальное исправление не будет разработано, проверено и одобрено для применения и реализации.

Последняя временная метрика — достоверность отчета — отражает фактическую достоверность сообщения об уязвимости и связанных с ней технических деталей. Например, первые отчеты могут поступать из непроверенных источников, но при этом быть достаточно проработанными для оценки поставщиками или экспертами по кибербезопасности. Эта метрика описывается такими значениями, как «неизвестно», «обосновано» и «подтверждено», представляющими возрастающий уровень уверенности в обнаруженной уязвимости и ее деталях. Все эти факторы могут влиять на общую оценку уязвимости.

Метрики среды

Метрики среды оцениваются в контексте каждой отдельной организации индивидуально. Для конкретного потребителя ПО они будут отличаться ввиду уникальности рабочей среды и ряда факторов, которые обычно влияют на оценку. К таким факторам могут относиться стек используемых технологий, архитектура и реализованные средства противодействия угрозам.

Метрики среды помогают организациям скорректировать базовые показатели на основе факторов, характерных исключительно для их рабочей

инфраструктуры. Вносимые изменения касаются триады CIA и позволяют аналитикам присваивать значения, которые соответствуют роли каждого ее аспекта в функционировании организации или ее миссии. Стоит отметить, что для внесения точных изменений требуется детальное понимание факторов базовых и временных метрик, чтобы степень угрозы той или иной уязвимости не оказалась занижена.

Шкала оценки CVSS

В CVSS используется количественная шкала оценки серьезности от 0,0 до 10,0, показанная в табл. 3.1.

Таблица 3.1. Шкала оценки CVSS

Степень	Баллы CVSS
Отсутствует	0,0
Низкая	0,1–3,9
Средняя	4,0–6,9
Высокая	7,0–8,9
Критическая	9,0–10,0

Все описанные метрики и методы оценки CVSS позволяют построить так называемую *векторную строку*, определенную в спецификации CVSS как «специально отформатированная текстовая строка, каждое значение которой соответствует некоей метрике, которая должна всегда отображаться вместе с показателем уязвимости». Векторные строки состоят из групп показателей, названий метрик, возможных их значений, а также отражают, являются ли они обязательными. В результате получается строка, которую можно представить следующим образом: «Вектор атаки: сетевой, сложность атаки: низкая, необходимые привилегии: высокие, участие пользователя: не требуется, масштаб: неизменяемый, конфиденциальность: слабо, целостность: слабо, доступность: не влияет».

Для вычисления этих метрик необходимо детальное понимание возможных полей, их значений и прочих факторов. Настоятельно рекомендуем изучить спецификацию CVSS 3.1, чтобы лучше разобраться с формулами оценки и руководствами (<https://clck.ru/3CKPpc>).

Критика

Несмотря на широкое распространение CVSS, эта система не обходится без критики. Эксперт в области кибербезопасности и автор книг Уолтер Хейдок опубликовал в своем блоге *Deploying Securely* (<https://clck.ru/3CKPqL>) статью под названием *CVSS: An (Inappropriate) Industry Standard for Assessing Risk* (<https://clck.ru/3CKPua>), в которой подверг критике CVSS. Уолтер отмечает, что CVSS не подходит для анализа угроз кибербезопасности. Он приводит цитаты из статей ведущих специалистов индустрии, в которых авторы рассказывают, почему не следует использовать эту систему для анализа рисков без дополнительных инструментов, хотя по факту так и происходит (<https://clck.ru/3CKPvd>). Как пишет Уолтер и другие критики, CVSS зачастую применяется организациями для приоритизации мер по устранению уязвимостей или их ослаблению, исходя из выявленных показателей простоты эксплуатации и потенциального ее влияния.

Исследования, проведенные такими организациями, как Tenable, указывают, что более 50 % уязвимостей оцениваются системой CVSS как высокие или критические без учета вероятности их эксплуатации в принципе (<https://clck.ru/3CKPwH>). В работе говорится, что для 75 % всех уязвимостей, оцененных как высокая или критическая, по факту нет опубликованных и связанных с ними эксплойтов, но специалисты по безопасности все равно ставят приоритеты на их устранении. Получается, что компании впустую тратят ограниченные ресурсы на решение проблем, которые вряд ли когда-либо возникнут, хотя могли бы направить силы на слабые места с активными эксплойтами. Организации потенциально подвергают себя значительным рискам, не сосредотачиваясь на уязвимостях с оценкой серьезности от 4 до 6, для которых есть активные эксплойты. Вместо этого они применяют комплексный подход для устранения уязвимостей, которые по системе CVSS получили высокую оценку. И это понятно, поскольку организации зачастую имеют дело с сотнями или тысячами активов, все из которых нередко имеют множество уязвимостей. Сложно отслеживать каждую и реализовывать точечную стратегию устранения, когда вы буквально утопаете в море уязвимых данных, не говоря уже о получении четкого представления об активах вашей компании, представляющих риски.

Более углубленно тема CVSS раскрывается в опубликованном NIST внутреннем отчете *8409 Measuring the Common Vulnerability Scoring System Base Score Equation* (<https://clck.ru/3CKPx2>).

Система прогнозирования эксплойтов

В свете острой критики CVSS некоторые специалисты призывают использовать Систему прогнозирования эксплойтов (Exploit Prediction Scoring System, EPSS) или объединить CVSS и EPSS для повышения эффективности метрик уязвимостей. Система EPSS разработана организацией FIRST и представляет собой открытый и основанный на данных проект, созданный для оценки вероятности эксплуатации уязвимости в реальных условиях. CVSS опирается на характерные особенности уязвимости, определяя их серьезность. Тем не менее один только показатель серьезности еще не отражает вероятности эксплуатации, что важно для специалистов по работе с уязвимостями. Эта информация необходима им для того, чтобы эффективно расставлять акценты по устранению уязвимостей с целью минимизации угроз для организации.

В рамках EPSS существует специальная открытая группа (Special Interest Group, SIG), в которую могут вступить все, кто желает принять участие в реализации проекта. EPSS является добровольным начинанием, реализуемым исследователями, специалистами по кибербезопасности, академиками и чиновниками. Тем не менее, несмотря на такой совместный подход, права на обновление модели и связанных с ней руководств принадлежат FIRST. На данный момент в группе работают представители таких организаций, как RAND, Cyentia, Virginia Tech, Kenna Security, и других. Группа EPSS опубликовала несколько работ по прогнозированию атак, моделированию и обнародованию уязвимостей, а также их эксплуатации (www.first.org/epss/papers).

Модель EPSS

Цель EPSS — помочь специалистам по безопасности и их организациям грамотно расставлять приоритеты при устранении уязвимостей. В современном цифровом мире возникает все больше разнообразных угроз, что обусловлено такими факторами, как непрерывная цифровизация систем и общественных процессов, более тщательный анализ цифровых продуктов, а также расширение возможностей для исследований и освещения новых достижений. EPSS показывает, что организации способны исправлять от 5 до 20 % уязвимостей в месяц. Также было установлено, что по факту в реальных условиях эксплуатируется менее 10 % опубликованных уязвимостей. Кроме

того, в этой связи отмечается дефицит кадров. В качестве примера можно взять ежегодно проводимое (ISC) исследование кадровых ресурсов в сфере кибербезопасности Cybersecurity Workforce Study (<https://clck.ru/3CKPy7>), согласно которому нехватка профессионалов составляет более двух миллионов по всему миру (рис. 3.5).



Рис. 3.5

Источник: модель EPSS (www.first.org/epss/model), последнее обращение 27.08.2024

Эти факторы побуждают организации применять целостный и эффективный подход к устранению в первую очередь тех уязвимостей, которые представляют наибольшую опасность, чтобы избежать нерационального использования ресурсов и времени. Модель EPSS помогает в этом вопросе, демонстрируя вероятность эксплуатации уязвимости в течение ближайших 30 дней. Отражаются эти показатели значениями в диапазоне от 0 до 1 либо в процентах от 0 до 100. Для предоставления этой информации в EPSS используются данные из различных источников, таких как список CVE компании MITRE, информация о CVE вроде количества дней с момента публикации, а также наблюдения от поставщиков вроде AlienVault и Fortinet, показывающие, насколько активно та или иная уязвимость эксплуатируется в реальных условиях.

Команда EPSS публикует данные, подкрепляющие их подход, состоящий в объединении показателей CVSS и EPSS для максимальной эффективности распределения усилий по устранению уязвимостей. Например, многие организации предпочитают устранять уязвимости с показателем CVSS 7 и выше. Но подобный подход позволяет устанавливать приоритеты только

на основе данных CVSS, независимо от того, эксплуатируется ли уязвимость в реальности. Интеграция EPSS с CVSS оказывается более продуктивным подходом, так как компании могут расставлять приоритеты не только на основе данных о серьезности уязвимостей, но также исходя из понимания того, эксплуатируются ли они активно в действительности.

Модель EPSS оперирует двумя основными метриками: эффективность и покрытие. Первая показывает, насколько грамотно организации используют ресурсы для разрешения уязвимостей. EPSS демонстрирует, что для большинства компаний эффективнее тратить ресурсы на закрытие самых активно эксплуатируемых уязвимостей. Такой подход отличается от выбора случайных уязвимостей, основанного исключительно на серьезности по шкале CVSS. Вторая же метрика отражает процент устраненных уязвимостей среди эксплуатируемых.

Чтобы продемонстрировать эффективность предлагаемого подхода, команда EPSS в 2021 году провела исследование, используя базовые показатели CVSS v3: данные CVSS v1 и CVSS v2. Они проанализировали 30-дневный период, определив общее число CVE, а также количество устраненных и эксплуатируемых CVE. В результате можно выделить два основных момента (см. рис. 3.5). Во-первых, большинство CVE просто не устраняются: организации не исправляют большую часть CVE, а среди исправленных присутствует лишь часть тех, которые реально эксплуатируются и представляют наибольшую угрозу. Во-вторых, что EPSS v2 дополнительно повышает эффективность мер по устранению уязвимостей, обеспечивая больший процент закрытия именно эксплуатируемых. Когда у компаний возникают кадровые проблемы по части кибербезопасности, важно максимизировать отдачу от имеющихся сотрудников, сосредоточив их усилия на тех проблемах, которые представляют наибольшую угрозу. EPSS стремится помочь организациям более эффективно распределять имеющиеся ресурсы, снижая при этом риски.

Критика EPSS

Подобно CVSS, EPSS тоже не обошлась без критики со стороны сообщества. В блоге SEI (<https://clck.ru/3CKPzy>) была опубликована статья *Probably Don't Rely on EPSS Yet*. Изначально сотрудники SEI выпустили работу под названием *Towards Improving CVSS*, раскритиковав CVSS, вскоре после которой появилась EPSS (<https://clck.ru/3CKQ2V>). В статье про EPSS

отмечаются проблемы с прозрачностью этой системы, а также сложности, касающиеся ее данных и результатов оценки. В ней подчеркивается недостаточная ясность процессов разработки модели, ее управления и выбора целевой аудитории. В ней также указывается, что эта система опирается на существующие CVE ID, то есть она не особо полезна для поставщиков ПО, команды реагирования на инциденты или группы охотников за багами, поскольку многие уязвимости, с которыми они работают, еще не имеют CVE ID и, возможно, никогда их не получат. Существует также проблема с тем, что EPSS окажется бесполезна при работе с уязвимостями нулевого дня, поскольку они раскрываются уже в процессе эксплуатации, несмотря на отсутствие связанных с ними CVE ID.

Помимо этого, автор публикации поднимает вопрос в отношении открытости и прозрачности EPSS. Хотя эта система и позиционирует себя как открытый и основанный на данных проект — и даже имеет действующую специальную группу (SIG), — EPSS и ее управляющая организация FIRST по-прежнему сохраняют за собой право на изменение сайта и модели в любое время и без каких-либо объяснений. Вдобавок к этому даже члены SIG не имеют доступа к коду или данным, используемым в модели EPSS. SIG также не может контролировать разработку модели или управлять ею, то есть процесс ее обновления и изменения остается скрытым для этой группы, не говоря уже о внешних потребителях. В статье также указывается, что модель EPSS и ее данные могут быть изъяты из публичного доступа и использования, так как управляются FIRST.

EPSS определяет вероятность того, что уязвимость будет эксплуатирована в течение ближайших 30 дней. Но для этой оценки необходимо знание нескольких фундаментальных аспектов. К ним относятся: существующие CVE ID из NVD вместе с вектором CVSS v3, сигнатура системы обнаружения вторжения (Intrusion Detection System, IDS), связанная с активным эксплойтом для CVE ID, участие со стороны AlienVault или Fortinet (которые предоставляют данные для EPSS) и, наконец, сама модель, привязанная к следующим 30 дням. Как отмечает автор, только 10 % уязвимостей с CVE ID имеют сигнатуры IDS. То есть остальные 90 % уязвимостей могут остаться непроверенными на предмет их эксплуатации. Это также создает зависимость от Fortinet и AlienVault в плане их IDS и сопутствующих данных, что может быть исправлено привлечением к процессу большего числа представителей сообщества поставщиков услуг и систем безопасности.

Подход CISA

В свете развития дискуссии на тему приоритизации и управления уязвимостями к ней начали подключаться такие организации, как CISA. В статье от ноября 2022 года под названием *Transforming the Vulnerability Management Landscape* (<https://clck.ru/3CKQ3F>) помощник директора CISA Эрик Голдштейн рассуждает на тему сложности современного цифрового ландшафта и появления все большего числа новых уязвимостей.

Исходя из содержания статьи, CISA подчеркивает три важнейших шага для развития экосистемы управления уязвимостями.

1. Расширение применения стандарта обмена информацией о проблемах безопасности (Common Security Advisory Framework, CSAF) для реализации автоматизированных машиночитаемых руководств.
2. Повсеместное внедрение стандарта обмена информацией об использовании уязвимостей (Vulnerability Exploitability eXchange, VEX), который помогает потребителям ПО понять, какие уязвимости действительно представляют риск для их окружения, а какие нет.
3. Помощь организациям в эффективном распределении ресурсов для управления уязвимостями с помощью методологии для приоритизации уязвимостей на основе их влияния на конкретные заинтересованные стороны (Stakeholder-Specific Vulnerability Categorization, SSCV) и разработанного CISA каталога известных эксплуатируемых уязвимостей (Known Exploited Vulnerabilities, KEV).

Каждый из этих шагов мы разберем подробнее.

Стандарт CSAF

Первым в списке CISA идет использование CSAF для автоматизации машиночитаемых руководств по безопасности. Как указывает CISA, при каждом обнаружении и раскрытии новой уязвимости поставщикам ПО необходимо пересматривать свои предложения, проверяя, касается ли это их продуктов. Если да, то они должны решить, какие меры требуется принять для устранения обнаруженной уязвимости, и сообщить эту информацию своим клиентам. И пока поставщики работают над исправлением, злоумышленники также стараются найти активный эксплойт до его устранения, будь

то исправление в контексте SaaS или выпуск патча для традиционного ПО. В свете все более частого обнаружения и обнародования уязвимостей возникает критическая потребность в автоматизации и ускорении перечисленных действий для доставки этой информации потребителям ПО, что делает машиночитаемость и автоматизацию особенно актуальными.

Инициативой CSAF управляет технический комитет некоммерческого консорциума OASIS. Этот консорциум помогает создавать и внедрять лучшие практики и стандарты. Он предоставляет прекрасные информативные ресурсы для тех, кто желает больше узнать о CSAF, включая видео *What Is a Common Security Advisory Framework (CSAF)?* (<https://clck.ru/3CKQ5E>) и подробную спецификацию CSAF 2.0 (<https://clck.ru/3CKQ6J>).

Как сказано в спецификации, «CSAF поддерживает создание, обновление и интероперабельный обмен между заинтересованными сторонами информационными сводками по безопасности, представленными в виде структурированной информации о продуктах, уязвимостях, а также статусе их влияния и устранения». Эти сводки предоставляются в формате JSON.

Традиционно сводки по безопасности публикуются в виде статических документов, например PDF-файлов и сайтов, и ориентированы на человека. Проблема этого подхода в нарастающих темпах обнаружения и обнародования уязвимостей наряду со сложностями их исправления до момента эксплуатации злоумышленниками. В связи с этим и появляется потребность в автоматизированной обработке этой информации. Существуют и другие сферы кибербезопасности, а также инициативы, которые аналогичным образом делают акцент на внедрении машиночитаемых артефактов. К ним относятся инструменты по управлению рисками и соответствием требованиям (Governance Risk and Compliance, GRC), внедряющие язык для анализа средств обеспечения безопасности (Open Security Controls Assessment Language, OSCAL), а также традиционные IT-решения, применяющие новые подходы «инфраструктура как код» (Infrastructure as Code, IaC) и «соответствие требованиям как код» (Compliance as Code, CaC). Современный технологический прогресс движется слишком быстро, чтобы человек продолжал оставаться информационным посредником.

CSAF также стремится предоставить надежный набор инструментов (<https://clck.ru/3CKQ7D>), среди которых присутствуют парсер CSAF (CSAF Parser), визуализатор (Visualizer), доверенный провайдер (Trusted Provider) и агрегатор

(Aggregator). Стоит отметить, что на момент написания книги эти средства еще находятся на стадии разработки и могут иметь ограниченные возможности. Например, парсер CSAF пока не поддерживает CSAF 2.0. У каждого из инструментов есть свой репозиторий GitHub, документация и кодовая база, что помогает потребителям и издателям ПО более эффективно использовать CSAF.

Схемы документов CSAF включают три основных класса информации:

- структура, агрегирование и справочная информация документа;
- информация о продукте, оцененная создателем сводки как актуальная;
- информация об уязвимости в отношении обсуждаемого продукта (продуктов).

Помимо этого, CSAF поддерживает возможность ссылаться на стандартизованную информацию вроде данных о платформе, классификации уязвимостей и их оценки. К примерам относятся CPE, CVSS и CWE, для каждой из которых есть своя схема использования.

Исходя из этого базового описания, становится понятно, как CSAF помогает нам перейти в эпоху машиночитаемых сводок по безопасности, что ускорит обмен информацией между всеми заинтересованными сторонами. Все это будет полезно как для поставщиков, так и для потребителей ПО и позволит быстрее принимать решения в отношении управления рисками в области кибербезопасности. Чтобы узнать больше об этой теме, рекомендуем изучить саму спецификацию или посмотреть ролики о CSAF (<https://clck.ru/3CKQ7D>).

Стандарт VEX

Исходя из сложившейся дискуссии вокруг CSAF, в качестве второго шага CISA предложило повсеместное внедрение формата VEX. Подробнее о нем мы поговорим в главе 4. Если коротко, то VEX позволяет поставщикам ПО выяснять, подвержен ли их продукт конкретным уязвимостям. Так как компании сталкиваются с недостатком грамотных специалистов в сфере кибербезопасности, VEX помогает им более эффективно расставлять акценты при устранении уязвимостей, не тратя ресурсы на менее опасные. Документы VEX дополняют SBOM, помогая понять, насколько уязвимы указанные компоненты. При этом новые уязвимости могут возникать не только в новом ПО, так как их по-прежнему продолжают находить и в уже существующем.

В связи с этим документы VEX в некоторых случаях будут представлять собственную ценность.

По части спецификаций и инструментов индустрия продолжает развиваться, и инновации касаются не только SBOM, но и VEX. Например, в начале 2023 года ведущая компания в области цепочек поставок ПО, Chainguard, анонсировала спецификацию OpenVEX, в работе над которой принимали участие HPE и TestifySec. OpenVEX создавалась с упором на минимализм, соответствие требованиям, встраиваемость и интероперабельность. Подробнее о ней можно почитать в репозитории GitHub (<https://github.com/openvex/spec>).

Методология SSCV

Один из ключевых вопросов в индустрии связан с поддержкой организаций в определении наиболее опасных для них уязвимостей.

Агентство CISA курировало создание каталога KEV и в итоге опубликовало его, выпустив оперативную директиву (Binding Operative Directive, <https://cisa.gov/3CKQ9o>), обязывающую федеральные агентства устранить указанные в этом каталоге уязвимости. Коммерческим организациям CISA рекомендует проделать то же самое. Каталог опубликован в виде веб-страницы, но доступен и в формате CSV или JSON. Организации и частные лица также могут подписаться на обновления каталога KEV и получать уведомления о добавляемых в него уязвимостях по электронной почте.

Для внесения в каталог уязвимость должна соответствовать трем критериям:

- иметь CVE ID;
- активно эксплуатироваться в реальных условиях с подтверждением в виде надежных свидетельств;
- иметь отчетливое руководство по устранению, например предоставленные поставщиком патчи или обновления.

Как уже говорилось, программа CVE спонсируется CISA, но фактически управляется MITRE — Федеральным бюджетным центром исследований и разработок (federally funded research and development center, FFRDC). Целью программы являются определение и каталогизация публично раскрытых уязвимостей. CVE ID присваиваются компаниями CNA и могут иметь один из трех статусов: «зарезервирована», «опубликована» или «отклонена».

Включение уязвимости в каталог KEV не происходит по факту одной только возможности ее эксплуатировать, но и от наличия надежных свидетельств попытки или успешной ее реализации. Например, может быть доказано, что злоумышленник попытался выполнить в отношении цели код, но его попытка провалилась, или же он проделал это на так называемой системе-приманке (honeypot system). Приманка — это механизм, предназначенный для выявления вредоносной активности посредством наблюдения. В отличие от попытки, успешная эксплуатация означает, что злоумышленник сумел воспользоваться уязвимым кодом на целевой системе, получив возможность совершать в ней неавторизованные действия. CISA специально отмечает, что эксплойты на стадии PoC не вносятся в каталог KEV, так как, хотя исследователи и могли продемонстрировать возможность эксплуатации некой уязвимости, по факту таких попыток в реальных условиях не происходило.

Третий критерий для включения в каталог подразумевает наличие четких инструкций по устранению уязвимости. Это означает, что существуют конкретные шаги, которые организации могут предпринять для исключения рисков, связанных с определенной уязвимостью. В качестве таких действий может выступать установка обновлений от поставщика или, в потенциально критических случаях, полное удаление продукта, подверженного уязвимости, если обновлений для него нет или не может быть в принципе из-за окончания срока поддержки. CISA также признает, что при отсутствии релевантных обновлений и патчей организации зачастую будут стремиться реализовать иные меры противодействия (чтобы воспрепятствовать эксплуатации уязвимости) или искать обходные пути, представляющие собой ручные изменения с целью защитить уязвимую систему от эксплуатации, пока не появятся необходимые патчи.

Опираясь на руководство по применению KEV, CISA стремится помочь организациям эффективно распределять ресурсы для управления уязвимостями, используя категоризацию на основе SSVC. Система SSVC была создана как совместный проект CISA и SEI. SEI, обычно критикующий организации, которые ограничиваются использованием исключительно таких источников, как CVSS, в 2018 году опубликовал работу под названием *Towards Improving CVSS*. В ней его специалисты указали на несколько недостатков широко используемой CVSS для приоритизации уязвимостей. И здесь стоит отметить иронию ситуации: несмотря на лоббирование таких фреймворков, как SSCV, государственные организации вроде CISA по-прежнему настаивают на применении CVSS, хотя при этом активно

сотрудничают с учреждениями вроде SEI, которые указывают на недостатки этой модели (<https://clck.ru/3CKQBD>).

Согласно руководству, SSVC является кастомизируемой моделью дерева решений, нацеленной на помощь Правительству США и связанным с ним субъектам в приоритизации мер по реагированию на обнаруженные уязвимости, но также доступной для использования другими организациями. Всего дерево SSVC включает четыре возможных решения, описанных в табл. 3.2.

Таблица 3.2. Решения SSVC

Отслеживание	В данный момент уязвимость не требует принятия каких-либо мер. Организация продолжает ее отслеживать и повторно оценивать в случае появления новой информации. CISA рекомендует исправлять уязвимости из этой категории <i>в соответствии</i> с действующим графиком обновлений
Отслеживание*	Уязвимость имеет конкретные характеристики, которые могут потребовать более пристального мониторинга изменений. Подобные уязвимости CISA рекомендует исправлять <i>в соответствии</i> с действующим графиком обновлений
Требуется внимания	Уязвимость требует рассмотрения на административном уровне организации. В качестве необходимых мер организация может запросить стороннюю помощь или дополнительную информацию об уязвимости, а также выпустить внутреннее либо внешнее оповещение о ней. CISA рекомендует устранять подобные уязвимости, <i>не дожидаясь</i> очередного обновления
Требуется действий	Уязвимость требует рассмотрения специалистами всех уровней. В качестве необходимых мер организация может запросить стороннюю помощь или информацию об уязвимости, а также опубликовать внутренние/внешние оповещения. Как правило, внутренние группы собираются вместе с целью определить тактику реагирования, после чего предпринимают утвержденные действия. CISA рекомендует устранять подобные уязвимости <i>как можно скорее</i>

Согласно руководству, организации должны определять область влияния уязвимости, поскольку это также затрагивает дерево решений. Например, следует выяснить, подвержена ей вся корпорация или же только часть критической системы.

При использовании SSVC и обходе дерева решений организациям следует учитывать несколько факторов, а именно Национальную базу данных уязвимостей (NVD) или Центры обмена и анализа информации (Information Sharing and Analysis Centers). Эти источники обычно предоставляют сведения о том, является ли угроза эксплуатируемой или находится на стадии PoC.

Помимо этого, организациям нужно понимать технические последствия эксплуатации уязвимости. Здесь можно провести параллели с используемым в CVSS принципом определения серьезности. Степень при этом может отражаться такими значениями, как «частичная», когда злоумышленник имеет ограниченный контроль или влияние на систему, и «тотальная», когда у него есть полный контроль над поведением уязвимого ПО или системы.

Еще один важный момент касается возможности автоматизации эксплуатации. Злоумышленникам гораздо проще масштабировать свои корыстные действия с помощью эксплойтов, которые можно автоматизировать, нежели тех, которые требуют ручного вмешательства и реализации. Описывающим этот аспект значением здесь может быть просто «да» либо «нет». Если ответ «да», значит, злоумышленник способен автоматизировать шаги 1–4 модели Lockheed Martin Cyber Kill Chain (<https://clck.ru/3CKQCP>): разведка, разработка оружия, доставка, эксплуатация. В руководстве отмечается, что, кроме автоматизации, нужно учитывать использование цепочек уязвимостей, поскольку для успешной эксплуатации иногда возможно или даже необходимо связать несколько уязвимостей.

После рассмотрения технических аспектов воздействия нужно учесть угрозу для выполняемых организацией задач, то есть влияние уязвимости на критически важные функции (mission-essential functions, MEF) или связанные с ними субъекты. В руководстве SSVC эти функции определяются как «напрямую связанные с реализацией миссии организации, прописанной в ее уставе». Формируются MEF в рамках мероприятий по планированию непрерывности бизнеса (business continuity planning). Здесь используются значения «минимальное», «поддерживается» и «критическое». Критическое влияние означает, что уязвимый компонент непосредственно обеспечивает возможности как минимум одной MEF или субъекта и его эксплуатация приведет к провалу выполнения задачи. Значение «поддерживается», в свою очередь, говорит о том, что уязвимый компонент поддерживает выполнение MEF, но не напрямую. Наконец, «минимальное» значение описывает ситуацию, в которой остальные два критерия неприменимы.

Далее в дереве решений идет влияние на благосостояние общества, или степень, до которой уязвимый компонент либо система воздействуют на людей в целом. Для благосостояния SSVC использует следующее определение: физическое, социальное, эмоциональное и психологическое состояние людей. Решения классифицируются в зависимости от влияния, которое может быть материальным или необратимым, а также типа ущерба, который может варьироваться от физического до психологического и даже финансового.

Последним этапом является анализ мер по противодействию. Здесь оценивается степень сложности противостояния уязвимости во временном контексте на основе следующих факторов:

- доступность средств противодействия;
- сложность внесения необходимых изменений в систему;
- наличие исправления или необходимость использования обходного пути.

В руководстве подчеркивается, что уровень требуемых мер противодействия не должен изменять приоритет решения SSVC, но дерево SSVC необходимо активно отслеживать и учитывать. На рис. 3.6 представлен пример дерева решений, но его также можно выразить и в виде таблицы.

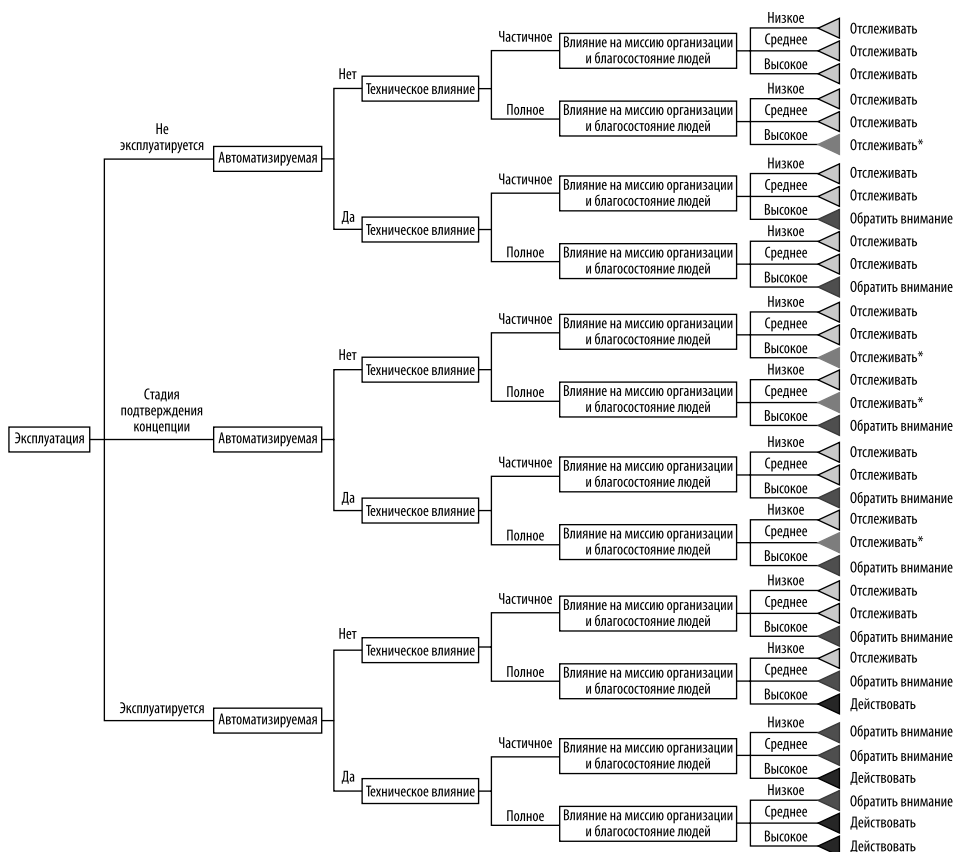


Рис. 3.6

Движение вперед

Руководство CISA акцентирует внимание на таких мерах, как автоматизация обмена отчетами по безопасности с помощью CSAF и информирование потребителей ПО об уязвимостях через формат VEX. Тем не менее также становится очевидной потребность в субъективном опыте обеспечения кибербезопасности, особенно в сложных деревьях решений. Автоматизация играет решающую роль в снижении когнитивной нагрузки на организацию и ее команды по кибербезопасности, но при этом важно иметь опыт для понимания корпоративного или системного контекста, подверженного уязвимостям, а также способов эффективного распределения ресурсов для их устранения.

Резюме

В этой главе мы познакомились с используемыми в индустрии базами данных безопасности и методиками оценки уязвимостей. Сюда относятся традиционные базы данных уязвимостей с их потенциальными ограничениями и альтернативные подходы. Кроме того, мы разобрали распространенные форматы идентификации ПО, связанные с ними проблемы и предлагаемые решения для некоторых из этих проблем. Мы также изучили системы категоризации, оценки и обмена информацией об уязвимостях. В следующей главе мы поговорим об истории появления и развития концепции SBOM и сопутствующих ей идей вроде VEX.

4

Появление концепции SBOM

<https://t.me/portalToIT>

В этой главе речь пойдет об истоках концепции SBOM. Здесь вы узнаете о ее ранних успехах и неудачах, а также о государственных и частных организациях, способствовавших ее взрослению. Помимо этого, мы разберем некоторые детали используемых в SBOM форматов, конкретные сферы ее применения и поговорим о появлении VEX.

SBOM в стандартах: провалы и успехи

Несмотря на то что в индустрии наблюдается активное внедрение концепции SBOM, для достижения этой точки потребовалось немало лет и усилий, включая участие различных правительственных и отраслевых организаций. Значительный прогресс в этом направлении произошел при поддержке NTIA.

Тем не менее, хотя NTIA и события вроде атак Log4j и SolarWinds могли сыграть решающую роль в распространении SBOM, более ранние случаи, связанные с уязвимостями Apache Struts 2 и OpenSSL, привели к появлению Закона об управлении и прозрачности цифровых цепочек поставок от 2014 года (Cyber Supply Chain Management and Transparency Act, <https://clck.ru/3CKQDN>). В этом документе акцент делается на целостность ПО, прошивок и продуктов, разработанных для Правительства США или приобретенных им и использующих сторонний код, в том числе открытый. Этот закон требует, чтобы ПО, прошивки и контракты на программные продукты, поставляемые для правительства, сопровождались перечнями используемого в них ПО и зависимостей, то есть SBOM. Одним из пионеров, участвовавших

в этой инициативе, стал Джошуа Корман, успевший поработать в различных компаниях-лидерах индустрии и CISA, а также основавший движение I am The Cavalry, работающее на пересечении сфер цифровой и общественной безопасности и человеческой жизнедеятельности.

Указанный закон так и не был полноценно реализован, отчасти вследствие сопротивления индустрии к призывам повышать прозрачность компонентов ПО в создаваемых продуктах. Тем не менее это все равно показывает, что, хотя концепция SBOM и стала активно обсуждаться недавно, дискуссия вокруг нее происходит уже около десятилетия.

К прочим ярким примерам относится отчет Оперативной группы по вопросам кибербезопасности в сфере здравоохранения от июня 2017 года. В нем специалисты призывают производителей и разработчиков к созданию «списка зависимостей и составляющих частей», описывающего компоненты медицинских устройств (<https://clck.ru/3CKQE9>).

NTIA: пропаганда потребности в SBOM

В последнее время наиболее активную позицию по части внедрения SBOM со стороны государства проявляло NTIA. Это управление начало работать над продвижением идеи прозрачности программных компонентов в 2018 году. В июле того же года они провели встречи (<https://clck.ru/3CKQFD>) со множеством участников индустрии, на которых обсуждали тему повышения прозрачности компонентов ПО, связанные с этим проблемы и варианты их решения. На этих встречах также присутствовал доктор Аллан Фридман, в то время работавший в NTIA. Фридман, как и упомянутый выше Джошуа Корман, играл заметную роль в обсуждениях идеи SBOM с самого ее зарождения. Сегодня он занимает должность старшего советника и стратега в CISA, продолжая содействовать продвижению идей прозрачности ПО и SBOM.

NTIA отмечает, что современное ПО разрабатывается с использованием многих компонентов и библиотек как из коммерческой, так и из открытой сферы (<https://clck.ru/3CKQFg>). Управление также осознает, что создание и обслуживание ведомостей этих компонентов и связанных с ними уязвимостей представляет невероятно трудную задачу. Эта организация, вместе с сообществом заинтересованных лиц, стремится выработать наиболее эффективные подходы к инвентаризации программных компонентов, практик, а также связанных с этим вопросов и рыночных сложностей.

В результате усилий NTIA был создан надежный пакет документов и руководств (<http://ntia.gov/SBOM>), связанных с прозрачностью ПО и SBOM. Эти документы нацелены на ознакомление с концепцией тех, кто с ней еще не знаком, углубленное разъяснение ее принципов, реализацию SBOM, а также включают сопутствующие технические ресурсы.

Пункт F раздела 4 Указа Президента США о повышении безопасности цепочек поставок программного обеспечения ставил NTIA задачу опубликовать список минимальных элементов, которые должны указываться в SBOM, в течение 60 дней с момента выхода самого указа. В итоге в июле 2021 года NTIA выпустило требуемое руководство под названием *The Minimum Elements for a Software Bill of Materials (SBOM)* (<https://clck.ru/3CKQGQ>). В этом руководстве установлен минимальный перечень необходимых элементов SBOM, определена сфера применения и описаны примеры использования концепции для повышения прозрачности цепочек поставок ПО. Помимо этого, в нем указано, что системный обмен и отслеживание метаданных компонентов помогают поддерживать прозрачность цепочек поставок.

В текущем разделе мы разберем это руководство, в котором описаны три категории элементов:

- поля данных;
- поддержка автоматизации;
- практики и процессы.

NTIA отмечает, что любое ПО можно представить в виде иерархического дерева, состоящего из компонентов и подкомпонентов. Подразумеваются как сторонние компоненты, происходящие из другого источника, так и собственные, которые отслеживаются в виде автономных, независимых единиц ПО. Первой категорией выступают поля данных. Они содержат базовую информацию о каждом компоненте, и основная их задача — помочь организации отслеживать эти компоненты вдоль всей цепочки поставок ПО. NTIA определяет базовый набор полей данных, представленный в табл. 4.1.

Организации могут использовать эти поля данных для сопоставления компонентов с другими источниками данных, производя так называемое *обогащение*. Как видно из табл. 4.1, смысл в том, чтобы предоставить базовую информацию вроде поставщика, наименования и версии компонента, после чего связать этот компонент с соответствующими вышестоящими узлами. Обычно это называется *связующей зависимостью*, поскольку большинство программных продуктов включают сторонние элементы, что ведет к формированию зависимостей между собственным кодом приложения и кодом этих сторонних

компонентов или приложений. Дополнительно стоит отметить, что в руководстве NTIA рассматриваются некоторые проблемы, такие как использование среди поставщиков ПО различных подходов к версионированию.

Таблица 4.1. Базовый набор полей данных

Поле	Описание
Поставщик	Имя субъекта, который создает, определяет и идентифицирует компоненты
Компонент	Обозначение, присвоенное единице ПО изначальным поставщиком
Версия компонента	Идентификатор, используемый поставщиком для указания изменений в сравнении с предыдущей версией
Прочие уникальные идентификаторы	Другие идентификаторы, используемые для идентификации компонента или в качестве ключа для связанных баз данных
Зависимая связь	Описание связи, по которой вышестоящий компонент X включен в ПО Y
Автор данных SBOM	Имя субъекта, создающего данные SBOM для этого компонента
Временная метка	Регистрация даты и времени сборки данных SBOM

Источник: <https://clck.ru/3CKQNk>. Министерство торговли США, открытая информация

Еще одна значимая категория в SBOM — необходимость автоматизации. Люди, скептически настроенные в отношении распространения и внедрения концепции SBOM, указывают на потребность в надежных инструментах и автоматизации для широкого внедрения этого процесса. Поддержка автоматизации даст организациям возможность интегрировать SBOM в используемые ими схемы управления уязвимостями, рисками цепочек поставок и программы по обеспечению кибербезопасности. В руководстве NTIA описываются три основных формата SBOM, которые мы рассмотрим в разделе «Форматы SBOM».

Наконец, в руководстве приводятся практики и принципы, отражающие механики использования SBOM внутри организаций:

- частота;
- глубина;
- известные неизвестные;
- распространение и поставка;
- управление доступом;
- учет ошибок.

Частота описывает, собственно, частоту создания новых SBOM. В руководстве подчеркивается, что их необходимо создавать после каждой очередной сборки или релиза ПО. Это также нужно делать, если поставщику становятся известны новые детали о компонентах его ПО, включая исправление ранее обнаруженных ошибок.

Глубина отражает уровень детализации информации в дереве зависимостей описываемых в SBOM компонентов. В руководстве говорится, что в SBOM должны быть указаны как минимум все компоненты верхнего уровня, а также список транзитивных зависимостей. *Транзитивные зависимости* можно определить как компоненты, необходимые для выполнения, компиляции или тестирования ПО. И хотя их полное перечисление не всегда целесообразно, руководство требует указания достаточной информации для нахождения подобных зависимостей в случае необходимости. В плане глубины каждая организация может устанавливать разные требования, даже ввиду внутренних политик или стандартов. При этом потребители SBOM будут стремиться указать глубину используемых программных продуктов, особенно при работе с поставщиками проприетарного ПО, включающей контракты и юридические аспекты.

Принцип *известных неизвестных* отражает обязательство составителей SBOM сохранять ясность своих намерений в отношении будущих зависимостей. Бывают случаи, когда не все зависимости известны, и в руководстве такая ситуация называется известными неизвестными. Авторы SBOM могут указать, что данные неполноценны, тем самым предупредив потребителей о том, что использование их ПО сопряжено с рисками ввиду неустранимых или неизвестных проблем.

Практики *распространения и поставки* нацелены на своевременное предоставление SBOM заинтересованным сторонам вместе с необходимыми разрешениями и доступом. Этот аспект внедрения SBOM все еще развивается, и здесь явно могут использоваться различные подходы. Метод распространения и поставки также будет отличаться в зависимости от природы распространяемого ПО и систем. Например, встраиваемые системы и онлайн-сервисы распространяются и развертываются для конечных потребителей по-разному. В указе, нацеленном на повышение уровня кибербезопасности, это требование прописано так: «Предоставление покупателю списка зависимостей компонентов программного обеспечения (SBOM) для каждого продукта непосредственно или путем публикации его на общедоступном сайте».

Управление доступом является важнейшей составляющей процесса внедрения и использования концепции SBOM. Поскольку практика предоставления SBOM продолжает стандартизироваться среди различных производителей ПО, то уровень открытости информации может варьироваться. Некоторые организации захотят сделать информацию публичной, например мейнтейнеры открытого ПО или частные компании вроде JupiterOne (<http://jupiterone.com>), разместившей свой SBOM у себя на сайте <http://jupiterone.com/sbom>. В то время как другие поставщики, особенно те, которые считают свои SBOM чувствительными или имеют особые обязательства в отношении своих клиентов, скажем, в сфере охраны или национальной безопасности, могут предпочесть предоставлять эту информацию только в соответствии с конкретными условиями и при наличии конкретных правил доступа. Этот аспект концепции SBOM и прозрачности ПО еще будет дорабатываться.

Последней в руководстве описывается практика *учета ошибок*. Если коротко, то она отражает возможность допускать ошибки. Внутренняя практика обеспечения безопасности цепочек поставок ПО продолжает развиваться, так что ошибки и оплошности неизбежно возникают. Это двунаправленный процесс, когда производители ПО могут при необходимости предоставлять дополнительную информацию или пояснения, а потребители понимают, что изначально не все бывает идеально. Такая модель взаимодействия позволяет повышать гигиеничность всей цепочки поставок.

Вклад NTIA в обсуждение и внедрение SBOM выходит далеко за рамки описанного руководства и включает развенчание мифов (<https://clck.ru/3CKQPu>), обучающие видео, а также ответы на часто задаваемые вопросы (<https://clck.ru/3CKQQA>). Одной из наиболее цитируемых публикаций NTIA является структурный документ SBOM (<https://clck.ru/3CKQQq>). В нем охватываются следующие аспекты: постановка задач в сфере прозрачности ПО и системных рисков, описание самой концепции SBOM, сопутствующих ей процессов и вариантов применения, а также общий обзор диалога с различными заинтересованными сторонами, который управление вело для создания описанного руководства. Но хоть NTIA и выступило первичным катализатором обсуждений вопросов внедрения SBOM, в том числе прозрачности цепочек поставок ПО, их инициатива продолжала развиваться и получила поддержку CISA (<http://cisa.gov/sbom>). Отчасти это вызвано переходом Аллана Фридмана из NTIA в CISA, учитывая, что он один из наиболее активных сторонников внедрения SBOM. CISA стремится расширить работу по внедрению SBOM за пределы изначальных инициатив NTIA за счет привлечения сообщества и дальнейшей разработки, ставя целью масштабирование и более глубокое внедрение SBOM в индустрии.

На момент написания книги самым ярким событием CISA в области SBOM с привлечением сообщества стало двухдневное мероприятие SBOM-a-Rama, прошедшее в декабре 2021 года. Первый день был посвящен текущему положению концепции SBOM, а второй — вопросам масштабирования и повышения эффективности мероприятий по ее внедрению. Среди участников присутствовали лидеры из государственного и частного секторов, такие как конгрессмен Джеймс Лангевин, давний сторонник SBOM и идеи прозрачности ПО, Джошуа Корман, а также представители отраслевых организаций вроде OWASP, Linux Foundation и других.

На встрече обсуждали, что такое SBOM и зачем эта концепция нужна, разбирали ведущие форматы SBOM, такие как SPDX и CycloneDX, а также ранние инициативы в сфере здравоохранения, энергетики и автомобилестроения, в рамках которых проводятся исследования. Записи и презентации с обоих дней мероприятия доступны по адресу <http://cisa.gov/cisa-sbom-rama>.

Инициативы индустрии: научно-исследовательские центры

В рамках семинаров NTIA по теме SBOM было сформировано несколько исследовательских групп, нацеленных на решение конкретных проблем индустрии. Исследования в сфере здравоохранения прошли не менее трех итераций и начались в области автомобилестроения, а также ряде других. Некоторые из этих проектов открыты, другие доступны только по приглашению. Во многих случаях даже открытые рабочие группы имеют ограничения в отношении того, как поставщики решений могут оказывать поддержку. Проект, изучающий возможность применения SBOM в сфере энергетики, был запущен INL (<http://inl.gov/sbom-poc>). Основоположниками этой инициативы выступили Вирджиния Райт из INL, Том Алрих из NERC CIP (North American Electric Reliability Corporation Critical Infrastructure Protection, Североамериканская корпорация по вопросам надежности энергоснабжения). Сейчас Тома сменил Аллан Фридман, но сама рабочая группа остается открытой к новым знаниям и обсуждению проблем, связанных с критической инфраструктурой.

Материалы, собранные этой рабочей группой в сфере энергетики, являются одними из наиболее детальных среди доступных публично и уступают лишь данным с официального сайта NTIA (www.ntia.gov/SBOM). В доступе есть полноценные записи первых встреч, а также множество других справочных материалов, ссылок и тематической информации, связанной с тематикой SBOM. И хотя основное внимание было уделено вариантам использования

концепции в сфере энергетики, основополагающий образовательный аспект этого проекта служит прекрасной стартовой точкой для всех, кто стремится ознакомиться с данной темой.

Научно-исследовательские центры, действующие под эгидой Министерства энергетики США, выполняют важные исследования и разрабатывают инновационные решения. Эти центры специализируются в том числе на таких направлениях, как климат, зеленая энергетика и кибербезопасность. Например, исследовательский проект под названием Blockchain for Optimized Security and Energy Management (Блокчейн для оптимизации управления безопасностью и энергетикой, <http://netl.doe.gov/BLOSEM>) был нацелен на применение технологии блокчейн для обеспечения безопасности цепочек поставок. Например, для подтверждения соответствия поставляемого коммунальным службам оборудования заявленным требованиям, проверки целостности конфигураций и установленных значений и, конечно же, безопасного распространения перечней компонентов аппаратного и программного обеспечения.

Спонсируется эта инициатива Министерством энергетики США по программе модернизации сетей (Grid Modernization Initiative), представляющей совместный проект Управления по обеспечению стабильных поставок электроэнергии США (Office of Electricity Delivery and Energy Reliability) и Управления энергетической эффективности и возобновляемой энергии США (Office of Energy Efficiency and Renewable Energy). Реализует же инициативу Национальная лаборатория возобновляемой энергии (National Renewable Energy Lab) совместно с Национальной лабораторией энергетических технологий (National Energy Technology Laboratory). Существует и много других партнеров по индустрии, включая Тихоокеанскую северо-западную национальную лабораторию (Pacific Northwest National Laboratory) и другие.

Кроме того, стоит отметить программу Cyber Testing for Resilient Industrial Control Systems (CyTRICS, <http://cytrics.inl.gov>), посредством которой была проделана масштабная работа по определению наилучшего способа представления компонентов системы, включая аппаратную и программную части. Все эти проекты являются закрытыми и доступны только для поставщиков продуктов, но созданная в этих рамках экосистема предоставляет фундаментальный пример использования концепции SBOM как совокупной идеи. Кроме того, CyTRICS создает общую систему для сбора и обмена информацией, связанной с безопасностью продуктов и цепочек поставок в критической инфраструктуре. Пока она не распространяется на коммерческую сферу, но является примером того, как сотрудничество частного и государственного секторов может создавать возможности для инноваций в области обеспечения прозрачности ПО.

Форматы SBOM

В рамках инициатив по стандартизации и широкомасштабному внедрению SBOM также решается вопрос относительно использования нескольких основных форматов. На момент написания книги тремя такими форматами являются SWID, CycloneDX и SPDX. У каждого из них есть свои плюсы и минусы, как и основные сценарии применения. Помимо этого, развитием CycloneDX и SPDX занимаются разные организации. Первый опирается на поддержку OWASP, а второй — на Linux Foundation. Как уже говорилось, за последние годы обсуждение концепции SBOM велось преимущественно со стороны NTIA, и стандарты не стали исключением. NTIA опубликовало работу, посвященную форматам и стандартам (<https://clck.ru/3CKQRH>), в которой обозначило проблемы с прозрачностью ПО и проанализировало доступные на тот момент форматы для SBOM. Вышла данная статья в 2019 году, и с тех пор дискуссия вокруг этих форматов, особенно CycloneDX и SPDX, продолжает развиваться. И хотя поначалу формат SWID тоже упоминался в различных источниках, таких как документация NTIA, он не отвечает требованиям NTIA по части минимального числа элементов SBOM. Говоря точнее, в нем отсутствует временная метка создания SBOM. Более того, одна из организаций, активно внедряющая концепцию SBOM — армия США, — заявила, что не будет принимать списки зависимостей в форматах, отличных от CycloneDX или SPDX (<https://clck.ru/3CKQTJ>).

Агентство NTIA сосредоточилось на необходимости документировать разные рабочие и эффективные машиночитаемые форматы, а также развивать экосистему, в которой эти форматы смогут сосуществовать. Каждый из ведущих форматов SBOM преподносит информацию в различных представлениях и форматах файлов. В табл. 4.2 показан пример каждого из трех упомянутых форматов с минимальными требованиями к числу элементов SBOM.

Таблица 4.2. Три формата SBOM

Атрибут	SPDX	CYCLONEDX	SWID
Имя автора	(2.8) Создатель:	Метаданные/авторы/автор	<Entity> @role (tagCreator), @name
Временная метка	(2.9) Создан:	Метаданные/временная метка	<Meta>
Название поставщика	(3.5) Поставщик пакета:	Издатель-поставщик	<Entity> @role (softwareCreator/publisher), @name

Атрибут	SPDX	CYCLONEDX	SWID
Наименование компонента	(3.1) Наименование пакета:	Наименование	<softwareIdentity> @version
Строка версии	(3.3) Версия пакета:	Версия	<softwareIdentity> @version
Хеш компонента	(3.10) Контрольная сумма пакета: (3.9): Верификационный код пакета:	Хеш alg	Payload>/../<File> @[hash-algorithm]: hash
Уникальный идентификатор	(2.5) Документ SPDX Пространство имен: (3.2) SPDXID:	bom/serialNumber component/ bom-ref	<softwareIdentity> @taqgID
Связь	(7.1) Связь: ОПИСЫВАЕТ: СОДЕРЖИТ:	(Присутствует во вложенных сборках/подсистемах и/или зависимых графах)	<link> @rel, @href

Источник: <https://clck.ru/3CKQUv>. Министерство торговли США, открытая информация

SWID

В основе использования SWID лежит поиск компонентов программного обеспечения по их тегам. Применяется этот формат при инвентаризации ПО и управлении правами доступа. Определяется он в соответствии со стандартом ISO/IEC 19779-2, реализуемым Международной организацией по стандартизации (International Organization for Standardization, ISO) и Международной электротехнической комиссией (International Electrotechnical Commission, IEC). На момент написания книги последней версией стандарта является ISO/IEC 19770-2:2015.

Документация SWID состоит из структурированных наборов элементов данных, используемых для идентификации программных продуктов, включая их версии, организации/частных лиц, участвовавших в создании и распространении, а также компоненты, из которых этот продукт состоит. Теги SWID могут в том числе применяться для установления связи между различными программными продуктами.

SWID зачастую используют инструменты управления программными активами, а также средства безопасности с целью автоматизации административных функций, связанных с продвижением программных продуктов по циклу разработки.

Спецификация SWID охватывает широкий круг участников и инстанций, включая создателей тегов, провайдеров платформ, программного обеспечения и инструментов для работы с тегами, а также потребителей тегов. Каждый из них использует SWID для своих конкретных задач.

Однако, несмотря на то что SWID в первой документации NTIA был признан в качестве формата SBOM, индустрия предпочла сосредоточиться на CycloneDX и SPDX. В связи с этим SWID часто воспринимается скорее как CPE, нежели формат SBOM.

CycloneDX

И если SPDX продвигают такие группы, как Linux Foundation, развитием CycloneDX (<http://cyclonedx.org>) занимается давний лидер сообщества по вопросам безопасности, организация OWASP. CycloneDX — это самоопределяемый «легкий стандарт SBOM, предназначенный для использования в контексте безопасности приложений и анализа компонентов цепочки поставок» (рис. 4.1). Костяк команды разработки этого стандарта составляют Патрик Дуайер (Patrick Dwyer), Джеффри Хессе (Jeffrey Hesse), а также ведущий специалист по цепочкам поставок и создатель Dependency Track Стив Спрингетт, который выступает в роли председателя группы (<http://dependencytrack.org>). Помимо OWASP, CycloneDX также поддерживают специалисты из Lockheed Martin, Contrast Security, Sonatype и других компаний.

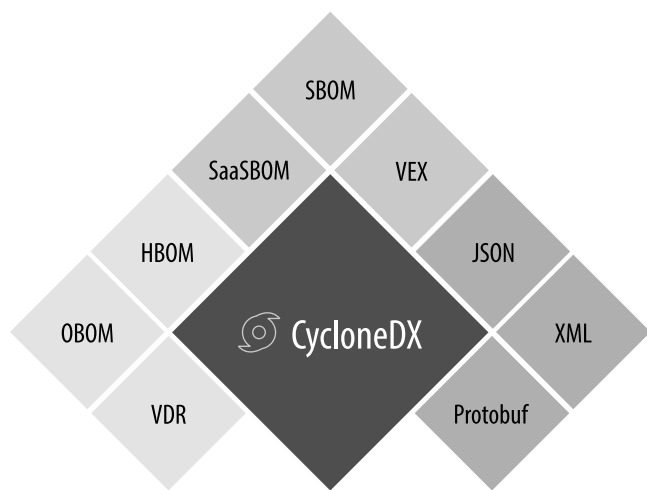


Рис. 4.1

Уникальным CycloneDX делает тот факт, что изначально он создавался как формат BOM. Формат CycloneDX можно применять в самых разных контекстах, включая управление составом программных услуг (SaaS BOM), конфигурациями и зависимостями. Говоря в целом, этот формат может использоваться в широчайшем спектре возможных вариантов вне сферы программного обеспечения.

CycloneDX позволяет ссылаться на компоненты, сервисы и уязвимости в других системах и BOM. Реализуется это с помощью механизма вложений, который согласуется со сложностью современной экосистемы ПО, когда речь идет об аппаратном обеспечении, облачных технологиях и SaaS. В CycloneDX эта возможность именуется как BOM-Link и поддерживается в форматах JSON и XML. Пользователи могут ссылаться на URL внешнего BOM или даже URI BOM-Link, в котором используются серийные номера и версии внешних BOM.

Кроме того, CycloneDX позволяет проводить полную и точную инвентаризацию всех собственных и сторонних компонентов с целью определения рисков. Реализуется это посредством составления подробного списка типов и классов компонентов, который выходит за рамки ПО и приложений, включая в себя устройства и даже сервисы. В итоге он позволяет идентифицировать уязвимости, используя три поля:

- CPE — спецификация, подходящая для уязвимостей операционных систем, приложений и оборудования;
- SWID — спецификация, применяемая для установленного ПО;
- PURL — спецификация, используемая для метаданных программных пакетов.

CycloneDX посредством хеш-значений и криптографии обеспечивает целостность верификации компонентов, связанных с соответствующими BOM. В контексте развития сферы обеспечения безопасности цепочек поставок все более распространенной рекомендацией становится подписывание ПО. Продвижением этой техники занимаются такие проекты, как Sigstore и Cosign. CycloneDX поддерживает возможность «внутреннего подписывания» (enveloped signing) как для XML, так и для JSON, чего на данный момент не может предложить Cosign. Кроме того, CycloneDX поддерживает *провенанс*, то есть возможность представлять авторов и поставщиков компонентов. Опираясь на принцип провенанса, CycloneDX описывает родословную компонента, отражая его предков и потомков, и предоставляет возможные варианты описания его происхождения. Если говорить о высоких

требованиях к устойчивости цепочек поставок, то руководство по управлению рисками кибербезопасности в цепочках поставок (Cybersecurity Supply Chain Risk Management, C-SCRM) рекомендует реализацию провенанса, родословной и цифровых подписей для повышения надежности.

CycloneDX также предоставляет поддержку формата VEX, который содержит информацию об эксплуатируемости известных уязвимостей в программных продуктах и компонентах, предоставленную издателями ПО. Мы подробнее рассмотрим VEX в следующих разделах.

SPDX

Целью проекта SPDX было создание общего формата обмена данными о программных пакетах, который позволил бы хранить эти данные и обмениваться ими. SPDX поддерживает крупнейшую коллекцию форматов файлов, включая RDF, XLSF, SPDX, JSON и YAML. SPDX также выступает в качестве динамической спецификации за счет возможности описать набор программных пакетов, файлов или фрагментов. Во многом SPDX, как и SWID, является одним из трех ведущих форматов, на данный момент достигших статуса сертификации ISO (<https://cck.ru/3CKQZU>). Об этом в Linux Foundation заявили в сентябре 2021 года. В заявлении также подчеркивалось, что SPDX начали внедрять такие крупнейшие корпорации, как Intel, Microsoft, Siemens, Sony и ряд других, также относящихся к сообществу SPDX.

На момент написания книги спецификация SPDX достигла версии 2.3 (<http://spdx.github.io/spdx-spec>). Чтобы документ SPDX считался валидным, он должен содержать конкретные поля, предусмотренные спецификацией этого формата. Такие документы могут состоять из разных полей и данных, таких как информация о создании документа, о пакетах, файлах, сниппетах, лицензии, связях и аннотации. Информация о создании документа применяется для обеспечения прямой и обратной совместимости при использовании инструментов обработки. Сведения о пакетах служат для описания различных субъектов, таких как продукты, контейнеры, компоненты и прочие, и могут использоваться для группировки связанных элементов с общим контекстом. Информация о файлах включает метаданные вроде имен, контрольных сумм и авторских прав. Сниппеты являются необязательным элементом и нужны в основном, когда данные происходят из иного источника или привязаны к другой лицензии. SPDX также поддерживает связи для документов, пакетов и файлов. Наконец, аннотации позволяют рецензенту включать в документ SPDX сведения касательно своего ревью.

Помимо этого, SPDX предлагает профиль SPDX Lite, который представляет собой часть основной спецификации SPDX. Его цель — обеспечить соответствие рабочих потоков отдельных отраслей при сбалансированном соблюдении всех требований стандарта SPDX. Профиль Lite включает поля из разделов «Создание документа» и «Информация о пакетах», а также сопроводительную базовую информацию.

Обмен информацией об уязвимостях (VEX) и их раскрытие

Последствия инцидента с SolarWinds (<https://clck.ru/3CKQaE>) и Log4j наряду с Указом о кибербезопасности вывели на передний план тему обеспечения безопасности цепочек поставок и концепции SBOM. Все это привело к осознанию важности SBOM как критического компонента современных программ по устранению уязвимостей в области кибербезопасности, а также породило готовность к скорейшему внедрению этой концепции (<https://clck.ru/3CKQan>).

Как показано на рис. 4.2, дерево SBOM может изображать утверждения о связях между компонентами, включая указание на неизвестность источника.

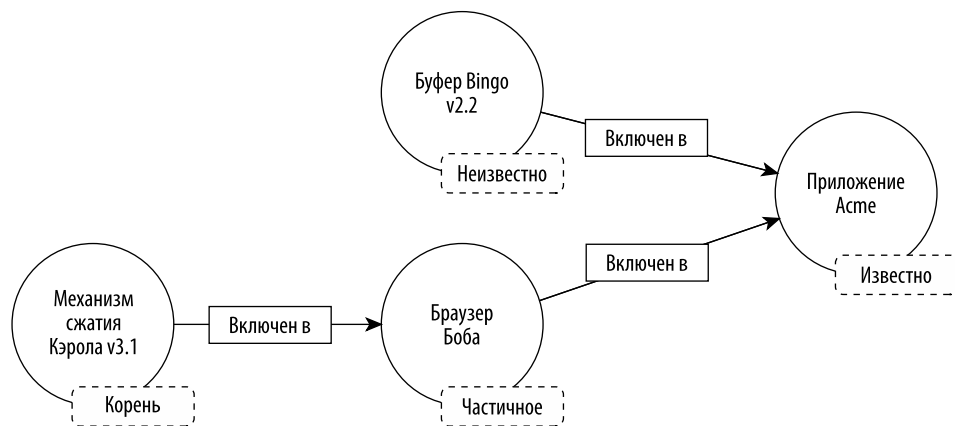


Рис. 4.2

Одно из главных преимуществ использования SBOM — возможность определять потенциально уязвимые элементы. Ведущие платформы и инструменты SBOM вроде Dependency Track помогают пользователям анализировать свои компоненты ПО и обращать внимание на связанные с ними уязвимости. Dependency Track и другие инструменты опрашивают такие ресурсы,

как NVD (<http://nvd.nist.gov>), Sonatype OSS Index (<http://ossindex.sonatype.org>), VulnDB (<http://vuln.db.cyberriskanalytics.com>) или OSV (<http://osv.dev>). Однако связь уязвимости с компонентом ПО еще не означает, что компонент может быть использован злоумышленниками, и здесь на помощь приходит VEX.

На сцену выходит VEX

Как сказано в руководстве NTIA (<https://clck.ru/3CKQbw>), основным сценарием использования VEX является

предоставление пользователям (например, операторам, разработчикам и поставщикам услуг) дополнительной информации о том, подвержен ли продукт уязвимостям его компонентов. И если да, какие есть рекомендации по устранению этой угрозы.

VEX, как и другие ведущие SBOM и руководства по обеспечению прозрачности цепочек поставок ПО, был разработан в рамках инициативы NTIA, которая включала обсуждения с различными заинтересованными сторонами. Здесь важно отметить: хотя в руководстве указано, что VEX создавался специально для использования в SBOM, его применение не ограничивается этой сферой, как и не является обязательным для нее.

Как уже говорилось, одно только присутствие уязвимости еще не означает, что она может быть использована. И эту информацию очень важно знать, поскольку на ее основе организации выстраивают стратегии по устранению уязвимостей и управлению рисками. В случае управления рисками кибербезопасности компании стремятся найти, проанализировать, оценить и устранить угрозы на основе установленного уровня толерантности. Все это обуславливает приоритизацию рисков на основе вероятности и серьезности их последствий. Без понимания того, является ли уязвимость эксплуатируемой, будет невозможно точно предсказать эти последствия.

VEX: добавление контекста и ясности

VEX позволяет поставщикам ПО рекомендовать статус уязвимости для конкретного продукта. VEX предлагает четыре возможных статуса:

- не подвержен — исправления данной уязвимости не требуется;
- подвержен — рекомендуются действия для устранения или ослабления уязвимости;

- исправлена — в этой версии продукта уязвимость устранена;
- исследуется — пока неизвестно, подвержена ли данная версия продукта обозначенной уязвимости, но в следующем обновлении будет предоставлена дополнительная информация.

На примере SBOM можно увидеть, как индустрия движется в сторону машиночитаемых артефактов и документации, что повышает автоматизацию, точность и скорость обработки информации. Похожие тенденции наблюдаются и в сфере соответствия требованиям. Здесь стоит упомянуть созданный NIST язык OSCAL (<http://pages.nist.gov/OSCAL>), который преобразует традиционные бумажные документы по управлению безопасностью и авторизацией в машиночитаемый формат.

VEX проделывает нечто аналогичное, избавляя от необходимости рассылать руководства и рекомендации по электронной почте. Вместо этого он позволяет переносить всю эту информацию в машиночитаемый формат, что способствует автоматизации и использованию современных инструментов безопасности, которые больше соответствуют актуальным угрозам. Поскольку прозрачность цепочек поставок и безопасность становятся все более важными, несложно представить мир, где инвентаризацию корпоративного ПО можно отобразить в виде таблиц вместе с релевантными уязвимостями и данными об их реальной опасности. Это создаст резкий контраст с современной экосистемой, в которой многие организации не имеют точной информации о потребляемых или развернутых ими компонентах, а также связанных с ними уязвимостях. И все это вопреки тому факту, что нынешнее ПО преимущественно строится из открытого исходного кода, объем которого, по некоторым оценкам, достигает 80–90 %.

В руководстве также утверждается, что, хотя VEX и может создаваться поставщиком ПО, эти данные также могут предоставляться сторонними лицами, вынуждая пользователей самостоятельно определять, как их использовать. В итоге возникнут ситуации, когда исследователи безопасности и поставщики инструментов управления уязвимостями начнут создавать VEX для продуктов в рамках собственных предложений.

VEX и VDR

Вдобавок к концепции VEX тема отчетов о раскрытии уязвимостей (Vulnerability Disclosure Reports, VDR) внесла некоторую путаницу, хотя фактически она очень проста. VDR уведомляет вас о наличии уязвимости и может содержать как известные, так и неизвестные уязвимости. Кроме того, эти

отчеты предоставляют дополнительные метаданные об уязвимости, лице, обнаружившем ее, информацию о публичных эксплойтах и некоторую другую. Стандарт ISO 29147, а также документы NIST 800-161r1 выделяют следующие элементы VDR.

1. **Общая информация об отчете** помогает целевой аудитории понять, касается ли он их продуктов.
2. **Идентификаторы.** Это могут быть уникальные идентификаторы для самого отчета, такие как используемая поставщиком схема оригинальной нумерации, идентификаторы уязвимостей, такие как CVE, или идентификаторы продуктов вроде CPE или PURL. В плане корреляции с SBOM данный атрибут может быть одним из наиболее подходящих для связывания результатов различных подтверждений.
3. **Дата и время.** Дата и время создания первичной сводки, а также возможные ее обновления. При сравнении временных меток VDR с другими подтверждениями это позволяет определить, был ли VDR выпущен раньше или позже и содержит ли он более актуальную информацию в сравнении с другими обрабатываемыми подтверждениями уязвимости продукта.
4. **Доверенная подпись.** Несмотря на то что в стандарт ISO этот момент не включен, потребность в подписанных подтверждениях была отмечена NIST. Данная техника может быть наилучшим вариантом связывания нескольких подтверждений уязвимости с любым уровнем доверия.
5. **Заголовок.** Должен быть краток и достаточно емок для отчетливого понимания содержимого сводки, включая имена продуктов и версии или категории CWE.
6. **Обзор.** Данный элемент должен раскрывать заголовок, предоставляя достаточно деталей для того, чтобы читатель понял, применима ли сводка к используемым им продуктам.
7. **Затрагиваемые продукты.** Продукты, на которые распространяется действие VDR, должны быть четко обозначены. Информация должна быть достаточной для определения читателем того, касается ли отчет его продукта. Сюда могут входить ссылки на онлайн-скрипты для тестирования или конкретные метаданные, которые допускается использовать, например хеш файла, строка символов или уникальный идентификатор версии.
8. **Затрагиваемые компоненты.** Хотя этот пункт не входит в стандарт ISO, он прописан в документации NIST. В идеале именно здесь следует отражать информацию о деталях на уровне компонентов, которую можно

будет совместить с SBOM и использовать по аналогии с пунктом «За-трагиваемые продукты» — например, для описания того, как компонент должен быть реализован, чтобы оказаться уязвимым. Наличие уязвимого компонента само по себе может быть недостаточно для создания уязвимых условий.

9. **Целевая аудитория.** Для кого предназначен отчет? Для разработчиков? Конечных пользователей? И хотя в VDR это поле необязательно, его можно использовать для передачи дополнительного контекста.
10. **Локализация.** Отчеты могут создаваться для конкретных регионов и языков. Текущий раздел позволяет указать любую подобную информацию или контекст, который нужно передать, даже если это будет просто перевод.
11. **Описание.** Описание должно сообщать дополнительную информацию, такую как класс уязвимости ПО, и при каких условиях угроза может быть реализована. Здесь также часто указываются ссылки на внешние источники информации вроде классификаторов CWE.
12. **Влияние.** Этот пункт обычно связан с техническим воздействием, таким как утрата конфиденциальности, целостности или доступности, подобно тому, которое отражается в CVE или CVSS.
13. **Серьезность.** Здесь с помощью стандартных шкал оценки угроз, таких как CVSS или их альтернатив, указывается количественный показатель серьезности уязвимости. Независимо от вашего отношения к системе оценки CVSS именно так в индустрии в целом измеряется серьезность, и именно этот показатель, скорее всего, будет использован.
14. **Исправление.** VDR должен указывать, как уязвимость можно устранить, пропатчить или перенастроить. Как минимум здесь прописывается обходной путь, который позволит снизить риск. Во многих случаях изначальный VDR может не включать рекомендации по устранению, но вскоре за ним должно последовать обновление, где эта информация уже будет отражена.
15. **Ссылки.** Здесь может быть указано множество внешних ссылок: на CVE, базу знаний поставщика, страницу продукта, исследовательский блок или другую сопроводительную информацию, полезную для упоминания в рамках VDR.
16. **Благодарности.** Благодарности должны быть выражены исследователям и организациям, участвовавшим в обнаружении, изучении, исправлении и прочих фазах цикла составления отчета об уязвимости. Этот пункт добровольный.

17. **Контактная информация.** Нужна, чтобы пользователи продукта могли связаться с поставщиком для получения дополнительных сведений.
18. **История ревизий.** Элемент, где указывается, опубликован данный отчет впервые или нет. Если это обновление, то должна сохраняться история, соответствующая указанной в VDR информации о дате и времени.
19. **Условия использования.** Наконец, отчет представляет собой лицензированный документ и должен содержать информацию об авторских правах, а также условия использования. Например, во многих случаях он предназначен только для владельцев лицензионного продукта, или же их вторичная передача полностью запрещается.

Тем не менее вы заметите, что, хотя VDR и информирует пользователя о проблеме, он не предоставляет отчета о том, что проблема *отсутствует*. И это важное отличие, поскольку по аналогии с VDR аналитический отчет для SBOM может создавать значительное число ложноположительных. Интеграция в существующий формат VDR указаний VEX, а именно так построена модель составления отчетов для CycloneDX — это лаконичный и согласованный способ определения существования новых уязвимостей и уведомления о том, представляют ли они угрозу для пользователей.

Уязвимость не всегда является проблемой, и на это есть множество причин. Согласно отчету компании Contrast Security (<https://clck.ru/3CKQg9s>), 80 % открытых компонентов фактически никогда не вызываются в процессе исполнения кода. Экстраполируя эти выводы, допустимо предположить, что 80 % уязвимостей, выявленных в подобных продуктах, могут не представлять риска, поскольку не будут использованы. Это определенно указывает на проблемы с раздуванием кодовой базы и ее эффективностью, а также на увеличение поверхности атаки, но все же не может свидетельствовать о наличии действительно уязвимого ПО. Кроме того многие поставщики вносят корректировки в код для предотвращения эксплуатации уязвимости, не изменяя при этом версию исправленного компонента. Это означает, что любой анализ с помощью NVD для поиска конкретных уязвимых версий ПО может привести к другому компоненту. На данный момент в NVD не существует принципа уникальной идентификации программного файла, только схема CPE, описывающая поставщика, продукт, его версию, архитектуру и соответствующие метаданные.

Наконец, конфигурация имеет большое значение. Зачастую программное обеспечение становится уязвимым только при определенных настройках или использовании конкретной библиотеки, которая не поставляется с этим

ПО изначально. В этом случае VEX может предоставить пользователю дополнительный контекст, поясняющий, когда эти ситуации возникают, а когда фактического риска эксплуатации нет.

И хотя VEX предполагается как машиночитаемый документ, мы рассматриваем его как более динамичный способ аттестации, чем SBOM. Несмотря на то что SBOM является преимущественно статичным документом, привязанным к конкретному релизу ПО, статус уязвимостей в нем со временем может меняться из-за обнаружения новых уязвимостей, выявления их нежизнеспособности, исправления и так далее. Мы считаем, что он больше подойдет для модели распространения на основе API, когда обновление статуса уязвимостей не только является машиночитаемым, но и автоматически непрерывно выполняется в свете раскрытия новой информации о той или иной угрозе.

Дальнейшие шаги

Как мы уже говорили, инициатива по внедрению SBOM перешла от NTIA к CISA, что совпало с переходом в него Аллана Фридмана, ярого сторонника SBOM. После этого CISA опубликовало в 2022 году два дополнительных документа VEX: *VEX Use Case Document* и *Vulnerability Exploitability eXchange (VEX) Status Justifications* (<https://clck.ru/3CKQgr>).

В первом указывается минимальный набор элементов данных документа VEX по аналогии с тем, как NTIA определило необходимый минимум элементов для SBOM (см. табл. 4.1). В этом руководстве сказано, что документ VEX должен включать метаданные VEX, подробности о продукте и его статус, а также сведения об уязвимости. Статус включает в себя информацию о присутствующей в продукте уязвимости и может быть представлен как «не подвержен», «подвержен», «исправлен» или «изучается».

Второй документ требует предоставлять пояснения к отчетам VEX от их создателей. Касается это только тех случаев, когда был присвоен статус «не подвержен». В качестве объяснений может быть указано, что уязвимый компонент или код отсутствуют, уязвимый код недоступен для злоумышленника, не находится на пути выполнения или в программе уже реализованы меры противодействия уязвимости. VEX способствует повышению эффективности SBOM за счет предоставления поставщиками деталей контекста и утверждений относительно эксплуатируемости присутствующих в их продуктах уязвимостей. Указывая в документах VEX минимум элементов

и давая при необходимости пояснения в соответствующем поле, разработчики ПО помогут потребителям принимать взвешенные решения при планировании мероприятий по управлению уязвимостями в рамках более обширных программ по обеспечению кибербезопасности.

Использование SBOM с другими методами аттестации

SBOM — это просто вид подтверждения. Это документ, содержащий подтверждение высказанных в нем утверждений. Его достоверность подкрепляется крипто-валидацией, которая может быть применена к этим утверждениям. По этой причине становится возможным связывание разных SBOM с помощью цифровой подписи. И именно благодаря этому неполный SBOM все равно представляет ценность, так как его можно объединить с другими, привносящими дополнительную подтверждающую информацию. Тем не менее нам нужно изучить и другие виды аттестаций, которые тоже иногда включают в SBOM для создания пакета подтверждающих свидетельств.

Достоверность источника

Достоверность источника — это принцип, основанный на требованиях соответствия, установленных стандартами NERC CIP 010 и 013. Суть в том, что вы можете проверить сам файл, используя его хеш или материал подписи, однако немаловажно проверять и подлинность провенанса этого файла. То есть данный механизм позволяет ответить на вопросы вроде «Откуда файл взялся?», «Можно ли доверять предоставившему его субъекту?», «Является ли этот субъект именно тем, за кого мы его принимаем?», «Не был ли этот субъект скомпрометирован или изменен каким-либо неожиданным образом?»

Некоторые поставщики стандартных решений в качестве выхода предложили проверять информацию сертификата TLS (Transport Layer Security), связанного с веб-доменом, откуда был получен файл, для подтверждения его валидности и достоверности. Установлен ли для него адекватный срок действия? Не изменялся ли он в последнее время неожиданным образом? Совпадает ли указанное имя компании с ожидаемым? Возможно, вчера это была Microsoft Corp., а сегодня уже Microsoft, Inc. Этот момент заслуживает

изучения. Самое же главное, что помогает организациям определять достоверность источника или фиксировать попытки его компрометации, — это формирование базового представления нормального состояния и выявление отклонений от него.

Помимо прочего, верификации требует и DNS-информация. Не менялся ли домен на новый набор IP-адресов и не был ли перемещен в другую часть мира? Этот нюанс становится трудно отследить с высокой степенью уверенности в случае CDN-сетей (Content Delivery Networks), которые способны распределять трафик из различных точек планеты на основе качества интернет-соединения или местоположения запрашивающего. Некоторые специалисты даже предположили, что для передачи зараженных файлов могут использоваться такие методы перехвата интернет-трафика, как BGP-перехваты, участвовавшие в последние годы. Эти сценарии также становится сложнее проверять, однако для них полезно проводить моделирование угроз.

Реализация проверки по этим критериям и создание файла аттестации, который можно криптографически привязать к SBOM, позволяют дополнительно обогатить уже содержащиеся в нем данные провенанса.

Аттестации сборки

Аттестация среды сборки также весьма полезна для повышения уверенности в программном артефакте. Она включает информацию о системе сборки и даже валидацию процесса или средств управления.

Обратите внимание, в рамках SLSA одной из целей атак выступает система сборки. Если существует угроза ее компрометации, как вы можете доверять программам, собранным на этом сервере? В идеале он должен быть отделен от рабочей среды и усилен как крайне чувствительный ресурс. Необходимо проводить проверку соответствия сервера таким руководствам, как Security Technical Implementation Guides (STIG), которые представляют собой шаблоны безопасности, снижающие вероятность того, что сервер сборки окажется источником вторжения. В индустрии существуют компании наподобие TestifySec, предоставляющие услуги по управлению цепочками поставок и которые используют эти средства аттестации в рамках своих автоматизированных процессов и применяют политики сборки, требующие прохождения проверки STIG для успешного завершения.

У вас также могут быть реализованы собственные политики и процессы, такие как необходимость согласия нескольких разработчиков на принятие кода

перед началом сборки. Подобные аттестации сборки позволяют включать неограниченное количество требований безопасности, которые вышестоящие процессы должны использовать для ее защиты. То же касается аттестаций, применяемых для подтверждения ее надежности. Одним из подходящих для этого инструментов является открытый проект Witness. Он помогает создавать аттестации и внедрять политики для обеспечения целостности цепочек поставок ПО (<https://clck.ru/3СКQoV>).

Управление зависимостями и их верификация

Поскольку разработчики все чаще используют программные компоненты из внешних источников, возрастает и потребность в управлении ими. Мы уже говорили, что зависимости зачастую интегрируются для экономии различного рода ресурсов организаций, а также для ускорения вывода продукта на рынок/реализации миссии и внедрения инноваций. При этом реальность такова, что чем больше зависимостей мы используем, тем большим их числом нужно управлять.

Обычно зависимости рассматриваются в двух контекстах и являются либо прямыми, либо транзитивными. Прямые зависимости, как и предполагает их название, представляют компоненты, на которые ваше приложение непосредственно ссылается и реализует. Транзитивные же зависимости — это компоненты, которые используют прямые зависимости, создавая многоуровневую цепочку. Оба типа зависимостей имеют свои риски и важные моменты. И хотя они обеспечивают множество преимуществ, с увеличением числа зависимостей, будь то прямых или транзитивных, растут и потенциальные риски. Конечно, здесь подразумеваются и вопросы лицензирования, но с точки зрения безопасности речь идет о возможных уязвимостях, вредоносном коде и векторах атаки.

Это отмечается в таких отчетах, как *The State of Dependency Management* от Endor Labs. По данным исследования этой компании, 95 % уязвимостей обнаруживается именно в транзитивных зависимостях, что повышает сложность их анализа и разрешения. Попутно с этим открытием было обнаружено, что 50 % наиболее популярных пакетов в 2022 году не получали обновлений, а 30 % обновлялись последний раз до 2018 года. Все это вынуждает организации работать со старым и потенциально уязвимым кодом (<https://clck.ru/3СКQpA>). Здесь также присутствуют сложности с административными издержками, связанными с использованием множества зависимостей. Сообщество разработчиков даже придумало специальный термин «ад зависимостей», отражающий недовольство, связанное с установкой программных

пакетов, имеющих зависимости от других конкретных версий пакетов. Эта проблема возникает, когда несколько пакетов имеют зависимости от одних и тех же общих компонентов, но обращаются к разным версиям или сталкиваются с проблемами совместимости.

Каждая зависимость или компонент сопровождается своими нюансами. Это и устаревшая и необслуживаемая библиотека, и плохо написанный код, и отсутствие документации, и, естественно, уязвимый код, который может или не может быть исправлен мейнтейнерами, если они все еще существуют и активно участвуют в проекте.

Тем не менее есть несколько практик, которые следует использовать для преодоления этих проблем или снижения рисков, связанных с управлением зависимостями. К ним относятся приоритизация обновления зависимостей и критичный подход к проекту или рискам безопасности. Другая легко организуемая на словах рекомендация заключается в минимизации неиспользуемых зависимостей путем их полного удаления с сохранением только необходимых. Если этого не сделать, мы получаем раздутый код, увеличенную поверхность атаки и проблемы с административными издержками. Во многих случаях также можно автоматизировать обновление программных компонентов, что позволит снизить объем ручных усилий, связанных с управлением зависимостями. Одним из наиболее популярных примеров подобной автоматизации является Dependabot, который помогает обновлять зависимости для разных языков программирования, таких как Ruby, JavaScript, Python и ряда других. Dependabot поддерживается крупнейшими платформами непрерывной интеграции вроде GitHub и GitLab.

Помимо автоматизации, организациям следует внедрять политику управления зависимостями, чтобы обеспечить соблюдение стандартизированного подхода и реализовать систему контроля, о чем говорится в руководстве S2C2F корпорации Microsoft или руководстве по управлению цепочками поставок NIST, которое мы разберем в главе 6. Наличие стандартизированной политики и процессов помогает отслеживать проблемы безопасности, повышать производительность приложения и обеспечивать соблюдение лицензионных требований. Программные зависимости уже никуда не денутся. В индустрии мы наблюдаем все более обширное повторное использование пакетов, библиотек и компонентов. Все это неизбежно создает потребность в управлении зависимостями и реализации процессов верификации. Если пренебречь реализацией структурированного подхода, риски безопасности останутся нерешенными, что создаст проблемы командам разработки, замедлит доставку продукта и оставит пробелы, способствующие нарушению лицензионных требований.

Sigstore

На фундаментальном уровне обеспечение безопасности цепочек поставок ПО связано с потребностью в соблюдении целостности, прозрачности и достоверности. И тут к нам на помощь приходит Sigstore. Как уже упоминалось ранее, после ряда ярких инцидентов вроде SolarWinds и Log4j мы наблюдаем повышенное внимание к вопросу безопасности цепочек поставки. Президент США издал соответствующий указ, NIST обновил свое руководство, изложенное в документе SP 800-161, посвященном управлению рисками кибербезопасности цепочек поставок. Обновило свое руководство и CISA. Кроме того, мы заметили интерес со стороны Белого дома, администрация которого в начале 2022 года организовала саммит, посвященный безопасности цепочек поставок. После чего Linux Foundation совместно с OpenSSF опубликовали План по повышению безопасности ПО с открытым исходным кодом, нацеленный на использование цифровых подписей для повышения уровня доверия к компонентам в цепочках поставок. Эта инициатива делает основной упор на внедрении и развитии технологии, называемой Sigstore. Sigstore — это стандарт для подписи, верификации и защиты ПО. Как мы уже говорили, программное обеспечение применяется повсеместно в каждой промышленной отрасли и сфере бизнеса. Но несмотря на то, что способы использования ПО меняли и продолжают менять мир, у нас по-прежнему недостаточно методов для обеспечения необходимого уровня целостности потребляемого ПО. В цепочках поставок зачастую недостает автоматизации, масштабирования и проверок цифровых подписей, так как обычно преобладают традиционные техники.

Sigstore был запущен в марте 2021 года в результате сотрудничества Red Hat, Google и Linux Foundation с целью закрыть вопросы относительно происхождения ПО. Задача Sigstore — повысить целостность цепочек поставок ПО и эффективность мероприятий по верификации.

По словам Дэна Лоренца, одного из создателей этого проекта, Sigstore — это бесплатный сервис подписи для разработчиков, который повышает безопасность цепочек поставок программного обеспечения и облегчает внедрение криптографических подписей благодаря технологии журналов прозрачности. Основатель проекта Люк Хиндс из Red Hat написал в своем блоге *Introducing Sigstore* (<https://clck.ru/3CKQpt>), что Sigstore задумывался на основе реального запроса индустрии, связанного с нехваткой сервисов для подписи кода и сложностями в управлении ключами. Его основатели постарались ответить на вопросы вроде «А что, если ключи не нужно будет аннулировать или хранить?» и «Что, если все события подписания будут публично доступны на безопасном ресурсе?»

Внедрение

С момента своего запуска Sigstore продолжает вызывать интерес и все чаще принимается на вооружение в индустрии. Несмотря на то что он стартовал в начале 2021 года, он уже привлек внимание некоторых крупнейших открытых проектов, одним из которых является Kubernetes. Kubernetes — это крупнейшая и наиболее широко используемая платформа оркестровки контейнеров на рынке. Ее разработчики заявили, что начнут стандартизировать платформу с помощью Sigstore, начиная с версии 1.24 (<https://clck.ru/3CKQqC>). В результате ее клиенты получили возможность убедиться, что используемый ими дистрибутив является доверенным. Более того, реализуемый OpenSSF план по повышению безопасности ПО с открытым исходным кодом включает раздел, посвященный цифровым подписям, который фокусируется на внедрении и использовании Sigstore для устранения недочетов по части целостности цепочек поставок. В начале 2022 года компания Sonatype объявила, что Maven Central будет внедрять Sigstore для решения вопросов происхождения компонентов в цепочках поставок (<https://clck.ru/3CKQqm>).

Компоненты Sigstore

Итак, что же представляет собой Sigstore? Как он работает? И почему это важно? Sigstore был создан для восполнения некоторых из существующих пробелов в цепочках поставок OSS: в обеспечении целостности, цифровых подписях и верификации достоверности компонентов OSS. Это крайне важно, потому что, по примерным оценкам, около 90 % ведущих IT-компаний используют OSS (<https://clck.ru/3CKQrE>). Организации делают ставку на найм талантливых разработчиков открытого ПО, и индустрия пережила несколько серьезных инцидентов, связанных с цепочками поставок. Sigstore совмещает несколько открытых инструментов, таких как Fulcio, Cosign и Rekor, помогающих в реализации цифровых подписей, верификации и проверки провенанса кода. Провенанс кода — это возможность отслеживать цепочку ответственности, показывающую происхождение кода. Специалисты по защите личных данных и обеспечению безопасности компании Uber опубликовали прекрасную статью, в которой описали свой подход к отслеживанию провенанса кода (<https://clck.ru/3CKQsC>).

Разбор ключевых компонентов Sigstore мы начнем с Fulcio (<https://github.com/sigstore/fulcio>). Fulcio — это бесплатный корневой центр сертификации (Certificate Authority, CA), отвечающий за подписание кода и выдающий

сертификаты с привязкой к OpenID Connect (OIDC). В нем зачастую используются существующие идентификаторы, с которыми разработчики уже связаны. В свете стремительного развития сферы облачных архитектур и развертывания контейнеров их подписание стало лучшей практикой по защите ключей (<https://clck.ru/3CKQv6>). Управление ключами — простое занятие, которое часто предлагают в виде облачного сервиса сами провайдеры таких услуг или сторонние компании. Sigstore позволяет снизить эту нагрузку благодаря поддержке Cosign, а также решает некоторые проблемы управления ключами, используя «подписи без ключей» или временные ключи (<https://clck.ru/3CKQvt>). Несмотря на применение временных ключей, вы все равно можете получать подтверждение валидности подписей через сервис временных меток, предлагаемый Fulcio (<https://clck.ru/3CKQwJ>).

Здесь и подключается Cosign, так как он поддерживает различные опции и способен легко обеспечивать генерацию пар ключей, а также подписание контейнеров для хранения в реестре контейнеров. Этот инструмент позволяет облачным средам проверять контейнер с помощью открытого ключа и гарантировать, что он подписан доверенным источником. Цифровое подписание образов во время сборки и проверка этих подписей являются лучшей практикой, что отмечается в работе Cloud Native Security фонда CNCF (<https://clck.ru/3CKQwq>).

Следующим идет Rekor (<https://clck.ru/3CKQyA>), представляющий неизменяемый и устойчивый к вмешательству журнал, создаваемый в рамках мероприятий по обслуживанию ПО и его сборке. Этот журнал дает возможность потребителям ПО проверять метаданные и принимать взвешенные относительно рисков решения в ходе цикла разработки ПО. Возвращаясь к предыдущему пункту касательно провенанса ПО, разработчики могут использовать Rekor для дополнения информации о провенансе программного компонента посредством журналов прозрачности.

Еще один важный момент связан с появлением руководства SLSA (<https://clck.ru/3CKQyg>) и SSDF организации NIST (<https://clck.ru/3CKQz7>). SLSA третьего уровня подчеркивает необходимость проверки источника и целостности провенанса ПО. Эту проблему, как уже говорилось, решает Sigstore. Конкретные практики, описанные в SSDF, также указывают на необходимость предоставления механизмов отслеживания провенанса и верификации. И это важно, поскольку Правительство США стремится обязать разработчиков и поставщиков ПО для государственных учреждений проходить аттестацию в соответствии с практиками, указанными в SSDF (<https://clck.ru/3CKR33>). Внедрение Sigstore поможет вашей организации соответствовать новым

стандартам и передовым практикам, а также снизить критические риски для цепочек поставок, которые могут привести к инцидентам безопасности и негативным последствиям.

Подписание коммитов

Говоря о современных методах управления исходным кодом, обычно подразумевается система Git. Git позволяет разным субъектам, таким как авторы, которые выполняют работу, и разработчики, делающие коммиты предложенных изменений, участвовать в процессе разработки. Зачастую в роли обоих этих субъектов выступает один и тот же человек, но не всегда. Тем не менее во многих случаях легко выдать себя за другого разработчика, указав чужое имя пользователя, электронную почту и так далее. По этой причине в чувствительных средах разработки рекомендуется подписывать коммиты Git. Это гарантирует, что вы являетесь автором конкретного фрагмента кода и его метаданных, таких как временные метки. Подписанные коммиты не являются панацеей, но могут снизить некоторые потенциальные риски, например, когда бывшие сотрудники хотят внедрить вредоносный код, выдав себя за действующего сотрудника, или создают вредоносные пул-реквесты под чужой личностью с хорошей репутацией.

Подписывание коммитов не остановит злоумышленников, поскольку они все равно могут прикидываться другими личностями, но без цифровой подписи это сразу же привлечет внимание.

Чтобы подписать коммиты в Git зачастую используются методы вроде GNU Privacy Guard (GPG). В контексте Git этот инструмент позволяет вам загружать открытый ключ на популярные платформы вроде GitHub и GitLab, после чего использовать для подписания коммитов закрытый ключ. На GitHub есть документация, объясняющая, как управлять процессом подписания и подписями коммитов (<https://clck.ru/3CKR4S>).

Критика и недочеты SBOM

Было бы упущением не уделить внимания и наиболее актуальной критике в отношении внедрения SBOM и связанным с этим проблемам. Наиболее полно они отражены в разделе Common Misconceptions & Concerns вопросов по SBOM на сайте NTIA (<https://clck.ru/3CKR5V>).

Далее мы рассмотрим основные тезисы, изложенные в документе NTIA и некоторых других источниках.

Видимость для атакующего

Один из вопросов, связанных со SBOM, заключается в том, что списки зависимостей могут служить «дорожной картой для атакующего», так как к ним часто обращаются. Это можно рассматривать в контексте более широкой дискуссии о «безопасности за счет маскировки», которая какое-то время велась в сфере кибербезопасности и отражала аргументы обеих сторон. Концепция обеспечения безопасности через маскировку обычно сводится к тому, что основным методом защиты системы или компонента выступает секретность, а не конкретные меры противодействия и грамотное проектирование решений. Наш личный опыт перекликается с высказыванием Даниэля Мисслера, давнего лидера индустрии, который разъяснил отличие хорошей маскировки от плохой. Он сказал, что «разница между хорошей и плохой маскировкой определяется тем, используется она поверх хорошей безопасности или вместо нее. Первый вариант хорош, а второй — плох» (<https://clck.ru/3CKRu6>).

Мисслер говорит, что в некоторых сценариях секретность определенно может быть полезна, но она не способна заменить надежную систему безопасности. Тем не менее другие участники индустрии решили, что SBOM можно использовать, например, для атак на API, о чем писал Дэн Эпп в своей статье *Can SBOM Help You Attack APIs?* (<https://clck.ru/3CKRuV>). Но даже в этой работе указывается, что SBOM необходим защитникам, чтобы понять, *уязвимы* ли они и *где* именно. Это перевешивает беспокойство в отношении нападающих, так как они в любом случае будут искать эти слабые места, даже при отсутствии SBOM.

Стараясь ответить на вопрос, является ли SBOM дорожной картой для атакующего, NTIA отметило, что злоумышленникам SBOM не нужен, учитывая, что предвидение не является необходимым условием для атак. NTIA также указывает, что хакеры уже имеют инструментарий для идентификации программных компонентов и не стесняются его использовать, в отличие от законопослушных пользователей ПО. Агентство утверждает, что SBOM «уравняют шансы», предоставив защитникам уровень понимания и прозрачности, которыми уже обладают злоумышленники. Хакеры могут применять инструменты анализа двоичного кода для понимания композиции ПО и структуры компонентов. С этим не поспоришь, однако наличие SBOM позволяет потребителям ПО лучше понимать как изначальный риск, связанный с использованием программных компонентов, так и новые уязвимости. Так что SBOM помогает сбалансировать «условия игры», когда злоумышленники видят то, чего не видят защитники.

Интеллектуальная собственность

Еще одно распространенное беспокойство в отношении SBOM касается интеллектуальной собственности и исходного кода. NTIA на этот счет возражает, утверждая, что SBOM не требуют от организаций раскрытия исходного кода и что он может быть предоставлен на усмотрение поставщика ПО. SBOM акцентируются на компонентах, применяемых для создания программного обеспечения, а не на раскрытии самого исходного кода. Программные компоненты представляют лишь часть картины того, как работает код. В руководстве NTIA сказано, что в случае ПО есть большая разница между знанием сторонних компонентов и всей структуры исполнения. Тем более что сторонние компоненты не являются интеллектуальной собственностью использующих их поставщиков ПО, а принадлежат создавшим их вышестоящим субъектам, которые ограничены определенными лицензиями. Стоит отметить, что вопреки ускорению темпов внедрения SBOM все соответствующие источники советуют должным образом защищать сами SBOM, прибегая к устоявшейся передовой практике предоставления доступа с минимальными разрешениями и минимальной осведомленностью. Такие документы, как памятка OMB 22-18, которые призывают федеральные ведомства требовать от поставщиков ПО предоставления SBOM, также рекомендуют им надежно хранить эти артефакты, если они не раскрываются публично поставщиком.

Инструменты и операционализация

Одно из главных опасений, связанных с повсеместным внедрением SBOM, заключается в отсутствии широкодоступных инструментов для создания, распространения, потребления, обогащения, анализа и функционирования SBOM. Сегодня многие согласятся, что существует достаточно инструментов, помогающих создавать SBOM, и в этой книге мы коснулись некоторых из них. Тем не менее на момент написания инструменты для реализации дополнительных действий, таких как распространение, потребление, обогащение и анализ, по-прежнему развиваются. Как уже говорилось, в этой сфере развиваются различные инновационные инициативы, за которыми стоят такие организации США, как Министерство внутренней безопасности и Агентство по кибербезопасности и защите инфраструктуры. В том числе это происходит и в коммерческой сфере, где мы наблюдали немало компаний, вкладывающих средства в исследования и разработку (Research & Development, R&D), а также создающих продукты для заполнения этого сегмента. К ярким примерам

относится основанная бывшими инженерами Google фирма Chainguard, которая за 2022 год получила финансирование в размере 50 миллионов долларов. В качестве других заметных компаний можно выделить Endor Labs, TestifySec и ряд других, включая лидеров индустрии вроде Google, которые предлагают услуги по управлению цепочками поставок и помогают организациям регулировать потребление и использование OSS. Здесь можно снова процитировать памятку OMB 22-18, призывающую правительство «установить централизованный репозиторий для программных аттестаций и артефактов», включающий SBOM.

Хотя определенные опасения вроде видимости для атакующих, интеллектуальной собственности и незрелости инструментов и возможностей остаются актуальными, подавляющая часть индустрии считает, что стремление к прозрачности и обеспечиваемым ей выгодам перевешивает эти проблемы. Это не значит, что указанные вопросы совсем не заслуживают внимания и не должны решаться. Напротив, речь идет о том, что движение в сторону непрозрачной экосистемы ПО без возможности видеть составные компоненты, а также связанные с ними риски и уязвимости больше недопустимо.

Резюме

В этой главе мы изучили историю развития SBOM и познакомились с начальными инициативами таких правительственных агентств, как NTIA, положившего начало этому процессу, и CISA, впоследствии перенявшего эстафету. Мы рассмотрели различные форматы SBOM, а также концепцию VEX, помогающую исключить из SBOM часть шума, создаваемого уязвимостями. Помимо этого, мы обсудили распространенную критику SBOM, связанные с этой концепцией опасения и области, которые продолжают развиваться по мере ее внедрения в индустрии.

5

Сложности в обеспечении прозрачности ПО

В NTIA на ранних этапах обсуждения инициатив по внедрению SBOM был выявлен усложняющий фактор в виде реализации SBOM для устройств. В данный момент все сходятся во мнении, что SBOM — настолько новая концепция для организаций, что усложнять ее обсуждение нежелательно. В конце концов, разве ПО и прошивки для устройств не одно и то же?

Прошивки и встраиваемое ПО

Мы разобьем эту категорию на несколько тематических областей: прошивка как операционная система, прошивка для встраиваемых устройств и сфера использования SBOM в некоторых специфических сценариях, например для медицинского оборудования.

Прошивки Linux

Тема прошивок для операционных систем (ОС), особенно для Linux, наиболее понятна, но важно отметить, что SBOM для них оказываются очень сложными. Linux представляет собой тысячи программных продуктов, объединенных в виде операционной системы. В связи с этим возникают трудности с достижением уровня ясности, необходимого для обеспечения прозрачности такого ПО. Но в данной области это одна из самых простых проблем. Многие из доступных для Linux инструментов обычно выполняют

не более чем команду Red Hat Package Manager (RPM), если только образ не обрабатывается масштабными процессами реверс-инжиниринга или SBOM не создаются на уровне объекта ПО при сборке. По сути, процесс сборки ОС Linux настолько фрагментирован, что полагаться на одну организацию для получения точной информации может быть затруднительно.

Прошивки для операционных систем реального времени

Прошивки для операционных систем реального времени (Real-time Operating System, RTOS) вызывают больше сложностей, поскольку в традиционном смысле RTOS фактически не имеет файловой системы. Зачастую RTOS больше напоминает двоичный файл, нежели образ прошивки в стиле Linux. Для автора RTOS это решаемая задача, и ранее мы уже сталкивались с публичными SBOM от VxWorks. Тем не менее в случае устаревших или проприетарных RTOS (которые создаются для некоторых промышленных систем защиты, не использующих популярные решения вроде QNX или VxWorks) этот вопрос остается открытым. Реверс-инжиниринг может быть единственным действенным подходом к устаревшему ПО или в ситуациях, когда поставщики не могут обеспечить необходимый уровень прозрачности.

Встраиваемые системы

Встраиваемое ПО часто является высоко оптимизированным кодом, занимающим небольшой объем памяти. Это может быть код для систем на кристалле (System On a Chip, SoC) или для BIOS, чаще всего представленный в настолько низкоуровневой форме, что для человека становится сложно проанализировать такую прошивку. Кроме того, поскольку из кода с целью оптимизации и уменьшения объема потребляемой памяти исключается множество строк и символов, идентифицировать программные компоненты традиционными способами довольно проблематично. При использовании в зонах с высоким уровнем безопасности, например в доверенных средах выполнения (Trusted Execution Environment, TEE), значительная часть этого кода выполняется в защищенной области, которая не только повышает безопасность, но и создает абстракции, усложняющие понимание того, как функционируют программа и среда ее выполнения. Это особенно актуально при попытках разобраться в уязвимости программных компонентов или в том, как злоумышленник может или не может повлиять на выполнение ПО.

SBOM для устройств

Если посмотреть, как SBOM используется в строго регулируемых средах, особенно в медицинском оборудовании, то вы увидите, что сертификация устанавливается на уровне устройств. Управление по контролю качества пищевых продуктов и лекарственных средств (FDA) может сертифицировать инсулиновую помпу в качестве устройства, однако SBOM для подобных приспособлений зачастую охватывает лишь один уровень в глубину. Обычно это больше похоже на инвентаризацию, нежели на полноценный SBOM. Поскольку эти изделия подлежат строгому контролю и изменения в них происходят нечасто, SBOM и итоговый анализ принимают скорее форму статистических данных. Но наиболее тревожным фактором должно быть то, что функциональность подобных устройств иногда больше опирается на облачную инфраструктуру и прочие внешние зависимости, чем на код, выполняющийся на самом оборудовании. Это касается многих устройств для интернета вещей (Internet of Things, IoT), когда внешние API поддерживают инфраструктуру, расположенную в опасных местах планеты, или иные внешние зависимости создают значительные риски. Несмотря на то что эти устройства представляют собой очень статичные и зрелые технологические модели, такая вариативность входов и отношений доверия образует непроработанные области в контексте прозрачности ПО и цепочек поставок. Отмечаются также проблемы, связанные с тем фактом, что различное оборудование получает обновления по-разному: по воздуху, по сети или даже через USB-накопители. Это может привести к ситуации, когда для устройств с одинаковыми номерами моделей в любой момент времени будут формироваться разные SBOM.

Открытое ПО и проприетарный код

В мире программного обеспечения код представлен в виде двух основных форм — открытый (OSS) и проприетарный. Это усложняет обсуждение прозрачности программного обеспечения в контексте цепочек поставок. OSS характеризуется тем, что код доступен для всех: его можно просматривать, использовать и изменять под свои нужды. Однако существует проприетарное ПО с «открытым кодом», когда исходный код можно просматривать или даже модифицировать, но при этом он не считается открытым. И все же проприетарный код не всегда доступен для просмотра, использования и изменения, а находится под контролем поставщиков ПО или организаций, разрабатывающих его для своих бизнес-задач, будь то внутренних или внешних.

Что же касается открытого кода, то его также могут использовать и разработчики проприетарного ПО, что они зачастую и делают. По некоторым оценкам, 97 % всего программного обеспечения включает в себя определенную долю открытого кода (<https://clck.ru/3CKRve>). И OSS не только содержится почти во всех кодовых базах, но и нередко составляет до 80 % от их объема.

В сфере ПО существует понятие *цифровое общественное достояние* (digital commons), являющееся подкатегорией общеизвестной концепции *общественного достояния*. Общественное достояние — это социальный инструмент, который способствует созданию и обмену ресурсами и информацией (<https://clck.ru/3CKRwL>). В этом контексте цифровое общественное достояние является его частью, когда ресурсами выступают данные, а также культура и знания, относящиеся к цифровой сфере. Код обычно становится открытым, когда разработчики ПО применяют существующие лицензии, дающие другим пользователям определенные права, например возможность редактировать и распространять код. Когда это происходит, все, у кого есть электронный доступ к проекту, могут копировать и использовать код, а также предлагать для него свои изменения.

Программные продукты зачастую привязаны к нескольким типам лицензий, которые определяют условия использования, ограничения и стоимость продукта. Лицензия программного обеспечения не только описывает правила распространения и использования, но и устанавливает права конечных пользователей на такие действия, как установка, гарантия и обязательства. В контексте обсуждения OSS и проприетарных продуктов лицензии на открытое ПО обычно позволяют пользователям изменять и повторно задействовать исходный код, а также дают к нему прямой доступ. При этом лицензии на проприетарное ПО обычно не предоставляют подобных возможностей. Существуют случаи, когда продукт не регулируется никакими лицензиями и считается либо открытым, либо закрытым нелицензированным ПО. Например, бизнес-приложения, которые все равно могут требовать соблюдения авторских прав. Нарушение требований лицензии может иметь последствия: владелец ПО вправе обратиться в суд с иском к нарушителю.

Лицензии стандартного программного обеспечения бывают следующих видов:

- **разрешительные лицензии** — содержат минимальное число ограничений на варианты изменения или распространения ПО, но при этом могут требовать указания авторства. К известным примерам можно отнести лицензии Apache, BSD и MIT;

- **лицензии Weak Copyleft** — часто называется общедоступной лицензией ограниченного применения GNU (<https://clck.ru/3CKRx2>). Она позволяет ссылаться на OSS библиотеки с минимальными обязательствами. При этом вся работа должна распространяться под собственными лицензиями, даже проприетарными, с минимальными требованиями;
- **общедоступные лицензии** — любой может использовать и изменять такое ПО без каких-либо ограничений; этот тип лицензии не является распространенным;
- **лицензии copyleft**, или **перекрестные лицензии**, — позволяют разработчикам изменять лицензированный код и встраивать его в другие проекты либо даже модифицировать проприетарный код и распространять его, но только под той же лицензией. В таком случае в проект включается исходное программное обеспечение и его изменения, что делает данный вид лицензии менее привлекательным для разработчиков с коммерческими интересами;
- **коммерческие/проприетарные лицензии** — используются для ограничения возможности копировать, изменять и распространять код. Считаются одними из самых ограничительных видов лицензий, так как предотвращают несанкционированное использование ПО и защищают коммерческие интересы его владельцев.

Если не брать во внимание вопросы лицензирования, то OSS используется повсеместно, в том числе и в проприетарных продуктах. И делается это по таким очевидным причинам, как экономия времени разработки, сокращение необходимой работы за счет привлечения готовых решений, снижение затрат и сроков выхода на рынок. Использование открытого кода разгружает разработчиков и команды, позволяя сфокусироваться на создании уникального кода или функциональности и решении основных задач бизнеса. Как говорилось в начале книги, OSS преобладает практически во всех сферах, включая критическую инфраструктуру (энергетика, финансы, коммуникации, здравоохранение и оборонно-промышленный комплекс). И хотя на первый взгляд доминирование OSS кажется логичным за счет инновационности и ориентированности на сообщество, его использование также несет свои сложности. Исследования показали, что 88 % кодовых баз включают открытые компоненты, которые не обновлялись более двух лет, 81 % имеют как минимум одну уязвимость, а около 90 % содержат неактуальные версии OSS.

Все это означает, что устаревшее, слабо обслуживаемое и уязвимое открытое ПО встречается *повсеместно*, включая критическую инфраструктуру.

В проприетарном сегменте существуют свои особенности. Несмотря на то что в таких источниках, как NVD для OSS, зачастую регистрируются CVE и их можно отыскать с помощью сканеров уязвимостей, проприетарный код имеет определенные сложности и может потребовать применения других техник для выявления уязвимостей. Кроме того, исходный код для проприетарного ПО обычно не раскрывается, что дополнительно затрудняет его анализ. Тенденция внедрения SBOM создает ситуацию, в которой разработчики и поставщики ПО начнут предоставлять SBOM, проливающие свет на содержащиеся в их продуктах OSS-компоненты и связанные с ними уязвимости.

Когда речь идет о раскрытии и исправлении уязвимостей, между OSS и проприетарным ПО всплывают яркие отличия. У поставщиков проприетарных продуктов обычно есть контракты и соглашения, которые обязывают их информировать пользователей о потенциальных угрозах и рисках, чего нельзя сказать об OSS. Поскольку такое ПО обычно открыто для использования и изменения кем угодно, становится гораздо сложнее наладить централизованный механизм уведомления всех сторон о подверженных уязвимостях в OSS-проекте или компоненте. Дополнительно усложняет ситуацию интеграция OSS-компонентов в другие проекты и код в виде зависимостей. В итоге для потребителей становится крайне важно понимание используемого в их средах ПО, а также получение уведомлений о любых связанных с ним рисках.

Зачастую бывает так, что OSS оказывается безопаснее проприетарного ПО, так как код доступен для любого разработчика, который может провести его ревью и анализ. В сообществе открытого ПО даже бытует выражение, называемое «Законом Линуса»: «При достаточном количестве глаз любые баги становятся поверхностными». Это кажется логичным с учетом того, что в случае анализа кода достаточным числом разработчиков и специалистов по безопасности вероятность сохранения в нем багов и уязвимостей минимизируется. И хотя это может быть актуальным для особо популярных проектов вроде Linux и Kubernetes, в целом в экосистеме OSS этот принцип практически не работает. Существуют миллионы открытых проектов, и, несмотря на то что у некоторых могут быть устойчивые развивающиеся сообщества, многие поддерживаются небольшими группами или даже отдельными людьми. Есть исследования, показывающие, что почти четверть OSS-проектов обслуживается разработчиками-одиночками (<https://clck.ru/3CKRxV>). Еще удивительнее то, что почти 95 % проектов развиваются менее чем десятком контрибуторами (<https://clck.ru/3CKRy6>). Стоит отметить, что поставщики проприетарного ПО тоже ограничены в ресурсах, в связи

с чем компании зачастую оказываются вынуждены жертвовать безопасностью в угоду функциональности и улучшений, которые принесут реальную прибыль. Реальность такова, что ограниченность ресурсов ведет к повышению рисков, и это следует учитывать всем потребителям ПО независимо от того, используют они OSS или проприетарный код. Устаревшая модель обеспечения безопасности путем маскировки также почти канула в Лету. Многие эксперты согласились, что преимущества маскировки больше выгодны злоумышленникам, нежели сообществу. Тем не менее это не останавливает некоторые организации от попыток скрыть свои слабые места. И здесь неясно, что ими движет: искренняя вера в повышенную безопасность или страх раскрыть свои уязвимые места, о которых они знают, но не хотят говорить.

Еще одна отличительная черта открытого и проприетарного кода заключается в том, что в случае первого единственным эффективным способом уведомления потенциально уязвимых потребителей и сторон является публичное раскрытие информации. Не существует единого списка всех пользователей опасного OSS-компонента, поэтому только открытое разглашение позволяет пользователям узнать об этом и принять меры. Но здесь также важно, чтобы потребители, получая эту информацию, понимали, какие именно открытые компоненты они используют. В случае проприетарного ПО организации должны уведомлять своих клиентов и потребителей, поскольку обычно между ними действуют определенные соглашения, такие как контракты, договоры о предоставлении услуг и прочие.

И хотя открытое программное обеспечение способствует формированию процветающей экосистемы инновационных решений, которые помогают организациям экономить время, ресурсы и избавляют их от разногласий, оно также стало причиной возникновения цепочек поставок с зависимостями, растущими экспоненциально. Согласно исследованиям, среднестатистическое приложение и проект содержат около 70 % зависимостей, связанных с используемым языком программирования, и не менее пяти критических уязвимостей. Многие потребители ПО не осведомлены обо всех зависимостях этого ПО, а значит, и об уязвимостях. Данные исследования показывают, что, несмотря на повсеместное использование открытого ПО, у половины опрошенных организаций нет политики безопасности, которая регулировала бы потребление этих продуктов. Организации, такие как NIST, рекомендуют применять передовые практики, например, устанавливать политики, регулирующие использование OSS, устанавливать для разработчиков внутренние репозитории известных и надежных открытых компонентов, запрещая неограниченное использование сторонних OSS-решений.

Пользовательское ПО

Пользовательское ПО, в отличие от прошивок устройств или корпоративных продуктов, используемых для управления сетями и безопасностью, зачастую рассматривается совсем под иным углом. Преимущественно воспринимаемое как некритическое, а значит, и недостойное нашего внимания, пользовательское программное обеспечение и распространенные утилиты нередко подвергаются атакам. Если принять во внимание, что любое ПО выполняется с разрешения пользователя и многие организации до сих пор наделяют своих сотрудников локальными правами администратора, становится ясно, что само программное обеспечение не обязательно должно быть предназначено для административных задач, чтобы получить возможность выполнять соответствующие функции.

В рамках Указа 14028 одной из возложенных на NIST обязанностей стало определение критического ПО. Цель заключалась в ограничении сферы влияния новых требований SBOM только наиболее критическими приложениями. И на первый взгляд это имеет огромный смысл, пока вы не поймете, что критичность определяется не самим программным обеспечением, а способом его использования. По мнению NIST, такое ПО:

- может быть запущено при наличии повышенных или административных привилегий;
- имеет прямой или привилегированный доступ к сетевым или вычислительным ресурсам;
- создано для управления доступом к данным или операционной технологии;
- выполняет функции, критические для доверия (<https://clck.ru/3CKRyw>);
- работает вне стандартных границ доверия с привилегированным доступом.

Пользовательское ПО вроде Adobe Reader или Notepad++ не отвечает ни одному из этих критериев, но, учитывая потенциальные сценарии угроз, такие как частый парсинг и просмотр пользователями сомнительных внешних документов, они могут представлять риски или служить вектором атаки. В частности, Adobe не привыкает к компрометации посредством фишинга, и эта программа может не входить в список зависимостей критического ПО, но предоставляет вполне реальный путь для атаки.

Пожалуй, самая серьезная проблема с таким ПО — необходимость устранять подобные недочеты в широких масштабах. После обнаружения уязвимости Log4j в декабре 2021 года одному из авторов книги прислали список из более чем миллиона CVE с вопросом: «Сколько из этих уникальных версий продуктов уязвимы к log4shell?» Приславшее список агентство понимало, что неважно, для чего используется программное обеспечение, — через любой экземпляр, в котором эта уязвимость могла столь же просто эксплуатироваться, как это произошло с log4shell, можно легко распространить атаку на более критические цели.

И это прекрасный пример того, как атаки на цепочки поставок позволяют злоумышленникам выйти за пределы системы неординарными способами. Это не прямые атаки, и за счет доверия к нашим внутренним сотрудникам без должной проверки их действий, слишком открытого доступа и других факторов даже самое невинное ПО может стать окном для проникновения. Как же решить эти проблемы? Определенно потребуется время, чтобы стандартизировать продукты на основе представленных в книге рекомендаций. Мы надеемся, что однажды эти практики реализации цепочек поставки станут такими же привычными, как и проверка пищевых продуктов в магазинах. Но мы также понимаем, что с практической точки зрения нам еще предстоит пройти немалый путь как индустрии и обществу в целом.

Устаревшее ПО

Если взглянуть на все прекрасные проекты, порождаемые экосистемой открытого ПО для решения проблем безопасности цепочек поставок, включая современные фреймворки разработки приложений и инструменты, то мы увидим отчетливую картину наших перспектив. Но что это значит в отношении устаревшего ПО?

Здесь мы сталкиваемся с новыми трудностями. Особенно это касается критической инфраструктуры или оборонной промышленности, где срок службы систем зачастую составляет 20–30 лет и более, а также случаев, когда в производстве ПО не используются менеджеры пакетов, исходный код оказывается недоступен, изначальные разработчики уволились или, хуже того, покинули этот мир. Иногда бывает, что программное обеспечение перестает обслуживаться или, возможно, получает лишь редкие кастомные обновления. В подобных сценариях зачастую реверс-инжиниринг оказывается единственным действенным способом определить структуру ПО.

Но давайте разберем другую проблему, касающуюся проприетарного ПО, которая становится еще более интересной в случае устаревших программных продуктов.

Когда-то давно почти все программное обеспечение было проприетарным. Им широко обменивались в научных кругах и других сферах, но по большому счету, когда мы говорим о текущем определении OSS, то в виде концепции с конкретными лицензиями это понятие оформилось лишь за последние 25 лет. Существует ряд исключений, таких как `glibc`, GNU-библиотека языка Си, и GNU Compiler Collection (GCC) с компилятором Си, разработанным аж в 1987 году. Но при сравнении старого ПО, созданного более 30 лет назад, с современным становится очевидно, что в последнем содержится намного больше узнаваемых и понятных компонентов.

Почему концепция открытого исходного кода настолько важна? Потому что сегодня основная часть инструментов обеспечения безопасности приложений не учитывают компоненты, используемые в производстве проприетарного ПО. Подавляющее большинство современных инструментов создания SBOM уведомят вас об уязвимости вашего ПО, только опираясь на известные открытые компоненты. И это огромный пробел в случае устаревшего ПО, в котором таких компонентов либо мало, либо нет совсем. В действительности многие из одних и тех же инструментов составления SBOM оказываются полезны только в случае использования менеджера пакетов при разработке ПО. Несмотря на то что в Linux управление пакетами реализуется уже более 20 лет, современные языки вроде Python и Node.js применяют современное поколение инструментов лишь в течение примерно 10 лет. Нынешние инструменты создания SBOM разрабатываются под сегодняшние реалии без учета того, что использовалось десятилетия назад и до сих пор в изобилии представлено во многих рабочих средах. Например, установщик пакетов `pip` появился в Python 2.7.9 в 2014 году.

Если исходный код недоступен, инструменты статического анализа не могут его проверить. Если отсутствует соответствующая среда выполнения, становится сложно выполнять валидацию с помощью современных инструментов IAST. А без доступа к исходному коду не будет и публично доступных CVE, позволяющих увидеть случаи повышенной уязвимости проприетарного приложения. Фактически, стремясь понять, как построено ПО, мы почти всегда имеем дело со своеобразным «черным ящиком». Единственным спасением в этом случае является тот факт, что присутствующая в ПО уязвимость может оказаться известной. Проблема же в том, что такое старое ПО является

одним из важнейших в мире. Оно отвечает за управление запуском ракет, ядерными реакторами, водоочистными сооружениями и прочими подобными объектами.

Безопасный транспорт

В основе управления рисками цепочек поставок лежит принцип доверия или проверки доверия. Имеется в виду уверенность в источнике, или провенансе, программного обеспечения, что ПО не содержит компоненты или библиотеки, которые превышают допустимый для нас уровень риска, и что оно попало к нам неизменным. Мы должны четко понимать, что устанавливаем именно тот продукт, который и планировали.

Безопасные механизмы транспортировки могут быть представлены в нескольких формах, но мы говорим о протоколе TLS, преемнике SSL (Secure Sockets Layer). Задача TLS — переместить биты из пункта А в пункт Б с высокой степенью доверия, гарантирующей, что:

- содержимое останется конфиденциальным и его целостность не будет нарушена;
- обменивающиеся информацией конечные точки являются именно теми, за кого себя выдают;
- биты и байты прибыли останутся неизменными.

И хотя TLS в том числе используется для шифрования и защиты конфиденциальности нашего потока данных, это усложняет задачу обеспечения прозрачности, а также значительно затрудняет управление самим потоком. Это сознательный компромисс между прозрачностью и безопасностью. Вы больше не видите, что конкретно происходит, но если вы доверяете источнику данных и данные за время передачи не изменились — не могли измениться, — то можете доверять и полученным данным.

К сожалению, в основе этих решений о доверии лежит центр авторизации (СА). Этот механизм устанавливает доверительную сеть, которая работает через Интернет. Это полностью централизованная экосистема в том смысле, что взлом СА может привести к компрометации выпускаемых этим центром сертификатов для TLS или даже операций подписания кода. Тем не менее СА обеспечивает достаточную степень уверенности в том, что сложность, необходимая для выполнения подобных атак, соответствует

моделям угроз большинства организаций, а значит, основанные на TLS решения считаются безопасными.

Как уже вскользь отмечалось, злоумышленники обычно внедряются в зашифрованные каналы, стараясь скрыть свой вредоносный трафик. Но ввиду самой природы современных протоколов TLS, особенно использующих такие концепции, как Perfect Forward Secrecy, взлом и восстановление зашифрованного соединения становятся сложными, если не невозможными, в том числе и для защищающейся стороны. Но это сужает спектр возможностей для компрометации пункта А и В, также устраняя из списка возможных точек взлома около 30 участков сетевой передачи.

При этом нам также необходимо гарантировать достоверность аттестаций, с которыми мы работаем при принятии решений в отношении рисков. Откуда нам знать, что SBOM или аттестация сборки легитимны? Использование журналов прозрачности и блокчейн-реестров обеспечивает прозрачность и доверие на одной стороне транзакции, в точке А. Иногда по всей транзакции. Но откуда нам знать, что данные, передаваемые по сети, не были изменены? К счастью, туннелирование TLS помогает нам ответить и на эти вопросы.

Но что, если ПО и аттестации не защищены таким образом? Значит ли это, что они небезопасны? Даже многие современные дистрибутивы Linux не защищают трафик репозитория таким образом, вместо этого опираясь на подписи уровня GNU Privacy Guard (GPG), обеспечивающие целостность ПО. Здесь вам нужно будет самим определить для себя допустимые риски, но важно учитывать следующее. Если вы потребуете применения TLS для передачи всего ПО даже в 2023 году, то потеряете возможность использовать различное корпоративное ПО.

Пока TLS не станет повсеместным, это решение в отношении всего ПО, кроме самого критического, можно рассматривать просто как «приятный бонус». Тем не менее, несмотря на ряд проблем, связанных с реализацией TLS, сторонники таких моделей, как «Нулевое доверие», выступают за использование сквозного шифрования в качестве лучшей практики, которой должны следовать организации.

Организации стремятся перейти на использование взаимного TLS (mutual TLS, mTLS), когда тот же протокол реализуется не с односторонней, а с двухсторонней верификацией. В этом случае субъекты со стороны сервера и клиента верифицируются до установки соединения и начала обмена данными.

Резюме

Несмотря на высокую активность в продвижении идеи прозрачности ПО, в этой сфере еще предстоит преодолеть ряд проблем. В текущей главе мы разобрали некоторые из них, такие как встраиваемое и устаревшее ПО, OSS и его различные виды лицензирования, а также потребность в защите передаваемых данных. Тем не менее над разработкой решений для каждой из этих проблем трудится как сообщество, так и отдельные поставщики. Продолжая работать в данном направлении, мы постепенно заполним пробел по части обеспечения прозрачности ПО вне зависимости от его расположения. В следующей главе мы обсудим, какую роль облачные технологии, включая контейнеры и Kubernetes, играют в вопросах цепочек поставок и прозрачности ПО, а также разберем некоторые связанные с этим сложности.

6

Облако и контейнеризация

Реализация концепции SBOM и прозрачности ПО в традиционной локальной инфраструктуре — сложная задача. Но если говорить о работе с облачными сервисами и архитектурами, то здесь возникают свои трудности. В текущей главе мы обсудим метрики, отражающие расширение облачной сферы и контейнеризации, а также вопросы прозрачности ПО и безопасности цепочек поставок в контексте облачных вычислений.

При обсуждении технологий зачастую желательно использовать общую терминологию. С учетом этого мы приведем наиболее популярное определение облачных вычислений из NIST 800-145 (<https://clck.ru/3CKSCa>).

Облачные вычисления — это модель, обеспечивающая повсеместный и удобный сетевой доступ по требованию к общему пулу настраиваемых вычислительных ресурсов (например, сетей, серверов, хранилища, приложений и сервисов), которые можно быстро получать и освобождать с минимальными усилиями или вмешательством сервиса. Эта облачная модель состоит из пяти основных характеристик, трех моделей сервисов и четырех моделей развертывания.

Поясню подробнее. Эти характеристики включают самообслуживание по необходимости, широкий сетевой доступ, пул ресурсов, высокую гибкость и измеримое обслуживание. Что касается обслуживания, то облако также несет в себе три модели: Infrastructure-as-a-Service (IaaS, инфраструктура как сервис), Platform-as-a-Service (PaaS, платформа как сервис) и Software-as-a-Service (SaaS, программное обеспечение как сервис). Каждая из них

имеет свои уникальные модели ответственности и связанные с ними следствия в виде того, как программное обеспечение доставляется конечному клиенту. При этом сфера облачных вычислений включает четыре основные модели развертывания: закрытую, сообщество, открытую и гибридное облако. Каждая из них имеет свои следствия в отношении таких концепций, как мультиарендность и безопасность.

Все эти уникальные модели также включают в себя программное обеспечение в той или иной форме и требуют, чтобы нижестоящие потребители имели определенную уверенность в отношении безопасности используемых ими сервисов и ПО.

Модель общей ответственности

Ни одно обсуждение безопасности и рисков в облачной среде не будет полноценным без учета модели общей ответственности (Shared Responsibility Model, SRM). Многие руководители компаний продолжают спрашивать: «А облако безопасно?», не осознавая некорректность этого вопроса. Правильнее было бы спросить: «Обеспечивает ли наша команда безопасности и организация свою долю безопасности при использовании облака?»

Подавляющее большинство взломов хранящихся в облаке данных и их утечек происходят по вине клиентов. К примеру, компания Gartner, которая занимается исследованиями и консультациями в области информационных технологий, прогнозирует, что в 2025 году 99 % нарушений безопасности облачной среды будут связаны с действиями клиентов (<https://clck.ru/3CKSDK>). Поэтому очень важно, чтобы все специалисты по безопасности четко понимали свою ответственность.

Разделение обязанностей в модели общей безопасности

В SRM четко описано, за что отвечает потребитель облачных услуг, а за что — их поставщик. В модели IaaS поставщик отвечает за безопасность облака — подразумеваются физические активы, средства, системы, аппаратное обеспечение и так далее. Клиент же отвечает за безопасность в облаке, то есть за управление сетью, идентификацией и доступом, настройку приложений и защиту данных.

Ответственность в рамках модели общей безопасности

При этом разделение зон ответственности может меняться в зависимости от модели сервиса. На базовом уровне сформированное NIST определение облачных вычислений описывает три модели сервисов:

- **инфраструктура как сервис (IaaS)**, где в рамках модели IaaS поставщик услуг отвечает за физические дата-центр, сеть и сервер/хостинг;
- **платформа как сервис (PaaS)**, где поставщик уже больше отвечает за такие процессы, как исправление ошибок и обновления (с этим клиенты всегда справлялись ужасно, и именно этот аспект чаще всего становится причиной инцидентов безопасности) и обслуживание операционных систем;
- **программное обеспечение как сервис (SaaS)**, где клиент может вносить изменения только в настройках приложения. Контроль над всем остальным остается за CSP (простой пример — Gmail). Стоит отметить, что даже в контексте SaaS еще многое предстоит сделать в плане управления и безопасности модели, о чем мы поговорим чуть позже.

Каждая из этих моделей представляет определенный компромисс. Клиент уступает контроль, получая услугу под ключ, а поставщик обрабатывает больше операционных процессов, позволяя клиенту сосредоточиться на своих основных задачах. Многие организации находят такую парадигму привлекательной, поскольку управление вычислениями и инфраструктурой обычно не является основой бизнеса, а представляет процессы, которые приходится выполнять для предоставления услуг и продуктов.

Каждый CSP работает со своей вариацией SRM. На рис. 6.1 показан пример из SRM Microsoft Azure (<https://clck.ru/3CKSDp>).

Помимо таких аспектов, как контракты и финансовые последствия, SRM также включает в себя ряд вопросов безопасности. Специалисты по безопасности должны понимать, за что они отвечают в SRM, исходя из того, какие сервисы они используют и какие реализации и архитектуры применяет их организация.

Напомним, что почти все инциденты с данными в облаке происходят на стороне клиента. Это основная причина для тщательного изучения SRM и выполнения своих обязательств в рамках модели. Еще одним крайне важным аспектом является то, что, несмотря на разделение ответственности,

собственную ее долю нельзя переложить на стороннего субъекта. Ваша организация остается единственным ответственным владельцем данных и связанных с ними лиц, даже если вы решите привлечь третью сторону, например CSP. И хотя контрактом могут предусматриваться последствия для CSP, причастных к утечке данных их клиентов, основной урон придется на репутацию и бренд.



Рис. 6.1

Ваша ответственность зависит от того, какую из двух основных ролей в области безопасности вы занимаете: специалист или руководитель. Специалисты вроде инженеров или архитекторов облачной безопасности должны понимать, какие облачные сервисы ваша организация использует, как безопасно выстроить эти решения, а также какие конфигурации, настройки и системы управления, на которые вы можете влиять, находятся в вашей компетенции. Приведем один яркий пример недоработки со стороны компаний. Поставщик облачных услуг SysDig в своем отчете *Cloud-Native Security and Usage Report* от 2023 года показал, что 90 % выданных в облачной среде разрешений не используются. Это идет вразрез со стремлением индустрии к схеме «нулевого доверия» и предоставляет злоумышленникам широкие возможности для использования взломанных аккаунтов (<https://clck.ru/3CKSEf>).

Специалисты по безопасности должны хорошо знать используемые их организацией платформы и сервисы, а также уметь реализовать их безопасным

образом. Инженеры/архитекторы облачной безопасности зачастую работают с командами разработчиков. Если вы не сориентируете их для реализации безопасного решения или выявления рискованных конфигураций (напомним, что недочеты в конфигурациях оказываются причиной большинства инцидентов безопасности), это может создать для организации серьезную угрозу.

Ресурсы для обеспечения безопасности ищите у своего CSP. Например, Amazon Web Services предлагает прекрасную базу документации по безопасности, разбитую на категории (вычисления, хранилище, безопасность, ID, соответствие требованиям и прочие). В ней вы отыщете детали, связанные с каждым используемым вашей организацией сервисом. Сюда входит широкий спектр информации, начиная с безопасной настройки сервисов и указания доступных для этого конфигураций, заканчивая руководством по устранению проблем.

К ключевым вопросам, решаемым руководителями служб безопасности, относятся: мониторинг использования сервисов (определение используемых организацией сервисов и зон, где обеспечение безопасности невозможно), слежение за соответствием сервиса установленным требованиям, а также понимание договорных/юридических аспектов вроде соглашений об уровне услуг (Service-Level Agreements, SLA), особенно когда дело касается планирования реагирования на инциденты. Одним из таких сервисов, к публикации которого был причастен автор книги, является стандарт Cloud Incident Response Framework от CSA (<https://clck.ru/3CKSFZ>). Многие организации сотрудничают с CSP и делят ответственность.

Сюда входит отслеживание соответствия используемых сервисов нормативным требованиям, которым вы должны следовать. Поставщики гипермасштабируемых облачных услуг делают эту информацию доступной. Например, AWS и Microsoft публикуют ее на странице *services-in-scope* (используемые сервисы), где вы можете определить, какие сервисы каким стандартам соответствуют, а какие находятся на стадии утверждения или только готовятся к анализу. Это помогает убедиться, что ваша команда не только создает в облаке надежные и безопасные архитектуры и рабочие нагрузки, но также использует сервисы, отвечающие требованиям применяемых вами документов.

Специалисты по безопасности всех уровней должны стремиться реализовывать в своих облачных средах современные стандарты. Это может означать внедрение лучших практик от вашего CSP или реализацию чего-то вроде

бенчмарка CIS (Center for Internet Security) для облачных сред (<https://clck.ru/3CKSG4>).

Основным элементом при работе с SRM является матрица ответственности клиентов (Customer Responsibility Matrix, CRM), которая определяет, какие системы контроля обеспечивает CSP, а какие обязанности остаются на плечах потребителя. Найти шаблон CRM и поуче лучше ознакомиться с этой темой можно через Государственную программу США по управлению рисками и авторизацией. FedRAMP занимается авторизацией предложений облачных услуг для потребления госструктурами США (<https://clck.ru/3CKSGi>).

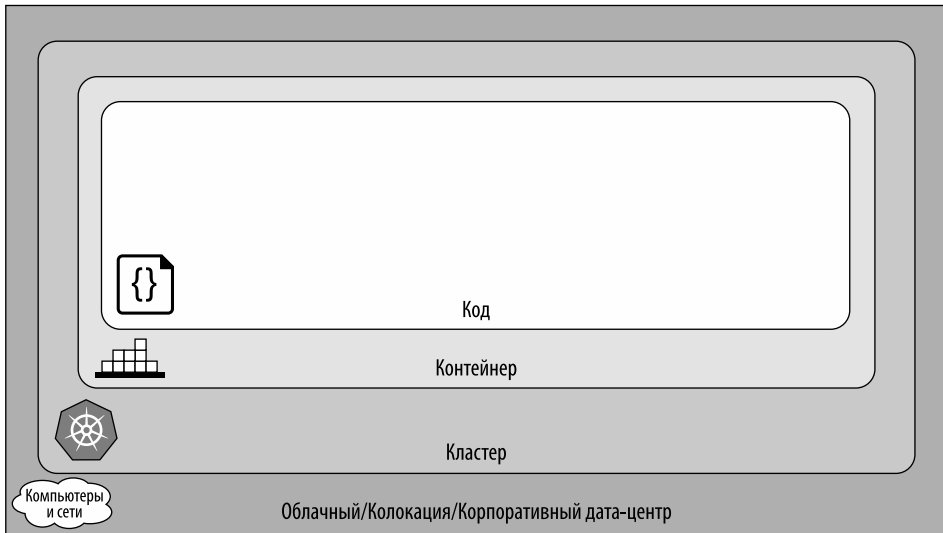
CRM — это критический инструмент для специалистов по безопасности. В модели общей ответственности системы управления безопасностью либо реализуются полностью CSP, либо совместно с потребителем облачных услуг, либо полностью возлагаются на клиента. Специалисты по безопасности могут использовать CRM для получения ясной картины этого разграничения зон ответственности. В рамках данной матрицы управление безопасностью со стороны потребителя обычно отмечается как полное, частичное или отсутствующее.

Используя облачные сервисы, можно перекладывать ответственность за некоторые меры безопасности на CSP, что позволяет организациям сосредоточиться на своих основных задачах. В результате возникает разделение ответственности, которое специалисты по безопасности должны отчетливо понимать. Напомним, что большинство инцидентов с данными в облаке при использовании модели общей ответственности происходят на стороне клиента. В итоге ваша организация несет полную ответственность, включая репутацию, а CSP выступает сторонним поставщиком услуг, от которого можно требовать стремления к прозрачности ПО и обеспечения безопасности цепочек поставок.

Четыре столпа облачной безопасности

Для удобства контекстуализации облачной экосистемы можно использовать четыре столпа облачной безопасности, представленные в документации Kubernetes в виде оркестровки контейнеров (рис. 6.2).

В этом контексте, который специфичен для контейнеров и их оркестровки, у вас есть облако, кластеры, контейнеры и код. Каждый из этих слоев зависит от вышестоящих (то есть предшествующих) элементов по части архитектуры в ее различных формах и кода, который в ней развертывается.

**Рис. 6.2**

Источник: <https://clck.ru/3CKSJ2>, авторы Kubernetes, CC BY 4.0

Как уже говорилось, парадигма облака имеет три утвержденные NIST модели сервисов. Основным способом потребления облачных услуг выступает IaaS, когда организации задействуют физические мощности поставщика для развертывания операционных систем и приложений. В этой парадигме CSP отвечает за внутреннюю инфраструктуру, утилиты, мощности, персонал и капитальные расходы (Capital Expenditure, CapEx). Ведущие CSP предоставляют надежные облачные сервисы в таких ключевых областях, как сетевые услуги, вычисления и хранение. Тем не менее эти сервисы, как и большая часть современной инфраструктуры, также работают на ПО, а значит, подвержены уязвимостям и эксплойтам. И хотя в модели IaaS потребители облачных услуг изолированы от внутренней инфраструктуры, это не избавляет их полностью от потенциальных рисков и угроз, связанных с ней. Именно это произошло в случае с Log4j. Ведущие CSP — AWS (<https://clck.ru/3CKSWH>), Azure (<https://clck.ru/3CKSXd>) и Google Cloud (<https://clck.ru/3CKSYB>) — все выпустили рекомендации по устранению этой уязвимости в своих продуктах. Они, как и IT-индустрия в целом, используют огромное множество открытых библиотек и компонентов, подкрепляющих их обширные портфолио облачных услуг. Мы видим, что одна-единственная уязвимость оказала влияние на весь спектр предоставляемых ими услуг. Но это влияние оказалось еще более масштабным. Все названные поставщики имеют ежегодный доход, исчисляемый миллиардами долларов, обслуживают тысячи компаний и миллионы

частных лиц. В свете того, что организации создают и распространяют приложения, а также сервисы на базе сервисов этих поставщиков, компрометация цепочки поставок или уязвимость во внутренних предложениях услуг может повлиять не только на сами эти организации, но и на их бизнес-партнеров, клиентов и стейкхолдеров. В случае поставщиков гипермасштабируемых облачных услуг среди клиентов присутствуют даже самые критические и чувствительные субъекты в США, поскольку услугами этих поставщиков пользуются такие организации, как Министерство обороны США и Федеральное агентство исполнительной власти США.

Эта реальность требует обеспечения гарантий для облачных сервисов, а также для лежащего в их основе ПО. И хотя существуют зрелые программы вроде FedRAMP (www.fedramp.gov) или SOC2 (<https://clck.ru/3CKSYn>), традиционно они не рассматривают программное обеспечение детально и не проводят его инвентаризацию для анализа внутренних цепочек поставок, лежащих в основе современных предложений облачных услуг, а также связанных с этим ПО рисков. Подобные программы используют более консервативные методы оценки угроз, такие как политики и процессы CSP, сканирование базовых вычислительных ресурсов на наличие угроз и тесты на проникновение. По мере того как цепочки поставок ПО становятся предметом озабоченности лидеров ИТ-сообщества и специалистов по безопасности, мы можем увидеть сдвиг, когда программы авторизации вроде FedRAMP начнут запрашивать у поставщиков такие данные, как SBOM для предлагаемых этими поставщиками управляемых сервисов на базе ПО. Правительство США уже обозначило такое намерение, выпустив меморандум OMB 22-18 (<https://clck.ru/3CKSZd>) с указанием агентств, которые могут запрашивать артефакты, включая SBOM, от сторонних поставщиков ПО для правительства. Естественно, это касается и облачных провайдеров. Меморандум также требует от сторонних поставщиков предоставления самоаттестаций, подтверждающих следование практикам безопасной разработки, SSDF и прочим соответствующим руководствам NIST.

Тем не менее это непросто, учитывая масштабы AWS, Azure и Google, в основе работы которых лежит огромное множество программных компонентов. Это также противоречит текущему подходу, преимущественно основанному на привлечении сторонних независимых организаций, оценивающих поставщиков услуг и предоставляющих артефакты/аттестации, связанные с используемыми ими практиками и системами безопасности. Облачные и управляемые сервисы, как правило, предназначались абстрагировать часть внутренней сложности от потребителя в обмен на повышение качества пользовательского опыта и удобство. Здесь стоит отметить, что

в меморандуме 22-18 поставщикам ПО разрешается проводить самоаттестацию на предмет должной реализации ими практик безопасной разработки. Тем не менее он также позволяет государственным агентствам требовать аттестации сторонней организацией, если они считают это необходимым. Однако с учетом текущего ландшафта, инструментов и возможностей организаций такой подход в крупных масштабах может оказаться непрактичным для каждого облачного провайдера.

Несмотря на то что IaaS неизбежно включает в себя программное обеспечение, обсуждения вокруг безопасности цепочек поставок достигли гораздо большей зрелости в других аспектах облачных архитектур, таких как контейнеры, Kubernetes, код и все больше SaaS.

Контейнеры

По аналогии с традиционными моделями вычислительных систем, таких как физические серверы и виртуальные машины (virtual machine, VM), в облачной экосистеме активно задействуются контейнеры. В связи с этим важно изучить тему контейнеров и соответствующих аспектов цепочек поставок в контексте таких систем оркестровки, как Kubernetes.

С развитием вычислительных абстракций произошел переход от виртуальных машин к контейнерам. Согласно определению лидера индустрии Docker, контейнер — это «стандартная единица программного обеспечения, в которую упаковываются код и все его зависимости, чтобы приложение выполнялось быстро и надежно в разных вычислительных средах». Контейнеры намного легче, динамичнее и портативнее в сравнении с виртуальными машинами, для каждой из которых требуется гостевая операционная система (рис. 6.3).

Одно из главных преимуществ контейнеров в том, что они позволяют разработчикам упаковывать код, связанные с ним библиотеки и зависимости в легковесный артефакт. Этот артефакт является портативным, то есть может выполняться в разных хост-системах и инфраструктурах, поддерживающих развертывание контейнеров.

И хотя такой процесс упаковки дает огромные преимущества в плане портативности и эффективности, он также создает условия для внедрения небезопасных или уязвимых зависимостей, которые впоследствии будут реплицированы в больших масштабах. И это происходит постоянно, например, в открытых хранилищах вроде DockerHub, Google Container Registry

и пр., где образы контейнеров доступны для потребления и повторного использования любым субъектом.

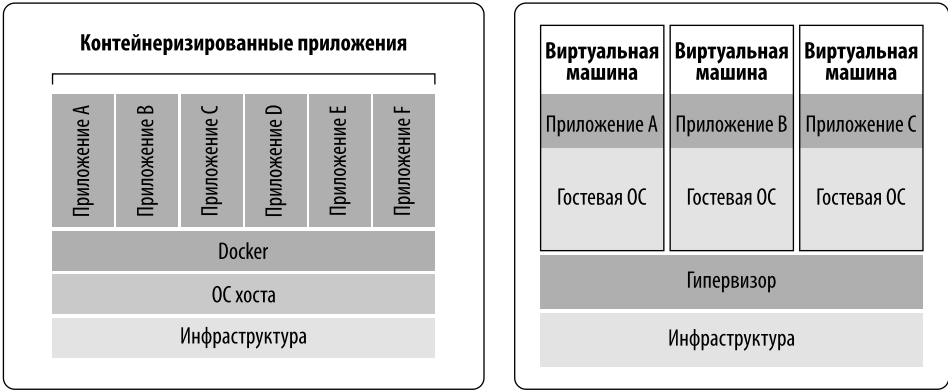


Рис. 6.3
Источник: www.docker.com/resources/what-container,
последнее обращение 27.08.2024

Так, в исследовании 2021 года, проведенном группой Unit 42 Threat Research компании Palo Alto Networks, в ходе изучения более 1500 публично доступных образов контейнеров было выявлено, что известные уязвимости присутствовали в 96 % образов. При этом 91 % образов содержал как минимум одну критическую или серьезную уязвимость по шкале оценки CVSS (<https://clck.ru/3CKSaR>).

Это исследование также показывает, что обычно число уязвимостей напрямую коррелирует с числом зависимостей в образе. Поэтому мероприятия по уменьшению поверхности атаки, или ограничению количества зависимостей и кода только теми, которые необходимы для функционирования контейнера и приложения, считаются наиболее эффективными.

По аналогии с другими формами доставки приложений и ПО развертывание контейнеров также требует прозрачности. Использование SBOM для контейнеров позволяет увидеть каждый артефакт в образе, облегчая обнаружение уязвимостей в цепочке поставок ПО и своевременное принятие мер в ходе цикла разработки продукта. Сюда относится все, начиная с исходного кода, сборки и индексирования, заканчивая развертыванием и выполнением. Уязвимости могут быть внесены на любой из этих стадий цикла разработки, поэтому генерация SBOM в ходе данного процесса обеспечит прозрачность образов и сделает видимыми изменения, которые создают риски.

Несмотря на обширное преобладание уязвимых образов, постепенно создаются инновационные инструменты, помогающие обеспечить прозрачность контейнера, а также программных компонентов и зависимостей, которые он содержит. Ведущие поставщики, такие как Anchore, создают открытые инструменты, позволяющие организациям генерировать SBOM. Например, инструмент Syft может похвастаться мощным набором возможностей и широкой языковой поддержкой. Среди его функций стоит отметить генерацию SBOM для образов контейнеров, файловых систем и не только, поддержку форматов Open Container Initiative (OCI) и Docker, а также возможность использования подписанных аттестаций SBOM с помощью фреймворка in-toto. (Концепцию аттестаций мы будем разбирать в главе 11.)

Syft поддерживает ведущие форматы SBOM, такие как CycloneDX и SPDX, о которых мы говорили в главе 4. Еще один популярный сканер OSS — Trivy от Aqua Security. Он работает с образами контейнеров, файловыми системами, репозиториями Git и ресурсами Kubernetes, осуществляя поиск CVE, пакетов открытого ПО и зависимостей. Это помогает создавать SBOM и обнаруживать проблемы в конфигурациях при использовании декларативного подхода «инфраструктура как код».

На платформе Docker даже появилась встроенная функция генерации SBOM. В начале 2022 года ее создатели объявили о появлении команды `docker sbom` в интерфейсе командной строки Docker Desktop для генерации SBOM для любого образа Docker (<https://clck.ru/3CKSbN>). Данная функция была разработана совместно с сообществом открытого ПО и командой Anchore на основе их проекта Syft. По словам Docker, она нацелена на повышение уровня доверия в цепочке поставок за счет возможности видеть содержимое образов и упрощения этого процесса для разработчиков.

С точки зрения цепочек поставок ПО контейнеры и связанная с ними экосистема представляют широкий спектр возможных векторов атаки и рисков, которые необходимо учитывать. И хотя сложно перечислить их все, мы обязательно разберем несколько основных сценариев, включая базовые образы, операционные системы, репозитории Git, код приложений и зависимости, а также компоненты OSS. Обеспечение безопасности контейнеров дополнительно усложняется тем, что в разработке, защите и операциях, реализуемых в облачных средах, задействуется много различных команд, а также то, что в защищаемых контейнерах создается несколько слоев безопасности. Сюда входят образ контейнера и содержащееся в нем ПО, взаимодействия между контейнером и ОС хоста, а также другие контейнеры, работающие на платформах оркестровки. Помимо этого, существуют и риски, связанные с передачей контейнеров по сети, хранением и средой выполнения, в которой они

развертываются при выводе в рабочую среду и которая обычно расположена поверх кластера Kubernetes.

Что касается базовых образов, то, как и в случае с любым вышестоящим компонентом в цепочке поставок ПО, здесь все нижестоящие потребители и пользователи тоже могут оказаться подвержены риску, возникшему выше. Базовые образы являются критическим этапом поставки ПО, поскольку приложения, работающие в контейнерах, наследуют все уязвимости и недостатки безопасности, включенные в базовый образ, на основе которого приложение было собрано. Рекомендуемый в руководствах нескольких ресурсов подход к контейнеризации подразумевает использование усиленных базовых образов. Как правило, это означает укрепление образа с целью уменьшения числа уязвимостей и его сокращение до минимально необходимого числа компонентов для поддержания функциональности. Обычно эта техника называется *уменьшением поверхности атаки* и заключается в удалении необязательных компонентов, которые могут быть использованы для компрометации контейнера и зависящих от него приложений. К наиболее популярным базовым образам относятся Alpine, Ubuntu и Debian.

Среди стремлений минимизировать поверхности атаки на контейнеры выделяется появление так называемых «образов контейнеров без дистрибутива» (distroless). Это образы, которые не содержат лишних компонентов операционной системы и включают только само приложение и его минимально необходимые зависимости для запуска. В них нет ни менеджеров пакетов, ни оболочек, ни других программ, которые обычно присутствуют в популярных дистрибутивах Linux. Минимизация поверхности атаки контейнеров нередко описывается как лучшая практика. Примеры тому можно найти в руководстве *Application Container Security Guide* (<https://clck.ru/3CKSc3>) от NIST, а также в книге Лизы Райс «Безопасность контейнеров»¹, где обсуждаются различные советы по защите контейнеров и уменьшению их поверхности атаки. При этом, согласно описанию в репозитории Google Container Tools Distroless (<https://clck.ru/3CKSeC>), подобные образы предоставляют и ряд дополнительных преимуществ, например минимизируют шум от сканеров, вызванный ложным обнаружением CVE, упрощают работу с провенансом и обеспечивают повышенную эффективность за счет меньшего размера.

Стартап Chainguard, ориентированный на обеспечение безопасности цепочек поставок ПО, разработал Wolfi — пакет образов без дистрибутива с гораздо более безопасным содержимым в контексте CVE по сравнению

¹ Райс Л. Безопасность контейнеров. Фундаментальный подход к защите контейнеризированных приложений. — СПб.: Питер, 2021.

с большинством традиционных образов контейнеров (<https://clck.ru/3CKSfa>). Команда Chainguard также начала развивать тенденцию создания образов, безопасных с точки зрения памяти. Эта инициатива перекликается с более общими рекомендациями таких организаций, как NSA, призывающих переориентировать индустрию на использование более безопасных по памяти языков и экосистем.

Как и в случае с другими OSS-компонентами, организации зачастую получают образы контейнеров из общедоступных ресурсов, и наиболее популярным источником выступает Docker Hub (<https://hub.docker.com>). Он управляется командой Docker и используется для поиска и обмена образами контейнеров. Если рассматривать ситуацию с позиции Docker Hub и размещаемых на нем образов, то на момент написания книги он содержит 20 образов, скачанных *не менее 1 миллиарда раз*. С учетом этого, по аналогии с любыми другими свободно распространяемыми компонентами, данные образы зачастую содержат в своих файлах по несколько уязвимостей, то есть могут унаследовать эти уязвимости и технический долг, если не будут соблюдать должную осторожность перед началом их использования. Как уже говорилось, исследования показали, что более 90 % публично доступных на Docker Hub образов содержат известные уязвимости. Причем среди них крайне высок процент тех, в которых есть как минимум одна критическая или серьезная брешь в безопасности. И если учесть те самые миллиарды скачиваний, то можно представить, сколько уязвимых контейнеров выполняются в корпоративных средах по всему миру.

В дальнейших исследованиях таких организаций, как Chainguard, использовались такие инструменты сканирования контейнеров на наличие уязвимостей, как Trivy, Snyk и Grype. С их помощью анализировались самые скачиваемые базовые образы на Docker Hub (<https://clck.ru/3CKSgp>). Специалисты Chainguard выяснили, что все самые популярные образы вроде Node, Debian, Ubuntu и Red Hat UBI имеют значительный технический долг. Число уязвимостей, связанных с этими образами, находилось в диапазоне от 28 до 800 в зависимости от применяемого сканера. Серьезность уязвимостей варьировалась от низкой до критической, и только образ Alpine: 3.15.0 не содержал уязвимостей, известных задействованным сканерам. Согласно исследованию, причина такого успешного результата Alpine в сравнении с другими в том, что их образ ориентирован на безопасность и содержит менее десяти пакетов. Это пример уменьшения поверхности атаки и создания базового образа с акцентом на безопасность. Естественно, каждый слой контейнеризированного приложения потенциально повышает

профиль риска, но рекомендуется начинать с достаточно безопасного и минимального базового образа.

Но беспокойство вызывает не только базовый образ. Организации все чаще используют практику создания внутренних репозиториев контейнеров с усиленной защитой, которые прошли процедуру сокращения поверхности атаки и были подписаны доверенными сущностями. Подобный подход сокращает потребность для команд скачивать образы контейнеров из публичных источников, так как они могут задействовать в своей работе внутренне утвержденные и доступные экземпляры, отвечающие установленным самой организацией требованиям безопасности и соответствия. Одним из наиболее наглядных и часто упоминаемых примеров в этом смысле является Iron Bank Министерства обороны США (<https://clck.ru/3CKShX>), представляющий хранилище усиленных, утвержденных и авторизованных образов контейнеров для использования ВВС США и в более широких масштабах самого министерства. Федеральные агентства США также начали использовать эти образы и, последовав примеру своих коллег, организовали собственный внутренний репозиторий безопасных контейнеров для применения и развертывания.

Публичные репозитории Git представляют еще одно направление, по которому в цепочку поставок контейнеров могут проникнуть уязвимости. И хотя выявление CVE, присутствующих в образах, выступает прекрасной метрикой для оценки безопасности контейнеров, знание репозиториев, откуда они были взяты, помогает организациям оценить потенциальные риски. Известным проектом, изучающим конфигурацию и действия, касающиеся публичных GitHub-программ и их репозиториев, является инициатива OpenSSF Scorecard, о которой речь пойдет в главе 7.

Код приложения, зависимости и OSS-компоненты представляют риски, которые могут повлиять на профиль уязвимости образа контейнера. Следовательно, важно прибегать к лучшим практикам обеспечения безопасности приложений, таким как SAST и SCA, путем сканирования контейнеров, проходящих через пайплайны CI/CD, до их включения в среды выполнения. Эти инструменты помогут выявить уязвимости в коде контейнера и устранить их до попадания в рабочую среду, где у злоумышленника появится возможность эти уязвимости эксплуатировать.

Однако, несмотря на то что смещение процессов безопасности влево и выявление уязвимостей в пайплайне или во время сборки до их попадания в рабочую среду и являются прекрасными практиками, они не исключают потребности в анализе и мониторинге контейнера в среде выполнения.

Может получиться так, что уязвимости будут упущены из виду при просмотре пайплайна или же они появятся после развертывания. Поэтому организациям, помимо обзора пайплайна, необходимо продолжать мониторить контейнеры в среде выполнения. Как указано в работе *Cloud-Native Security Best Practices* от CNCF (<https://clck.ru/3CKSiQ>), облачные рабочие нагрузки требуют реализации мер защиты на протяжении всего жизненного цикла, включая среду выполнения. Используя подходящие инструменты и проводя мониторинг, команды безопасности смогут выявлять уязвимости в среде выполнения, обновлять образ контейнера и заново развертывать уже его безопасную версию.

Важно отметить, что современные приложения содержат огромное количество открытого кода и компонентов. По данным компании Snyk, лидера в области безопасности, современные приложения на 80 % состоят из OSS (<https://clck.ru/3CKSjf>). Этот код, естественно, включает как прямые, так и транзитивные зависимости, каждая из которых имеет собственные уязвимости. Помимо прочего, образы контейнеров создаются слоями, каждый из которых использует инструменты, библиотеки и дополнительные компоненты, что тоже вносит свои риски. Регулируя использование дополнительного кода и слоев, которые добавляются в файлы контейнеров, организации могут контролировать возможные риски.

Еще одним важным моментом при работе с контейнерами является сама инфраструктура. Как уже говорилось, это означает, что кластеры Kubernetes работают как оркестратор, но еще есть такие важные аспекты, как внутреннее виртуальные машины. И здесь в качестве задач выступает усиление VM и ее операционной системы, а также управление сетевыми процессами, связанными с экземпляром (экземплярами) хоста. Подробнее об этом мы поговорим в следующем разделе.

Kubernetes

Оркестровка контейнеров зачастую определяется как операционный процесс по выполнению контейнеризированных рабочих нагрузок и сервисов. И хотя для этого существует несколько инструментов, Kubernetes, бесспорно, является наиболее популярным решением (<https://kubernetes.io>).

Kubernetes зародился в Google как проект по оркестровке контейнеров и довольно быстро распространился по индустрии. В ходе опроса, проведенного в 2021 году проектом CNCF, об использовании Kubernetes сообщили 96 %

респондентов, 70 % из которых сказали, что применяют его в рабочей среде (<https://clck.ru/3CKSKi>). По всему миру насчитывается примерно 3,9 миллиона разработчиков, использующих эту платформу, причем их число с каждым годом значительно растет. В аналогичном опросе, проведенном Red Hat (<https://clck.ru/3CKSmE>), 85 % IT-лидеров сообщили, что Kubernetes является крайне или очень важным инструментом в их стратегиях развертывания облачных приложений.

Как говорилось ранее, корни Kubernetes произрастают из ранних проектов Google, в частности системы Borg, созданной в 2003 году. Позднее в 2013 году это начинание трансформировалось в проект Omega, ставший очередной системой Google по управлению кластерами. И наконец, в 2014 году корпорация открыла исходный код Borg, представив миру инструмент под названием Kubernetes. С того момента его внедрение и развитие ускорились. Начали появляться дополнительные версии и возможности, а также устойчивое сообщество энтузиастов. В рамках таких масштабных мероприятий, как конференция KubeCon, посвященная Kubernetes и облачным экосистемам, выделяются лидеры индустрии, а также подчеркиваются новые возможности и инновации в ней.

Несмотря на быстрый рост Kubernetes, несколько организаций, занимающихся поставками ПО, продолжают использовать и внедрять Kubernetes и связанные с ним технологии. Архитектурно эта платформа состоит из нескольких компонентов, составляющих кластер. Одним из таких компонентов является API-сервер Kubernetes. Этот сервер проверяет и настраивает данные объектов Kubernetes, включая поды, сервисы, контроллеры репликации и прочие сущности.

В 2022 году организация Shadow Server, которая ежедневно сканирует все пространство IPv4, выявила 380 000 раскрытых API-серверов Kubernetes (<https://clck.ru/3CKSnM>). Исследование показало, что 84 % из обнаруженных ими экземпляров Kubernetes являются публично раскрытыми и возвращают ответы HTTP 200 ОК. Многие эксперты по безопасности и Kubernetes отметили, что это не говорит о небезопасности или уязвимости данных кластеров, а объясняется стандартными конфигурациями, установленными управляемыми CSP сервисами Kubernetes. Тем не менее это показывает, что малейшие ошибки в настройке могут раскрыть кластеры, контейнеры и выполняющийся в них код. За счет подобных недочетов конфигурации злоумышленники могут скомпрометировать кластеры и их рабочие нагрузки, а также потенциально переключиться на другие активы организации.

Еще один фактор, связанный с угрозами цепочкам поставок Kubernetes, состоит в том, что эта платформа реализуется посредством манифестов. Манифесты представляют собой декларативные файлы конфигурации в формате YAML или JSON, которые описывают, как кластеры Kubernetes должны выполняться, устанавливая нужное состояние различных объектов Kubernetes API. Это означает, что они написаны в виде кода и их можно сохранять, а также передавать аналогично другим традиционным формам файлов кода и конфигурации.

Управление сложным развертыванием Kubernetes с различными ресурсами является непростой задачей. Как правило, организации используют Helm-чарты, представляющие формат упаковки коллекций файлов, описывающих набор связанных ресурсов Kubernetes. Такие чарты, как и ресурсы Kubernetes, обычно описываются в формате YAML. Их можно сохранять и передавать друг другу на таких площадках, как Artifact Hub. Artifact Hub — это веб-приложение, позволяющее находить, устанавливать и публиковать пакеты, а также конфигурации для разных проектов CNCF, включая Kubernetes.

И хотя использование готовых чартов и файлов конфигураций способно ускорить реализацию и развертывание, это также создает риски, связанные с вредоносным или уязвимым кодом либо конфигурацией. К примеру, исследовательская группа Unit 42 компании Palo Alto Networks проанализировала Helm-чарты и файлы YAML на Artifact Hub (<https://clck.ru/3CKSnw>) с помощью инструмента под названием «helm-сканер». Результаты показали, что более 99 % контейнеров чартов имели небезопасную конфигурацию, а почти 10 % — как минимум одну критическую либо очень рискованную.

Помимо этого, специалисты Unit 42 отметили, что Helm-чарты, как и код, зачастую содержат зависимости от других чартов и что 62 % из выявленных ими ошибочных конфигураций были вызваны именно зависимостями. Они также указали, что количество зависимостей напрямую связано со средним числом уязвимых конфигураций. К таким конфигурациям относились элементы вроде излишне привилегированных контейнеров и небезопасных настроек сети. Поэтому, несмотря на то что современные приложения и рабочие нагрузки зачастую развертываются в контейнерах Kubernetes, файлы конфигураций и чарты нередко сами реплицируются и передаются со своими уязвимостями и ошибочными настройками, потенциально угрожая всем размещенным в этих кластерах рабочим нагрузкам.

В контексте Kubernetes настройки и декларативные манифесты несут собственные риски, но злоумышленники зачастую преследуют рабочие

нагрузки, оркестрируемые Kubernetes, и используют жизненный цикл образов контейнеров для проникновения в цепочку поставок этой платформы. В качестве векторов атаки здесь могут выступать базовые образы, код приложения и зависимости, репозитории, OSS-компоненты и прочие связанные ресурсы. Если злоумышленник внедрит вредоносный код в контейнеризированные рабочие нагрузки, особенно в отсутствие должных настроек безопасности Kubernetes, то он сможет повлиять не только на одну рабочую нагрузку, но и потенциально на другие такие нагрузки в кластерах или даже в корпоративных системах организации, в зависимости от используемой архитектуры и конфигураций. Подходящая защита цепочек поставок в контексте Kubernetes должна подразумевать использование безопасной архитектуры и конфигурации Kubernetes.

Мы рекомендуем организациям в этой сфере ознакомиться с документацией Kubernetes и лучшими практиками, а также использовать отраслевые стандарты, в частности Kubernetes CIS Benchmark (www.cisecurity.org/benchmark/kubernetes) или STIG, выпущенный Агентством по защите информационных систем (Defense Information Systems Agency, DISA). Эти стандарты акцентируют внимание на ключевых мероприятиях, таких как защита компонентов плоскости управления архитектуры Kubernetes, включая мастер-узел, менеджера контроллеров, планировщика и так далее. Помимо этого, они включают методы управления и конфигурации, связанные с правильной реализацией безопасных политик и контроля доступа на основе ролей (Role-Based Access Control, RBAC), сетевых политик и управления секретами. Неграмотная реализация этих практик и конфигураций может привести к компрометации рабочих нагрузок в узлах и кластерах, позволив атакующим использовать горизонтальное перемещение для взлома других систем в корпоративных средах, к которым они подключены.

Организации зачастую задействуют предлагаемые провайдером управляемые сервисы Kubernetes, такие как Google Kubernetes Engine (GKE) или Amazon Elastic Kubernetes Service (EKS). Их применение позволяет снять часть административной нагрузки, связанной с самостоятельным ведением Kubernetes (например, при реализации всех мероприятий плоскости управления и конфигураций). Тем не менее использование управляемых сервисов сопряжено со своими проблемами. В среде поставщиков облачных услуг по-прежнему есть безопасные конфигурации, которые необходимы в таких областях, как управление идентификацией и доступом (Identity and Access Management, IAM), шифрованием и ключами, в сетевых взаимодействиях и службе метаданных для выполняющих нагрузки узлов. Все это также сопровождается характерными аспектами доверия и надежности облачного провайдера.

Одним из лучших ресурсов для изучения темы обеспечения безопасности кластеров Kubernetes является официальная документация (<https://clck.ru/3CKSp4>). К полезным источникам также относятся Kubernetes Benchmark от CIS, Kubernetes STIG от DISA и Kubernetes Security Guide от NSA. Однако пользователям нужно также убедиться, что эти ресурсы учитывают лучшие практики безопасности, специфичные для используемой облачной платформы, если потребители работают в облаке.

Организации могут использовать инструменты для обеспечения защиты кластеров и развертываний Kubernetes в соответствии с ранее упомянутыми руководствами и лучшими практиками. К примерам таких инструментов относится проект с открытым исходным кодом kube-bench от Aqua Security (<https://clck.ru/3CKSqK>). Он проверяет, соответствуют ли развернутые конфигурации Kubernetes требованиям CIS Kubernetes Benchmark. Как и сам Kubernetes, kube-bench использует написанные на YAML тесты, которые по мере развития бенчмарка могут изменяться. У Aqua есть еще один интересный инструмент, kube-hunter, который ищет уязвимости безопасности в кластерах Kubernetes. Он может выполняться на машине или в конечной точке и использует IP для выполнения удаленного сканирования. Кроме того, его можно запускать на машине в кластере или даже в виде пода внутри кластера. Уникальность kube-hunter заключается в его поддержке не только режимов аудит/анализ, но и режима «активной охоты», который может менять состояние кластера для эксплуатации обнаруженных уязвимостей. В частности, по этой причине в документации kube-hunter подчеркивается, что его не следует применять в кластерах, которые вам не принадлежат. Это также касается управляемых сервисов Kubernetes.

Компании, которые хотят разобраться в различных тактиках и техниках атак, связанных с Kubernetes, также могут использовать ресурсы вроде Матрицы угроз для Kubernetes от Microsoft (<https://clck.ru/3CKSqM>), показанной на рис. 6.4.

Аналогично MITRE ATT&CK, этот ресурс помогает организациям определить поверхность атаки их сред, а также различные тактики, которые могут применять злоумышленники. В Microsoft при создании Матрицы угроз пошли похожим путем, но реализовали ее в отношении Kubernetes, включая такие действия, как раскрытие секретов, эксплуатация внутренней сети кластера и прочие. Как и в случае MITRE ATT&CK, эти тактики простираются от этапов «Первичного доступа» вплоть до «Влияния».

Первичный доступ	Выполнение	Закрепление	Повышение привилегий	Обход защиты	Доступ к учетным данным	Обнаружение	Горизонтальное перемещение	Влияние
Использование облачных учетных данных	Выполнение в контейнере	Контейнер-бэджор	Привилегированный контейнер	Очистка логов контейнера	Вывод списка секретов K8S	Доступ к API-серверу K8S	Доступ к облачным ресурсам	Полное уничтожение данных
Скомпрометированные образы в реестре	Выполнение bash/cmd в контейнере	Перезаписываемый том hostPath	Привязка кластера к админу	Удаление событий K8S	Мониторинг субъект-службы	Доступ к kubelet API	Аккаунт контейнеризированной службы	Перехват ресурсов
Файл Kubeconfig	Новый контейнер	Kubernetes CronJob	Том hostPath	Сходство имен под и контейнера	Доступ к аккаунту контейнеризированной службы	Карта сети	Сетевое взаимодействие в кластере	Отказ в обслуживании
Уязвимость приложения	Эксплуатация приложения		Доступ к облачным ресурсам	Подключение через прокси-сервер	Учетные данные приложения в файлах конфигурации	Доступ к информационной панели Kubernetes	Учетные данные приложения в файлах конфигурации	
Раскрытая информационная панель	SSH-сервер, выполняющийся в контейнере					API метаданных экземпляра	Перезаписываемые смонтированные тома на хосте	
							Доступ к информационной панели Kubernetes	
							Доступ к конечной точке Tiller	

Рис. 6.4

Еще один похожий ресурс — это Матрица MITRE ATT&CK для Kubernetes (<https://clck.ru/3CKSrM>), которую продвигает компания Weaveworks. В ней используются те же фазы, что и в Матрице угроз Microsoft, плюс добавлена фаза «Сбор», которая акцентируется на техниках, применяемых атакующими для сбора информации, в частности для получения образов из закрытого реестра. Между матрицами Microsoft и Weaveworks есть как сходства, так и отличия, но они помогают организациям понять различные фазы атак на Kubernetes и связанные с ними тактики, которые атакующие могут использовать для эксплуатации сред Kubernetes и оркестрируемых рабочих нагрузок. Помимо реализации ранее упомянутых стандартов CIS, STIG и матриц угроз Kubernetes, организациям следует также внедрять новые фреймворки, например SLSA, о котором речь пойдет в следующей главе.

Kubernetes также поддерживает различные модели мультиарендности, которые используют структуру кластеров и пространств имен для повышения эффективности, сокращения затрат и масштабирования облачных рабочих нагрузок (<https://clck.ru/3CKTQC>). Для организаций важно разворачивать рабочие нагрузки в Kubernetes, чтобы понимать задействованные модели владения, а также потенциальные проблемы безопасности и выполнения требований. Особенно это актуально, когда произошел инцидент безопасности или взлом данных, который в случае слабой защиты и недостаточного контроля радиуса последствий мог оказать каскадный эффект. Подробно риски мультиарендности в облачных средах рассматривает один из лидеров в сфере безопасности, компания Wiz, в своем фреймворке PEACH Framework. PEACH — это мнемоника, отражающая такие факторы, как Privilege (привилегии), Encryption (шифрование), Authentication (аутентификация), Connectivity hardening (укрепление систем связи), а также Hygiene (гигиена). Данный фреймворк можно применять к разворачиваниям Kubernetes и более обширным мультиарендным облачным средам (<https://clck.ru/3CKTQk>).

В экосистеме Kubernetes также присутствует поддержка SBOM. В частности, она включает утилиту с открытым исходным кодом bom, которая генерирует SBOM для проектов Kubernetes через код, написанный для этой платформы (<https://clck.ru/3CKTRJ>). Инструмент bom поддерживает создание SBOM из файлов, образов и образов Docker в форме tar-архивов, представляющих множества файлов, упакованных в один. Вы можете получать образы из реестров и анализировать их с помощью этого инструмента. Он создает артефакты SBOM в формате SPDX и может быть экспортирован в систему

подтверждения провенанса in-toto. In-toto — это фреймворк, который выходит за рамки проверки всех артефактов и создан для проверки целостности программного продукта на всем пути, начиная с его зарождения и до установки конечным пользователем. Он позволяет подробно пронаблюдать, какие шаги, кем и в какой последовательности были на этом пути выполнены (<https://in-toto.io/in-toto>). Эти проверки провенанса дают возможность генерировать проверяемые утверждения в отношении любого аспекта создания программного компонента. При использовании инструмента bom манифесты SPDX можно создавать с помощью простой команды `bom generate`.

Бессерверная модель

Развивая системы на базе вычислительных абстракций, а также виртуальных машин и контейнеров, некоторые организации пошли дальше и начали использовать так называемую *бессерверную модель*. На верхнем уровне она позволяет командам разработки создавать и выполнять облачные приложения без необходимости управлять их серверами. В отличие от SaaS, где используется готовое приложение от поставщика, в бессерверной модели разработчик развертывает свой код на облачной платформе, размещает его и выполняет у облачного провайдера. Облачный провайдер управляет внутренней инфраструктурой, включая административную нагрузку вроде масштабирования и обновления серверов и экземпляров для размещения работающего кода.

Одним из самых популярных примеров является сервис AWS Lambda, который позволяет выполнять код без необходимости управлять серверами. Код выполняется в высокодоступной вычислительной инфраструктуре и также обрабатывает внутреннее логирование экземпляров. Аналогичные бессерверные возможности предлагают и другие облачные провайдеры вроде Google и Microsoft Azure.

Тем не менее даже в бессерверной модели по части обеспечения безопасности есть свои нюансы. Код, который выполняется в функциях такой модели, по-прежнему может содержать открытые компоненты или уязвимости, представляющие риски для организаций, работающих на базе бессерверной архитектуры. Кроме того, внутренний шаблон «Функция как услуга» (Function-as-a-Service) и облачная инфраструктура работают на ПО, содержащем различные программные компоненты, потенциально с открытым кодом.

SaaSБOM и сложность API

Генерировать SBOM для контейнеризированных сущностей, например для доставленного в организацию ПО или образа контейнера, бывает непросто. Создание же SBOM для динамичных и быстро меняющихся сред со множеством взаимосвязей, таких как облако и SaaS, повышает эту сложность еще на одну ступень.

Использование концепции SBOM для SaaS начали активно обсуждать, назвав это направление *SaaSБOM*. Облачная среда и SaaS вводят управляемые поставщиком модели развертывания, где клиент не владеет или не управляет внутренней физической инфраструктурой, платформой хостинга, операционной системой или даже приложением. В этом случае приложение предоставляется как услуга, когда клиент имеет ограниченный контроль над доступными ему конфигурациями и модификациями.

Более формальное определение облачных вычислений дано в NIST SP 800-145.

Возможность, предоставляемая потребителю, заключающаяся в использовании приложений поставщика, которые выполняются в облачной инфраструктуре. Эти приложения доступны с различных клиентских устройств через интерфейс тонкого клиента, такой как веб-браузер (например, веб-почта), или программный интерфейс. Потребитель не контролирует внутреннюю облачную инфраструктуру, включая сеть, серверы, операционные системы, хранилище или даже отдельные функции приложения, разве что за исключением некоторых настроек конфигурации.

В начале 2021 года темпы распространения и роста популярности SBOM увеличились, и многие специалисты в индустрии задумались о том, как эта тенденция будет разворачиваться в динамичных и комплексных средах, таких как SaaS (другой метод использования приложений, отличающийся от традиционной схемы доставки и потребления ПО). Один из авторов книги был причастен к написанию статьи под названием *The Case for a SaaS Bill of Materials* (<https://clck.ru/3CKTSh>), вышедшей в сентябре 2021 года. В ней отмечается сложность определения ПО в контексте SaaS. Современная сфера SaaS построена на базе существующих сред IaaS и PaaS, зачастую принадлежащих другим CSP. Внутренние IaaS и PaaS обычно определяются «как код» и включают физические и виртуальные компоненты, все из которых содержат собственное ПО. Это затрудняет определение ПО

в контексте SaaS ввиду огромного объема кода и программных компонентов вкуче с различными связями и субъектами владения. Тем не менее многие представители отрасли склоняются к рассмотрению SaaSБOM в контексте сервисов и потоков данных. В рабочую группу по SBOM агентства CISA входит подразделение, занимающееся вопросами облачной среды, и многие ее специалисты поддержали эту позицию. Использование SBOM для SaaS также упоминается в работе CSA под названием *SaaS Governance Best Practices for Cloud Customers*, к публикации которой был причастен один из авторов книги (<https://clck.ru/3CKTYU>). Дополнительные сложности привносит то, что поставщики часто задействуют инструменты для поддержания и управления внутренними облачными средами, а также реализуемыми поверх них SaaS. К наиболее известным относятся Ansible, Chef и Terraform. Все они помогают настраивать, развертывать и управлять облачной инфраструктурой, лежащей в основе SaaS. Естественно, есть и другие системы и ПО, способствующие доставке SaaS, например пайплайны CI/CD и программное обеспечение IAM.

Все эти субъекты состоят из ПО и обычно являются очень динамичными и изменчивыми, что затрудняет точное определение их компонентов и составление SaaSБOM для потребителей приложения. Один из предлагаемых подходов заключается в том, чтобы поставщик SaaS определил все технологические компоненты, находящиеся под его контролем, на которые опирается приложение SaaS. И, несмотря на свою сложность, подобный подход может сработать, ведь поставщик SaaS должен как минимум разбираться в компонентах, используемых в схеме доставки, контролируемого им ПО. Но, как мы уже говорили, это не решает проблему с предоставлением потребителям SaaS-приложения подробного списка SBOM внутреннего ПО и систем поставщиков IaaS и PaaS, которые являются критически важными зависимостями для приложения SaaS.

CycloneDX SaaSБOM

Мы не единственные, кто осознает проблемы внедрения и утверждения SBOM в индустрии. Один из ведущих проектов, CycloneDX, управляемый OWASP, разработал собственную модель SaaSБOM, показанную на рис. 6.5.

Передаваемая CycloneDX SaaSБOM информация учитывает, что современное ПО опирается на ряд внешних сервисов и даже может состоять полностью из них. По этой причине разработчики сделали так, что CycloneDX может представлять различные типы сервисов, включая микросервисы, FaaS,

системы систем (Systems of Systems, SoS) и, естественно, SaaS. CycloneDX указывает, что SaaSBOM дополняет модель IaC, которая также формирует логическое представление сложных систем. В случае IaC сюда включаются компоненты облачной инфраструктуры, отвечающие за сетевое взаимодействие, вычисления и управление доступом, определенные в модели типа «как код».

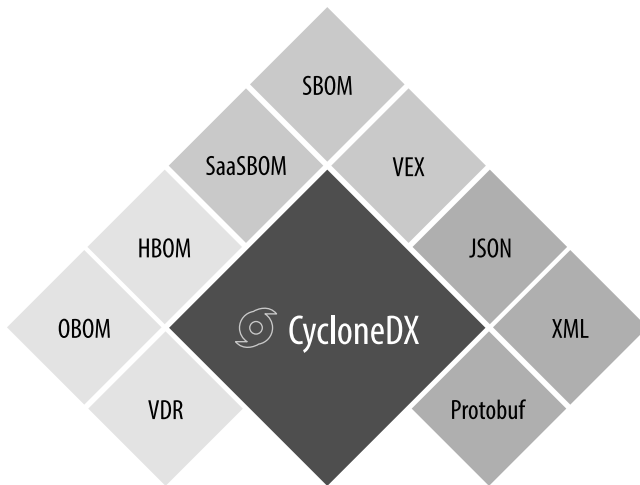


Рис. 6.5

Источник: OWASP Foundation, Inc. <https://clck.ru/3CKTdB>,
последнее обращение 27.08.2024

CycloneDX утверждает, что SaaSBOM способны обеспечить такое же логическое представление, включая все сервисы, их зависимости, связанные URL и даже направленный поток данных между сервисами и системами.

Как ранее отмечалось, в каждой архитектуре SaaS содержится коллекция независимых приложений и сервисов, необходимых для доставки приложений потребителям. Предлагаемая CycloneDX возможность создания SaaSBOM позволяет проводить инвентаризацию сервисов и программных компонентов в одном BOM или независимых BOM. CycloneDX рекомендует отделять SaaSBOM от SBOM для крупных систем, так как облако и SaaS включают в себя динамичные сервисы и сценарии развертывания. SaaSBOM можно рассматривать как единую сущность, с которой сопряжено множество SBOM, представляющих связанные сервисы, работающие согласованно для доставки приложения SaaS.

Поскольку каждый независимый сервис, очевидно, имеет собственные уникальные уязвимости, CycloneDX также поддерживает возможность ссылаться на компоненты, сервисы и уязвимости внутри BOM из других систем и BOM. Этот принцип называется Bom-Link (<https://clck.ru/3CKTba>). Он поддерживает два основных сценария: ссылка из одного BOM на другой и ссылка из одного BOM на конкретный компонент или сервис в другом BOM.

CycloneDX также рекомендует разделять VEX и BOM, так как BOM не изменяется, пока не изменится состав системы, в то время как VEX делает это по мере возникновения новых уязвимостей, связанных с существующими и используемыми программными компонентами.

Инструменты и обсуждения

Из всего сказанного становится очевидно, что в контексте сложной и динамичной облачной среды реализация SaaSБOM или SBOM в целом является нелегкой задачей. Учитывая, что люди не поспевают за столь быстрым изменением состава ПО и появлением связанных с ним уязвимостей, необходимы новые процессы и инструменты, способствующие масштабному внедрению.

В связи с этим становится все более актуальным обсуждение темы инструментов и возможностей, которые помогут решить эти проблемы SBOM в контексте облачных технологий и SaaS. В июле 2022 года CISA провело серию слушаний по нескольким вопросам, в том числе в сфере облачных технологий и онлайн-приложений (<https://clck.ru/3CKTdz>). В рамках этих событий не стояла задача найти некий консенсус, но эти дискуссии послужили началом открытого диалога по теме SBOM для SaaS и облачных приложений.

Более того, Управление по науке и технологиям (Science & Technology, S&T) Министерства внутренней безопасности США призывает технологические компании содействовать разработке автоматизированных инструментов SBOM. Это связано с тем, что масштабное управление SBOM представляет собой сложную задачу, в том числе для SaaS. В своем заявлении S&T подчеркивает, что атаки, направленные на эксплуатацию уязвимостей в ПО, могут привести к выходу из строя и нарушению безопасности критических систем, от которых зависит общество. Более подробную информацию можно найти в документе *DHS S&T Seeks Solutions to Software Vulnerabilities* (<https://clck.ru/3CKTex>).

Применение в DevOps и DevSecOps

По мере развития и усиления роли облачных систем и архитектур возникла еще одна тенденция: переход от традиционной каскадной модели разработки ПО к эпохе DevOps, а в последнее время — DevSecOps. Не углубляясь в философские дебаты, DevOps можно определить как набор практик, которые служат связующим звеном между разработкой ПО и IT-операциями. Это движение уходит корнями в начало 2000-х, когда Патрик Дебуа, Эндрю Шафер, Джен Ким и Джек Хамбл только начинали работать в этом направлении и вносить свой вклад в экосистему. Эта тенденция совпала с эволюцией облачных вычислений, и организации стали чаще использовать их, способствуя развитию бизнеса с помощью таких методик, как Agile и DevOps. Для лучшего понимания DevOps мы рекомендуем почитать такие книги, как «Руководство по DevOps» и «Проект “Феникс”».

Многие технологии, такие как облако, SaaS, CI/CD, Kubernetes, контейнеры и «инфраструктура как код», сыграли свою роль в успешном развитии и реализации DevSecOps. В контексте безопасности цепочек поставок ПО и DevSecOps развилась надежная экосистема. Беглый обзор ландшафта CNCF (<https://landscape.cncf.io>) демонстрирует увеличение разнообразия инструментов в областях CI/CD, автоматизации и конфигурации, а также планирования и оркестровки, играющих важную роль в современных средах DevSecOps. При этом, несмотря на то что инструменты и технологии способствовали успешной трансформации DevOps, сама эта концепция не является ни инструментом, ни технологией. DevOps — это методология и набор практик. Для поддержания ее развития было задействовано множество инструментов, а некоторые из распространенных подходов вроде CI/CD позволили интегрировать инструменты для решения сложных вопросов по части цепочек поставок ПО.

Как правило, в пакетах инструментов присутствуют Syft и Grype, которые позволяют создавать и сканировать SBOM на предмет наличия уязвимостей, связанных с программными компонентами, до их развертывания в рабочей среде. В DevOps задействуются управление исходным кодом и репозитории Git, пайплайны CI/CD, стили декларативного развертывания GitOps и не только. Объединение всех этих инструментов сделало возможной итеративную доставку ПО, начиная с исходного кода и до среды выполнения. Они также позволяют командам и организациям добиваться лучшей видимости компонентов, используемых в исходном коде, образах контейнеров, небезопасных IaC или конфигурациях Kubernetes, которые достигают кульминации в облачных архитектурах. Причем каждый из этих аспектов имеет свои особенности в плане цепочек поставок.

Такие организации, как NIST, начали отстаивать идею, что практики DevSecOps помогают выявлять, анализировать и снижать риски в цепочках поставок (<https://clck.ru/3CKTfs>). В работе *Software Supply Chain and DevOps Security Practices* сказано о том, что рекомендации для защиты сред DevOps должны учитывать риски. Помимо этого, в ней говорится о согласовании с другим руководством NIST, SSDF и 800-161/C-SCRM. Это руководство акцентирует внимание на использовании пайплайна CI/CD для интеграции безопасности цепочек поставок ПО в среды DevSecOps.

Одним из фундаментальных аспектов Agile и DevOps является смещение от традиционной каскадной модели разработки в сторону более итеративной и поэтапной поставки ПО. Некоторые специалисты считают, что поэтапная природа Agile и DevOps потенциально способна минимизировать проблемы с защитой цепочек поставок. Организации, создавая и выпуская ПО поэтапно, смогут использовать пакеты инструментов для идентификации программных компонентов, обнаружения уязвимостей, сбора метрик и принятия решений о рисках в отношении цепочек поставок. Инструменты DevSecOps помогут выявить уязвимые компоненты на ранних стадиях жизненного цикла ПО, до их попадания в среду выполнения. Многие утверждают, что такой подход эффективнее с точки зрения затрат, а также снижает вероятность эксплуатации, учитывая, что уязвимость не попадет в рабочую среду, где ей мог бы воспользоваться злоумышленник.

DevSecOps не только позволяет выявлять уязвимые компоненты ПО на более ранних этапах цикла разработки и улучшает прозрачность, но и помогает командам DevSecOps эффективнее реагировать на проблемы и инциденты безопасности в цепочках поставок. Все это подтверждается исследованиями, особенно программой DevOps Research and Assessment (DORA), реализуемой Google. В одном из своих отчетов команда DORA показала, что такие практики, как сканирование безопасности до развертывания, оказались более действенными в поиске уязвимых зависимостей, что привело к их сокращению в приложениях, выпущенных на рынок (<https://clck.ru/3CKThD>). Команда DORA проводила исследования в течение нескольких лет и разработала специальные метрики, которые теперь называются метриками DORA. Вот четыре ключевые из них, которые используются для оценки продуктивности команд разработки ПО:

- **частота развертывания (Deployment Frequency)** показывает частоту успешных релизов;
- **скорость внесения изменений (Lead Time for Changes)** — количество времени, необходимое коммиту для попадания в рабочую среду;

- **частота сбоев после изменений (Change Failure Rate)** — процент развертываний, вызывающих сбой в рабочей среде;
- **время до восстановления сервиса (Time to Restore Service)** отражает скорость восстановления организации после сбоя в рабочей среде. Зачастую эта метрика также называется «среднее время восстановления» (mean time to recovery, MTTR).

Эти четыре метрики отражают две ключевые темы: темпы развертывания и внесения изменений, а также стабильность в плане частоты сбоев и скорости восстановления. Они позволяют оценить способность команды выпускать ПО в темпах, соответствующих интересам бизнеса или миссии организации, и убедиться, что эти релизы не нарушают отказоустойчивости системы. На первый взгляд может показаться, что рассматривать динамичные команды как стабильные — контрпродуктивно, однако эксперты индустрии отмечают, что именно практика разработки ПО и его поставки без ущерба для стабильности способствует успеху подобных команд. Команды, которые редко выпускают ПО в рабочую среду, зачастую сталкиваются с изменениями, приводящими к сбоям, а также с невозможностью восстановить систему после таких инцидентов. В этом контексте оперативную работу команд DevOps можно рассматривать как наработку мышечной памяти или, точнее говоря, компетентности.

Резюме

Строго говоря, вышедший в 2022 году отчет о состоянии DevOps команды DORA, а также фреймворки SLSA и SSDF института NIST определяют процессы и практики, связанные с защитой цепочек поставок ПО. Мы узнали, что DevOps и DevSecOps — это не инструменты, а набор культурных практик и методологий, которые помогают командам в решении вопросов цепочек поставок, и последние исследования подтверждают эту идею.

Мы разобрали некоторые сложности и особенности, связанные с прозрачностью ПО в облачной среде. Сюда входят фундаментальные модели облачных сервисов, четыре столпа облачной среды, а также SBOM в контексте SaaS. Помимо этого, мы выяснили, что динамичные команды разработки, использующие зрелые практики DevSecOps, способны обеспечить более высокий уровень безопасности цепочек поставок.

7

Актуальные рекомендации для коммерческих разработок

Растущий в сообществе интерес к безопасности цепочек поставок побудил различные организации разрабатывать новые надежные руководства, фреймворки и ресурсы, чтобы повысить уровень защиты. В последующих главах мы разберем некоторые из этих ресурсов и познакомимся с организациями, которые их предоставляют, включая CNCF, NSA, NIST, а также некоторые другие.

Уровни безопасности в цепочках поставок компонентов программного обеспечения

С ростом числа атак на цепочки поставок ПО стала очевидной потребность в полноценном фреймворке для определения самих атак и методов противодействия им. В июне 2021 года команда Google по обеспечению безопасности открытого ПО запустила проект SLSA (<https://clck.ru/3CKTiw>). Цель этого проекта — обеспечить целостность программных компонентов на протяжении всего цикла поставки ПО.

SLSA включает четыре уровня, каждый из которых предоставляет более высокую гарантию целостности, но лишь в соответствии с уровнями зрелости и усилий со стороны внедряющих его организаций. Организации имеют различные потребности, определяющие интересующие их уровни SLSA, которые зависят от требований отрасли, закона или безопасности.

Аналогично многим другим похожим проектам SLSA является сложным процессом, требующим времени и сил для реализации. На рис. 7.1 показаны четыре уровня SLSA с описанием.

Уровень	Описание	Пример
1	Документация процесса сборки	Неподписанный провенанс
2	Защищенность от взлома сервиса сборки	Развернутый исходный код/сборка, подписанный провенанс
3	Дополнительная защита против конкретных угроз	Меры безопасности на хосте, нефальсифицируемый провенанс
4	Максимальные уровни уверенности и доверия	Двухстороннее ревью + герметичные сборки

Рис. 7.1
Источник: The Linux Foundation, <https://slsa.dev/spec/v0.1/levels>,
последнее обращение 27.08.2024

SLSA предоставляет схему, которая охватывает весь путь ПО от изначального разработчика до конечного потребителя и пользователя, включая взаимодействие этого ПО с репозиториями исходного кода, системами сборки, репозиториями пакетов и все промежуточные шаги. На рис. 7.2 показаны все точки в описанной цепочке поставки, где злоумышленник может скомпрометировать целостность программных компонентов, которые потребители используют в рабочих средах.

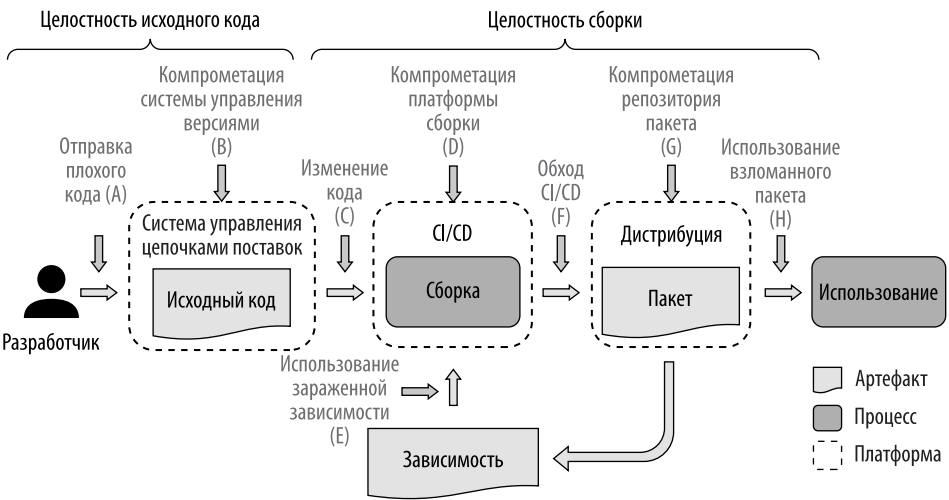


Рис. 7.2
Источник: <https://clck.ru/3CKTpA>, Google

Если злоумышленник отправит зараженный код в репозиторий исходного кода, это создаст прямую угрозу внедрения уязвимости в нижестоящие этапы процесса сборки. Однако проблема не ограничивается только этим: атаке и компрометации может подвергнуться сама система контроля версий (SCM), что влияет на хранящийся в ней код и нижестоящие процессы сборки и дистрибуции.

Эта угроза касается как локальных SCM, так и их облачных аналогов. В случае локального размещения организация отвечает за безопасность SCM с позиции инфраструктуры и хостинга. Если же размещение происходит в облаке, то уже провайдер отвечает за внутреннюю инфраструктуру, платформу и само приложение, при условии, что оно предоставляется по модели SaaS.

Исследователь Франсуа Пру (François Proulx) в статье *SLSA Dip – At the Source of the Problem!* (<https://clck.ru/3CKTpf>) построил дерево атаки, связанное с компрометацией SCM. На рис. 7.3 показаны примеры векторов атаки, которые хакеры могут использовать для взлома репозитория SCM, таких как GitHub.

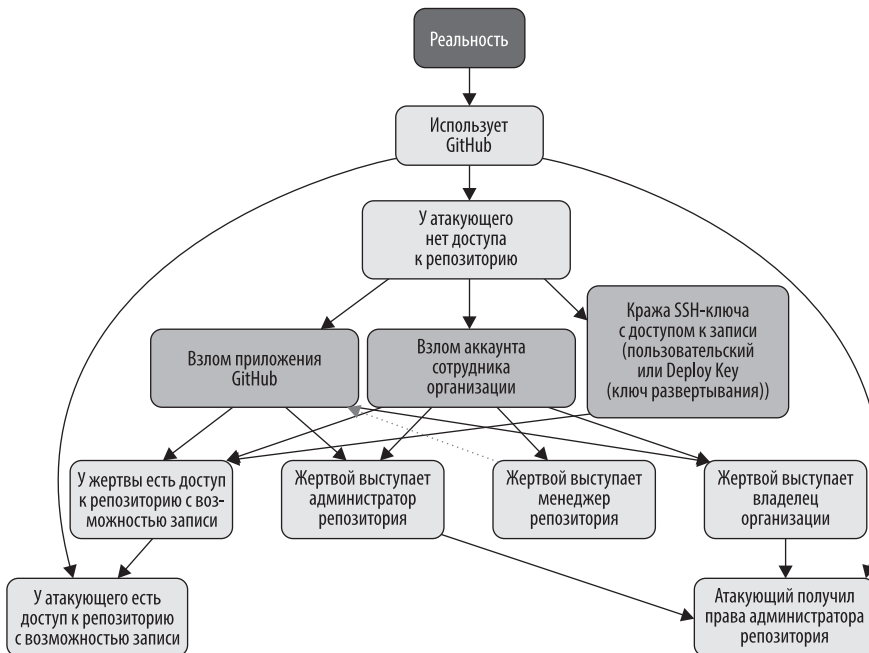


Рис. 7.3

Продолжая рассматривать потенциальную модель угрозы для цепочек поставок ПО, злоумышленник может использовать утвержденный процесс

сборки и инфраструктуру, но при этом внедрить код, не соответствующий тому, который должен использоваться в SCM. Это пример, когда информация о провенансе кода необходима, чтобы подтвердить происхождение кода и исключить возможное нарушение его целостности. SLSA определяет провенанс как «проверяемую информацию о программных артефактах, описывающую, где, когда и как что-либо было создано». Чем выше мы поднимаемся по уровням SLSA, тем более жесткими становятся требования к проверке.

Платформа сборки — это еще одна критическая часть цепочки поставок ПО, представляющая собой привлекательную цель для злоумышленников. Этот вектор атаки был использован в инциденте с Solarwinds, когда произошел взлом системы сборки и вредоносный код был внедрен в процесс компиляции инструмента управления ИТ-ресурсами SolarWinds Orion. Далее эту программу скачали и установили 18 000 нижестоящих потребителей Solarwinds, в том числе правительственные учреждения. Данный факт, бесспорно, способствовал последующей разработке Указа Президента США, нацеленного на усиление кибербезопасности, о чем мы ранее неоднократно упоминали.

Еще одна потенциальная линия атаки — возможность внедрения хакером безобидной зависимости, которая впоследствии может быть изменена для реализации злонамеренного сценария. Реальная история связана с пакетом `prpm event-stream`, когда кто-то выступил добровольцем на роль мейнтейнера проекта, но в итоге добавил вредоносную зависимость под названием `flatmap-stream`. Этот прием оказался особенно эффективен, так как пакет `event-stream` еженедельно скачивали по 1,9 миллиона раз.

Злоумышленники также использовали прием с загрузкой зараженных пакетов, которые скачивали и устанавливали нижестоящие потребители. Наиболее ярким примером является инцидент с Codecov, в котором хакеры использовали скомпрометированные учетные данные для загрузки зараженных пакетов в облачное хранилище Codecov, откуда атаковали уже сети клиентов сервиса. Если бы клиенты использовали верификацию провенанса, то увидели бы, что скачанный пакет не был тем, который они ожидали получить из репозитория исходного кода.

SLSA также подчеркивает *аттестацию программного обеспечения*, определяемую как «аутентифицированное описание (метаданные) программного артефакта или коллекции таких артефактов» (<https://clck.ru/3CKTqw>).

SLSA предлагает общую модель для представления аттестаций ПО, показанную на рис. 7.4.

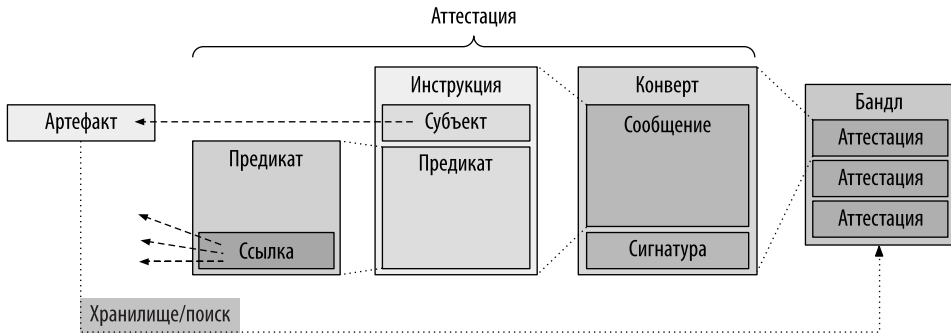


Рис. 7.4

На начало 2023 года SLSA находится на завершающей стадии очередной версии (Release Candidate, RC), которая стала первым значительным обновлением после выхода SLSA в июне 2021 года. Одно из существенных изменений заключается в разделении SLSA на несколько направлений для оценки конкретных аспектов цепочек поставок ПО. Спецификация RC также упростила и прояснила некоторые моменты изначальной спецификации, в том числе предоставив новое руководство по верификации провенанса. Тем не менее на момент написания книги специфика каждого отдельного направления еще не полностью определена и будет уточняться в последующих версиях SLSA. Более подробную информацию о RC 1.0 можно найти по адресу <https://clck.ru/3CKU6J>.

Граф для понимания состава артефактов

Следом за фреймворком SLSA компания Google вместе с Citi и Университетом Пердью объявили о запуске открытого проекта GUAC (Graph for Understanding Artifact Composition) — графа для понимания состава артефактов (рис. 7.5). Компания заявила, что «GUAC перекрывает потребность в генерации метаданных о сборке, безопасности и зависимостях программного обеспечения, возникшую у развивающихся в экосистеме проектов» (<https://clck.ru/3CKU9r>).

В заявлении отмечаются множественные инициативы с открытым исходным кодом под управлением таких групп, как OpenSSF, а также проекты по внедрению форматов SBOM для цепочек поставок ПО. Однако здесь необходимо сопоставлять и анализировать большой объем информации. Организации пытаются разобраться во всех этих данных, начиная с таких форматов SBOM, как SPDX и CycloneDX, и заканчивая аттестациями для

фреймворков вроде SLSA и различных рассмотренных нами баз данных уязвимостей наподобие NVD, GSD и OSV.

Политики и аналитика

Автоматизация, управление рисками и соответствие требованиям в течение всего цикла разработки ПО. Управление, помощь разработчикам и смещение политик безопасности влево

Агрегация и синтез

Грамотная агрегация с преобразованием данных в осмысленную информацию. Продуманное связывание проекта, ресурсов, разработчиков, артефактов, репозитория и набора инструментов

Аттестации ПО

Схемы и источники обширных метаданных о безопасности, SBOM, провенанс SLSA, VEX, OSV, оценочные листы безопасности, репутация разработчиков и проприетарные данные

Фундамент доверия

Децентрализованная, гибкая и устойчивая структура доверия. Подписи, сильные субъекты, распределенная система временных меток, объединение

Рис. 7.5

И хотя вся эта информация полезна, проект GUAC указывает на необходимость ее совмещения и синтеза для построения общей картины рисков в цепочках поставок ПО. GUAC объединяет различные источники метаданных в графовые базы данных и связи, которые пригождаются в случае аудита, управления рисками и помогают повысить эффективность разработки.

Согласно заявлению Google, GUAC акцентируется на четырех основных аспектах работы с информацией:

- сбор;
- потребление;
- сопоставление;
- запрос.

Процесс сбора информации включает такие источники, как OSV, GSD и даже внутренние репозитории. Потребление позволяет получать ее из вышестоящих источников, таких как артефакты, проекты и ресурсы. Сопоставление

подразумевает получение сырых метаданных из разных источников с последующей их компоновкой в нормализованный граф, отражающий связи между такими субъектами, как проекты/разработчик, уязвимости и версии ПО. Наконец, с помощью запросов можно выполнять поиск по собранным графам и связанным метаданным, запрашивая артефакты вроде SBOM, оценочных карт безопасности, уязвимостей и прочих. Схематично эта система показана на рис. 7.6.

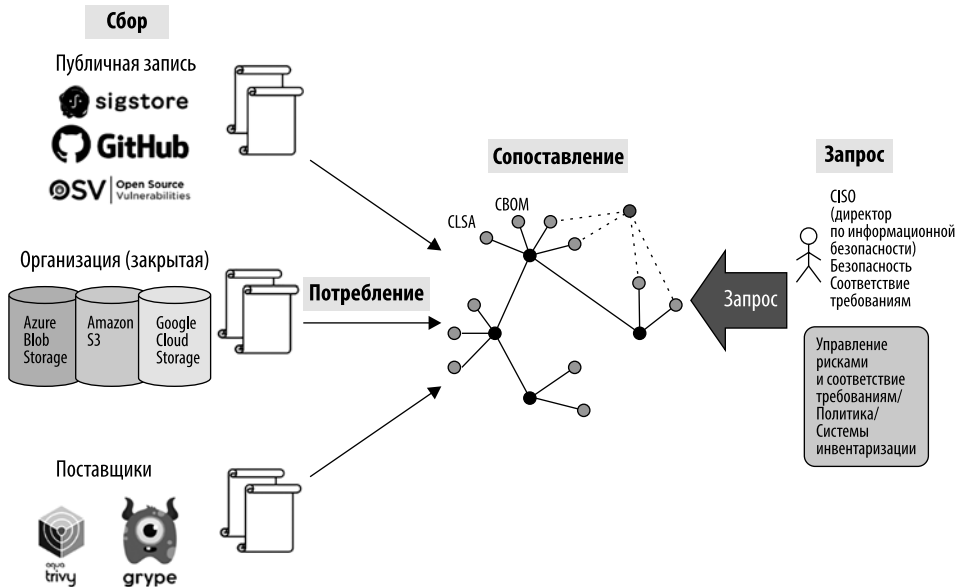


Рис. 7.6

Как показано на рис. 7.6, «Сбор», «Потребление», «Сопоставление» и «Запрос» включают разные субъекты и сценарии использования: руководство, управление рисками и соответствие требованиям (GRC), специалистов по внедрению политик, директоров по информационной безопасности (CISO) и даже разработчиков. Как сообщается в заявлении, понимание влияния конкретной уязвимости открытого ПО требует сбора и анализа невероятного объема данных по всей корпорации и экосистеме. Задача GUAC — сделать всю эту информацию действенной, чтобы пользователи могли ответить на три фундаментальных вопроса.

1. Как можно предотвратить взлом цепочек поставок?
2. Реализованы ли нужные меры защиты?
3. Если происходит инцидент, влияет ли он на нашу организацию?

Эти вопросы охватывают три тематические области, показанные на рис. 7.7:

- упреждающая;
- превентивная;
- реактивная.



Рис. 7.7

Упреждающий аспект нацелен на понимание наиболее критических компонентов в вашей цепочке поставок, а также основных слабых точек ПО. Превентивный аспект подразумевает получение уверенности, что развертываемые приложения соответствуют политикам и требованиям вашей организации. Реактивный же позволяет организации понять, каким образом ее затрагивают обнаруживаемые уязвимости.

Руководство по обеспечению безопасности цепочек поставок CIS

По мере развития практик обеспечения безопасности цепочек поставок начали появляться руководства от таких групп, как NIST, OpenSSF и CIS. В этом разделе мы разберем конкретно руководство CIS (<https://clck.ru/3CKUCJ>). Оно было создано в сотрудничестве с компанией Aqua Security, которая разработала открытый инструмент `chain-bench`, помогающий проверять стек

программ цепочки поставок на соответствие требованиям этого руководства (<https://clck.ru/3CKUD5>).

Цель бенчмарка **chain-bench** заключается в предоставлении высокоуровневого, независимого от платформы набора лучших практик, которые впоследствии можно использовать для создания платформо-ориентированного руководства для площадок вроде GitHub, GitLab и других.

Состоит это руководство из пяти ключевых блоков:

- исходный код;
- пайплайны сборки;
- зависимости;
- артефакты;
- развертывание.

Оно также следует фазам цепочки поставок ПО, от исходного кода до развертывания, и затрагивает потенциальные векторы угроз, присутствующие на каждом этапе этого процесса. Как видно по рис. 7.8, руководство напоминает другой развивающийся фреймворк — SLSA, а также существующий фреймворк SCVS (Software Component Verification Standard) проекта OWASP.

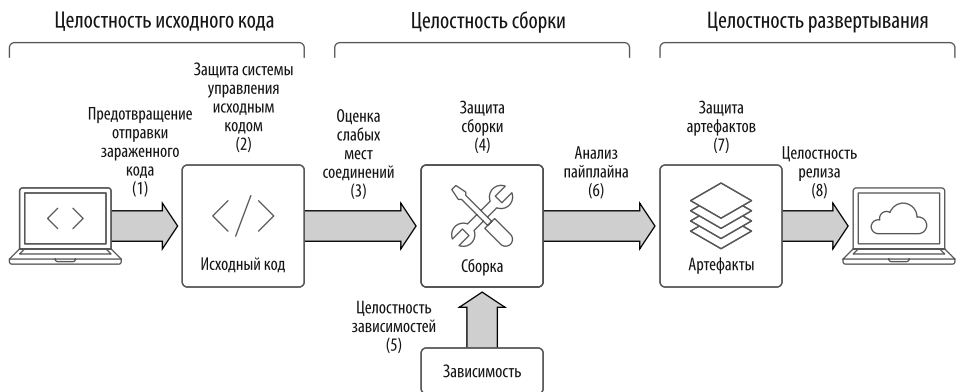


Рис. 7.8

В целом это руководство включает более 100 рекомендаций в пяти названных категориях. Далее мы разберем каждую из них с пояснением ее роли и предлагаемых ею важных рекомендаций.

Исходный код

Как уже говорилось, руководство CIS соответствует жизненному циклу цепочки поставок ПО, поэтому список неизбежно начинается с исходного кода — первой фазы этого цикла. Как сказано в руководстве, исходный код выступает источником истины для всех последующих процессов/этапов.

Обеспечение его безопасности также подразумевает защиту системы управления исходным кодом. В руководстве отмечается, что защита исходного кода включает все элементы, начиная с разработчиков, чувствительных данных в коде и самой платформы версионирования.

При этом данный блок также делится на подразделы:

- изменения кода;
- управление репозиторием;
- контроль доступа для участников;
- третьи стороны;
- риски кода.

Эти подразделы включают критические рекомендации, связанные с безопасностью исходного кода. В «Изменении кода» особенно важны следующие аспекты: отслеживание изменений в системе контроля версий, сканирование слияний кода на наличие рисков и требование подтверждения изменений двумя строго аутентифицированными пользователями. Подобные меры защиты позволяют гарантировать отслеживание изменений, устранять потенциальные проблемы до принятия этих изменений, а также проводить ревью со стороны пользователей, имеющих соответствующий уровень доступа.

В подразделе «Управление репозиторием» рекомендуются такие меры защиты, как размещение файлов `SECURITY.md` в публичных репозиториях и очистка неактивных репозиториев. Это помогает защитить целостность репозиториев, собирать обратную связь по безопасности и уязвимостям, а также не допускать расползания репозитория и обеспечивать достаточный уровень надзора.

Еще одной фундаментальной мерой для обеспечения целостности кода является «Контроль доступа для участников». Она включает в себя требования мультифакторной аутентификации (multifactor authentication, MFA) при внесении кода, реализацию доступа к репозиториям с минимальными

разрешениями и предоставление доступа к Git только конкретным IP-адресам из белого списка. Эти меры повышают уровень безопасности при внесении кода в репозиторий, открывая доступ только тем пользователям, которые успешно прошли аутентификацию и используют подтвержденный IP-адрес.

Присутствующие в репозиториях сторонние приложения представляют риск для организаций. Сюда относятся интеграции со сторонними инструментами для расширения функциональности или повышения продуктивности разработчиков. В этом случае весьма полезными будут такие меры, как обязательное подтверждение администратором устанавливаемых приложений, удаление устаревших программ, а также установка доступа с минимальными разрешениями. Эти фундаментальные способы защиты из раздела «Сторонние компоненты» минимизируют поверхность атаки организации и ограничивают возможный радиус ущерба в случае компрометации стороннего приложения.

Снижение рисков является ключевым направлением, которое организация должна реализовывать в рамках защиты своего исходного кода. В подразделе «Риски кода» приводятся рекомендации сканировать код на наличие чувствительных данных, например секретов, а также на предмет их бесконтрольного распространения, особенно в облачных средах DevOps. Сюда же относится необходимость сканировать код на уязвимости посредством SAST, SCA, а также инструментов и практик SBOM для поиска уязвимостей в открытых компонентах.

Пайплайны сборки

Согласно руководству CIS, пайплайн (конвейер) сборки — это серия задач по преобразованию файлов исходного кода в конечный продукт, который представляет собой последнюю версию ПО или приложения, подлежащего развертыванию. Разделы «Пайплайны сборки» включают среду сборки, рабочие узлы сборки, инструкции сборки и целостность пайплайна. В руководстве подчеркивается, что многие из рекомендаций здесь связаны с отличием собственных платформ сборки от предложений SaaS. При этом предложения SaaS несут свои уникальные риски, о которых мы говорили в главе 6.

В отношении среды сборки руководство советует применять такие меры защиты, как выделенные пайплайны для отдельных репозиторий, использование неизменяемой инфраструктуры и конфигурации, а также

логирование происходящих в среде процессов. Эти меры позволят организации избежать таких проблем, как дрейф и отклонения конфигурации, недостаточная видимость/логирование в случае неполадок, а также повысят уровень автоматизации при создании сред сборки, минимизировав риски, связанные с ручными действиями. Реализация подобных мер помогает избежать ситуаций, связанных с человеческим фактором, способным привести к раскрытию данных и ошибкам в конфигурациях, которыми могут воспользоваться злоумышленники.

Рабочие узлы сборки, иначе называемые *раннеры* (gunners) или *воркеры* (workers), представляют ключевой компонент инфраструктуры пайплайна и зачастую выступают целью злоумышленников. Воркеры проверяют код, выполняют тестирование и даже отправляют код в реестр, то есть отвечают за задачи, посредством которых можно нанести серьезный вред. Рекомендации безопасности включают использование одноразовых рабочих узлов сборки, разделение обязанностей между ними, а также защиту среды выполнения. Все эти меры нацелены на обеспечение мониторинга среды выполнения на предмет вредоносных паттернов поведения, ограничение потенциальной атаки раннера его сферой ответственности и минимизацию риска кражи данных.

Инструкции пайплайна используются для преобразования исходного кода в конечные артефакты. Получение доступа к ним позволяет злоумышленникам совершать вредоносные действия. В качестве мер защиты здесь рекомендуется определять шаги сборки как код, отчетливо обозначать входящую часть и получаемый результат, а также использовать автоматическое сканирование пайплайнов на недочеты в конфигурации. Все это позволит сделать этапы сборки неизменяемыми и повторяемыми, обеспечить ожидаемые вводы/выводы, а также отследить возможные ошибки в конфигурациях, чтобы избежать появления бэкдоров или компрометации.

Сохранение целостности пайплайна является критическим элементом защиты. Данный процесс подразумевает проверку достоверности и неизменности самого пайплайна, а также всех необходимых зависимостей и создаваемых им артефактов. Основные меры включают отслеживание наличия подписей артефактов перед релизом, проверку зависимостей перед использованием и контроль согласованности генерируемых пайплайном артефактов, позволяющий убедиться в их неизменности. При этом в руководстве говорится о необходимости не только создания SBOM для каждого компонента процесса сборки, но и подписания SBOM для дальнейшей проверки его валидности. Все эти рекомендации согласуются с руководствами от NIST и OpenSSF и используют технологию Sigstore, о которой мы говорили в главе 4.

Зависимости

Руководство CIS учитывает, что зависимости играют в цепочках поставок фундаментальную роль. В нем отмечается, что зависимости обычно появляются из сторонних источников, например Log4j, и в случае эксплуатации могут наносить огромный урон. В действительности исследования компаний Sonatype и EndorLabs показывают, что шесть из семи уязвимостей являются транзитивными.

Использовать сторонние пакеты нужно грамотно и с должной осмотрительностью. Сторонние пакеты влияют не только на ваше ПО, но и на его потребителей, что наглядно демонстрирует случай с Log4j. Здесь в качестве мер безопасности проводится проверка сторонних артефактов и открытых библиотек, запрашивается SBOM от сторонних поставщиков, а также требуются/проверяются подписанные метаданные о процессе сборки. Эти шаги помогут снизить риски использования вредоносных или опасных сторонних компонентов, прояснят содержимое программного обеспечения поставщика и позволят убедиться, что артефакты не были скомпрометированы в процессе сборки.

Руководство призывает проверять пакеты, чтобы понимать, как их применять и стоит ли вообще это делать. Помимо этого, оно включает комплекс мер по технической защите и управлению политиками, в том числе определение правил использования зависимостей, сканирование пакетов на известные уязвимости и поддержание осведомленности об изменении владельцев. Эти меры помогают контролировать использование зависимостей, проверять, не являются ли существующие пакеты уязвимыми, а также отслеживать возможные негативные последствия из-за смены владения.

Артефакты

Согласно руководству CIS, артефакты представляют собой упакованные версии ПО, которые хранятся в реестрах пакетов. Авторы подчеркивают необходимость защиты артефактов на протяжении всего их жизненного цикла.

Здесь требуется применение мер верификации, а именно проверка подписей артефактов пайплайном, их последующее шифрование перед распространением и контроль за тем, кто может выполнять расшифровку. Подобные действия помогают обеспечить целостность и конфиденциальность артефактов, попутно ограничив список тех, кому доступен просмотр их расшифрованных копий.

В этом случае важно реализовать надежную защиту доступа. Зачастую это касается работы с реестрами, где хранятся артефакты. Необходимо предотвратить изменение этих данных до поставки клиенту. Здесь в качестве мер защиты используется контроль списка авторизованных пользователей, имеющих право загружать новые артефакты, и мультифакторная аутентификация для доступа к ним. Цель этих действий — отделить администрирование пользователей от реестра пакетов, поручив этот процесс внешнему механизму/серверу аутентификации.

Реестры пакетов представляют еще один объект атаки. Именно здесь хранятся артефакты организации. Причем нужно иметь в виду, что защитить артефакты не удастся, попутно не защитив реестры, в которых они хранятся. Здесь потребуются реализовать аудит изменений конфигурации реестра, а также криптографическую проверку находящихся в нем артефактов.

Еще одной эффективной практикой, которая набирает популярность, является отслеживание источника (провенанса) кода. Этот процесс помогает гарантировать, что организация и ее клиенты понимают, откуда происходит артефакт и что это место является доверенным источником. Очень важно, чтобы артефакты содержали информацию о своем происхождении. Достичь этого можно с помощью составления SBOM и сопровождения компонентов метаданными. Организациям также следует использовать промежуточные реестры для перенаправления запросов внутренних пакетов из публичных реестров. Эта рекомендация согласуется с приложением F руководства NIST SP 800-161r1 (<https://clck.ru/3CKUEy>), которое призывает организовывать внутренние реестры известных проверенных компонентов. У этого приложения даже есть своя веб-страница (<https://clck.ru/3CKUJc>).

Развертывание

Развертывание — последняя фаза цепочки поставок ПО, на которой артефакты развертываются в среде выполнения. Защитить на этом этапе необходимо конфигурацию и среду развертывания.

Для обеспечения безопасности конфигурации рекомендуется отделить ее файлы от исходного кода, отслеживать вносимые в конфигурацию изменения, а также использовать сканеры для защиты манифестов IaC. Неудачная конфигурация или наличие в ней уязвимостей может поставить под угрозу саму среду развертывания.

Для защиты среды рекомендуется сделать развертывания автоматизированными и воссоздаваемыми, а также ограничить доступ к ним только теми, кому он необходим. Эти рекомендации позволят обеспечить неизменность развертываний, избежать отклонений конфигурации и ограничить доступ для минимизации угроз.

И хотя область обеспечения безопасности цепочек поставок еще только развивается, руководство CIS вместе с рекомендациями других упомянутых организаций служит своеобразным маяком в сфере кибербезопасности. Цепочка поставок ПО является невероятно сложной и хрупкой экосистемой, представляющей системный риск ввиду огромного числа организаций, потребителей, поставщиков и отраслей, на которых она влияет. Обязательно следите за появлением новых платформи-ориентированных бенчмарков CIS, основанных на методах защиты и рекомендациях, указанных в разобранном выше руководстве CIS.

Лучшие практики CNCF по защите цепочек поставок

Еще одним источником рекомендаций, регулярно упоминаемым в дискуссиях на тему безопасности цепочек поставок, является работа *Software Supply Chain Best Practices* (<https://clck.ru/3CKUQW>), выпущенная в 2021 году CNCF. В ней описываются способы защиты исходного кода, материалов, пайплайнов, артефактов и развертываний. В качестве ключевого инцидента, привлечшего массовое внимание к вопросу обеспечения безопасности цепочек поставок ПО, в этой работе приводится случай с SolarWinds. Сама же она построена вокруг четырех основных принципов:

- каждый шаг в цепочке поставок должен быть надежным и безопасным;
- в каждом шаге цепочки поставок должна использоваться автоматизация;
- среды сборки должны быть грамотно определены и защищены;
- для всех субъектов в среде цепочек поставок должна применяться взаимная аутентификация.

В работе также признается, что в организациях и отрасли присутствуют разные гаранты надежности и допустимые уровни рисков. В каждой отдельной компании могут действовать свои требования к надежности. Например, для систем вооружения Министерства обороны будут установлены гораздо

более жесткие требования безопасности, чем для простого веб-приложения без обработки чувствительных данных. Организации могут использовать приведенные в работе рекомендации для снижения угроз цепочкам поставок ПО, исходя из собственного уровня допустимых рисков и требований надежности.

В работе рассматриваются три уровня надежности: низкий, средний и высокий. Среды с низким уровнем — это такие, в которых защите целостности (или безопасности) продукта уделяется мало внимания. Средний уровень подразумевает адекватные требования надежности, соответствующие большинству развертываний. В работе он отмечается как основной. Наконец, среды с высоким уровнем надежности требуют, чтобы продукты сохраняли целостность, имели защиту от неавторизованного изменения, а также проходили детальные аттестации и верификации.

Как видно из рис. 7.9, в работе проводятся параллели с традиционными цепочками поставок, которые действуют в сфере сырьевых материалов, товаров и производства. В ней отмечается, что технологическая индустрия позаимствовала из области малозатратного производства практики Agile и DevOps и сфера цепочек поставок тоже может пойти этим путем.



Рис. 7.9

Источник: CNCF Supply Chain Best Practices Whitepaper (<https://clck.ru/3CKYuA>). GitHub

Однако, помимо сходств с производственной цепочкой поставок, в документе также подчеркиваются и ключевые отличия. Во-первых, цепочки поставок ПО неосязаемы, так как включают виртуальные и цифровые компоненты. Во-вторых, цепочки поставок ПО ввиду своего акцента на технологиях развиваются более активно. В-третьих, повторное использование

и сложность цифровых цепочек поставок. Многие описывают это как «глубинную медлительность», подразумевая, что во многих случаях здесь наблюдается бесконечная регрессия с несколькими уровнями прямых и транзитивных зависимостей. Некоторые продукты и цепочки поставок являются частями или компонентами других продуктов или цепочек поставок. Данный факт порой создает сложности, особенно в свете того, что большая часть уязвимостей чаще наблюдается именно в транзитивных зависимостях, о которых организации обычно осведомлены недостаточно хорошо.

В цифровой среде, как и в физическом мире, каждый этап цепочки поставок связан с определенными рисками. В связи с этим и появились такие фреймворки, как SLSA, и сопутствующие им методики, нацеленные на снижение рисков на каждой стадии цикла поставок. Как и в сфере кибербезопасности и технологий в целом, устойчивость цепочек поставок ПО определяется их наиболее уязвимым звеном. Поэтому слабые места в одном из аспектов могут оказать каскадное влияние на всю последующую часть цепочки и всех нижестоящих потребителей и стейкхолдеров, что мы неоднократно наблюдали на примере компонентов, получаемых как от поставщиков, так и из открытых источников.

Руководство CNCF подчеркивает некоторые ключевые аспекты, такие как верификация, автоматизация, авторизация в контролируемых средах и аутентификация. Все это очень важно для обеспечения высокой надежности на протяжении всего цикла жизни цепочки поставок.

Верификация необходима для подтверждения сохранности продукта на всех этапах цепочки, а автоматизация помогает наладить повторяемость и неизменяемость процессов. Грамотное определение разрешений для машин и людей гарантирует, что каждый субъект и этап выполняют только предписанные действия. И хотя этот подход с минимумом разрешений является давней практикой безопасности, он также подчеркивается в инициативе по внедрению модели нулевого доверия, так что в этом смысле между ними есть сходства.

Наконец, безопасная аутентификация подразумевает взаимную проверку субъектов в цепочке поставок с применением методов, соответствующих уровню надежности среды и требованиям организации. В руководстве рассматривается пять этапов защиты цепочки поставок: защита исходного кода, материалов, пайплайнов сборки, артефактов и развертываний. Далее мы подробно разберем каждый.

Защита исходного кода

Цепочки поставок ПО берут начало в исходном коде. Создание и защита кода являются фундаментальной задачей, которая влияет на целостность всей последующей цепочки. В руководстве CNCF указывается, что наиболее критическим вектором атаки для любой платформы или поставщика выступает IAM к репозиториям исходного кода со стороны людей и машин. Подкрепляется этот довод отчетами вроде *Data Breach Investigations Report* компании Verizon от 2022 года. В нем показано, что более 60 % случаев взлома данных сопряжено с компрометацией учетных записей.

К рекомендациям по защите исходного кода относятся процессы верификации, такие как подписание коммитов для подтверждения целостности и неподдельности получаемого исходного кода или изменений. Существуют более традиционные методы вроде ключей GNU Privacy Guard (GPG) или сертификатов Secure/Multipurpose Internet Mail Extensions (стандарт для шифрования и подписания электронной почты с помощью открытого ключа, S/MIME). Но также есть и инновационные решения, например GitSign (<https://github.com/sigstore/gitsign>), который использует Sigstore для реализации подписей «без ключей». Этот метод подписания артефактов и метаданных весьма привлекателен, поскольку снижает издержки, связанные с традиционными способами подписания и управления ключами.

Для защиты исходного кода и его репозитория не лишней будет автоматизация. Одной из таких областей является проблема раскрытия секретов. В данном контексте в качестве секретов могут выступать файлы учетных данных, ключи SSH, токены доступа и ключи API. Распространение конфиденциальных учетных данных — серьезная проблема, особенно в облачных средах DevSecOps, которые включают декларативный подход и возможность отправлять секреты в различных манифестах и файлах конфигурации.

Компания GitGuardian, специализирующаяся на управлении секретами, в 2022 году выпустила очень информативный отчет *State of Secrets Sprawl* (<https://clck.ru/3CKUSS>), подробно освещающий эту тему. Среди тревожных показателей можно отметить более 3000 эпизодов обнаружения секретов в 2021 году на одного инженера по безопасности и более 6 миллионов публично раскрытых секретов в том же году, что вдвое больше показателей 2020 года. Один из авторов данной книги написал статью, в которой осветил

состояние сферы управления ключами и отметил, какое влияние могут оказать неудачно реализованные практики (<https://clck.ru/3CKUTA>). Этими раскрытыми секретами могут воспользоваться злоумышленники и нанести серьезный урон организации. Причем ситуация ничуть не улучшается. Как показывает выпущенный в 2023 году компанией GitGuardian очередной отчет *State of Secrets Sprawl*, в открытых репозиториях GitHub было обнаружено более 10 миллионов раскрытых секретов. В 2022 году также ряд секретов был вовлечен в инциденты безопасности, повлиявшие на ряд крупнейших организаций отрасли, таких как Uber, NVIDIA, Microsoft и Slack (<https://clck.ru/3CKUTh>).

Реализация управляемых сред для репозитория исходного кода позволяет снизить некоторые риски за счет ряда практик, включая внедрение политик внесения кода. Вдобавок к ранее отмеченным рекомендациям по выделению только необходимых разрешений также можно связать роли сотрудников и допуски с функциональными обязанностями внутри организации. Организации могут дополнительно расширить эту идею с помощью механизмов контроля доступа на основе контекста, которые проверяют такие факторы, как время суток и возможности устройства. Благодаря этой информации они предоставляют динамический доступ с учетом времени, что является типичным решением для сред с нулевым доверием. Еще одна ключевая рекомендация в области кибербезопасности заключается в разделении обязанностей, чтобы автор запроса не мог утверждать его.

Аутентификация — это процесс предоставления первоначального доступа к репозиторию исходного кода на основе субъекта, будь то человек или машина. Для защиты этого процесса рекомендуется использовать мультифакторную аутентификацию (MFA) для входа и ключи SSH для упрощения доступа разработчикам. Для этих ключей должна быть реализована политика обновления, чтобы ограничить возможное влияние хакера на систему в случае взлома. Для таких субъектов, как машины и сервисы, например агентов пайплайна, организациям следует применять краткосрочные учетные данные. Это позволит снизить влияние взломанных учетных данных, поскольку ключи будут постоянно генерироваться, использоваться и удаляться. Как уже говорилось, взлом учетных данных является одним из самых распространенных векторов атаки, так что их ограниченный срок действия уменьшит этот риск.

Защита материалов

В контексте цепочек поставок ПО в руководстве CNCF материалами выступают зависимости и библиотеки, будь то прямые или транзитивные. Данный аспект наиболее тесно связан с тематикой книги. В руководстве отмечается, что некоторые высоконадежные среды могут блокировать соединение с недоверенными репозиториями. Стоит также обозначить, что в руководстве NIST SP 800-161r1 рекомендуется устанавливать внутренние репозитории известных доверенных компонентов для использования в разработке, о чем мы поговорим в главе 8.

В руководстве подчеркивается, что организациям следует применять методологии управления рисками в отношении потребляемого ими стороннего ПО. И хотя публикация информации об уязвимом коде в виде CVE является типичной практикой для организаций и проектов с открытым исходным кодом, руководство отмечает, что CVE являются «запоздалой» метрикой, так как информируют об уязвимости только после ее обнаружения и публикации. То есть потребляемый пользователями код уже является уязвимым. Поэтому организациям важно использовать и другие метрики, которые будут уведомлять потребителей о рисках, связанных с проектами и кодом, отражая, например, их операционное состояние. Один из таких инструментов под названием Scorecard разрабатывает фонд OpenSSF (подробнее о нем мы поговорим чуть позже). Организации потребляют компоненты с открытым исходным кодом от внешних субъектов, поэтому проверка этих сторонних библиотек критически важна и включает такие методы, как использование контрольных сумм и проверка подписей. Организации могут (и должны) также применять методы вроде генерации SCA и SBOM для выявления присутствия в потребляемом ими ПО уязвимых открытых компонентов. Внедрение SCA и SBOM позволит компаниям отслеживать зависимости между потребляемыми ими OSS-компонентами и системами, к которым они относятся. Организация может начать проводить инвентаризацию цепочки поставок, включающую не только OSS-компоненты, но также поставщиков и источники, используемые внутри самой организации.

Еще одно руководство по управлению рисками предостерегает о возможности внедрения вредоносного кода в исполняемые файлы или TAR-архивы. По этой причине организациям рекомендуется собирать библиотеки из исходного кода, а не использовать скомпилированные решения, которые могут оказаться предварительно взломанными. В руководстве также рекомендуется создавать список доверенных менеджеров пакетов и репозиториев,

контролировать доступ к ним, чтобы предотвратить внедрение кода из ненадежных источников.

Организациям также следует применять автоматизацию для защиты материалов, получаемых извне. Эта область включает сканирование ПО на уязвимости, уточнение условий лицензий, а также выявление его прямых и транзитивных зависимостей.

Защита пайплайнов сборки

Руководство CNCF SSCP проводит параллели между сборочной линией в промышленности и пайплайнами (конвейерами) сборки в контексте цепочек поставок ПО, называя последние «сердцем фабрики по производству программного обеспечения». В пайплайне собираются различные материалы, поступающие из репозитория исходного кода и от сторонних поставщиков. Эти сборочные конвейеры включают такие компоненты, как этапы сборки, воркеры, инструменты и оркестратор. Каждый из них также будет рассмотрен в разделе «Эталонная архитектура защищенной фабрики ПО».

Защита пайплайнов требует усиления этапов сборки и промежуточных результатов во избежание компрометации как самого пайплайна, так и его процессов. И хакеры понимают, что, если им удастся взломать пайплайн, то они смогут распространить зараженное ПО на ничего не подозревающих нижестоящих потребителей.

Для защиты пайплайна сборки CNCF предлагает принять следующие меры безопасности:

- использовать один репозиторий для всех компонентов сборки;
- документировать все этапы и связанные с ними входные/выходные материалы;
- использовать такие методы, как подписание артефактов и метаданных, для обеспечения целостности процесса сборки.

Помимо этого, организации должны выполнять в отношении своих пайплайнов сборки и инфраструктуры моделирование угроз и автоматизированное тестирование безопасности точно так же, как делают это для рабочих систем. Грубо говоря, ваши системы сборки с точки зрения безопасности должны рассматриваться как системы в рабочей среде, поскольку производимые ими

артефакты в итоге попадают в среду выполнения. Если данные артефакты будут скомпрометированы, это коснется и среды выполнения.

И хотя руководство CNCF не ссылается на проект OWASP Top 10 CI/CD Security Risks напрямую, последний — один из лучших источников рекомендаций на эту тему. В нем говорится, что CI/CD-среды, процессы и системы не являются основой современных схем поставки, а также указывается, что компрометация CI/CD-сред привела к ряду громких инцидентов безопасности, таким как SolarWinds, Codecov и других, которые повлияли на тысячи конечных пользователей (<https://clck.ru/3СКUX8>). Руководство акцентирует внимание на таких ключевых компонентах современных сред сборки, как оркестратор пайплайна, воркер и защищенные репозитории артефактов (рис. 7.10). Все эти сущности объединяются для выполнения этапов сборки, связывания зависимостей, компиляции приложения, его тестирования и публикации в защищенном репозитории с последующим развертыванием. Компрометация этих процессов и элементов окажет каскадное воздействие на среды развертывания.

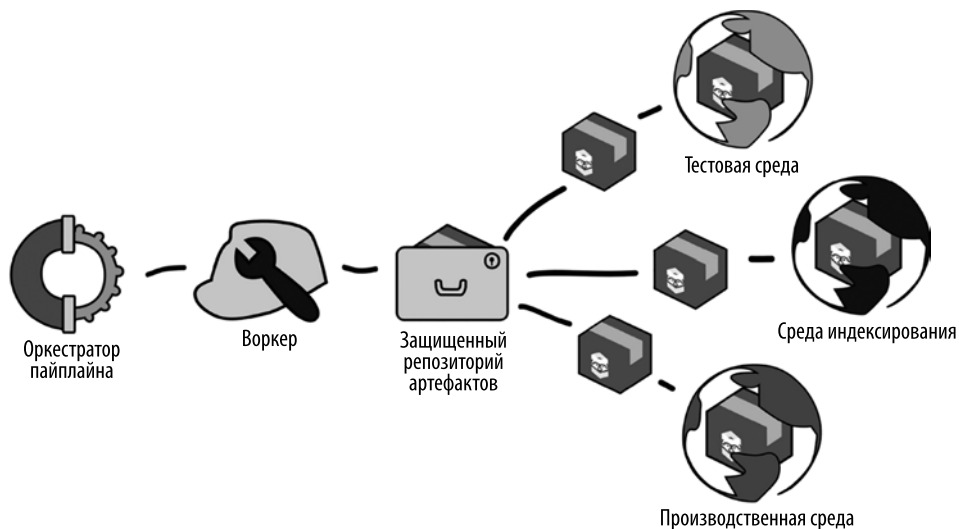


Рис. 7.10

Источник: <https://clck.ru/3СКUZ2>. The Linux Foundation

Еще одна критическая мера, выделяемая в руководстве, — это верификация. Данная область включает такие техники, как применение криптографии для оценки соответствия политикам, проверка сред и зависимостей перед использованием и проверка безопасности среды выполнения воркеров

(см. рис. 7.10). В качестве одного из вариантов в руководстве приводится создание сквозных, криптографически подтвержденных гарантий.

Дополнительно акцент делается на использовании воссоздаваемых сборок для высоконадежных сред и сред с высокими рисками. Если обобщить, то воспроизводимые сборки позволяют получать входные материалы и создавать из них артефакты, которые можно криптографически проверить. Это позволяет своевременно выявлять вредоносные или незапланированные изменения и предпринимать соответствующие меры. Данная практика приводится и в других руководствах, таких как SLSA. При этом, как многие отмечают, она является затратной в плане времени и ресурсов, в связи с чем больше распространена в высоконадежных средах.

Вот еще несколько эффективных практик и возможностей:

- автоматизация создания среды сборки;
- распространение сборок по разным инфраструктурам;
- использование автоматизации для стандартизации пайплайнов среди команд и проектов;
- обеспечение защищенной среды оркестровки для размещения фабрики ПО (подробнее в разделе «Эталонная архитектура защищенной фабрики ПО»).

Защита артефактов

После завершения этапа сборки артефакты и ПО создаются вместе с сопровождающими их метаданными. Для защиты этих артефактов и сохранения их целостности можно применять различные методы, например рекомендуемое CNCF подписание, которое следует выполнять на каждом этапе жизненного цикла артефакта.

Согласно руководству CNCF, организациям рекомендуется проводить верификацию, подписывая каждый этап процесса сборки и проверяя эти подписи. В качестве примеров проектов, которые можно использовать для выполнения аттестаций, необходимых в течение этого процесса, в нем приводятся The Update Framework (TUF), SPIFFE и SPIRE. SPIFFE и SPIRE обеспечивают единую плоскость управления идентификацией по всей современной разносторонней инфраструктуре. Эта поддержка включает защиту взаимодействия микросервисов, возможность безопасной аутентификации, а также межсервисную аутентификацию для моделей безопасности с нулевым доверием (<https://spiffe.io>). Организации могут задействовать

автоматизацию, используя TUF и Notary для управления подписанием артефактов, а также сохранения связанных с ними метаданных и выводов.

Кроме того, организациям следует принимать меры по ограничению артефактов, которые могут быть подписаны каждой конкретной стороной, и разделять эти возможности по времени, выполняя для них периодические ревью. Разграничение по времени подразумевает установление временных рамок на сроки действия учетных данных и доступа. После создания организации должны зашифровывать артефакты до начала их распространения, причем сделать это таким образом, чтобы расшифровать их могли только авторизованные платформы и пользователи. Выполнение этих мер обеспечит сохранение целостности артефактов на протяжении всего жизненного цикла, а также позволит предоставить конечным потребителям все необходимые разрешения и должный контроль доступа.

Защита развертываний

Заключительный раздел работы CNCF SSCP посвящен защите развертываний. Здесь CNCF еще раз подчеркивает важность TUF в этом процессе и необходимость наличия возможности обнаружения и предотвращения вредоносной активности.

В качестве ключевой меры руководство приводит верификацию, которая должна производиться после развертывания артефактов ПО. Под верификацией понимается то, что клиент, получающий этот артефакт, может проверить его целостность и сопутствующие метаданные. То есть он сможет проверить подпись SBOM, если таковой был создан, и убедиться, что он был подписан авторизованной стороной.

Эталонная архитектура защищенной фабрики ПО от CNCF

Для многих термин *фабрика* в отношении разработки ПО может показаться неуместным. Чаще люди ассоциируют фабрику со сбором, обработкой и производством физических материалов вроде стали, автомобилей или бытовой электроники. Тем не менее процесс создания программного обеспечения, которое по иронии жизни управляет многими процессами на современных фабриках, обретает с этими самыми фабриками все больше сходств.

Выражение *фабрика ПО* описывает набор инструментов, активов и процессов, необходимых для создания программного обеспечения эффективным, воспроизводимым и безопасным способом.

Этот термин закрепился как в частной, так и в государственной сферах и был признан такими организациями, как MITRE, VMware и другие. Министерство обороны США наладило устойчивую экосистему из 29 фабрик ПО, в частности Kessel Run, Platform One и Army Software Factory. Структура подобных фабрик также распространяется по федеральным агентствам США в таких программах, как Centers for Medicare and Medicaid (CMS) и batCAVE. Данный термин в том числе признали лидеры отрасли вроде CNCF, которая даже разработала собственную эталонную архитектуру защищенной фабрики ПО (Secure Software Factory Reference Architecture). О ней мы и поговорим далее.

Эталонная архитектура защищенной фабрики ПО

CNCF определяет цепочку поставок как «серию шагов, выполняемых при создании, тестировании, упаковывании и распространении программного обеспечения до конечного потребителя». Фабрика ПО — это логическая конструкция, которая помогает поставлять ПО и в случае правильной реализации обеспечивает безопасность как ключевой компонент процесса поставки.

Руководство Secure Software Factory Reference Architecture (<https://clck.ru/3CKUc2>) строится на прежних публикациях CNCF, таких как Cloud Native Security Best Practices (<https://clck.ru/3CKUdt>) и Software Supply Chain Best Practices. Эталонная архитектура выделяет существующие инструменты с открытым исходным кодом с акцентом на безопасность. Она также выстраивается вокруг четырех комплексных принципов:

- защита в глубину;
- подписание и верификация;
- аналитика метаданных артефактов;
- автоматизация.

Каждый из них необходим для обеспечения безопасной доставки ПО, начиная с его зарождения и до ввода в эксплуатацию.

Эталонная архитектура также дает понять, что основное внимание уделяется провенансу кода и процессам сборки, а не сканированию и подписанию кода. Это связано с тем, что такие методы, как SAST/DAST, опираются на проверку провенанса и доверенности источника (например, идентификаторы, привязанные к живому пользователю или устройству), что является ключом к обеспечению надежности провенанса.

Каждый субъект в защищенной фабрике ПО (secure software factory, SSF) имеет зависимости, которые могут относиться как к более обширным системам IAM организации, так и к управлению исходным кодом или зависимости нижестоящих потребителей от самой SSF, необходимой для аттестации и подписания используемых артефактов.

В SSF есть несколько видов компонентов, одни из которых считаются ключевыми, другие — компонентами управления, а третьи — компонентами распространения. Ключевые отвечают за получение входных материалов и их использование для создания выходных артефактов. Компоненты управления обеспечивают, чтобы SSF выполнялась в соответствии с установленными политиками. И наконец, компоненты распространения перемещают продукцию фабрики для последующего безопасного потребления.

Ключевые компоненты

К ключевым компонентам относятся платформа планирования и оркестровки, фреймворк пайплайна и инструменты, а также среды сборки. Все компоненты SSF используют платформу и связанную с ней оркестровку для выполнения своих задач. Пайплайн и сопутствующий инструментарий помогают наладить рабочий поток для сборки программных артефактов. Руководство подчеркивает, что к пайплайну должны предъявляться те же требования, что и к рабочим нагрузкам. Цель — подчеркнуть, что пайплайн является частью поверхности атаки и в результате компрометации может повлиять на потребителей, как было в случае с SolarWinds. На этом акцентируют внимание и новейшие фреймворки вроде SLSA.

Наконец, у вас есть среда сборки, в которой ваш исходный код преобразуется в машиночитаемые программные продукты, называемые *артефактами*. Зрелые среды сборки устроены так, чтобы предоставлять автоматизированные аттестации в отношении входных материалов, действий и инструментов, используемых во время сборки для проверки целостности этого процесса и его выходных артефактов.

Организации вроде TestifySec (www.testifysec.com) разрабатывают инновационные решения, чтобы помочь организациям выявлять вмешательство в процессы или компрометацию сборки. Ярким примером выступает инструмент Witness, нацеленный на предотвращение вмешательств в материалы сборки и проверку целостности этого процесса на всем пути от источника до цели (<https://clck.ru/3CKV9F>).

Компоненты управления

Компоненты управления включают в себя структуру управления политиками, а также проверяющих и наблюдателей. В контексте SSF ваша структура управления политиками помогает кодифицировать организационные требования и требования безопасности, такие как IAM, назначенные рабочие узлы и авторизованные образы контейнеров. Эти политики будут отличаться для каждой организации ввиду различий в допустимых рисках и используемых регулирующих фреймворках. Система управления политиками особенно важна в условиях активного внедрения концепции нулевого доверия.

Определение того, кому, что и в каком контексте разрешено делать, является ключом к реализации таких принципов нулевого доверия, как предоставление доступа только с минимально необходимыми разрешениями. Вы явно не захотите развертывать контейнеры, полученные от неавторизованных субъектов и недоверенных источников, не имеющие надежной подписи и так далее. Учитывая, что облачная среда зачастую использует контейнеры и инструменты оркестровки вроде системы Kubernetes, вы работаете с проверяющими узлов и рабочих нагрузок и наблюдателями пайплайнов. Они верифицируют подлинность ваших узлов и рабочих нагрузок, а также метаданных, связанных с процессами пайплайна.

Компоненты распространения

Последними ключевыми составляющими SSF выступают компоненты распространения, включая репозиторий артефактов и контроллер доступа. На этапах пайплайна создаются различные артефакты, которые сохраняются в специальном репозитории. Среди них образы контейнеров, манифесты Kubernetes, SBOM и связанные с ними подписи. Все чаще мы наблюдаем использование решений вроде Sigstore для подписания не только кода, но также SBOM и аттестаций. Это подчеркивается в рассмотренном ранее

Плане по повышению безопасности ПО с открытым исходным кодом, разработанном Linux Foundation совместно с OpenSSF (<https://clck.ru/3CKV9c>). Следом за репозиторием артефактов идут контроллеры доступа, которые отвечают за то, чтобы компоненты планирования и оркестровки выполняли только авторизованные рабочие нагрузки. Эти контроллеры обеспечивают соблюдение политик, определяющих допустимые источники для сборки и компоненты для хоста узлов, а также подтверждают, что применяемые компоненты являются доверенными и верифицируемыми.

Переменные и функциональность

Руководство по SSF учитывает тот факт, что входные материалы и создаваемые из них артефакты будут отличаться. Если на вход поступают программные зависимости, исходный код, пользовательские учетные данные, криптографическая информация и определения пайплайна, то на выходе создаются программные артефакты, открытые ключи подписей и документы с метаданными. В руководстве также рассматривается функциональность SSF, включая прохождение проекта через SSF и производство защищенных выходных продуктов и артефактов, прошедших проверку и имеющих достаточный уровень надежности для конечных пользователей.

Подводя итоги

На первый взгляд эталонная архитектура SSF может показаться сложной, и это действительно так. Процесс доставки программного обеспечения в современных облачных средах состоит из множества элементов и сопутствующих процессов, обеспечивающих такой уровень надежности для потребляемых и производимых компонентов, соответствующий допустимым рискам организации.

Эта сложность также подчеркивает, насколько трудно связать все воедино и насколько сильно система может быть подвержена ошибочным действиям и конфигурациям, негативно влияющим на конечных потребителей. Есть известное изречение, что защитникам необходимо всегда все делать правильно, в то время как злоумышленникам достаточно сделать все верно один раз. В сложных облачных средах, изобилующих проблемами с персоналом и когнитивной перегрузкой, это может быть подобно поиску иголки не в стоге сена, а в куче других иголок.

Фреймворк Secure Supply Chain Consumption от Microsoft

Поскольку Microsoft является одним из крупнейших контрибьюторов и потребителей OSS, не удивительно, что эта корпорация разработала собственный фреймворк для обеспечения безопасности цепочек поставок ПО под названием Secure Supply Chain Consumption Framework (S2C2F). О выходе этого фреймворка было объявлено в августе 2022 года, компания также заявила, что использовала его с 2019 года, но внутри организации. Сделав дополнительный шаг, Microsoft в ноябре 2022 года передала S2C2F в распоряжение сообщества, а конкретно организации OpenSSF (<https://clck.ru/3CKVHw>).

Далее мы рассмотрим этот фреймворк, изучим его функции и возможности. По заявлению Microsoft, задача S2C2F состоит в определении основных принципов безопасного использования открытых зависимостей и реализации фундаментальных практик безопасного потребления OSS. Аналогично другим похожим фреймворкам, он подчеркивает важную роль OSS в мире ПО и в других отраслях, а также в контексте повышения продуктивности и развития инноваций. Предоставляемое им руководство разбито на два раздела:

- ориентированный на зрелость без акцента на какие-либо конкретные решения и предназначенный для руководителей служб безопасности;
- раздел реализации, который фокусируется на разработчиках ПО и их коллегам по безопасности, что делает его информативным на уровне организаций и полезным на тактическом уровне.

В целом S2C2F преследует три цели:

- предоставить комплексную программу управления OSS;
- сократить среднее время устранения (mean time to remediate, MTTR) уязвимостей в OSS;
- препятствовать использованию скомпрометированных и вредоносных OSS-пакетов.

Эти три цели работают в комплексе с тремя основными принципами:

- масштаб;
- непрерывное совершенствование процессов;
- контроль всех входных материалов артефактов.

На рис. 7.11 эти цели и принципы показаны наглядно.

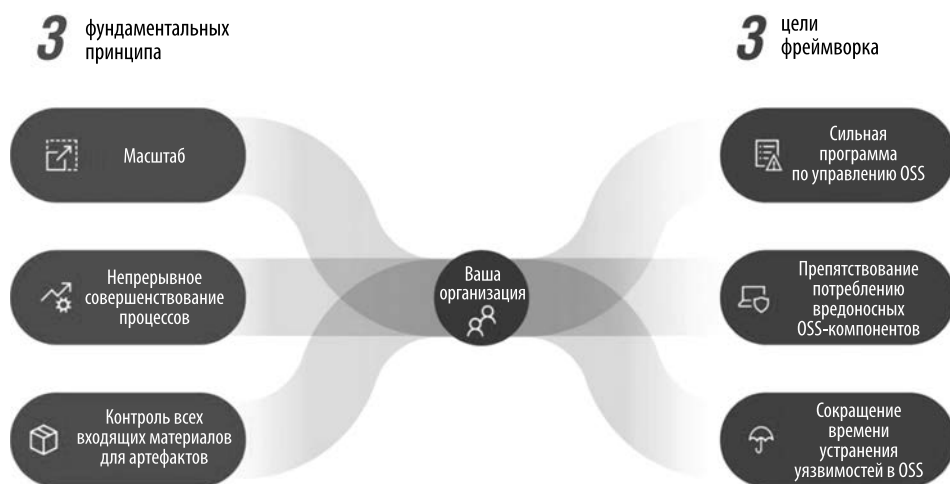


Рис. 7.11

Источник: <https://clck.ru/3CKVJi>. GitHub

И хотя базовые принципы на первый взгляд кажутся простыми, их реализация в корпоративных средах разработки ПО может быть невероятно сложной и трудоемкой. Согласно S2C2F, современные разработчики могут использовать OSS множеством разных способов. Обеспечить защиту OSS в таких условиях помогает разработка стандартизированных методов потребления этих компонентов и контроль за их исполнением со стороны разработчиков.

Крайне важно внедрить управляемый и структурированный подход к потреблению OSS во *всех* командах разработки в организации. Как и в случае с большинством фреймворков безопасности, организации должны следовать S2C2F во всех аспектах, поэтому он сформирован как зрелая модель с акцентом на непрерывное совершенствование процессов. Организации могут устанавливать приоритетные требования для ранней реализации и вносить изменения по мере появления новых угроз и рисков.

Наконец, неэффективно реализовывать практики и усовершенствования для отдельных частей организации, так как в этом случае не покрывается вся поверхности атаки и рисков. По этой причине в S2C2F среди ключевых принципов заложено масштабирование. Одним из конкретных примеров, которые приводит руководство, является установка внутреннего централизованного реестра для использования разработчиками. Причем то же самое рекомендует и руководство NIST SP 800-161 r1.

Как указывает фреймворк, эта модель рушится, как только один разработчик решает подтянуть OSS-компоненты из какого-либо стороннего источника, а не утвержденного реестра, который, в свою очередь, также требует затрат на обслуживание и управление. Создатели S2C2F утверждают, что данный фреймворк разрабатывался для обеспечения безопасного потребления OSS в больших масштабах без централизованного подхода, рекомендуемого в других рассмотренных нами руководствах. Эта модель мышления согласуется с видением компаний, выступающих за децентрализацию.

S2C2F представляет собой комбинацию инструментов и требований, описанных в дорожной карте развития возможностей и ориентированных на безопасное потребление OSS за счет реализации контроля. Руководство S2C2F описывает распространенные угрозы для цепочек поставок OSS, попутно приводя реальные примеры атак и сопоставляя их с конкретными требованиями самого фреймворка. Приводимые примеры включают как случайные уязвимости в OSS-коде или контейнерах, так и намеренно организованные бэкдоры и вредоносные действия в репозиториях и пакетах. Источником информации для руководства выступает работа *Software Supply Chain Threats*, в которой разбираются угрозы со стороны исходного кода, сборки, развертывания, среды выполнения и зависимостей (<https://clck.ru/3CKVKb>). Кроме того, существуют и более безобидные источники угроз, такие как устаревшие OSS-компоненты, которые больше не обслуживаются или имеют уязвимости, которые разработчики не успевают устранять в желаемые для потребителей сроки.

Практики S2C2F

Как уже говорилось, S2C2F включает в себя набор решений и практик, способных помочь организациям обезопасить цепочки поставок OSS. Различного рода руководители, инженеры и специалисты, а также непосредственно директора по информационной безопасности могут обращаться к этим практикам для выявления пробелов в мерах защиты их использования организациями OSS. Далее мы разберем эти практики, чтобы понять их суть.

Начнем с практики «Потребление». Согласно S2C2F, первым шагом защиты цепочек поставок выступает контроль за входными артефактами, к которым относятся упакованные артефакты и артефакты исходного кода. Цель данной практики — позволить организациям поставлять активы, несмотря на возможную компрометацию или недоступность внешних OSS-источников. К упакованным артефактам относятся элементы из репозиторий пакетов Linux и реестров Open Container Initiative (OCI). Компаниям

нужно создавать посредников для получения чего-либо из этих внешних источников и сохранять копии всех необходимых материалов. Что касается артефактов исходного кода, практика рекомендует создавать внутренние зеркала для внешних репозиторий и кэшировать пакеты локально, чтобы иметь возможность сохранять работоспособность в случае инцидента, демонстрируя тем самым устойчивость бизнеса. Это также позволяет организациям проводить сканирование безопасности и даже вносить исправления в исходный код либо локально, либо отправляя их в исходный репозиторий, в зависимости от ситуации.

После «Потребления» идет практика «Сканирование». Ее цель — сделать так, чтобы потребляющая организация и отдельные лица могли видеть уязвимости, связанные с OSS-компонентом. В S2C2F утверждается, что доверие создается за счет видимости в схеме безопасности используемых компонентов. Сканируя такой компонент, организации могут выявить недочеты в его конфигурации, уязвимый код, присутствующие известные уязвимости или лишний код, который может увеличить поверхность атаки. Существует множество открытых и проприетарных инструментов сканирования, с помощью которых организации способны реализовать эту практику.

После потребления и сканирования OSS-компонента необходимо провести его инвентаризацию, чтобы понимать, где их компоненты находятся в рабочих средах. В качестве прекрасного примера S2C2F приводит инцидент с Log4j. Компании, в которых налажены точные инвентаризационные описи OSS-компонентов, смогли гораздо эффективнее отреагировать на это происшествие в сравнении с теми, у которых такой описи не было. Поскольку в открытых компонентах постоянно появляются новые критические уязвимости, организации, ведущие их опись, могут быстро реагировать, исправляя или отбраковывая как уязвимые, так и скомпрометированные компоненты и системы.

Сканирование и инвентаризация потребляемых компонентов являются прекрасным началом, но, как нам известно, ПО портится так же быстро, как молоко, — то есть все компоненты, находящиеся в рабочей среде, неизбежно требуют обновлений. Хорошо известно, что при публичном раскрытии уязвимости зачастую между защитниками и хакерами возникает гонка: первые спешат уязвимость закрыть, а вторые — успеть ее эксплуатировать.

По этой причине практика обновления и исправления всех уязвимых OSS-компонентов в организации является критической. Важно отметить, что OSS-компоненты обычно обслуживаются энтузиастами, не имеющими соглашений об уровне услуг (SLA) для устранения уязвимостей. Поэтому

потребители должны быть готовы реализовывать виртуальные патчи и прочие меры противодействия, если мейнтейнер OSS-компонента не предоставит непосредственного исправления.

Наличие стандартизированного процесса потребления и использования OSS — это одно, но вам также необходимо проводить аудит рабочих сред для выявления компонентов, отклоняющихся от этого процесса или представляющих риск для организации.

Практика «Аудит» подразумевает проверку OSS-компонентов на предмет прохождения ими стандартизированного процесса потребления, сканирования и инвентаризации. В противном случае система контроля начнет давать сбои.

И хотя аудит — это здорово, организациям также нужна возможность как-то обрабатывать неконтролируемые OSS-компоненты, которые она обнаруживает. По этой причине следующей практикой идет «Обеспечение соблюдения». Если разработчик обойдет установленные процессы и внесет неконтролируемые OSS-компоненты из недоверенных источников, тогда организация должна иметь возможность устранить этот недочет. В качестве примера в руководстве приводится контроль за перенаправлением DNS-трафика или установка шлюзов в процессе сборки для разрыва этого процесса в случае внедрения не одобренных OSS-компонентов. Все чаще злоумышленники пытаются скомпрометировать процесс сборки или инфраструктуру, чтобы добраться до исполняемых файлов и артефактов.

Следующим шагом в руководстве является практика воссоздания каждого OSS-артефакта, развертываемого из исходного кода. Она может стать ключевой, если злоумышленники умудрятся внедрить зараженный OSS-пакет с бэкдором или внести неавторизованные изменения в генерируемые исполняемые файлы во время сборки.

В руководстве S2C2F подчеркивается, что для ключевых артефактов или критических для бизнеса активов может потребоваться создание цепочки ответственности, начиная от исходного кода и заканчивая каждым артефактом, используемым для создания рабочей версии приложения или сервиса. Здесь акцент делается на предоставлении разработчикам, зависящим от ключевых OSS-компонентов, возможности получать исходный код, пересобирать его, при необходимости изменять, например путем подписания, а затем кэшировать обновленный артефакт для внутреннего использования.

Данная практика указывает конкретно на *воссоздаваемые сборки*, то есть методы, используемые в разработке ПО для создания независимо верифицируемого

пути от исходного до исполняемого кода (<https://clck.ru/3CKVLe>). В результате мы получаем идентичные копии всех релевантных артефактов. Вы заметите, что эта практика играет важную роль и в руководстве SLSA, в частности за счет более высокого уровня ее зрелости.

В заключении в S2C2F приводится практика исправления проблемы и внесения изменений в основной репозиторий. В S2C2F говорится: «Я могу лично пропатчить, собрать и развернуть любой внешний артефакт в течение трех дней с момента обнаружения проблемы и конфиденциально отправить это исправление вышестоящему мейнтейнеру». Это очень зрелый принцип, так как он переносит ответственность с мейнтейнера на потребителя ПО, выступающего активным контрибьютором OSS-проекта или компонента. За реализацию именно такой схемы выступали исследователи, о которых мы говорили в других частях книги, например Чинмайя Шарма в своей работе *A Tragedy of the Digital Commons*.

Тем не менее руководство указывает, что данная модель должна применяться только в крайних случаях и для временного устранения угрозы, когда вышестоящий мейнтейнер не может предоставить исправление в приемлемые для вас сроки. В этом случае потребитель как бы признает, что мейнтейнер может предпринять необходимые действия, но при этом хочет по возможности помочь, что и является сутью идеологии открытого сообщества. В S2C2F также приводится несколько способов, которыми организации могут вносить вклад в OSS-сообщество, такие как прямая финансовая помощь или внесение средств в фонды, участие в программах поощрения, распространение лучших практик и активное участие в ключевых OSS-проектах.

Руководство по реализации S2C2F

По аналогии с другими фреймворками, такими как SLSA, S2C2F выстроен вокруг четырех уровней зрелости, начиная с минимальной системы контроля OSS и заканчивая продвинутой защитой от угроз. При этом каждый уровень несет собственный набор мероприятий, перечисленных на рис. 7.12.

Уровень 1 включает базовые меры, такие как внутреннее кэширование пакетов, проведение простой инвентаризации и фундаментального сканирования, а также обновление OSS в вашей среде. Уровень 2 развивает все эти процессы, сокращая среднее время исправления уязвимых компонентов и реагирования на инциденты (MTTR). Уровень 3 подразумевает создание черного списка известных вредоносных или ненадежных компонентов и источников, сканирование на наличие вредоносного ПО в OSS, а также отслеживание

провенанса. Последний, уровень 4, уже считается амбициозным, то есть большинству организаций такая степень зрелости окажется не по силам, за исключением самых богатых и чувствительных к взломам. На этом уровне реализуется воссоздание OSS в доверенной инфраструктуре, а также создание и цифровое подписание SBOM для всех воссозданных компонентов.

Уровень 1	Уровень 2	Уровень 3	Уровень 4
 Программа минимального контроля OSS • Использование менеджеров пакетов • Локальное копирование артефактов • Сканирование на известные уязвимости • Инвентаризация OSS • Ручное обновление OSS	 Безопасное потребление материалов и уменьшение MTTR • Сканирование на предмет завершения жизненного цикла • Наличие плана реагирования на инциденты • Автоматическое обновление OSS • Уведомление об уязвимостях в пул-реквестах • Проверка, чтобы потребление происходило утвержденным способом • Проверка целостности OSS • Защита конфигурации исходных файлов пакетов	 Защита от вредоносного ПО и атаки нулевого дня • Черный список • Клонирование исходного кода OSS • Сканирование на вредоносное ПО • Превентивные ревью безопасности • Отслеживание провенанса OSS • Потребление только из утвержденных источников	 Продвинутая защита от угроз • Проверка SBOM потребляемых OSS • Воссоздание OSS в доверенной инфраструктуре • Цифровое подписание воссозданных OSS • Генерация SBOM для воссозданных OSS • Цифровое подписание защищенных SBOM • Реализация исправлений

Рис. 7.12

Источник: <https://clck.ru/3CKVMT>. GitHub

Но, несмотря на всю полезность этих уровней зрелости и сопутствующих мер, организациям следует понимать, как правильно проводить собственный анализ в соответствии с ними. S2C2F предлагает два основных шага: подготовка к анализу и его фактическое выполнение. В ходе подготовки необходимо убедиться, что организация может эффективно взаимодействовать с разработчиками и инженерами по вопросам инструментов, возможностей и рабочих потоков. В руководстве приводятся несколько примеров вопросов, которые можно использовать в рамках анализа. Эти вопросы проясняют, как OSS потребляется в текущих проектах, откуда поступает, а также уточняют существующие системы контроля и практики безопасности, о которых говорилось ранее в контексте принципов, целей и уровней зрелости фреймворка.

После анализа организации должны получить представление об их текущем уровне зрелости в отношении рекомендуемых возможностей и практик. Это понимание позволит им спланировать дальнейшие улучшения для ликвидации обнаруженных недочетов или слабых сторон. Затем организации смогут делать целевые вложения и реализовывать инициативы для расширения

наиболее критических, по их мнению, возможностей и практик, исходя из поставленных задач и допустимых рисков.

S2C2F излагает требования фреймворка в виде матричной таблицы, которая включает практику, ID требования, уровень зрелости, название требования и его пользу для организации. Учитывая, что инструментарий — это ключевой компонент, который должен отвечать многим требованиям и уровням зрелости, руководство также предоставляет справочную информацию и рекомендации, включая существующие бесплатные инструменты, инструменты Microsoft и решения, над которыми Microsoft также работает.

S2C2F является желанным дополнением в контексте обеспечения безопасности цепочек поставок, и передача Microsoft этого фреймворка проекту OpenSSF показывает приверженность корпорации к сообществу открытого ПО, а также стремление помочь отрасли с решением этой проблемы.

OWASP Software Component Verification Standard

В отличие от фреймворков вроде SSDF института NIST, Software Component Verification Standard (SCVS) реализуется сообществом и акцентируется на защите цепочек поставок ПО. В нем основной упор делается на снижение рисков в цепочках поставок за счет выявления релевантных мероприятий, систем защиты и лучших практик, которые можно в ходе этого процесса реализовать. SCVS также неоднократно упоминается в руководстве NIST SSDF, о котором речь пойдет в главе 8. Версия 1.0 SCVS вышла в 2020 году под руководством Стива Спрингетта и при содействии нескольких контрибьюторов и ревьюеров.

SCVS разбит на шесть разделов, каждый из которых содержит определенные техники защиты для различных аспектов верификации программных компонентов и процессов:

- инвентаризация;
- SBOM;
- среда сборки;
- управление пакетами;
- анализ компонентов;
- происхождение и провенанс.

Уровни SCVS

По аналогии с SLSA SCVS выстроен по уровням — чем он выше, тем больше в нем заключено мер защиты. Всего в этом фреймворке три уровня. Уровень 1 предназначен для требований с низкой степенью надежности и базовых мер защиты. Уровень 2 — для умеренно чувствительного ПО и ситуаций, в которых требуется повышенная строгость. А уровень 3 ориентирован на высоконадежные среды, работающие с чувствительными данными и являющиеся критическими для реализации миссии организации. Далее мы более подробно разберем каждый из этих уровней.

Уровень 1

Первый уровень закладывает основу для всех последующих и включает ряд фундаментальных мер, в частности ведение детальной инвентаризационной описи компонентов. В мире ПО это зачастую означает создание SBOM для лучшего понимания всех используемых в вашем ПО компонентов. Данный уровень также включает применение непрерывной интеграции (CI) с целью создания повторяемых сборок. Наиболее популярными платформами в этом случае выступают CircleCI, Jenkins, GitLab и GitHub. Непрерывная интеграция предполагает совмещение всех рабочих копий кода в общем основном репозитории, для чего используются различные технологии и практики. В качестве технологий выступают CI-платформы или системы сборки, а в качестве практик — всевозможные культурные аспекты вроде выработки разработчиками привычки часто интегрировать изменения кода. Наконец, первый уровень призывает выполнять анализ сторонних компонентов с помощью публично доступных инструментов и средств сбора информации. Эти инструменты помогают проводить анализ и выявлять публично раскрытые уязвимости, о чем мы более подробно поговорим в последующих разделах.

Уровень 2

Второй уровень SCVS ориентирован на программно-нагруженные организации с достаточным уровнем зрелости в управлении рисками. Он строится на мерах защиты первого уровня и сопряжен с появлением дополнительных стейкхолдеров и участников, в частности нетехнических профессионалов, которые могут быть задействованы в процессах снабжения и заключения контрактов.

Уровень 3

Третий уровень является наиболее жестким. Он акцентирует внимание на проверяемости и сквозной прозрачности всей цепочки поставок и зачастую применяется в наиболее контролируемых и чувствительных отраслях в самых зрелых компаниях.

OWASP рекомендует организациям использовать SCVS в целях поэтапного повышения безопасности цепочек поставок, а также подстраивать эту схему под наиболее актуальные меры защиты и требования соответствия. Такое видение является уникальным и ориентировано на гибкость, что отличает его от ригидной обобщенной основы, которая зачастую реализуется другими фреймворками и стандартами безопасности.

Познакомившись с уровнями, пора перейти к рассмотрению разделов и связанных с ними мер защиты.

Инвентаризация

Наличие точной инвентаризационной описи всегда выступало критической мерой защиты, о чем говорится, например, в руководстве *Critical Security Control om CIS*. Но, несмотря на свою широкую известность, эта практика всегда представляла для организаций сложности, поэтому неудивительно видеть «Инвентаризацию» первым разделом в стандарте SCVS. В контексте SCVS упор делается на ведении списка всех компонентов, используемых в создании ПО, а также на рекомендуемых мерах защиты для каждого отдельного приложения, составлении описей по всей организации и повышении прозрачности потребляемого ПО. При этом инвентаризация должна покрывать как собственные, так и сторонние программные компоненты, в том числе открытый код.

Первый уровень «Инвентаризации» фокусируется на знании прямых и транзитивных компонентов в процессе сборки, а также на использовании менеджеров пакетов для управления сторонними исполняемыми компонентами. Кроме того, данный уровень включает ведение полной инвентаризации сторонних компонентов в машиночитаемом формате и генерацию SBOM для публичных и коммерческих приложений. Второй уровень основан на этих мерах и также требует создания SBOM для новых программных продуктов. Это пример защиты, которая выходит за рамки обязанностей специалистов по кибербезопасности и включает уже нетехнических участников, вроде снабженцев, запрашивающих эти артефакты от поставщиков.

Третий уровень инвентаризации привносит более жесткие меры защиты, такие как непрерывное ведение SBOM для всех систем, в том числе уточнение типов компонентов. Этот уровень требует не только знания типа компонентов, но также понимания их функций и, самое главное, источника происхождения. В сфере поставок ПО это обычно называется *провенансом*.

Перечень программных компонентов ПО и их зависимостей (SBOM)

SBOM в SCVS выступает ключевым элементом и мерой защиты. SCVS утверждает, что организации, реализующие зрелые практики разработки, создают SBOM в рамках своего пайплайна сборки, причем в машиночитаемом формате. SCVS учитывает, что существуют разные форматы SBOM, такие как CycloneDX и SPDX, и что организациям нужно ориентироваться на тот, который больше им подходит, а иногда и не на один, чтобы иметь возможность работать с разносторонней экосистемой поставщиков.

Первый уровень здесь описывает ряд фундаментальных мер защиты, таких как наличие структурированного и машиночитаемого SBOM, присваивание SBOM уникальных идентификаторов и использование метаданных, например временных меток. Основываясь на этих техниках, SCVS призывает создавать SBOM с полноценными инвентаризационными описями всех компонентов и анализировать эти SBOM на присутствие любых актуальных уязвимостей. Наконец, фреймворк требует следить, чтобы идентификаторы компонентов по возможности происходили из родной экосистемы и несли в себе точную лицензионную информацию.

Второй уровень расширяет требования к информативности и достоверности SBOM. Теперь эти списки должны не просто быть подписаны издателем, поставщиком или сертифицирующей инстанцией, но также проходить проверку подписей. Данный уровень также призывает к составлению в рамках SBOM точных описей всех тестируемых компонентов приложения и сопровождению их метаданными. Наконец, указанные в SBOM компоненты должны иметь валидные ID или положения лицензии SPDX, если таковое требование применимо.

Третий уровень добавляет дополнительные меры защиты. К ним относятся определение происхождения компонентов в машиночитаемом формате, таком как PURL, а также присутствие валидных записей об авторских правах

для всех определенных в SBOM компонентов. Кроме того, этот уровень требует наличия подробной информации о провенансе и происхождении компонентов, а также хеширования файлов с помощью SHA-256.

Среда сборки

SCVS признает сложность современных сред сборки, включая репозитории исходного кода и пакетов, процессы CI/CD, мероприятия по тестированию, а также сопутствующую сетевую инфраструктуру и сервисы, которые делают возможными сборку и поставку ПО. В руководстве отмечается, что каждый из этих субъектов и мероприятий в среде сборки и пайплайне несет собственные потенциальные векторы атаки, а также возможность обычных сбоев и ошибок в конфигурации. Эта проблема наиболее ярко отражена в рамках фреймворка SLSA, в котором описываются различные векторы атаки, включая потенциальные точки взлома в современных пайплайнах и средах сборки.

Данный раздел — самый обширный в стандарте SCVS. Он содержит более 20 мер. Первый уровень включает фундаментальные техники, такие как реализация повторяемых процессов сборки, ведение инфраструктурной документации и использование пайплайнов CI. Среди прочих мер защиты этот уровень требует, чтобы при сборке приложений использовался исходный код только из систем контроля версий и чтобы изменения исполняемых файлов и исходного кода в процессе сборки были прозрачными и понятными. Наконец, он призывает к созданию контрольных сумм для всех прямых и сторонних компонентов, задокументированных для каждой сборки.

На втором уровне управления средой сборки добавляются такие меры, как запрет в пайплайнах сборки корректировок, выходящих за рамки выполняющей сборку задачи, а также запрет на изменение настроек управления пакетами или выполнение кода вне контекста скрипта сборки. Помимо этого, он предписывает внедрять в пайплайн аутентификацию, авторизацию и предустановленный черный список компонентов. Учитывая, что сам пайплайн тоже может быть скомпрометирован по причине устаревших систем и ПО, второй уровень требует наличия графика обслуживания используемого в нем стека технических средств. Наконец, существует требование, чтобы для всех компонентов, доступных и доставляемых извне, когда они упакованы или поступают по распределенной системе, генерировались контрольные суммы.

Заключительный, третий уровень содержит такие условия, как разделение обязанностей по изменению системных настроек в пайплайне сборки и ведение проверяемого журнала аудита для всех изменений системы и процессов сборки. На этом уровне также действуют требования по мониторингу компиляторов, систем контроля версий, утилит разработки и наборов разработки на предмет вмешательств и внедрения вредоносного кода. Стандарт предписывает выявлять и удалять из приложения неиспользуемые прямые и транзитивные компоненты, уменьшая тем самым поверхность атаки и минимизируя число возможных векторов нападения.

Управление пакетами

SCVS отмечает, что современные компоненты с открытым исходным кодом зачастую публикуются в специализированных репозиториях вроде Maven, .NET и NPM. Организации ориентируются на создание внутренних репозиториях не только для собственных, но и для доверенных сторонних компонентов, причем подобный подход рекомендуется в таких руководствах, как NIST 800-161r1 и Secure Guidance for Developers от NSA. SCVS подчеркивает, что менеджеры пакетов часто вызываются во время сборки. Их использование обеспечивает немало преимуществ, но и несет риски для безопасности.

Средства контроля первого уровня в управлении пакетами требуют отслеживать, чтобы компоненты поступали из репозитория пакетов, а в случае OSS-компонентов — чтобы их содержимое происходило из доверенного источника. Репозиторий также должен поддерживать аудит/логирование при обновлении этих компонентов и проверку целостности пакетов при их получении из удаленного репозитория или файловой системы. Менеджеры пакетов также должны применять шифрование, например протокол TLS, при обмене данными и отслеживать, чтобы цепочки TLS-сертификатов проверялись или вели к безопасному отказу в случае, когда осуществить проверку невозможно. Безопасный отказ означает, что, если цепочку TLS-сертификатов нельзя подтвердить, система по умолчанию переходит в безопасное состояние, не позволяя выполнить какие-либо функции или действия. Менеджеры пакетов также блокируют исполнение кода компонентов, а данные об установке пакетов должны быть доступны в машиночитаемом формате.

Расширяя меры защиты первого уровня, второй уровень требует строгой аутентификации для репозитория пакетов, в том числе с помощью MFA.

Учитывая недавние инциденты, связанные с компрометацией MFA (<https://clck.ru/3CKVPb>), организациям, возможно, стоит обратить внимание MFA с защитой от фишинга (<https://clck.ru/3CKVQY>). Помимо усиленной аутентификации, второй уровень призывает предоставить репозиториям пакетов возможность создавать отчеты об инцидентах безопасности, а также уведомлять издателей об обнаруженных проблемах. Кроме того, он предписывает, чтобы репозитории пакетов могли проверяемым образом сопоставлять версии компонентов с исходным кодом в VCS, а также обязательное подписание кода при публикации пакетов в рабочем репозитории.

Наконец, третий уровень в группе управления пакетами добавляет ряд дополнительных контрольных мер. Сюда входит обеспечение того, чтобы компоненты репозитория пакетов публиковались с использованием многофакторной аутентификации и автоматизированного отчета об инцидентах, включая уведомление пользователей о проблемах безопасности. Здесь также приводится требование, чтобы репозиторий пакетов выполнял анализ состава ПО до публикации компонентов, делая результаты доступными для потребителей.

Анализ компонентов

Пятым разделом в SCVS выступает «Анализ компонентов» — процесс выявления потенциальных областей риска при использовании OSS и сторонних компонентов, к которым относятся как прямые, так и транзитивные. Организациям нужно понимать все риски, связанные с OSS и другими компонентами, которые они применяют в своих приложениях и системах. Такие компоненты задействованы повсеместно, и их можно встретить в большинстве современных приложений. Организациям необходимо осознать, с какими компонентами они работают и какие с этими компонентами сопряжены риски и уязвимости.

Для выяснения присущих компонентам уязвимостей организации зачастую прибегают к таким источникам, как NVD. Помимо известных уязвимостей, организации также должны быть знакомы и с актуальными версиями компонентов, типами компонентов, их назначением и количеством. Речь идет об отслеживании устаревших компонентов, которые могут быть уязвимы, а также о типе компонента и последствиях и рисках, связанных с его обновлением. Организациям необходимо осознать назначение компонента, чтобы выявлять дублирующиеся и оставлять только наиболее качественные для минимизации рисков. Им также нужно понимать количество компонентов

в инвентаризационных описях, поскольку в случае неконтролируемого разрастания управлять ими становится все сложнее. Наконец, организациям стоит разобраться с типами лицензий, связанных с используемыми ими компонентами, поскольку могут существовать правила распространения, ограничения и нежелательные для бизнеса конфликты.

Первый уровень в этом разделе включает такие аспекты, как возможность проверки компонентов с помощью линтеров и статического анализа. Акцент здесь делается на автоматизации процессов выявления в компонентах публично раскрытых уязвимостей, а также неуказанных версий или устаревших компонентов. Организации также должны автоматизировать процесс выявления количества используемых компонентов и связанных с ними лицензий.

Второй уровень требует дополнительной проверки компонентов с помощью линтеров и статического анализа, в том числе при любых их обновлениях, а также автоматизации процесса определения типов используемых компонентов.

Ключевое отличие третьего уровня состоит в автоматизации процесса выявления подтвержденной эксплуатируемости, завершения срока службы или поддержки компонентов, а также их функциональности.

Происхождение и провенанс

Эти аспекты крайне важны, поскольку сложно доверять или осознавать риски, связанные с использованием ПО, не зная качества потребляемого материала, его происхождения и цепочки ответственности по всем этапам поставки.

Первый уровень в этом разделе включает документирование провенанса изменяемых компонентов, а также их анализ с той же строгостью, что и неизменяемых. Помимо этого, здесь применяются меры для выявления рисков, связанных с изменяемыми компонентами и их вариациями.

Второй уровень привносит меры защиты, нацеленные на документирование проверяемой информации о происхождении изменений компонента и уникальную идентификацию компонентов, подвергнутых преобразованию.

Третий уровень привносит единственную меру, включающую отслеживание цепочки ответственности для исходного кода и исполняемых компонентов.

Политика использования открытых компонентов

В SCVS этот раздел формально не относится к мерам защиты, но в нем описываются более обширные рекомендации для организаций в отношении использования OSS. Несмотря на то что потребление OSS процветает и свойственно для большого количества современных приложений, многие организации еще не наладили в отношении процессов его использования зрелые уровни защиты.

SCVS рекомендует устанавливать политику OSS, которая будет поддерживаться и подкрепляться различными стейкхолдерами. Эта политика должна охватывать критические вопросы, связанные с OSS, такие как понимание возраста применяемых компонентов, определение правил работы со старыми и новыми версиями, а также непрерывное обновление компонентов с помощью средств автоматизации. Организациям следует разрабатывать руководства для исключения компонентов с известными уязвимостями или, по крайней мере, для поддержания приемлемого уровня рисков. Для опасных компонентов должен быть установлен критерий MTTR, а для устаревших — ограничения по их использованию ввиду характерных рисков. Некоторые организации могут даже создавать списки компонентов, запрещенных к использованию по причине наличия уязвимостей, угрозы национальной безопасности и других факторов.

Как уже говорилось, несмотря на то что популярность применения OSS растет, многие компании не уделяют достаточно времени установке собственных политик и руководств в отношении открытого ПО. Однако в последнее время мы стали замечать, что ситуация постепенно меняется, и некоторые организации выходят за рамки простого использования политик и внедряют специальные подразделения по управлению ПО с открытым исходным кодом (open source program office, OSPO, <https://clck.ru/3CKVSD>), нацеленные на управление потреблением OSS для создаваемых ими продуктов.

OpenSSF Scorecard

Более десяти лет назад Марк Андрессен сказал: «Программное обеспечение поглощает мир» (<https://clck.ru/3CKVTm>). Сегодня ПО проникло практически во все сферы жизни общества. Причем оно, бесспорно, является критическим для современной экономики и даже национальной безопасности.

В свете этих реалий можно сказать, что ПО с открытым исходным кодом поглощает индустрию программного обеспечения. По примерным оценкам таких групп, как Linux Foundation, свободное и открытое ПО (free and opensource software, FOSS) составляет от 70 до 90 % любого современного программного продукта (<https://clck.ru/3CKVUm>). И дело не только в насыщении продуктов открытым ПО, но и в том, что IT-лидеры также более склонны работать с поставщиками, участвующими в жизнедеятельности OSS-сообщества (<https://clck.ru/3CKVVd>).

Существует немало причин для активного использования OSS. Это и гибкость, и экономия, и инновации, и даже повышение безопасности за счет возможности проводить ревью кода большим «числом глаз», особенно в случае крупных проектов. Тем не менее OSS имеет и проблемы, включая уязвимости и CVE, присваиваемые подверженному им коду. CVE — это проект MITRE, нацеленный на выявление, определение и каталогизацию публично раскрытых уязвимостей кибербезопасности (cve.mitre.org). Однако, как отмечается в работе *Software Supply Chain Best Practices on CNCF*, CVE являются «запоздалой метрикой», то есть этот каталог перечисляет уязвимости, которые были публично раскрыты. Они выступают лишь одним из потенциальных рисков, связанных с ПО.

По этой причине организациям рекомендуется использовать и другие методы для оценки состояния безопасности конкретного OSS-проекта, который они потребляют. Одним из наиболее ярких инструментов для этого является OpenSSF Scorecard (<http://openssf.org>), о котором мы поговорим далее.

Система оценки безопасности проектов с открытым исходным кодом

В конце 2020 года OpenSSF объявила о запуске проекта Scorecard, предназначенного для автоматической генерации показателей безопасности OSS-проектов, чтобы потребителям было проще принимать решения об их использовании с учетом рисков. Организации все чаще потребляют OSS-зависимости, но определение сопряженных с этим рисков в масштабах всей экосистемы ПО остается преимущественно ручным процессом. Проект Scorecard нацелен на частичное упрощение данной задачи с помощью автоматизированной эвристики и проверок безопасности, выдающих показатель в диапазоне от 0 до 10 (рис. 7.13).

Наименование	Описание	Уровень риска
Исполняемые артефакты	Есть ли в этом проекте зарегистрированные исполняемые файлы?	Высокий
Защита веток	Использует ли проект защиту веток?	Высокий
Тесты CI	Выполняет ли этот проект тесты CI, например, с помощью GitHub Actions, Prow?	Низкий
Лучшие практики CII	Имеет ли проект значок CII Best Practices?	Низкий
Код-ревью	Требуется ли в проекте выполнение код-ревью перед слиянием кода?	Высокий
Контрибьюторы	Участвуют ли в проекте контрибьюторы хотя бы из двух организаций?	Низкий
Опасный рабочий поток	Избегает ли проект опасных паттернов программирования в рабочих потоках GitHub Actions?	Критический
Инструмент обновления зависимостей	Используются ли в проекте инструменты для обновления зависимостей?	Высокий
Фаззинг	Используются ли в проекте инструменты фаззинга, например OSS-Fuzz?	Средний
Лицензия	Заявляет ли проект о лицензии?	Низкий
Обслуживание	Обслуживается ли проект?	Высокий
Закрепленные зависимости	Есть ли у проекта объявленные и закрепленные зависимости?	Средний
Пакеты	Создает и публикует ли проект официальные пакеты из пайплайнов CI/CD, например через GitHub Publishing?	Средний
SAST	Использует ли проект инструменты статического анализа кода, например CodeQL, LGTM, SonarCloud?	Средний
Политика безопасности	Есть ли у проекта политика безопасности?	Средний
Подписание релизов	Сопровождает ли проект свои релизы криптографическими подписями?	Высокий
Разрешения для токенов	Объявляет ли проект токены рабочих потоков GitHub как «только для чтения»?	Высокий
Уязвимости	Есть ли в проекте неустраненные уязвимости по базе данных OSV?	Высокий

Рис. 7.13

Проект Scorecard огромен. Его специалисты еженедельно сканируют более миллиона наиболее критических OSS-проектов на основе прямых зависимостей и публикуют результаты в виде открытого набора данных. Организации могут использовать этот открытый набор данных, а также выполнять тест

Scorecard в отношении собственных проектов GitHub, используя GitHub Actions. В дальнейшем в случае изменения репозитория GitHub Actions уведомляет об этом его мейнтейнеров.

В проекте Scorecard применяется шкала оценки со следующими значениями: «критический», «высокий», «средний» и «низкий». Она отражает уровень серьезности, понятный для многих. Этот инструмент прогоняет стандартный список проверок в отношении всех проектов, на которые настроен. Ими могут являться как публичные, так и ваши собственные репозитории GitHub. Выполняемые проверки включают фундаментальные практики безопасности, такие как защита веток и криптографическое подписание релизов. Для обнаружения неисправленных уязвимостей в Scorecard используется база данных OSV (<http://osv.dev>) — распределенная БД для OSS, работающая с форматом OpenSSF OSV. Эта БД представляет собой собрание других баз данных уязвимостей, применяющих схему OSV, например GHSA и GSD. При этом OSV поддерживает как API, так и инструменты командной строки (CLI) для сканирования SBOM в форматах CycloneDX или SPDX, о которых мы говорили в главе 3.

Как организации могут использовать Scorecard

Как уже неоднократно отмечалось, организации все чаще прибегают к OSS. Тем не менее такие аспекты, как реализация подходящего контроля и управления рисками при потреблении открытых компонентов, все еще находятся на ранней стадии развития. При этом наблюдается значительный интерес к повышению устойчивости цепочек поставок ПО и внедрению зрелых практик обеспечения их безопасности. В свете этого появилось множество всевозможных руководств и фреймворков, включая проекты Cybersecurity Supply Chain Risk Management Practices for Systems and Organizations и SSDF от NIST, OSS Security Mobilization Plan от OpenSSF, SLSA и многие другие. Все они касаются необходимости управления потреблением OSS организациями с целью обеспечения соответствия этого процесса допустимым уровням рисков.

На первый взгляд звучит просто, однако, взглянув на масштаб экосистемы OSS-проектов и компонентов, мы понимаем, что это не так. Scorecard предоставляет автоматизированный способ получения информации о рисках и уровнях безопасности с учетом миллиона ведущих OSS-проектов. При этом организациям также доступен этот инструмент для их собственного ПО и проектов.

Организации могут применять Scorecard посредством CLI для управления проектами, которые им не принадлежат, а также через менеджеры пакетов вроде npm, PyPi или RubyGems. Scorecard в том числе доступен в виде контейнера Docker, то есть может быть развернут еще и таким образом.

Разработчики Scorecard проводят еженедельные встречи и активно ведут канал в Slack. Координаторами проекта выступают специалисты из Google, Datto, Cisco и других компаний. С момента своего появления Scorecard набрал высокую популярность, и его добавили в закладки уже более 3000 пользователей. По мере того как организации все больше внимания уделяют развитию практик управления потреблением OSS, популярность этого проекта будет неуклонно расти. У организаций и отдельных контрибьюторов также есть возможность принять в нем участие, включая отправку собственных проверок, которые могут быть учтены при анализе показателей безопасности. При этом организации могут настраивать использование Scorecard под себя и выполнять только конкретные проверки, потенциально соответствующие их требованиям или требованиям отрасли.

Scorecard предоставляет возможность автоматизировать сложные аналитические методы, ручная реализация которых является непрактичной, будь то публичные проекты или внутренние процессы. Хорошо известно, что, несмотря на привносимую OSS ценность и инновационность, большинство FOSS-проектов страдают от недостатка кадров и управляются недостаточно компетентными добровольцами.

И это говорит не о том, что организации не должны использовать OSS-продукты, а о необходимости строгого контроля за потребляемыми проектами и отслеживания представляемых ими рисков. Scorecard прекрасно компенсирует эту потребность, причем весьма удобным образом. Он берет на себя анализ OSS-проектов на предмет проблем с безопасностью, связанных с такими практиками, как подписание артефактов, SAST и другие, которые уже продвигаются представителями частного и государственного секторов.

Предстоящий путь

Несмотря на то что OSS обладает значительными преимуществами, многие исследования показывают, что разработчики FOSS часто пренебрегают вопросами безопасности. Например, исследование OpenSSF и Лаборатории по инновациям Гарвардского университета показало, что средний разработчик FOSS тратит лишь 2,3 % своего времени на повышение безопасности кода (<https://clck.ru/3CKVWd>). Это подчеркивает необходимость принимать

различные меры безопасности при потреблении OSS-компонентов. К ним относится анализ компонентов перед потреблением, а также использование SBOM для выявления связанных с ними уязвимостей и определения места расположения компонентов в инфраструктуре корпорации. Данные действия помогают эффективнее реагировать на инциденты вроде Log4j.

Для получения более конкретного руководства организации могут обратиться к практикам, которые NIST рекомендует в работе *Open Source Software Controls*, опубликованной в ответ на требования Указа 14028 о повышении уровня национальной кибербезопасности (<https://clck.ru/3CKVXN>). Эти практики разделены на уровни, основанные на зрелости организации: фундаментальные, поддерживающие и укрепляющие. Они включают такие меры, как рецензирование исходного кода для выявления уязвимых компонентов, приоритизация использования языков программирования со встроенными защитами и автоматизация процесса сборки, хранения и сканирования OSS-компонентов в усиленных внутренних репозиториях до их внесения в рабочие среды.

Теперь должно быть очевидно, что не существует панацеи для безопасного использования OSS или любого иного программного обеспечения. Тем не менее при правильной комбинации людей, процессов и технологий, именно в такой последовательности, организации смогут получить все преимущества OSS, минимизируя сопутствующие риски.

Резюме

В этой главе мы разобрали такие руководства для обеспечения безопасности цепочек поставок ПО, как SLSA, руководства Microsoft, CNCF и пр. В следующей главе речь пойдет о руководствах от правительственных субъектов.

8

Актуальные рекомендации для государственных разработок

Эта глава посвящена существующим публикациям на тему безопасности цепочек поставок, выпущенным правительственными и общественными организациями. Эти публикации строятся на основе коммерческих руководств, рассмотренных в предыдущей главе, и касаются ряда уникальных требований Министерства обороны, учреждений исполнительной власти, Агентства национальной безопасности и других правительственных структур США.

Практики по защите цепочек поставок ПО для систем и организаций

В начале 2020 года NIST выпустил специальную публикацию SP 800-161 под названием *Cybersecurity Supply Chain Risk Management Practices for Systems and Organizations*. Тем не менее, как и в случае многих других руководств, о которых шла речь ранее, Указ 14 028 потребовал обновления изначальной публикации NIST. Конкретно вопросам безопасности цепочек поставок в указе были посвящены разделы 4(b), 4(c) и 4(d). На основе этого требования NIST затем разместил ответ в виде 800-161 Revision 1 Appendix F, Response to Executive Order 14028's Call to Publish Guidelines for Enhancing Software Supply Chain Security. Вместо того чтобы вносить это руководство

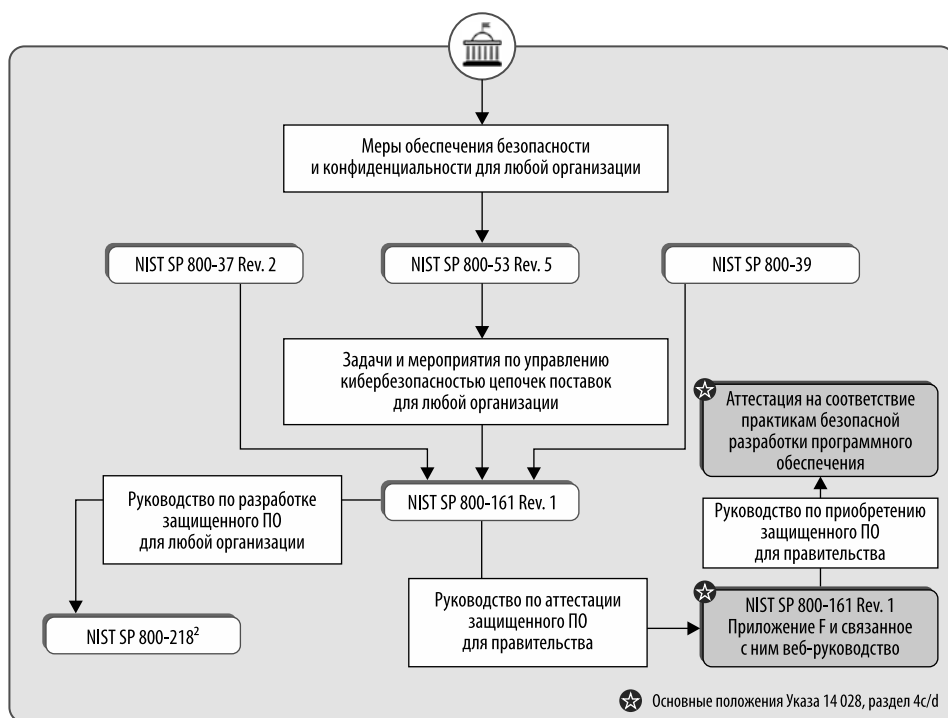
в более обширный документ 800-161, NIST опубликовал его онлайн в виде отдельного материала (<https://clck.ru/3CKVY7>).

Прежде чем разобрать это руководство более подробно, нам следует рассмотреть раздел 4 Указа 14 028. Он призывает министра торговли, действующего через NIST, к сотрудничеству с правительством, промышленностью и академической сферой для выявления существующих или разработки новых стандартов, инструментов и лучших практик, отвечающих прописанным в разделе 4 требованиям. В нем также приводятся критерии для оценки используемых разработчиками и поставщиками практик обеспечения безопасности ПО. Кроме того, указ требовал проведения этой оценки в течение 30 дней с момента его публикации. А в течение 180 дней директор NIST должен был опубликовать основные руководства по повышению безопасности цепочек поставок в соответствии с требованиями того же раздела.

В течение года с момента выхода Указа директор NIST также должен был опубликовать дополнительные рекомендации и процедуры для периодических проверок и корректировок руководств, опубликованных по этой теме ранее. Для создания первого стандарта, который был опубликован онлайн, NIST воспользовался информацией из SP 800-161r1 и позиционных документов, присланных в рамках подготовки к проведенному в июне 2021 года семинару по безопасности цепочек поставок (<https://clck.ru/3CKVYT>), а также от рабочих групп и, естественно, с самой страницы указа.

В руководстве 800-161r1 содержится информация о правилах получения, использования и обслуживания стороннего ПО и сервисов, которые агентства применяют в своих IT-проектах. Данное руководство может быть интегрировано в их программы C-SCRM, а также использоваться для обоснования деятельности по закупкам. Оно применимо не только для правительственных учреждений, но и для поставщиков, стремящихся к развитию практик и процессов, связанных с защитой цепочек поставок. Это руководство станет актуальным, когда его требования оформятся в утвержденную правительством контрактную форму и станут обязательными для поставщиков ПО, желающих продавать свои продукты государственным структурам США.

NIST также предоставил карту связей, иллюстрирующую взаимосвязь между различными публикациями, такими как 800-37r2, 800-53r5 и SSDF. Эти документы могут использоваться как для закупки защищенного ПО правительством, так и для проведения организациями аттестаций на предмет соответствия указанным в них требованиям. На рис. 8.1 показана связь между различными публикациями.



Фреймворк для разработки защищенного ПО (SSDF), версия 1.1: Рекомендации для устранения рисков, связанных с уязвимостями

Рис. 8.1

Источник: <https://clk.ru/3CKYxV>, Министерство экономики США, публичная информация

Критическое ПО

Одним из ключевых требований к NIST в рамках указа было определение *критического программного обеспечения*. В итоге институт дал этому термину такое определение: «Любое программное обеспечение, которое имеет прямые программные зависимости от одного или более компонентов с указанными атрибутами».

Критическое ПО:

- предназначено для работы с повышенными или административными привилегиями;
- имеет прямой или привилегированный доступ к сетевым или вычислительным ресурсам;

- создано для управления доступом к данным или операционным технологиям;
- выполняет функцию, критическую для обеспечения доверия;
- работает вне штатных границ доверия с привилегированным доступом.

Очевидно, что это сложный набор критериев, которые сделают практически любое программное обеспечение критическим. По этой причине мы разберем некоторые из них подробнее. Для начала нужно разобраться, что такое *прямая программная зависимость*. NIST определяет ее как сторонний программный компонент, такой как библиотека или пакет, который напрямую интегрирован или необходим для работы рассматриваемого ПО (<https://clck.ru/3CKVYy>). Институт также отмечает, что к таким компонентам не относятся интерфейсы или сервисы других независимых продуктов. Еще одной неоднозначной фразой может быть «*критический для обеспечения доверия*». NIST уточняет, что под ней подразумеваются категории ПО, используемого для функций безопасности, таких как сетевое управление, защита конечных точек и сети (<https://clck.ru/3CKVZz>). NIST подчеркивает, что их определение применимо к программному обеспечению в любой форме, если оно приобретается для развертывания в рабочих средах. Это означает, например, что оно не относится к ПО, используемому для R&D.

Кроме того, NIST рекомендует, чтобы во время реализации требований к критическому ПО в первую очередь внимание уделялось отдельному локальному программному обеспечению, выполняющему критические функции безопасности или способному нанести ощутимый вред в случае компрометации. Последующие этапы должны акцентироваться уже на других видах ПО, например облачном и гибридном, а также на управлении исходным кодом и других аспектах.

NIST также предоставляет предварительный список видов ПО, которые считает критическими, попутно аргументируя причины их включения. В этом списке перечислены такие категории, как «Идентификация», «Управление учетными данными и доступом», «Операционные системы», «Браузеры», «Безопасность конечных точек», «Сетевое управление» и ряд других. Объясняется их включение тем, что они обеспечивают критический доступ и функции, которые потенциально могут нарушить безопасность и целостность системы в случае наличия уязвимости и эксплуатации.

Для тех, кто желает более подробно ознакомиться с темой критического ПО, рекомендуем посетить страницу с ответами на часто задаваемые вопросы (<https://clck.ru/3CKVaa>).

Меры по обеспечению безопасности критического ПО

Помимо определения критического ПО, NIST также предоставил руководство по мерам безопасности под названием *EO-Critical Software*. Чтобы понять, какие именно меры необходимо применить к критически важному ПО, институт опирался на позиционные документы состоящих в сообществе организаций, проводил виртуальные семинары и получал информацию от CISA и OMB.

Основной сферой применения мер безопасности, определенных NIST, было использование критически важного программного обеспечения, а не его разработка и приобретение. Тем не менее есть и другие руководства, такие как SSDF, которые ориентированы именно на разработку, включая саму работу 800-161r1, которая более широко охватывает тематику C-SCRM и потребления ПО. Перед NIST стоят задачи по защите программного обеспечения и связанных с ним платформ от несанкционированного доступа. Институт также стремится предотвратить эксплуатацию и защитить конфиденциальность, целостность и доступность данных, используемых программным обеспечением и платформами.

NIST признает, что инциденты неизбежны, поэтому организации должны иметь возможность быстро обнаруживать, реагировать и восстанавливаться после них. Кроме того, им необходимо совершенствовать действия и работу персонала, который может повлиять на критическое ПО и платформы. И это крайне важно в свете отчета *Data Breach Investigation Report* компании Verizon в 2022 году, согласно которому более 60 % взломов данных связаны с человеческим фактором. Тем не менее организации расходуют лишь 3 % своих бюджетов безопасности на проработку этого вопроса.

Руководство NIST дополнительно определяет набор надежных практик безопасности и процессов, которые могут защитить критическое ПО. При этом подчеркивается, что данный перечень не является исчерпывающим и по мере изменения ландшафта угроз будет расширяться.

Далее мы разберем некоторые из приведенных NIST мер безопасности, которые институт считает фундаментальными для защиты критического ПО. Стоит отметить, что NIST черпает свои идеи из множества источников, в том числе других своих публикаций, CISA, OMB, NSA и других.

Одна из первых рекомендаций в руководстве — мультифакторная аутентификация (MFA). Как правило, факторы аутентификации включают следующие элементы:

- что-то, что вы знаете;
- что-то, что у вас есть;
- что-то, чем вы являетесь, например имена пользователей и пароли;
- физические токены;
- биометрические данные.

Организации частного и государственного секторов нередко упоминают MFA, когда речь заходит о лучших практиках безопасности. Этот вид аутентификации выходит за рамки базовой проверки, например по имени пользователя и паролю, добавляя такие факторы, как СМС-коды, телефонные звонки, подтверждение через специальные приложения на мобильных устройствах и даже физические устройства вроде карт идентификации личности (personal identity verification, PIV) или карт общего доступа (common access cards CAC), а также популярных аппаратных ключей YubiKey (подробнее о технологии YubiKey читайте на www.yubico.com). Добавление этих факторов значительно затруднит для хакеров компрометацию учетных данных. Сложность в этом случае будет зависеть от того, какой способ аутентификации использован в качестве второго.

Однако важно понимать, что даже MFA не панацея и может быть взломана с помощью, например, СМС-фишинга (<https://clck.ru/3CKVbY>). В связи с этим NIST рекомендует применять варианты MFA, устойчивые к подмене личности, причем не только для администраторов, но и для пользователей. Еще такие методы MFA называют *устойчивыми к фишингу*. При этом аутентификация с помощью паролей, СМС и вопросов безопасности таковой не считается. Устойчивые к фишингу способы аутентификации включают установку строгой связи между аутентификатором и личностью пользователя, отказ от использования общих секретов и обеспечение передачи ответов только доверенным сторонам. К примерам таких MFA относятся смарт-опции FIDO2 и PIV (<https://clck.ru/3CKVc5>).

Дополнительно NIST советует уникальным образом идентифицировать и аутентифицировать каждый сервис и следовать принципам доступа с минимумом привилегий. Эти меры позволяют реализовать эффективное

управление доступом и ограничат радиус угрозы на случай атак. Доступ с минимумом привилегий также является фундаментальной мерой при внедрении принципов нулевого доверия, о чем говорится в публикации NIST 800-207, *Zero Trust Architecture* (<https://clck.ru/3CKVe5>).

И хотя подобная модель доступа способна ограничить потенциальное влияние злоумышленников, она также способствует выполнению еще одной рекомендации NIST — реализации защиты периметра, а именно сегментации сети и использованию программно-определяемых периметров. Разграничение периметра позволяет локализовать влияние инцидента той областью, в которой он произошел, оградив от каскадных последствий остальную систему или даже внешние системы, с которыми она связана.

Для реализации второй задачи по защите конфиденциальности, целостности и доступности данных, используемых критическим ПО и платформами, NIST предлагает ряд ключевых мер и практик, включая создание и ведение реестра данных. Организация не сможет защитить свои данные, не имея должной их описи. Как говорится: «Нельзя защитить то, о существовании чего не знаешь».

По аналогии с ранее рассмотренными мерами обеспечения безопасности на уровне сети, данные и ресурсы организации необходимо защищать путем реализации детального контроля доступа. NIST рекомендует защищать данные в состоянии покоя, используя шифрование, соответствующее криптографическим стандартам (<https://clck.ru/3CKVen>). То же самое касается передаваемых данных. В этом случае институт советует использовать не только шифрование, но и технику взаимной аутентификации, если возможно.

Взаимная аутентификация — это когда две стороны аутентифицируют друг друга одновременно с помощью таких протоколов, как Internet Key Exchange (IKE), Secure Shell (SSH) и TLS. Это гарантирует шифрование передаваемых данных, в результате чего они, во-первых, не окажутся раскрыты для неавторизованных сторон, если те получат к ним доступ, и, во-вторых, смогут противостоять некоторым видам атак, таким как «человек посередине», атака повторного воспроизведения и спуфинг. Они основаны на попытке злоумышленника перехватить передаваемые данные с целью их воссоздать или имитировать валидного получателя. Последняя рекомендация NIST касается резервных копий данных, а именно необходимости практиковать восстановление из бэкапов, чтобы быть к этому всегда готовыми. Руководство признает, что, несмотря на присутствие таких мер защиты, как шифрование и взаимная аутентификация, инциденты с компрометацией данных все равно

могут возникать. Поэтому организации должны быть готовы оперативно восстановить свои данные из резервных копий.

Третья задача ориентирована на обязательную идентификацию и обслуживание платформ критического ПО и выполняющихся на них программ. То есть нужно внедрять и вести инвентаризационную опись программных активов. Эту же практику рекомендует CIS в своей работе *Critical Security Controls guidance* (www.cisecurity.org/controls).

Инвентаризация и управление программным обеспечением помогают более эффективно контролировать установку и использование только разрешенного ПО. Естественно, инвентаризация выходит за рамки описи программных продуктов и включает в том числе составление SBOM на уровне компонентов. Вдобавок к ведению инвентаризационной описи организациям стоит задействовать лучшие практики управления патчами, идентифицировать, документировать и устранять известные уязвимости, причем делать все это в документируемой форме с контролем изменений. Программные платформы и ПО в целом также требуют применения передовых методов управления конфигурацией, включая внедрение усиленных систем безопасности и мониторинг платформ и ПО на предмет несанкционированных изменений.

Как говорилось ранее, NIST осознает, что даже при реализации всех этих мер инцидентов не избежать. В связи с этим четвертая задача представляет собой возможность быстрого обнаружения, реагирования и восстановления после происшествий, повлиявших на критическое ПО и платформы.

Чтобы успешно реагировать и восстанавливаться, организациям нужно подобающим образом регистрировать всю важную информацию о событиях, связанных с безопасностью. Для этого следует придерживаться рекомендаций NIST по управлению журналами компьютерной безопасности и инструкций от поставщиков в отношении их продуктов и платформ (<https://clck.ru/3CKVg6>). Необходимо постоянно мониторить критическое ПО параллельно с использованием продуктов и сервисов по обнаружению и реагированию на угрозы для конечных точек, зачастую называемых инструментами EDR (endpoint detection and response). Эти инструменты помогают выявлять, анализировать и минимизировать поверхность атаки, снижать предрасположенность к известным угрозам, контролировать выполнение ПО и проводить восстановление в случае инцидентов. Организации должны реализовывать сетевую защиту для критического ПО и платформ с целью обнаружения и предотвращения угроз на всех уровнях технологического стека, а также

предоставлять всю необходимую телеметрию специалистам по безопасности и другим сотрудникам, задействованным в реагировании на инциденты. Для эффективного реагирования NIST советует обучать сотрудников, исходя из их конкретных ролей и ответственности.

Последняя задача ориентирована на работу с людьми и заключается в повышении уровня понимания и эффективности их действий в области защиты критического ПО и платформ. Для реализации этой задачи рекомендуется проводить инструктажи с пользователями критического ПО и платформ в соответствии с их ролями и уровнем ответственности, особенно с администраторами, которые обычно имеют привилегии. Наконец, обучение должно быть направлено на повышение осведомленности всех пользователей и администраторов критического ПО и платформ с периодической оценкой эффективности этого обучения.

Верификация ПО

Еще одним ключевым моментом раздела 4 Указа была публикация минимальных требований к тестированию исходного кода поставщиками. Сюда входят ручные и автоматизированные тесты, включая использование инструментов код-ревью, SAST, DAST и пентесты. Как и в случае других пунктов того же четвертого раздела, сотрудники NIST прибегли к сбору позиционных документов от организаций сообщества и проведению семинаров (<https://clck.ru/3CKVh7>).

NIST подчеркивает, что ключевым аспектом обеспечения защищенности ПО является его частое и тщательное тестирование на ранних этапах разработки. Эту верификацию NIST определяет как

дисциплину, реализуемую для повышения безопасности ПО, которая содержит в себе множество техник статической и динамической проверки, а также инструменты и сопутствующие процессы для обнаружения и устранения дефектов безопасности наряду с непрерывной доработкой используемых методов.

Источник: <https://clck.ru/3CKYyL>

Цитируя SSDF, NIST утверждает, что верификация необходима для выявления уязвимостей и проверки соответствия требованиям безопасности. Институт подчеркивает, что ни один отдельный вид тестирования или

стандарт не является всеобъемлющим, и называет свои руководства высокоуровневыми.

Прежде чем углубляться в специфику предлагаемых методов тестирования, стоит обратить внимание на один момент. NIST отмечает, что, хотя в указе в контексте тестирования и используется термин *поставщик*, разработчики зачастую потребляют ПО из внешних источников, в случае чего будут проводить собственную верификацию.

В своем руководстве NIST предлагает 11 видов тестирования и методик, а также приводит вспомогательные техники, которые мы вскоре рассмотрим (<https://clck.ru/3CKVnQ>). В целом руководство NIST поднимает вопрос о необходимости контроля того, чтобы разработанное ПО выполняло именно то, для чего создавалось, оставаясь при этом достаточно свободным от уязвимостей на протяжении всего жизненного цикла. Для достижения таких гарантий нужно использовать различные меры, например моделирование угроз, автоматизированное тестирование (SAST/DAST), динамический анализ, исправление неприемлемых багов и прочих недочетов, а также применение аналогичных техник для всех сопутствующих библиотек, пакетов и сервисов. И хотя многие из этих вопросов уже затрагивались ранее в контексте коммерческого руководства, мы вкратце разберем их здесь с позиции NIST, ориентированной на требования указа.

Моделирование угроз

NIST рекомендует применять моделирование угроз на ранних этапах разработки для выявления проблем безопасности на уровне структуры. Это даст вам принципиальное понимание и визуальное представление системы, после чего вы сможете начать профилировать потенциальные атаки наряду с их возможными целями и методами эксплуатации. Посредством такого моделирования организации выявляют потенциальные угрозы, слабые места и уязвимости, после чего определяют план их устранения. NIST приводит наглядную схему «Эталонная архитектура DevSecOps» Министерства обороны США (рис. 8.2) в качестве примера того, как моделирование угроз вписывается в методологию DevSecOps при разработке систем и планирования.

DevSecOps — это не линейная активность (например, каскадная модель), и моделирование угроз тоже не должно быть таковым. Данная практика должна реализовываться в рамках частой и повторяющейся схемы поставки систем и ПО.

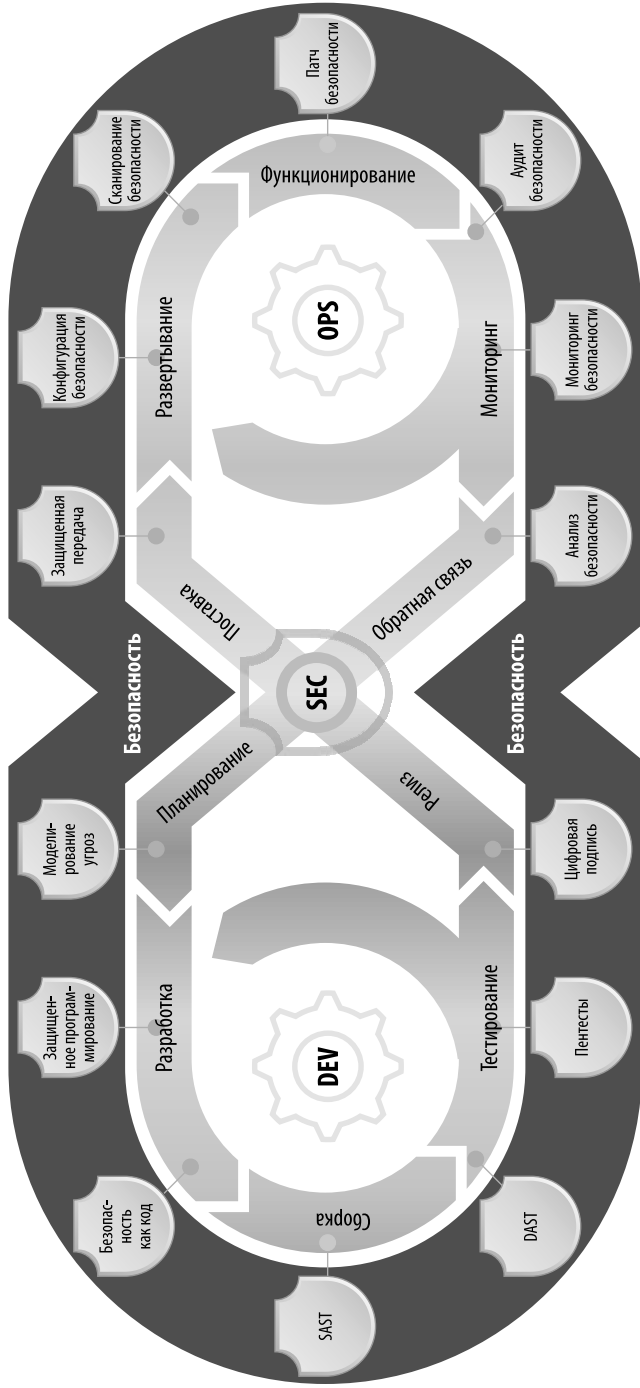


Рис. 8.2

Источник: DoD Enterprise DevSecOps Reference Design v1.0 (<https://clck.ru/3CKVoN>, Министерство обороны США, публичная информация)

Автоматизированное тестирование

В своем руководстве NIST проясняет, что автоматизированное тестирование может быть как обычным скриптом, выполняющим автоматический анализ, так и представлять собой сложный процесс создания целых сред, включая само тестирование и проверку его успешности. Организации могут использовать простые инструменты для тестирования веб-ориентированных приложений и областей или более сложные, состоящие из модулей и подсистем. В качестве рекомендаций приводится автоматизированная верификация, цель которой — обеспечить, чтобы статический анализ не выводил сообщения об обнаружении новых уязвимостей, чтобы тесты выполнялись итеративно, а результаты были точными. Кроме того, автоматизация уменьшает потребность в ручном анализе, который может оказаться ресурсозатратным и подвержен ошибкам. Организации, использующие современные системы разработки, такие как Git и пайплайны CI/CD, при фиксации коммитов и пул-реквестов могут сделать процессы верификации автоматизированными и повторяемыми.

Статический анализ и динамическое тестирование

NIST проводит различие между подходами статического анализа, или анализа на основе кода, и тестированием при выполнении. В качестве примеров этих подходов приводятся SAST и DAST соответственно. При статическом анализе код рассматривается как есть в своем изначальном формате. Динамическое же тестирование производится в процессе выполнения программы. Как и упомянутая выше методика DevSecOps в отношении автоматизированного тестирования, NIST рекомендует выполнять статический анализ сразу после написания кода, небольшими итерациями. Это эффективнее, чем дожидаться выполнения SAST для готового продукта, поскольку недочеты и уязвимости можно устранить на более ранних этапах разработки.

Анализ на наличие в коде жестко прописанных секретов

NIST рекомендует использовать инструменты для поиска в коде жестко прописанных секретов. В качестве таких секретов могут выступать учетные данные, ключи шифрования, API-ключи и токены доступа. Особенно

сложно это делать в облачных средах ввиду обширного использования в них API, токенов и учетных данных. Один из авторов книги затрагивал эту тему в статье под названием *Keeping Secrets in a DevSecOps Cloud-Native World* (<https://clck.ru/3CKVsw>). Согласно исследованиям, с компрометацией учетных данных связана значительная часть взломов, поэтому организациям следует расширять свои меры по защите секретов для снижения сопутствующих рисков.

Запуск кода с проверками и защитой, обеспечиваемыми языком

В руководстве NIST также рекомендуется использовать готовые проверки, которые предлагают различные языки программирования, как компилируемые, так и интерпретируемые. В руководстве также подчеркивается, что некоторые языки не являются *безопасными по памяти*, поэтому необходимо соблюдать в этом отношении осторожность. Безопасный по памяти означает, что программист не может внести в код определенные виды ошибок, связанных с потреблением памяти в приложении. Помимо использования встроенных возможностей, организации могут применять статические анализаторы, или линтеры, для обнаружения опасных функций и параметров. Стоит отметить, что в разработанном OpenSSF плане по повышению безопасности ПО с открытым исходным кодом, о котором шла речь в предыдущей главе, рекомендуется вести отрасль в сторону безопасных по памяти языков, уходя от устаревших решений, несущих повышенные риски, связанные с памятью.

Тестирование по принципу «черного ящика»

В отличие от видов тестирования, рассмотренных выше, тесты «черного ящика» не привязаны к конкретным участкам кода, а фокусируются на функциональных требованиях и проверяют, не делает ли ПО то, чего делать не должно. Эти тесты включают проверку на определение границ отказа в обслуживании и границ ввода. Выполняются они без необходимости какого-либо понимания структуры приложения или допущений, которые сопровождают анализ исходного кода и моделирование угроз.

Тестовые случаи в отношении кода

Такое тестирование подразумевает проверку особенностей самого кода. Это может быть тестирование количества элементов и взаимодействий, которое способно обработать ПО, и подтверждение этого с помощью тестовых случаев. NIST рекомендует наборы тестов, покрывающие не менее 80 % инструкций кода.

Тестирование в хронологическом порядке

Под фразой *тестирование в хронологическом порядке* руководство NIST подразумевает так называемое *регрессионное тестирование*. Данный вид тестирования чаще применяется зрелыми командами разработчиков, которые с его помощью убеждаются в *отсутствии* ранее обнаруженной уязвимости после ее устранения. Этот вид тестирования важен в случае частого изменения конфигурации.

Фаззинг

Еще одним рекомендуемым методом верификации выступает *фаззинг*. Это вид автоматизированного тестирования, который работает путем внедрения в программу недействительных, искаженных или неожиданных входных данных для выявления дефектов и уязвимостей. После их внедрения в систему тестировщик наблюдает, как она реагирует, например, не падает ли и не допускает ли утечку информации, которая не должна быть раскрыта. Для тех, кому эта тема интересна, у проекта OWASP есть обширная страничка, посвященная фаззингу (<https://clck.ru/3CKVuJ>).

Сканирование веб-приложений

Эта техника зачастую называется динамическим или интерактивным тестированием безопасности приложений (DAST или IAST) и приговорается, когда программное обеспечение предоставляет веб-услуги. По сути, это то же самое, что и фаззинг, но только для веб-приложений. Самым популярным в мире сканером является Zed Attack Proxy (ZAP, www.zaproxy.org) проекта OWASP.

Проверка компонентов ПО

В качестве последнего метода, который должен присутствовать в минимальном наборе инструментов тестирования разработчика, NIST рекомендует осуществлять проверку компонентов ПО. Цель этого мероприятия — убедиться, что привнесенный извне код *не менее* безопасен, чем код, разработанный локально. Как уже говорилось, техники вроде SCA помогают обнаружить OSS-библиотеки, пакеты и зависимости, которые использует ваше ПО. Подобные инструменты делают запрос к базам данных уязвимостей, таким как NVD, в поиске *известных уязвимостей* в этих компонентах. Учитывая, что значительная часть современного ПО состоит преимущественно из открытых компонентов, это мероприятие является особенно важным.

NIST также приводит дополнительную информацию обо всех описанных нами техниках. И она заслуживает внимания, если вас интересуют дополнительные подробности, в том числе о том, как их правильно реализовать.

Фреймворк SSDF

Как уже говорилось ранее, Указ 14 028 сильно повлиял на модель безопасности с нулевым доверием, облачные вычисления и, естественно, на защиту цепочек поставок ПО. В рамках этого указа президент потребовал «приобретать только такое программное обеспечение, которое разрабатывалось с акцентом на защищенность». При этом он поставил NIST задачу выпустить руководство, определяющее практики, повышающие безопасность цепочек поставок ПО. Что NIST и сделало. В сотрудничестве с представителями отрасли институт опубликовал SSDF 1.1, а также еще одно руководство по защите цепочек поставок. В текущем разделе мы более углубленно познакомимся с SSDF и выясним, почему он важен.

SSDF указывает, что для решения вопросов безопасности ПО существует несколько конкретных моделей цикла разработки. Про кибербезопасность в отрасли часто говорят так: «Прикручена, а не встроена». При разработке цифровых систем об этом вопросе обычно задумываются уже на последних этапах SDLC вместо того, чтобы изначально интегрировать в ПО и системы лучшие практики безопасности и требования. SSDF 1.1, вышедший в 2022 году, построен на оригинальном документе, опубликованном в апреле 2020-го. В рамках подготовки новой версии NIST провел семинар, в котором участвовали

представители частного и государственного секторов. На этом семинаре было собрано 150 докладов на тему возможных доработок SSDF.

Целевая аудитория SSDF включает как потребителей ПО, так и несколько категорий его производителей: поставщики продуктов, правительственные разработчики и внутренние команды разработки. И хотя SSDF специально создавался для использования федеральными агентствами, изложенные в нем методики и задачи применимы к командам разработки во всех отраслях и могут использоваться самыми разными организациями. Кроме того, SSDF носит не предписывающий, а описательный характер. В нем не говорится, как конкретно нужно реализовывать ту или иную практику, а вместо этого основное внимание уделяется результатам безопасного программного обеспечения, что позволяет организациям внедрять методики, способствующие достижению этих результатов. И это разумно, учитывая бесчисленное множество способов защиты ПО и уникальность людей, процессов и технологий, которые входят в состав каждой организации, производящей и использующей программное обеспечение. Руководство также проясняет, что при определении подходящих практик и необходимых ресурсов нужно учитывать такие факторы, как допустимые риски.

NIST сформировал перечень минимальных рекомендаций для федеральных агентств, которые потребляют ПО или продукты, его содержащие. Этот набор включает несколько ключевых положений, нацеленных на то, чтобы оградить правительство от получения незащищенного ПО и продуктов. Причем NIST советует агентствам при обсуждении вопросов, связанных с реализацией требований защищенной разработки, использовать терминологию SSDF. Кроме того, поставщикам рекомендуется проводить аттестацию на соответствие практикам разработки SSDF на протяжении всего SDLC. Однако вопрос аттестации является дискуссионным. Дело в том, что аттестацию в отношении процессов можно выполнить либо самостоятельно, то есть провести *самоаттестацию*, либо привлечь независимую стороннюю организацию.

Привлечение подобных организаций повышает надежность аттестации, поскольку в этом случае оценка производится непредвзятой стороной. Тем не менее в этом случае возникают дополнительные издержки в виде времени и средств. Например, FedRAMP требует, чтобы CSP подвергались проверке сторонней организацией, если те хотят поставлять свои услуги государственному заказчику.

Данная программа существует уже порядка десяти лет. Тем не менее на момент написания книги ее аттестацию прошло всего около 250 CSP. Если бы

такой подход решили применить и в рамках SSDF, это оказало бы аналогичное влияние в виде ограничения общего числа поставщиков, уполномоченных поставлять программное обеспечение правительству. Однако NIST в своем руководстве указывает, что в зависимости от уровня допустимых для агентства и потребителей ПО рисков в некоторых ситуациях сторонняя аттестация может быть обязательной. Критики призывают правительство не использовать такой подход ввиду его пагубного влияния на внедрение SSDF, приводя примеры с задержкой внедрения других похожих программ вроде продвигаемой Министерством обороны США сертификации СММС. Эта программа пережила несколько откатов (<https://clck.ru/3CKVvs>), часть которых была связана именно со сложностью прохождения аттестации для получения сертификата соответствия.

Подробности SSDF

Как уже говорилось, цель SSDF — продвигать использование фундаментальных и признанных лучших практик разработки защищенного ПО. Уникальность SSDF в том, что вместо того, чтобы создавать руководство с нуля, его авторы использовали множество известных и прошедших проверку временем источников, таких как BSIMM компании Synopsys, SAMM от OWASP и других.

В SSDF список надежных практик разработки защищенного ПО разбит на группы:

- подготовка организации (Prepare the Organization, PO);
- защита ПО (Protect the Software, PS);
- создание надежно защищенного ПО (Produce Well-Secured Software, PW);
- реагирование на обнаруженные уязвимости (Respond to Vulnerabilities, RV).

Эти практики основаны на таких элементах, как «Практика», «Задача», «Теоретический пример реализации» и «Привязка», которая связывает практики с задачами. А поскольку последняя версия SSDF учитывает требования Указа 14 028, в ней также предусмотрено сопоставление с конкретными требованиями раздела 4е. Цель использования практик SSDF в том, чтобы сократить число уязвимостей в финальной версии ПО и влияние тех из них, которые останутся незамеченными или неисправленными.

Подготовка организации (РО)

Подготовка к защищенной разработке — это первый логический шаг для любой организации, желающей создавать именно защищенное ПО. Находящиеся в этой категории практики подразумевают определение требований безопасности, которым должны отвечать инфраструктура разработки и само создаваемое организацией ПО. Естественно, об этих требованиях необходимо уведомлять всех сторонних поставщиков, которые предоставляют этой организации коммерческие программные компоненты.

На втором шаге необходимо распределить роли и ответственность. Сюда относятся роли на всех этапах SDLC, а также процесс подбора подходящего обучения связанных с ними людей. Здесь очень важен ответственный подход высшего руководства или уполномоченных должностных лиц в отношении защищенной разработки.

Современный процесс поставки ПО включает цепочки инструментов, использующих автоматизацию для минимизации человеческих усилий, связанных с разработкой ПО, и получения более последовательных, точных и воссоздаваемых результатов. В этом контексте организации необходимо определить инструменты для снижения рисков и понять, как они взаимодействуют друг с другом. Следует также установить рекомендации по безопасности при использовании этих инструментов и убедиться, что они настроены в соответствии с требованиями практик защищенной разработки.

Кроме того, организациям нужно задать критерии для проверки безопасности ПО. Сюда относится реализация процессов и инструментов для защиты информации на протяжении всего SDLC. С помощью цепочек инструментов можно наладить автоматическое информирование руководства о рисках безопасности и генерировать метрики, связанные с управлением уязвимостями.

Наконец, организациям нужно создать безопасные среды для разработки ПО. Обычно это предполагает такие среды, как «Разработка», «Тестирование», «Индексирование» и «Эксплуатация». Обычно они разделены для ограничения радиуса поражения в случае компрометации, а также для применения к ним различных требований безопасности и более эффективного управления конфигурацией, включая контроль ее дрефта. Под дрефтом подразумевается ситуация, когда фактическая конфигурация не соответствует желаемой.

Эти среды можно защитить с помощью таких методов, как MFA, условный контроль доступа или доступ с минимумом необходимых привилегий, а также путем мониторинга и регистрации всех происходящих в них действий для более эффективного обнаружения, реагирования и восстановления. Защита среды также подразумевает применение мер в отношении конечных точек, используемых разработчиками и другими лицами, взаимодействующими со средой, чтобы исключить возможные риски. Вы заметите, что эта рекомендация во многом согласуется с текущим руководством и лучшими практиками в контексте модели нулевого доверия.

Защита ПО (PS)

Практики этой группы включают в себя защиту кода от неавторизованных изменений, проверку его целостности и защиту каждого отдельного релиза.

Защита всех этих форм кода необходима, чтобы код не был намеренно или ненамеренно модифицирован таким образом, который нарушил бы его целостность. При хранении кода также должны соблюдаться правила доступа с минимумом привилегий на основе его требований безопасности, которые для открытого и проприетарного кода отличаются. Для предотвращения несанкционированных изменений в коде организации могут использовать репозитории с поддержкой контроля версий, подписание коммитов, а также проводить код-ревью. Кроме того, для обеспечения целостности кода его можно подписывать, применяя, например, криптографические хеши.

Однако нужно не только поддерживать целостность кода, но и предоставлять пользователям ПО методы проверки этой целостности. В этом случае принесет пользу практика размещения хешей на хорошо защищенных сайтах. Подписание кода должно реализовываться доверенными сертифицирующими инстанциями, что будет говорить потребителям ПО о надежности такой подписи.

Наконец, защищать необходимо и каждый релиз ПО, в том числе путем резервного копирования. Это позволяет выявлять, анализировать и устранять уязвимости, связанные с конкретными версиями. Кроме того, упрощаются откат (в случае компрометации релиза) и восстановление системы до стабильного состояния. Защита и резервное копирование помогают потребителям отследить провенанс кода, а также убедиться в его целостности.

Создание надежно защищенного ПО (PW)

Закрыв вопросы требований, защиты сред разработки и предоставляющих к ним доступ конечных точек, организация может сосредоточиться на разработке защищенного ПО. Естественно, это не значит, что все описанные практики не применяются постоянно на протяжении всего существования организации или программы, но при этом они строятся поверх друг друга и подразумевают периодическое к ним возвращение в случае необходимости.

Вы заметите, что в SSDF в разделе ПО определены и задокументированы требования безопасности. С этого момента программное обеспечение должно создаваться в соответствии с ними. Именно здесь организации могут применить методы моделирования угроз и их сопоставления с поверхностью атаки, чтобы проанализировать риски, сопряженные с разработкой ПО. Обучая подобным методам свои команды разработки, организации повысят их квалификацию в том плане, что они смогут распознавать угрозы для создаваемого ими ПО и принимать необходимые меры противодействия. Используя методы классификации данных, организации повысят качество анализа высокочувствительных и опасных областей. Кроме того, организациям следует регулярно проводить ревью структуры ПО, проверяя ее на соответствие любым актуальным требованиям. Причем касается это не только программного обеспечения, разработанного собственными силами, но и того, что приобретается у третьих сторон. В зависимости от природы потребляемого ПО организации могут работать с его разработчиками для устранения несоответствий требованиям безопасности, но это не относится к OSS, где не действуют какие-либо контракты или соглашения вроде SLA.

Организациям желательно повторно использовать существующее, надежно защищенное ПО, нежели дублировать его функциональность. Повторное использование имеет множество преимуществ, таких как снижение затрат на разработку, ускорение поставки и снижение вероятности появления новых уязвимостей в среде. Во многих крупных корпорациях нередко наблюдается *разрастание кода*, то есть неконтролируемый его рост, особенно с учетом наших реалий, когда инфраструктура и даже безопасность в облачных средах могут определяться «как код». Модель реализации «как код» поддерживает такие концепции, как модульность, повторное использование, конфигурация как код, а также усиленные шаблоны кода и манифесты, которые можно безопасно использовать где-либо еще в организации или даже вне нее.

Тем не менее здесь стоит вспомнить исследование Unit 42 компании Palo Alto, описанное в главе 6. В нем отмечалось, что если манифесты и шаблоны кода содержат уязвимости, то эти уязвимости тоже масштабируются, в связи с чем необходимы тщательный контроль и обеспечение безопасности.

Организации или даже их команды, которые повторно используют существующее ПО и код, должны проводить его ревью и анализ на предмет безопасности и ошибочных конфигураций. Кроме того, им необходимо понимать историю происхождения повторно используемого кода. SSDF дает аналогичную рекомендацию — создавать и поддерживать надежно защищенные программные компоненты и репозитории внутри организации для их вторичного применения в разработке, что также согласуется с рекомендацией NIST, описанной в документе 800-161r1.

Создаваемый организацией исходный код должен соответствовать практикам защищенного программирования, которые приняты в компании и которые рекомендует сама отрасль.

Эти практики включают проверку всех входных данных, отказ от небезопасных функций и вызовов, а также использование инструментов для выявления уязвимостей в коде.

Реагирование на уязвимости (RV)

Даже если организации будут устанавливать требования безопасности, подготавливать свои среды и стремиться создавать защищенное ПО, полностью уязвимостей все равно не избежать. Правда в том, что выявить их все во время разработки просто невозможно и с течением времени они всплывут. Как уже говорилось, чем дольше живет ПО, тем выше вероятность найти в нем уязвимости.

Организациям необходимо непрерывно работать над выявлением и подтверждением уязвимостей. Это подразумевает мониторинг баз данных уязвимостей, новостей от исследователей и автоматическую проверку всех программных компонентов на предмет новых уязвимостей. Эти практики являются ключевыми, так как новые угрозы неизбежно будут возникать. Кроме того, организациям следует внедрять политики, ориентированные на раскрытие и устранение уязвимостей, а также определять роли специалистов и обязанности, связанные с этим процессом. В отношении раскрытия уязвимостей организациям стоит создавать команды реагирования на

инциденты безопасности (product security incident response team, PSIRT), о чем мы подробно поговорим в главе 10.

Организациям потребуются не только методы выявления и подтверждения уязвимостей, но и способы их устранения, соответствующие рискам, которые несут уязвимости. Это подразумевает наличие отработанного процесса для анализа, приоритизации и устранения программных уязвимостей. Используя нужные инструменты и меры контроля, организации могут принимать решения с учетом рисков, например об устранении, принятии, а в некоторых случаях и об их переносе. Разработчикам ПО также необходимо создавать и публиковать рекомендации по безопасности для пользователей их продуктов. Эти рекомендации помогут понять, какие уязвимости присутствуют в потребляемом ПО и какие меры можно принять для их устранения, если возможно.

Наконец, организациям следует выявлять не просто отдельные уязвимости, но и их причины, чтобы снизить вероятность их появления в будущем.

Важно отметить, что ни одна организация, будь то небольшая или очень крупная, не сможет постоянно реализовывать все эти практики правильно. Тем не менее организации должны стремиться систематизировать их, ориентируясь на рекомендации SSDF и контролируя выполнение необходимых шагов в рамках SDLC.

NSA: серия руководств по защите цепочек поставок ПО

В 2022 году NSA совместно с CISA и ODNI выпустили руководство по безопасности цепочек поставок ПО. Оно состоит из трех частей, предназначенных для разработчиков, поставщиков и потребителей. Каждая часть — это отдельное руководство, основанное на конкретных ролях и действиях в рамках цепочек поставок ПО.

Серия была выпущена в рамках проекта *Enduring Security Framework* (ESF, фреймворк устойчивой безопасности). Этот проект представляет собой рабочую группу из представителей частного и государственного секторов под руководством NSA и CISA. Ее цель — предоставить руководство по устранению особо опасных угроз для критической инфраструктуры.

Далее мы изучим каждую из этих публикаций и их рекомендации. Данное руководство было выпущено в порядке, соответствующем жизненному циклу ПО, то есть начиная с разработчиков, продолжая поставщиками и заканчивая потребителями. И мы будем рассматривать его именно в таком порядке.

В аннотации к данному руководству (<https://clck.ru/3CKVyH>) подчеркивается роль разработчиков в создании защищенного ПО и говорится, что оно поможет им в этом процессе благодаря изложенным в нем рекомендациям от правительства и отрасли. Вторая и третья части фокусируются на поставщиках и конечных потребителях, учитывая их уникальную роль в обширной цепочке поставок и обеспечении ее устойчивости.

Документ разбит на три части:

- руководство по безопасности для разработчиков ПО;
- рекомендуемые практики для поставщиков;
- рекомендуемые практики для потребителей.

На рис. 8.3 схематично отражены роли и процесс взаимодействия разработчиков, поставщиков и потребителей в общей экосистеме цепочек поставок ПО.



Рис. 8.3

Руководство по безопасности для разработчиков

Атаки на цепочки поставок ПО уже далеко не новость. Например, CNCF ведет подсчет взломов цепочек поставок с 2003 года. Помимо этого, есть датасет Software Supply Chain Compromises от IQT Labs, который тоже берет начало в 2003 году (<https://clck.ru/3CKW3Q>).

Этот датасет представляет собой набор данных публично раскрытых случаев компрометации цепочек поставок ПО. Но, несмотря на то что об атаках на цепочки поставок уже известно на протяжении 20 лет, исследования показывают лишь увеличение их частоты. К примеру, об этом говорится в статье Дэна Гира, Бенца Тозера и Джона Спиды Майерса под названием *For Good Measure — Counting Broken Links: A Quant’s View of Software Supply Chain Security* (рис. 8.4) (<https://clck.ru/3CKW4B>).

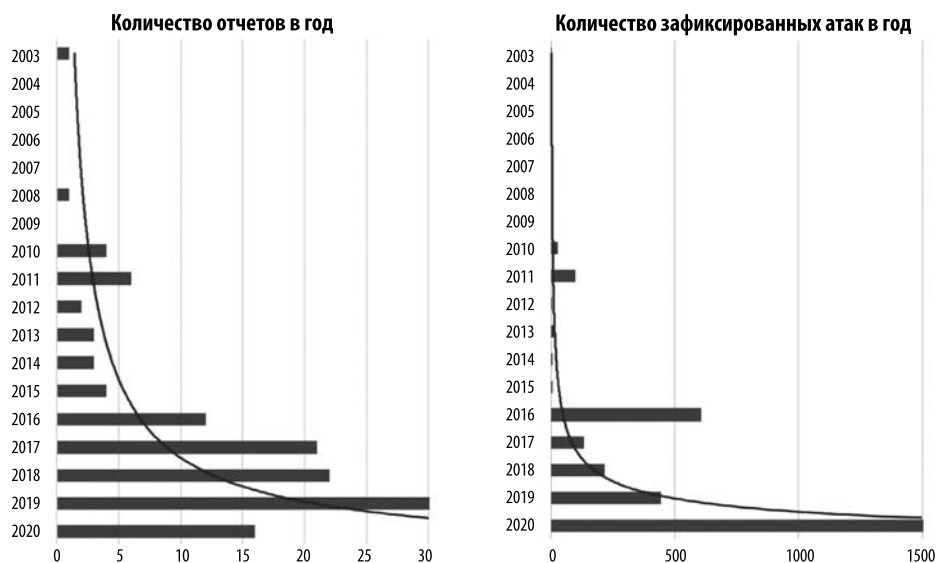


Рис. 8.4

Если смотреть с позиции производителей ПО, то они выступают в роли вспомогательного звена, предоставляя потребителям функциональные возможности, реализуемые разработчиками. Это неизбежно затрудняет для поставщиков и их команд разработки процесс ускоренного вывода

продукта на рынок, так как он противоречит созданию защищенных и устойчивых к отказам программных решений.

Защищенное ПО начинается с SDLC, и в руководстве NSA приводится множество доступных для команд подспорий в этом вопросе, таких как SSDF (<https://clck.ru/3CKWme>), Secure Software Development Life Cycle Processes Университета Карнеги — Меллона (Carnegie Mellon University) (<https://clck.ru/3CKWny>) и прочие инициативы вроде курсов Secure Software Development Fundamentals, о запуске которых было объявлено в 2022 году (<https://clck.ru/3CKWoj>).

В руководстве подчеркивается не только важность использования защищенных процессов разработки ПО, но и создание осязаемых артефактов и аттестаций, которые могли бы применять производители и потребители для получения гарантий безопасности и устойчивости ПО.

Эти процессы и мероприятия включают такие лучшие практики, как моделирование угроз, SAST, DAST и пентесты, а также задействуют техники вроде цифровых подписей. Ярким примером является активное распространение Sigstore, стандарта для подписания, верификации и защиты ПО. О внедрении и использовании этого стандарта также говорится в разработанном OpenSSF плане по повышению безопасности ПО с открытым исходным кодом. В нем Sigstore описывается как метод обеспечения повышенного доверия в рамках цепочки поставок ПО.

Далее мы подробнее разберем разделы предлагаемого NSA руководства для разработчиков.

Критерии защищенного продукта и управление

Основным аспектом в создании защищенных продуктов является наличие в организации практик и процедур безопасной разработки, подкрепляемых необходимым бюджетом и графиком. Естественно, это может создавать некоторые трудности, например, когда делается акцент на скорости поставки или когда не хватает ресурсов, поскольку реализация лучших практик безопасности требует времени, опыта и определенных усилий.

В знак признания успехов, достигнутых Министерством обороны США в сфере DevSecOps, NSA в своем руководстве использует схему процесса, представленную в методичке под названием *DevSecOps Fundamentals Playbook* (<https://clck.ru/3CKWpU>). Тем не менее, в отличие от примера Министерства

обороны США, где Dev, Sec и Ops описываются как три составляющие в замкнутом цикле, пример NSA фокусируется на трех следующих субъектах, указанных во введении руководства:

- разработчики;
- поставщики;
- потребители.

В отличие от команд, вовлеченных в процесс разработки ПО (как в случае Министерства обороны США), эти субъекты участвуют в создании и потреблении защищенного ПО, выходя за границы одной организации в более обширный контекст цепочек поставок.

Руководство NSA указывает на кое-какие потенциальные угрозы, которые возникают при создании и поставке программных продуктов: действия злоумышленников, внесение уязвимых сторонних компонентов, эксплуатация процесса сборки и компрометация механизма поставки продукта потребителю. Для снижения этих рисков разработчикам следует применять методы, ориентированные на безопасность, например, моделировать угрозы, определять критерии релиза, а также документировать и публиковать информацию о применяемых в процессе работы процедурах защиты. В руководстве приводятся и другие заслуживающие внимания примеры.

И хотя они далеко не новы, некоторые рекомендуемые практики становятся все более популярны. Так, моделирование угроз уже несколько лет рекомендуется такими лидерами отрасли, как Адам Шостак, Роберт Херлбут и многие другие. (*Моделирование угроз* — это попытка предугадать, что может пойти не так и как этого избежать.) По данной теме есть прекрасные руководства вроде «Манифеста о моделировании угроз» (Threat Modeling Manifesto) и проектов «Моделирования угроз» от OWASP.

Моделирование угроз можно применять на протяжении всего цикла разработки. NIST советует задействовать эту технику в отношении критического ПО, используемого правительством. Что касается определения критического ПО, то оно было дано NIST в соответствии с требованием указа. Подробнее об этом можно почитать на сайте института: <https://clck.ru/3CKWq3>. В качестве OSS-решения выступают инструменты вроде Threat Dragon от OWASP, которые помогают в составлении диаграмм потока данных, оценке угроз и поиске способов противодействия.

В руководстве NSA настоятельно рекомендуется использовать планы тестирования безопасности и критерии релиза. Планы и требования тестирования зачастую включают типичные методы вроде SAST/DAST, SCA и пентесты. Для реализации этих техник в отрасли существует несколько открытых проектов и платных инструментов. Причем последние все чаще интегрируются в пайплайны CI/CD, что позволяет *сдвинуть* усилия по обеспечению безопасности влево, то есть на более ранние этапы SDLC, где, по мнению некоторых специалистов, это более экономично и эффективно. Выражение «сдвиг влево» обычно используется для того, чтобы подчеркнуть необходимость реализации безопасности на более ранних стадиях SDLC и перехвата уязвимостей до их проникновения в рабочую среду.

Применение этих инструментов и методов тестирования позволит организациям определить критерии релиза, которые должны включать такие аспекты, как уязвимости, превышающие допустимые уровни рисков, отсутствие практик защищенной разработки или, в свете повышенного стремления к прозрачности ПО, создание, сопоставление и валидация SBOM.

И хотя критерии релиза крайне важны в целях снижения рисков для потребителей, их выработка представляет непростую задачу и требует признания того, что:

- широко распространенные методы оценки уязвимостей вроде CVSS не совершенны;
- инструменты часто дают ложноположительные показатели, которые снижают скорость поставки;
- многие опубликованные уязвимости по каталогу CVE не являются эксплуатируемыми.

Более подробно о проблемах, связанных с CVSS, читайте в статье *CVSS: Ubiquitous and Broken* (<https://clck.ru/3CKWqX>). Отсевание шумов при сокращении числа уязвимостей с сохранением скорости — это тонкое искусство, которое требует грамотного комбинирования эффективных специалистов, процессов и технологий. Специалист в области безопасности и писатель Уолтер Хейдок в одной из своих статей разбирает некоторые риски, сопряженные с неудачной реализацией критериев безопасности релиза (<https://clck.ru/3CKWr7>).

NSA рекомендует организациям придерживаться зрелых и систематизированных политик поддержки продуктов и обработки уязвимостей. Они

включают систему передачи уязвимостей на рассмотрение, команды реагирования на инциденты (PSIRT), а также безопасную поставку ПО через защищенные протоколы, такие как HTTPS/TLS.

Учитывая, что уязвимостей в ПО не избежать, очень важно проводить анализ кода и обучение специалистов. Для начала следует определить политики, которые позволят понять, кого необходимо обучать, как часто и по каким темам. Обучение обычно включает темы защищенной разработки ПО и ревью кода, а также верификации ПО и тестирования на уязвимости. Важно также следить, чтобы процедуры безопасности и процессы рассматривались как постоянно развивающиеся документы, которые улучшаются и дорабатываются по мере накопления опыта командами и организацией на основе анализа результатов того или иного решения (after-action review, AAR).

Все рассмотренные выше меры противодействия и лучшие практики, вроде моделирования угроз, планов по тестированию безопасности и критериев релиза, в руководстве NSA сопоставляются с SSDF. Организациям, продающим ПО частному сектору, рекомендуется ознакомиться с SSDF, так как вскоре соответствие указанным в нем требованиям и практикам разработки станет обязательным.

Разработка защищенного кода

Закрыв вопросы критериев и управления защищенным продуктом, мы переходим к разработке защищенного кода. Этот процесс начинается с самых базовых решений вроде выбора языка программирования и реализации основ защиты цифровых систем и данных, например безопасных предустановленных параметров и доступа с минимумом необходимых привилегий.

Организации не должны упускать из виду и внутренние угрозы, будь они случайные или намеренные. В обоих случаях они могут подразумевать изменение или эксплуатацию исходного кода. Это может произойти в результате недостаточного обучения разработчиков или вследствие намеренного внедрения инженерами бэкдоров. Здесь рекомендуется сдерживать разрастание функциональности, сопоставляя ее с требованиями, чтобы не допускать внесения лишних рисков. Люди порой оказываются в жизненных обстоятельствах, делающих их уязвимыми для шантажа или компрометации. Бывает, что они создают функции, которые облегчают им рабочий процесс, но снижают безопасность продукта. NSA также предупреждает о возможной

компрометации удаленных систем разработки. В свете развития парадигмы удаленной работы и размывания сетевого периметра основными целями злоумышленников становятся удаленные конечные точки разработки. К тому же организации часто пренебрегают гигиеной конечных точек, особенно при использовании в работе личных устройств сотрудников. Векторами атаки могут выступать и сторонние команды разработки. К тому же угрозе подвержены и заброшенные аккаунты и учетные данные. Такое происходит, когда пользователи покидают команды или организации, но их аккаунты остаются активными. Данный факт должен подталкивать организации к применению зрелых политик управления аккаунтами и доступом.

И хотя эти сценарии вносят беспорядок в процесс разработки и способны создать опасные прецеденты, компании могут предпринять шаги для снижения рисков, связанных с небезопасным кодом. К ним относится использование аутентифицированных процессов внесения исходного кода, сканирование на уязвимости и укрепление фактической среды разработки, которая может отличаться в зависимости от условий работы организации.

NSA также говорит о том, что подчеркивается NIST в руководстве 800-161, а именно о создании защищенного потока процессов в репозитории. Этот шаблон (рис. 8.5) позволяет организациям выбирать внешние программные компоненты, попутно реализуя рабочий поток, включающий сканирование SCA для определения состава ПО и обнаружения уязвимостей. Такой поток помогает анализировать внешние компоненты и устанавливать защищенный репозиторий, где проверенные компоненты будут доступны для внутреннего использования в ходе разработки.

Использование ранее рассмотренных инструментов вроде SAST и DAST должно соответствовать критериям безопасности релиза, а также позволять выявлять уязвимости и принимать соответствующие меры. Командам также необходимо выпускать промежуточные сборки (nightly builds — «ночные сборки». — *Примеч. пер.*), включая тестирование безопасности и регрессионное тестирование, для выявления и устранения ошибок или опасных изменений. Как мы уже говорили, разработчики могут добавлять функциональность для собственного удобства, но это порой ведет к появлению багдоров. Из-за широкого развития удаленной работы NSA рекомендует организациям укреплять свои среды разработки. Под этим подразумевается усиление не только сред сборки, но также конечных точек или систем, задействованных в рабочих потоках или процессе создания ПО. В таких случаях меры защиты вроде доступа через VPN, MFA, EDR, а также ряд других могут до определенной степени сократить риски удаленной работы.



Рис. 8.5

Источник: <https://clck.ru/3CKZ33>, Департамент кибербезопасности США, публичная информация

Для написания, тестирования и резервного копирования исходного кода необходимо применять практики защищенной разработки. Сюда входят вышеупомянутые меры по защите среды разработки и использование безопасных конфигураций сборки, а также защищенных сторонних цепочек инструментов и компонентов.

Поскольку компоненты и программное обеспечение внедряются в поставляемый продукт, это требует следования зрелым и безопасным практикам интеграции кода. Речь идет о тестировании сторонних компонентов на наличие CVE из NVD и использовании анализаторов безопасности зависимостей при интеграции кода.

Когда клиенты обнаруживают дефекты или уязвимости, команды разработки должны быть готовы реагировать на эти инциденты и отчеты. Должен существовать публичный процесс приема отчетов, а также внутренний процесс для ревью, диагностики и разрешения выявленных проблем. Также есть

вероятность, что другие команды добавят в продукт внешние расширения, которые могут иметь собственные уязвимости. Расширения от команд по созданию решений или реселлеров (*value-added resellers*) также должны разрабатываться с применением надежных практик. При этом ПО необходимо сопровождать SBOM для отслеживания добавленных пакетов и компонентов, отсутствующих в исходном продукте.

Верификация сторонних компонентов

Разработчики все чаще задействуют в своем коде и продуктах сторонние компоненты. Таким образом они экономят время, снижают затраты и усилия, будь то с помощью готовых коммерческих решений или свободного и открытого ПО (FOSS). Обсуждение процессов интеграции кода на ранних этапах может снизить риск внесения небезопасного стороннего кода. Прочие сложности сопряжены с присутствием сторонних исполняемых файлов, которые требуют выполнения SCA для определения содержащихся в них неизвестных элементов и OSS-компонентов, а также всех связанных с ними уязвимостей. Здесь может пригодиться проверка SBOM в отношении исходного кода, получаемого извне.

При выборе и интеграции сторонних компонентов необходимо оценивать все связанные с этим риски, используя SCA и другие методы сканирования, такие как SAST и DAST, когда исходный код доступен. Команды разработки также должны быть осведомлены относительно провенанса потребляемого кода и быть уверены в надежности применяемого для его поставки метода. Для понимания провенанса прослеживается весь путь цепочки поставки ПО вплоть до его изначального источника. Имейте в виду, что здесь могут присутствовать различные уровни требований, которые определяются политикой безопасности организации. Например, в SCVS от OWASP требования к провенансу более строгие.

NSA рекомендует приобретать компоненты у известных и доверенных поставщиков. И хотя у нас есть богатая экосистема разработчиков и поставщиков ПО, это не означает, что они равны в плане качества защиты своих программных продуктов. Так, Джошуа Корман выступает за то, чтобы брать пример с крупнейших автомобилестроительных концернов и обращаться лишь к небольшому списку поставщиков, использующих только высококачественные компоненты. Об этом он подробно говорил в своем выступлении *Continuous Acceleration: Why Continuous Everything Needs a Supply Chain Approach* на конференции USENIX (<https://clck.ru/3CKWrg>).

OSS привносит сложности в виде необходимости контроля сторонних компонентов. Как отмечается в отчете Sonatype от 2021 года *State of the Software Supply Chain* (<https://clk.ru/3CKWsG>), число скачиваний разработчиками OOS-компонентов ежегодно растет на 73 % (свыше 2,2 триллиона загрузок), а также на 650 % увеличивается число атак на цепочки поставок посредством таких методов, как путаница зависимостей (dependency confusion), тайпсквоттинг и внедрение зараженного кода (рис. 8.6). О каждом из этих видов атак мы говорили в главе 2.

Новейшие виды атак на цепочки поставок ПО (2015–2020 годы)

Путаница зависимостей, тайпсквоттинг и внедрение зараженного кода.

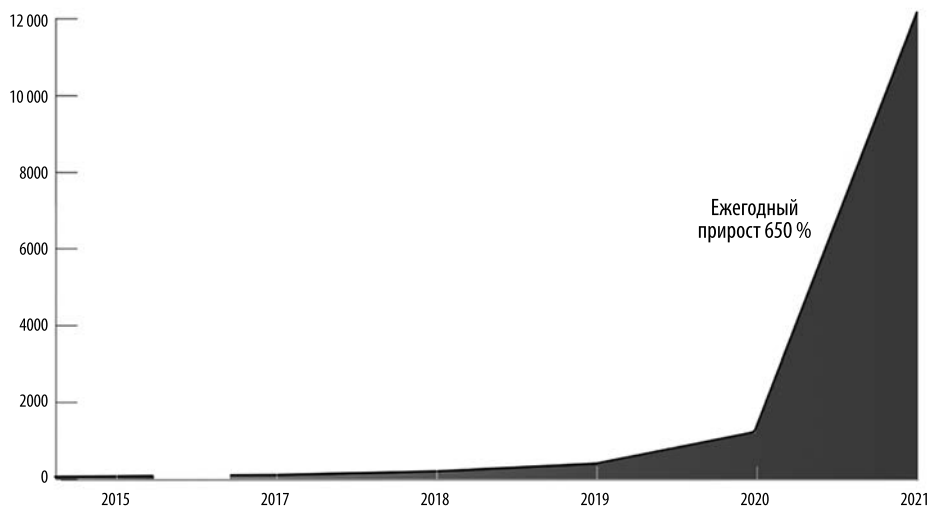


Рис. 8.6

При выборе и потреблении сторонних компонентов или открытого ПО организациям необходимо осознавать связанные с этим риски и уязвимости. Один из способов справиться с этой задачей — создать список доверенных поставщиков и проверенных сторонних компонентов. Важно учитывать качество не только самого компонента, но и сопутствующих артефактов, таких как SBOM. Кроме того, стоит учитывать, насколько оперативно владелец компонента реагировал на отчеты об угрозах в прошлом. Организациям также нужно понимать, что обслуживанием компонентов нельзя пренебрегать. Поставщики могут выпускать обновления для устранения найденных уязвимостей, например, предлагая нижестоящим потребителям обновить компонент.

Усиление среды сборки

Но усиления требуют не только среды разработки и системы, а также и сама среда сборки. Среда сборки позволяет воссоздавать продукты, но в случае компрометации становятся очень эффективным средством распространения скомпрометированного ПО, как это было в случае SolarWinds. Руководство NSA подчеркивает, что система сборки должна разрабатываться и обслуживаться с тем же уровнем безопасности, целостности и устойчивости, с каким это делается в отношении исходного кода и самого продукта.

Злоумышленники используют для компрометации цепочек сборки различные эксплойты, например, внедряются в сети, сканируют систему на предмет уязвимостей, после чего создают и развертывают эксплойт. Далее этот эксплойт может быть эффективно распространен по нижестоящим потребителям без их ведома. Руководство NSA рекомендует ограждать и защищать от неавторизованных сетевых действий не только пайплайн сборки, но и любые другие системы, задействованные в разработке и сборке продукта. При этом также рекомендуется применять защищенные конфигурации, реализовывать системы контроля, такие как сегментация версий, вести аудит и фиксировать все операции доступа и действия.

Основываясь на данных мерах защиты, NSA советует расширить их такими продвинутыми техниками, как *герметичные* и *воспроизводимые сборки*. Герметичные сборки полностью декларативны, неизменяемы и могут выполняться без доступа к сети, а также включать верификацию хеша. Воспроизводимые сборки подразумевают контроль за тем, чтобы исполняемые продукты собирались из одного источника, несмотря на изменчивые метаданные. В данном случае воспроизводимость означает, что они идентифицируют входные артефакты, которые ведут к идентичным побитовым выводам. Развиваются и новые фреймворки вроде in-toto (<https://github.com/in-toto/in-toto>). Он нацелен на защиту целостности цепочки поставок путем отслеживания, чтобы каждая задача в цепочке выполнялась запланированным образом и только авторизованным персоналом, а также чтобы продукт не был подделан во время передачи.

Некоторые определяемые NSA процессы и практики строятся на основе не только фреймворка SLSA, но и SCVS от OWASP.

И хотя подписание ПО становится все более распространенным и признается важной частью защиты цепочки поставок, о чем говорят рост

популярности Sigstore и его использование крупнейшими платформами вроде Kubernetes, существует угроза *эксплуатации сервера подписания*. Криптографические подписи помогают защитить целостность программных артефактов и предоставить гарантии для потребителей. Тем не менее, если подписывающий сервер и сам механизм подписания окажутся скомпрометированы, эти гарантии станут бесполезными. В такой ситуации злоумышленники смогут подписывать вредоносные артефакты и выводы, потребляемые ничем не подозревающими клиентами. Для снижения подобных рисков NSA рекомендует применять строгие методы MFA, контролировать физический доступ к инфраструктуре подписания и в целом усиливать ее, а также совершать подписания в изолированных сегментах сети.

Стоит отметить, что некоторые практики вроде герметичной сборки могут ограничивать возможность использования доверенных инстанций создания временных меток, которые необходимы для определения того, что подписание произошло в момент, когда соответствующие сертификаты были действительны. Относительно недавние инциденты вроде того, что повлиял на Twillio и многих других поставщиков SaaS (<https://clck.ru/3CKWtJ>), показывают, что MFA недостаточно и организациям следует реализовывать устойчивые к фишингу формы защиты, такие как YubiKeys.

Поставка кода

Теперь рассмотрим процесс поставки кода посредством распределенных систем. Разработчики ПО для верификации поставляемого ими ПО, а также всех сопутствующих метаданных должны использовать анализ двоичных компонентов. С помощью инструментов SCA организации могут убедиться, что в комплект поставки не входит ПО с неизвестным провенансом и что в конечные артефакты не были включены чувствительные данные. Опять же, в этом отношении тоже звучал призыв к созданию и включению в пакет поставки SBOM.

Злоумышленники могут пытаться задействовать тактики для компрометации программных пакетов и обновлений по мере их прохождения по системам распространения. Это могут быть, к примеру, атаки типа on-path с целью внедрения зараженного кода или уязвимостей для последующей эксплуатации. Организации могут снижать эти риски с помощью защищенных механизмов передачи, хеширования и цифровой подписи продукта и программных компонентов.

Наконец, существует вероятность прямой компрометации системы распространения через репозиторий, менеджер пакетов или других методов с целью гарантированной доставки взломанных пакетов потребителям. Команды разработки и организации могут противостоять подобным угрозам, защищая от неавторизованного доступа все виды кода, включая репозитории и менеджеры пакетов, а также используя протокол TLS, обеспечивающий шифрование системы распространения.

Приложения к руководству NSA

Прежде чем завершить эту часть главы, стоит добавить, что руководство NSA содержит несколько ценных приложений (рис. 8.7), освещающих следующие темы.

«Зебра». Часть между рассмотренными сценариями угроз и SSDF, которая связывает их конкретными практиками и мерами противодействия, ориентированными на разработчиков, поставщиков и потребителей.

Зависимости. Зависимости и артефакты, которые могут быть предоставлены поставщиком, сторонними организациями или потребителями в помощь самим поставщикам и разработчикам.

SLSA. В этом приложении разбирается, что такое SLSA, каковы его требования, описания и уровни.

Артефакты и чек-листы. Примеры артефактов, описания и цели, такие как чек-листы готовности продукта, дизайн-документы и логи сборки. Руководство приводит примеры того, какую ценность могут принести эти артефакты и кому.

Информативные ресурсы. Полезный набор ссылок из следующих источников:

- BSIMM;
- отдел по информационным технологиям Министерства обороны США;
- Указ Президента США, посвященный повышению уровня кибербезопасности;
- OWASP.

Зеленый — зависимости/артефакты, рекомендуемые для предоставления поставщику в интересах разработчика.

Темно-зеленый — зависимости/артефакты, рекомендуемые для предоставления сторонним поставщикам в помощь разработчику.

Розовый — зависимости/артефакты, которые заказчик рекомендует предоставлять в помощь поставщику/разработчику.

#	Зависимость
1	Передача потребителями отчетов о проблемах
2	Передача хешей по требованию
3	Политики и процедуры SDLC
4	Защищенная архитектура, высокоуровневый дизайн
5	Набор команды профессионалов, а также их обучение особенностям работы с кодом и безопасности
6	Независимое анкетирование отдельных специалистов или команд
7	Независимый аудит безопасности в отношении отдельных специалистов или команд
8	Отдел по надзору за открытым ПО и сопутствующий репозиторий
9	Ресурсы и управление выпуском продукта
10	SBOM
11	Место разработки и сопутствующая информация
12	Сторонние SBOM
13	Сторонние лицензии
14	Примечания к релизу (описание исправленных уязвимостей)
15	Уведомление об уязвимостях
16	Публикация обновлений и патчей для потребителей с целью устранения в продукте обнаруженных уязвимостей
17	Требования и критерии успеха
18	Предполагаемые требования отрасли по части безопасности
19	Передача из операционной среды отчетов о проблемах, получение обновлений и патчей
20	Уведомления об уязвимостях и получение отчетов от пользователей

Рис. 8.7

Многие из этих источников неоднократно упоминаются в документе в виде примеров руководств, от которых отталкивалось NSA.

Рекомендуемые практики для поставщиков

Согласно рекомендациям NSA, поставщики «осуществляют связь между потребителем и разработчиком». Это означает, что они обеспечивают безопасность ПО и сохранение его целостности посредством таких механизмов, как контракты, релизы ПО и уведомление подверженных угрозам потребителей.

Руководство NSA определяет для поставщиков ряд конкретных ответственностей:

- сохранение целостности безопасно поставленного ПО;
- проверка программных пакетов и обновлений;
- поддержание осведомленности об известных уязвимостях;
- принятие отчетов потребителей о проблемах или новых обнаруженных уязвимостях с последующим уведомлением разработчиков для их устранения.

Как и в рекомендациях разработчикам, в данном документе представлены различные примеры угроз, основанные на публикации SSDF. Напомним, что SSDF состоит из всевозможных практик безопасности, разбитых на четыре группы: подготовка организации (PO), защита ПО (PS), создание защищенного ПО (PW) и реагирование на уязвимости (RV). Руководство NSA использует аналогичную структуру и приводит конкретные сценарии угроз для каждой группы вместе с рекомендациями для устранения или снижения связанных с ними рисков.

Подготовка организации

Среди основных требований к подготовке организации на роль поставщика — разработка политик и процессов, обеспечивающих защищенную поставку ПО. Однако недостаточно просто создать их, они должны активно применяться во всей организации и быть знакомы ключевым участникам SDLC.

Когда в компании нет четко определенных политик и процессов, а внутренние команды не знают, как их реализовать, могут возникнуть проблемы с уведомлением потребителей о появляющихся уязвимостях. В результате поставщик не может оказывать поддержку на протяжении жизненного цикла продукта.

Чтобы избежать подобных проблем, необходимо использовать такие меры, как подписание образов и кода, хеширование для сверки отправленных и полученных продуктов. Поставщикам также нужны налаженные процессы исправления ошибок и обновления ПО через защищенные каналы связи.

Защита ПО

Наличие политик и процессов — это хорошо, но организациям также нужно принимать активные меры по защите ПО от неавторизованного доступа, изменения и даже удаления. На этом этапе возможны различные риски компрометации сборки и поставки ПО до его получения потребителем. К примеру, в программу может быть внедрен вредоносный код, создающий бэкдоры.

Для предотвращения подобных рисков NSA рекомендует:

- реализовать RBAC;
- обеспечить защиту конечных точек разработчиками;
- проводить код-ревью;
- осуществлять цифровое подписание кода в процессе поставки потребителю, что включает использование ключей, соответствующих Федеральным стандартам обработки информации серии 140, а при необходимости даже аппаратных модулей безопасности (hardware security modules, HSM).

В руководстве NSA также говорится о том, что поставщикам стоит налаживать механизмы проверки целостности релизов ПО посредством цифрового подписания кода на протяжении SDLC. Это способствует повышению доверия со стороны потребителей и сохранению надежности провенанса. Если поставщики пренебрегают данными мерами, возникает риск появления таких угроз, как вмешательство в ПО до его поставки, что мы наблюдали в некоторых ранее описанных историях атак. И хотя подписание кода помогает повысить уровень доверия и надежность провенанса, для этого требуются контрольные верифицируемые процессы, предшествующие самому подписанию, — такова рекомендация NSA. В противном случае вы можете столкнуться с тем, что потенциально вредоносный или скомпрометированный код окажется подписан и будет рассматриваться как легитимный, что и произошло в инциденте SolarWinds. NSA также советует зеркалить среды сборки, чтобы выполнять независимую компиляцию и упаковку кода, после чего сравнивать полученные хеши. Подобную практику мы встречали и в других руководствах, таких как SLSA, которые компания SolarWinds теперь осуществляет.

Естественно, подписание кода требует грамотного управления ключами, поэтому NSA рекомендует поставщикам хранить закрытые ключи в HSM для снижения рисков их кражи или подмены, а также защищать сопутствующие системы обработки, реализуя, например, доступ с минимумом привилегий.

Организациям также нужны стратегии, которые будут регулировать архивирование и политику длительности хранения устаревших версий ПО. Эти стратегии могут основываться на нормативных требованиях, определяемых отраслью или внутренними установками компании. Как отмечает руководство NSA, архивированное ПО полезно по нескольким причинам: цифровая криминалистика, восстановление гарантированно исправной версии на случай взлома текущей или после какого-то инцидента.

Тем не менее архивированию и защите угрожают определенные риски, такие как небезопасные или недоступные устройства хранения или редкие ревизии архивов. Это может вести к ложному чувству уверенности, которое не оправдается, когда потребуются реально восстановить архивную версию ПО.

Для ослабления некоторых из перечисленных угроз руководство NSA рекомендует задействовать долговременные хранилища и ограничивать доступ правами «только для чтения» во избежание несанкционированных изменений. И хотя организации могут перейти от использования устаревших устройств хранения вроде магнитных лент и сетевых дисков на облачные сервисы, им потребуется учесть уже новые риски, связанные с этой средой. Кроме того, организациям необходимо внедрить конкретные политики архивирования и сохранения, а также автоматизировать их работу с помощью доступных технологий.

Создание надежно защищенного ПО

Создание защищенного ПО требует следования принципам безопасной разработки на протяжении всего SDLC. Если разработчики не применяют защищенных практик, не проходят качественного обучения анализу рисков, не понимают проекта и операционных требований для создаваемого ПО, это определенно создает угрозы безопасности.

Чтобы избежать этого, NSA рекомендует использовать принцип моделирования угроз. Кроме того, важно убедиться, что предполагаемая операционная среда понятна, а требования к ПО включают спецификации защищенной разработки.

Сфера SaaS несет уникальные риски в области создания защищенного ПО ввиду своей модели функционирования. Среди приводимых NSA примеров упоминается возможность случайного попадания уязвимого кода в репозитории, отвечающие за автоматизированные сборки, или отличия в требованиях безопасности среди разных моделей одного решения SaaS, что приводит к неполной проверке компонентов. Эти проблемы можно решить путем

внедрения, соблюдения и верификации общих требований безопасности между поставщиками SaaS и связанными с ними субподрядчиками.

Как мы говорили, современное ПО зачастую содержит большой объем сторонних программ и компонентов. В связи с этим руководство NSA подчеркивает, что нужно проверять, соблюдают ли поставщики такого ПО существующие требования безопасности и не превышают ли компоненты допустимый риск для организации. Использование продуктов сторонних поставщиков несет различные угрозы, такие как отсутствие четких соглашений, неполная информация о реализуемых подрядчиком мерах защиты и его прошлом, а также недостаточная ясность в установлении минимальных требований безопасности для поставщиков.

Чтобы избежать этих проблем, NSA рекомендует:

- следить за выполнением третьими сторонами требований безопасности;
- использовать контрактные соглашения для передачи и обеспечения соблюдения требований безопасности;
- налаживать процессы для противодействия рискам, которые могут внести сторонние поставщики (случайно или намеренно).

В договорах необходимо учитывать раскрытие уязвимостей и уведомления, а также соответствие требованиям защищенной разработки ПО, прописаным в SSDF NIST.

По аналогии с руководствами от NIST NSA рекомендует создавать отдельное хранилище для подтвержденного стороннего ПО, соответствующего требованиям безопасности поставщика, и не использовать стороннее ПО из других источников. Кроме того, NSA советует поставщикам обновлять свои внутренние политики, внося в них требование об обязательном обновлении ПО в случае выхода новых версий, обнаружения уязвимостей или раскрытия рисков.

Среди прочих угроз руководство NSA отмечает неправильную конфигурацию процессов компиляции и сборки, отсутствие ревью и анализа кода, а также недостаточно тщательное тестирование исполняемого кода. Эти проблемы могут привести к внедрению вредоносного ПО в компоненты, компрометации процесса сборки, отсутствию оценки требований безопасности во время ревью и использованию недостаточного тестирования и сканирования на уязвимости в исполняемом коде. NSA рекомендует поставщикам запрашивать у потребителей ясный список требований и моделировать угрозы для всех критических компонентов ПО и систем в пайплайне сборки. Кроме того,

если поставщик заинтересован в сотрудничестве с правительственными учреждениями, агентство советует ему разобраться в требованиях к контролю и соответствию, установленных политиками FedRAMP или OMB.

Придерживаясь отраслевых тенденций по созданию защищенных механизмов, руководство NSA подчеркивает важность изначальной настройки ПО с помощью безопасных методов. В противном случае возникают такие угрозы, как установка ПО без надлежащей конфигурации, ограничение административного доступа и отсутствие регистрации административных действий.

Чтобы устранить подобные риски, NSA рекомендует поставщикам вести логи всех административных действий (и хранить эту информацию не менее 30 дней), использовать MFA для административного доступа, а также отключать сервисы и функциональность, которые не являются необходимыми для ПО.

Реагирование на уязвимости

Наконец, после того как организации подготовились и наладили меры по защите и созданию защищенного ПО, им нужен механизм для реагирования на уязвимости. Многие специалисты считают, что вопрос не в том, появятся ли уязвимости, а в том, когда это произойдет. Уязвимости практически неизбежны, поэтому поставщикам крайне важно иметь механизмы реагирования на них.

Для этого им следует стандартизировать непрерывные процессы их выявления, анализа и устранения. В противном случае возможны такие проблемы, как релиз ПО с известными или нераскрытыми уязвимостями, недостаточные меры по удалению известных уязвимостей или использование компонентов неизвестного происхождения.

Чтобы этого не произошло, NSA рекомендует:

- организовать междисциплинарную команду по анализу уязвимостей;
- выявлять уязвимости с помощью таких мероприятий, как фаззинг и SCA;
- создать PSIRT (команду реагирования на инциденты безопасности), включая публично доступную веб-страницу, где клиенты или исследователи смогут сообщать об уязвимостях, связанных с продуктами поставщиков.

Кроме того, NSA советует отслеживать известные проблемы безопасности и уязвимости, включая связанные с ними показатели CVSS и конкретное

влияние на продукты или компоненты. Учитывая тенденцию индустрии признавать риски со стороны OSS-компонентов, NSA предлагает поставщикам применять SBOM для лучшего понимания таких компонентов и относящихся к ним уязвимостей. Это дополняет корпоративное руководство по улучшению OSS-компонентов в случае обнаружения проблем и уязвимостей, чтобы оградить от них нижестоящих потребителей. Данная рекомендация согласуется с предыдущими нашими высказываниями.

В руководстве NSA также подчеркивается, что внедрение SaaS-продуктов и инструментов способно создать риски для безопасности организации. Об этом мы уже немало говорили в разделе главы 6, посвященном SaaS, поэтому здесь лишь повторим некоторые ключевые рекомендации NSA для поставщиков, использующих SaaS. Они включают применение строгих политик в отношении SaaS-приложений (например, систем контроля) и разработку эффективных моделей анализа, устраняющих пробелы между CSP и потребителем облачных услуг (в данном случае поставщиков).

Как уже упоминалось, руководство NSA для поставщиков, аналогично руководству для разработчиков, содержит ряд приложений, включая «зебру» из сценариев угроз, описанных в SSDF NIST.

Рекомендации для потребителей

Как правило, потребители — это организации, которые получают и развертывают программные продукты в своих средах и архитектурах. В процессе получения, развертывания, использования и обслуживания ПО им необходимо учитывать риски, связанные с цепочкой поставок. Как отмечает NSA, роль потребителя в этой цепочке начинается на этапе получения ПО. В рамках данного процесса должны быть установлены требования, основанные на внутренних политиках организации, а также актуальных внешних нормативных документах.

Для снижения рисков, связанных с отсутствием контроля за соблюдением таких требований, NSA предлагает потребителям ряд мер:

- постоянное обновление требований организации и их интеграция в бизнес-процессы;
- контроль за тем, чтобы специалистам, ответственным за обеспечение безопасности, на этапе получения ПО присваивались отдельные конкретные роли, соответствующие их обязанностям;

- независимая верификация хешей и подписей как исходных продуктов, так и любых последующих их обновлений от поставщика;
- создание групп по обеспечению безопасного получения ПО (acquisition security, ACQSEC) для контроля известных рисков, связанных с ситуациями, когда степень иностранного владения, контроля или влияния (foreign ownership, control or influence, FOCI) на поставщиков в их цепочках поставок является достаточно высокой, чтобы вызывать беспокойство.

Руководство NSA подчеркивает важность того, чтобы потребители проводили оценку как текущего рынка, так и предложений поставщиков. Эти мероприятия обычно происходят во время запросов информации (requests for information, RFI) или запросов предложений (requests for proposals, RFP), когда клиент может определить, соответствуют ли предложения потенциальных поставщиков его требованиям безопасности (рис. 8.8).

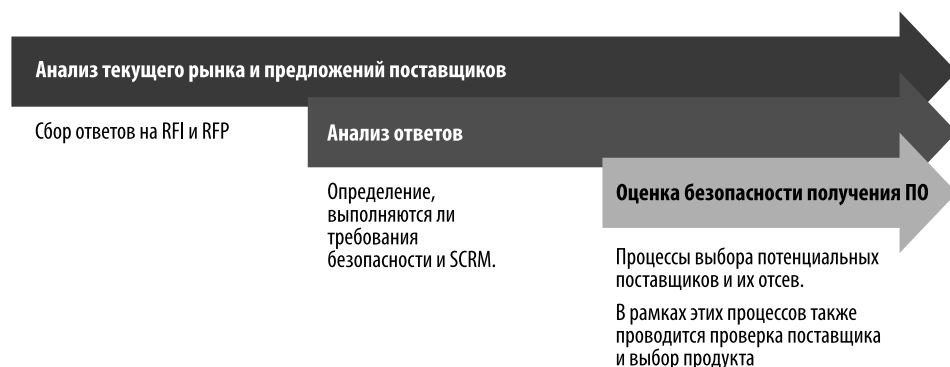


Рис. 8.8

NSA подчеркивает, что оценка угроз является ключом в анализе и управлении рисками в отношении конкретных поставщиков и продуктов. В качестве примеров, которые могут повлиять на потребителей, приводятся получение продукта, не прошедшего оценку, обнаружение уязвимостей, упущенных при первичной оценке, и изменение правил владения, которое привело к неизвестному внешнему влиянию на продукт или поставщика.

В руководстве указывается, что одной из основных проблем для потребителей является недостаточная видимость компонентов продукта, что затрудняет его оценку и принятие взвешенных решений. Речь идет о встроенных компонентах, библиотеках, модулях и других элементах, влияющих на

прозрачность ПО и необходимость понимать его состав, в том числе при оценке продуктов от поставщиков.

Для устранения этих рисков руководство рекомендует потребителям верифицировать содержимое ПО с помощью SBOM, а также анализировать внешние службы оценки, используемые поставщиком, чтобы убедиться в их независимости и объективности. Стоит также проверять наличие у внешних служб необходимого опыта в предметной области и следить за тем, чтобы указанные в SBOM сторонние поставщики подвергались той же оценке, что и прямые. Все это позволяет осознать масштаб рисков, которые представляет обширная цепочка поставок ПО, а также понять, что подобные риски могут быть внесены поставщиками любого уровня, не всегда прямыми или даже третьими лицами.

Здесь ключевым механизмом противодействия являются контракты. Они требуют от потребителей устанавливать требования к цепочкам поставок в форме условий договора, например, в виде технического задания, и различных запросов. Необходимо также привлекать квалифицированных специалистов, отвечающих за получение сторонних продуктов, обладающих достаточными знаниями для того, чтобы решать соответствующие задачи и корректно составлять условия контрактов.

В руководстве NSA отмечается потенциальный риск, связанный с заключением договоров с поставщиками, находящимися в иностранной юрисдикции или имеющими подозрительных субподрядчиков. Подобная ситуация может привести к более обширной проблеме, когда в контракте не отражены требования к SCRM, такие как обязательное предоставление поставщиком SBOM. Сложности также возникают, когда потребители недостаточно тщательно анализируют поставщиков, чье ПО было скомпрометировано, особенно если сопутствующие артефакты вроде хешей и подписей дают ложное чувство безопасности.

Чтобы избавиться от этих рисков, NSA рекомендует требовать от поставщиков историю происхождения их продуктов и протоколы самоаттестации системы безопасности в процессах разработки и сопутствующей инфраструктуре. Самоаттестация должна охватывать все практики защищенной разработки и включать подпись официального представителя поставщика, ответственного за эти процессы, а также хеширование и электронные подписи предоставляемых продуктов. Потребителям, в свою очередь, следует стремиться устанавливать требования для повышения прозрачности цепочки поставки и принадлежности продуктов, а также видимости вышестоящих поставщиков.

Более того, пользователям нужно требовать от поставщиков отправки всех артефактов в стандартизированном формате SBOM (например, SPDX или CycloneDX), сам SBOM для обновлений ПО, а также своевременного информирования в случае изменения основных атрибутов, связанных с правами владения поставщика, его расположением и контролем со стороны другого государства. Потребители должны запрашивать от поставщиков уведомления о киберинцидентах, расследованиях, мерах противодействия и любом влиянии на используемый ими продукт или среду его разработки.

Еще один ключевой аспект для пользователя — развертывание ПО в собственных средах и системах. Этот процесс начинается с получения продукта от поставщика и требует соблюдения мер предосторожности. Пренебрежение ими создаст различные риски. Например, клиент может получить не тот продукт, который заказывал, или без соответствующей документации и артефактов, а также стать жертвой компрометации продукта в цепочке его поставки. Поэтому заказчикам нужно внедрять политики и механизмы для снижения рисков, связанных с возможным получением скомпрометированных продуктов. Для этого можно проверить безопасность инфраструктуры, используемой поставщиками для распространения ПО, с помощью хешей и подписей, целостность поставленного продукта, включая его компоненты, а также проанализировать SBOM данного продукта.

Помимо прочего, потребителям ПО рекомендуется применять функциональное тестирование. Обычно оно подразумевает создание тестов и тестовых сред для продуктов с последующим выполнением этих тестов и сбором результатов. Если функциональное тестирование не проводить, пользователь рискует стать жертвой таких угроз, как неизвестное изменение функциональности продукта либо присутствие в нем подозрительных компонентов. Подобных угроз можно избежать, если сохранять тесты и тестовые среды для будущего использования, а также верифицировать содержимое SBOM проверяемого продукта.

NSA рекомендует потребителям в рамках эффективного управления ИТ-инфраструктурой создавать комиссию по надзору за конфигурацией (configuration control board, ССВ) для анализа выявленных проблем функциональности и надежности, определения рисков ПО и принятия конечного решения об использовании продукта. Тем не менее в случае слабой реализации схемы работы ССВ могут возникнуть угрозы, например, комиссия будет получать неполные или неточные отчеты от поставщиков или окажется в затруднении при принятии объективных решений из-за различных

факторов вроде отсутствия опыта в какой-либо сфере. Чтобы этого избежать, потребителям следует убедиться, что ССВ состоит из соответствующих специалистов или имеет связь с таковыми. Необходимо также обеспечить документирование требований SCRM и передачу отчетов в ССВ по защищенным каналам. Риски для пользователей существуют и на других этапах внедрения продукта, включая интеграцию, обновления, усовершенствования и даже фазу завершения эксплуатации (end-of-life, EoL). Для минимизации рисков, связанных с компрометацией, потребителям стоит применять хорошо задокументированные процессы использования программных продуктов на каждой из этих стадий. Злоумышленники перед активацией вредоносной функциональности будут стремиться отключить меры защиты, как было сделано в случае SolarWinds. Кроме того, разработчики продукта могут либо намеренно, либо случайно создавать бэкдоры, подвергающие потребителей риску.

При этом клиентам недостаточно мер защиты интеграции, обновлений и апгрейдов, нужно учитывать еще и этап EoL продукта. Данный этап наступает по разным причинам, в том числе переход пользователя на альтернативные решения, исчезновение потребности в продукте или прекращение его обслуживания разработчиками. Тем не менее интеграция продуктов в средах потребителя зачастую сопряжена с отношениями доверия и может включать учетные данные, разрешения, сетевой доступ и не только. Для устранения этих рисков клиент должен обеспечить надлежащее удаление из своей среды всех учетных данных, разрешений и доступа, связанных с поставщиком и его продуктом. Более того, он также может удалить продукты поставщика из списка допустимых приложений или даже явно отклонить их посредством сетевых инструментов и средств защиты. Команды службы безопасности (SecOps) обычно хорошо подготовлены к созданию и реализации планов по устранению рисков, связанных с EoL продуктов в средах потребителя.

В качестве еще одного актуального для потребителей аспекта руководство NSA называет обучение и передачу навыков, связанных с безопасным развертыванием и использованием программных продуктов. Исследования безопасности подтверждают, что недочеты конфигурации являются ключевым вектором атаки, повышающим риск взлома данных. Злоумышленники могут воспользоваться слабыми местами и векторами атаки независимо от того, намеренно или случайно поставщик предоставил вредоносное или небезопасное ПО. Поэтому, чтобы не подвергнуть себя опасности, стоит всегда придерживаться отраслевых стандартов и рекомендаций производителя.

Последний и, пожалуй, наиболее важный момент, который подчеркивает руководство NSA, — это решающая роль мер по управлению рисками цепочек поставок в снижении общих угроз, связанных с потреблением организацией ПО. Развитые команды и процессы SCRM помогают потребителям сохранять безопасное состояние программного обеспечения в своих сетях и инфраструктуре, а также эффективно отслеживать изменения продуктов и события безопасности, которые несут риски.

Без реализации грамотных процессов SCRM потребители могут стать жертвами вредоносного ПО, которое не подвергается проверке инструментами безопасности, а также лишено подходящего мониторинга. Завершается же руководство NSA обширным списком рекомендаций, касающихся SCRM. Среди них использование проверки целостности во всех развернутых агентствах безопасности, развертывание инструментов мониторинга за пределами периметра организации и реализация моделирования угроз на основе конкретных рисков продукта и данных разведки.

И хотя очевидно, что далеко не каждая организация сможет внедрить все эти меры, особенно в секторе малого и среднего бизнеса, многие потребители могут начать двигаться в сторону укрепления собственной системы использования ПО, внедряя практики и процессы, указанные в руководстве NSA.

Резюме

В этой главе мы разобрали существующие руководства, связанные с общественным сектором и правительственными субъектами. Сюда входили руководства от NIST и NSA. И хотя некоторые требования и рекомендации могут быть применимы только к федеральным агентствам США и их промышленной базе, все же важно понимать, как к решению вопросов безопасности цепочек поставок подходит публичный сектор. В следующей главе мы разберем ключевые вопросы, связанные с прозрачностью ПО в отношении операционных технологий (operational technology, OT) и ICS.

9

Прозрачность ПО в операционных технологиях

Операционные технологии отвечают за работу важнейших процессов в мире, включая шахтные пусковые установки, системы обороны, водоочистку, аэропорты, энергоснабжение и прочее. Зачастую эти среды надежно изолированы от Интернета и могут иметь определенные ограничения в отношении внешних подключений и использования облачных или мобильных возможностей. В связи с этим многие техники, которые мы привыкли применять для валидации ПО, здесь не подходят.

Например, как можно проверить сертификат подписи кода на присутствие в списке отозванных сертификатов, когда Интернет недоступен? Как найти и определить хеш компонента в SBOM, если он совпадает с известными записями репозитория вредоносного ПО? Будете ли вы по-прежнему считать проблемой чрезмерно длительные сроки действия сертификатов TLS, если обновление доверенной информации затруднено?

Кроме того, многие из этих продуктов создаются или обслуживаются на территории государств, которые мы можем считать враждебными. Тема провенанса ПО становится особенно актуальной в свете того, что требования соответствия Закона о полномочиях в сфере национальной обороны США (National Defense Authorization Act, NDAA) и различные указы президента США определяют, каким государствам разрешено поставлять продукты для критической инфраструктуры, а каким — нет.

Например, в 2020 году вышел Указ 13 920 о «Защите магистральной сети энергоснабжения США» (<https://clck.ru/3CKWvN>), нацеленный на снижение

рисков получения продуктов от иностранных государств. За этим указом вскоре последовал запрет на получение оборудования для энергосетей от Китая.

Впоследствии оба этих указа были аннулированы и включены в единый Указ 14 017, посвященный «Защите цепочек поставок США» (<https://clck.ru/3CKWvg>). Несмотря на это, по-прежнему очевидно, что разработка политики управления рисками цепочек поставок для критической инфраструктуры и обороны является государственным приоритетом.

Указ 14 028, который вызвал повышенную активность в сфере прозрачности ПО, включая SBOM и маркировку программного обеспечения, был разделен на три основные части, в одной из которых NTIA была поставлена задача определить атрибуты SBOM. Эта и другие задачи, такие как определение форматов для обмена информацией об использовании уязвимостей, с тех пор были делегированы CISA. Становится отчетливо ясно, что среды операционных технологий в ближайшем будущем будут оставаться в центре внимания движения за повышение прозрачности ПО в США.

Кинетический эффект ПО

Одним из необычных пунктов Указа 14 028 было требование дать определение «критическому ПО». Как мы уже говорили, эти определения довольно сложно трактовать. В каком-то смысле любое ПО может стать критическим — все будет зависеть от того, как его использовать. Одним из ранее приведенных примеров является библиотека отрисовки изображений, применяемая в Интернете для показа фотографий милых котиков или видео. Мало кто стал бы рассматривать ее как критическое ПО. Но что, если использовать ее для отрисовки данных телеметрии на электростанции? Это уже совсем иной сценарий. Одно и то же ПО, применяемое различным образом, будет иметь различную степень критичности. Одним из наиболее интересных для специалистов аспектов операционных технологий является возможность таких систем выходить за традиционные виртуальные границы и оказывать физическое влияние на реальный мир. Программное обеспечение контролирует замки тюремных камер, отвечает за смесь химических растворов для очистки питьевой воды, управляет энергосетями и даже системами вооружений. Если бы эта библиотека для отрисовки котиков использовалась в системах наведения ракет и от нее зависели бы жизни солдат, такое применение определенно считалось бы особо критическим.

В операционных технологиях опасность того, что дефекты ПО приведут к гибели людей, может быть катастрофической.

За последние несколько лет было множество отчетов о том, что вирусы-вымогатели замедляли работу медицинских учреждений, а в одном из таких случаев, произошедшем в 2020 году в Германии, предположительно, это даже привело к смерти пациента. И хотя прямых доказательств нет, исследование компании Pew Charitable Trusts (<https://clck.ru/3CKWwJ>) показало, что эти события могут повлечь за собой человеческие жертвы. Чем больше наше программно-контролируемое общество совершенствует средства дистанционной медицины и прочие цифровые услуги, тем больше оно создает возможностей для киберпреступников.

Поэтому число атак на энергосети тоже увеличилось. Например, если нанести удар по системам водоочистки или энергоснабжения, это непосредственно скажется на здоровье населения. Кроме того, военная машина государства зависит от портов, железных дорог, электричества, водоснабжения и прочих аспектов жизнедеятельности, что делает крайне привлекательным тактику киберсаботажа. Как еще можно наносить физический вред государству из разных точек планеты?

Одним из лучших примеров, в котором физический эффект был достигнут посредством ПО, является атака вирусом Stuxnet. И хотя анализ показал, что изначальное проникновение произошло через зараженную USB-флешку, именно вредоносное ПО вызвало массивную задержку процессов обогащения ураном в иранской ядерной программе. Некоторые аналитики предположили, что это примерно на два года отодвинуло прогресс обогащения в 984 зараженных центрифугах (<https://clck.ru/3CKWwg>).

Наиболее тщательный анализ атаки Stuxnet провела компания Langer, Inc, имеющая глубокий опыт в области ICS. В 2013 году с последующим обновлением в 2017-м ее специалисты выпустили работу *To Kill a Centrifuge* (<https://clck.ru/3CKWwz>). В ней они описали, как конкретно реализовывалась атака и что она означала для ядерной программы Ирана и остальной части индустрии в контексте стремления предотвратить будущие атаки на критическую инфраструктуру.

Вирус Stuxnet создавался для снижения эффективности центрифуг. И хотя его внедрение было осуществлено в нескольких точках, считается, что первичной целью был ядерный комплекс в Нетензе. Летом 2010 года в центрифугах было обнаружено аномальное число сбоев, но не столь явных, чтобы

вызвать подозрение. Очевидно, что здесь все было очень тщательно и тонко продумано, поскольку разработчики вируса легко могли сделать так, чтобы он просто уничтожил центрифуги.

Предполагается, что изначальное заражение произошло через машины под управлением Windows, скорее всего, через те же мобильные устройства, с помощью которых инженеры программировали и управляли программным обеспечением и оборудованием Siemens, отвечавшим за процесс обогащения. Причем это вредоносное ПО еще несколько лет не выявлялось антивирусами, пока его не использовали для реализации эксплойтов нулевого дня. А поскольку он не передавался по сети до первичного заражения, традиционные методы анализа сетевого трафика и антивирусные решения не могли предотвратить его распространение.

Но стоит отметить, что, помимо трудностей в обнаружении этого вируса с помощью традиционных методов, он был написан таким образом, чтобы незаметно и постепенно ухудшать качество работы и в конечном счете приводить к износу роторов, используемых в системе давления. Меняя местами циклы медленного и быстрого вращения, вирус заставлял работать систему на пределе физических возможностей. И такое изменение в поведении системы показывает, как легитимная модификация кода может оказать на систему серьезное влияние.

Риски устаревшего ПО

Не улучшает ситуацию и то, что значительная часть ПО в сфере операционных технологий является очень старой и представляет те же проблемы, что и в случае проверки устаревших продуктов. Важнейшей характеристикой систем ОТ является отказоустойчивость. Как сказал эксперт по ICS Даниэль Эренрайх, эти среды заточены под безопасность, надежность и продуктивность, а не конфиденциальность, целостность и доступность (<https://clck.ru/3CKX2u>).

Так как же все это реализовать? В основном это достигается путем глубокого инжиниринга в сочетании со строгой культурой безопасной разработки, когда характеристики системы подвергаются моделированию угроз. К подобным средам активно применяются стандарты вроде ISA84 (<https://clck.ru/3CKXDx>), и проекты чаще фокусируются на базовой безопасности, нежели на киберзащите. В компании с критической инфраструктурой каждая встреча начинается с инструктажа по технике безопасности, чтобы сотрудники уделяли этому особое внимание. В таких организациях строго

обязательно ношение защитных касок и костюмов. Кроме того, некоторые среды являются довольно статичными, поскольку изменения могут повлечь риски. После того как инженер провел заводские приемочные испытания (Factory Acceptance Test, FAT), чтобы убедиться в правильном функционировании системы, а также испытания на месте эксплуатации (Site Acceptance Test, SAT), в которых оцениваются безопасность и меры защиты, система преимущественно считается «зафиксированной», то есть никаких дополнительных изменений не допускается. При этом типичные изменения происходят, например, вносятся небольшие корректировки в программируемые логические контроллеры, исправления для некритических или вспомогательных систем или когда возникают новые задачи, но существенно обычно ничего не меняется.

И хотя это ограничивает возможность внедрения мер защиты, также снижается и вероятность сбоя при запланированном изменении в контексте компрометации цепочки поставок. Отсутствие изменений в сочетании с ощущением (не всегда оправданным) изолированности среды, порой ведет к ложному чувству безопасности. Однако верно и то, что модификации снижают вероятность атаки через запутанные зависимости, влияющие на систему ОТ, по крайней мере, до момента запланированного изменения. Одна из сложностей заключается в том, что для действительно изолированных от Интернета систем бывает трудно проверить достоверность и безопасность ПО. Зачастую программное обеспечение попадает на объект через ноутбук инженера, а не скачивается с доверенного источника. В некоторых случаях, например на ядерных электростанциях, принято требовать установки пункта сканирования на вредоносное ПО. Но, как мы видели на примере Stuxnet, данный подход не всегда эффективен.

В начале 2000-х бытовало мнение, что системы ОТ слишком сложные и закрытые, поэтому никто вне самого предприятия не понимал, как они работают. Инструментов для атаки было мало, а документация, которая помогла бы злоумышленникам в атаке тысяч потенциальных протоколов ICS на объектах, встречалась и того реже. Однако цифровизация и стандартизация стека протоколов ОТ вкупе с информационным обменом в Интернете снизили эффективность методов обеспечения безопасности, основанных на маскировке. В настоящее время существуют стабильные эксплойты и инструменты атаки в таких фреймворках, как Metasploit, регулярно проводятся хакерские соревнования в отношении ICS, например Pwn2Own в рамках ежегодной конференции S4 ICS Security Conference (Майами-Бич), и многие другие аналогичные события.

Ландшафт меняется, и новые подходы к защите программного обеспечения зачастую не учитывают область устаревшего ПО. Большинство инноваций в сфере безопасности цепочек поставок применяются к SaaS, контейнерам и современным фреймворкам разработки ПО, но редко учитывают проблемы программ, разработанных давно, за исключением реверс-инжиниринга инструментов, ориентированных на обнаружение уязвимостей в прошивках аппаратных средств ICS. В связи с этим возникает вопрос: как быть с продуктами, поддержка которых прекращена, а сам поставщик перестал функционировать? Обычно приходится выбирать между огромными затратами на замену старого ПО, использованием виртуальных патчей или защитных мер в надежде уменьшить поверхность атаки.

Релейная логика и заданные значения в системах управления

Программируемые логические контроллеры, удаленные терминалы и другие аналогичные средства ОТ выполняют простые автоматические программы и зачастую программируются с помощью реле, языка функциональных блоков и прочих несложных методов. Они описывают серию входов или выходов и включают логику либо параметры для принятия решений. Это может быть простейший выключатель или же устройство со сложной логикой, основанной на вспомогательных входных данных вроде температуры, вращений в минуту, напряжения, давления и других. Программа зачастую определяет диапазон допустимых значений и принимает решение о том, являются ли они нормальными.

При обсуждении кинетических эффектов ПО стоит также уточнить, что представляет собой уязвимость. Нужен ли нам эксплойт или модуль Metasploit, чтобы нанести урон? Что, если простое состояние данных может вызвать катастрофическое влияние? Нам не нужна система CVE, но, возможно, мы можем манипулировать данными датчиков так, что они будут сообщать ложное состояние. Или, возможно, мы можем изменять заданное значение, определяющее «нормальность» показателя. К примеру, нам известно, что состояние сбоя для события чрезмерного давления наступает при 34,47 бар, а заданное значение представляет безопасные 24,13 бар. В таком случае, если произойдет неавторизованное изменение этого значения на 1000 бар, оно может оказаться более катастрофичным, чем эксплойт Log4Shell для этой системы, вызвав взрыв котла.

Программируемые логические контроллеры обычно содержат очень простые механизмы защиты, такие как физический ключ, определяющий, находится ли устройство в состоянии работы или программирования, и использующийся для внесения изменений. Примером одной из известных атак на механизмы защиты является вирус Trisis, который поразил противоаварийную защиту Triconex компании Schneider Electric. Эта атака оказалась успешной отчасти из-за того, что устройство было переключено в режим программирования, то есть допускало внесение изменений, а отчасти потому, что организация недостаточно хорошо понимала, как должны выглядеть нормальные показатели, и игнорировала сбои. Но были и другие ситуации, когда на многих устройствах реализовывалось изменение аппаратной прошивки в обход физического ключа, контролирующего программирование. И хотя лучшей практикой здесь является сохранение устройства в активном режиме, это не гарантирует, что комбинированная компрометация цепочки поставки его прошивки не приведет к подобным атакам в будущем.

К примеру, в случае с Triton атака была реализована посредством многоэтапной операции, о чем пишут исследователи Idaho National Laboratory Cybersecurity for the Operational Technology Environment (CyOTE), изучающие методологию сортировки наблюдаемых показателей и принятия действий на их основе. Специалисты CyOTE в своем практическом исследовании Triton (<https://clck.ru/3CKXEm>) установили следующую цепочку событий.

Шаг 1. Эксплуатация компьютерной сети: этот шаг стал возможен благодаря слабому брандмауэру, разрешающему доступ к компьютерной сети.

Шаг 2. Переход с компьютерной сети на сеть операционных технологий: хакеры скомпрометировали инженерную рабочую станцию внутри ОТ-сети и развернули полезную нагрузку, имитирующую легитимное приложение Triconex, `trilog.exe`.

Шаг 3. Разработка атаки на ОТ: затем хакеры организовали точку контроля, определив целевые системы для последующей атаки, а также установили, какие устройства противоаварийной защиты находятся в активном состоянии.

Шаг 4. Реализация атаки на ОТ: на устройства Triconex несколько раз был передан неавторизованный код, сначала для тестирования, а затем для внедрения вредоносного шелл-кода и новой релейной логики, изменившей их поведение.

Шаг 5. Сопутствующая атака — маскировка: используя механизмы уклонения от обнаружения, вредоносная программа отключила проверку чтения

памяти с произвольным доступом и памяти «только для чтения» (RAM/ROM), а также те устройства, которые находились в режиме программирования, но только после того, как вирус изменил их прошивку и допустил выполнение даже в активном режиме.

Шаг 6. Результат атаки на ОТ: конечным результатом стало полное отключение мер защиты системы с возможностью включения на основе программной логики.

И хотя при этой атаке использовался традиционный подход, когда сначала компрометируется компьютерная сеть, а потом происходит переключение на ОТ, отсутствие валидации прошивки и кода, которое поспособствовало ее реализации, полностью аналогично атакам на цепочки поставок, которые мы видели. На наш взгляд, это образцовый случай, показывающий, как отсутствие валидации ПО и прошивок может поставить под угрозу безопасность и потенциально привести к человеческим жертвам. Из него можно сделать и другие выводы, например о необходимости правильной сегментации, эффективного мониторинга и реализации в средах процессов для обнаружения аномалий и реагирования на инциденты. Но правда в том, что если бы данный продукт для выполнения требовал подписанного исполняемого файла, то эта атака наверняка оказалась бы предотвращена.

Поверхность атаки ICS

Как уже говорилось, поверхность атаки ICS в сравнении с корпоративной ИТ-инфраструктурой обычно минимальна. Большинство сред в этой сфере сильно сегментированы, и Министерство энергетики по указанию раздела 5726 Закона о полномочиях в сфере национальной обороны США на 2020 год проспонсировало исследование, проведенное Оперативной группой по обеспечению безопасности энергетической инфраструктуры (Securing Energy Infrastructure Executive Task Force, SEI ETF). В результате было создано несколько эталонных архитектур, включая рекомендованную стратегию сегментации для энергосетей на основе вида объекта. Пример такой стратегии показан на рис. 9.1.

Кроме того, стандарты по защите критической инфраструктуры (Critical Infrastructure Protection, CIP) корпорации NERC (North American Electric Reliability Corporation), регулирующие Североамериканскую систему энергоснабжения, и требования семейства стандартов IEC 62443 предписывают использовать сегментацию для защиты ОТ-систем от прямого воздействия и недоверенных корпоративных ИТ-сред.

Профиль подстанции

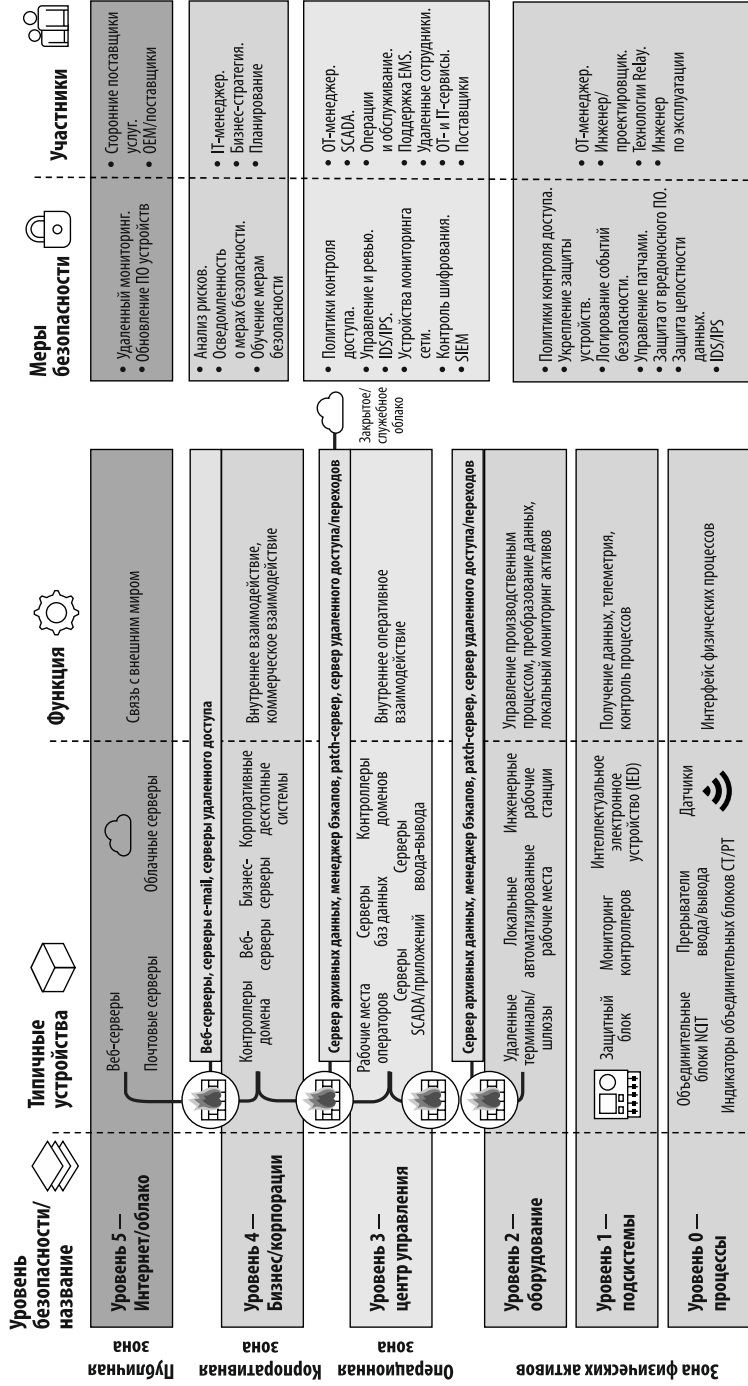


Рис. 9.1

Источник: <https://clck.ru/3CKZ4g>, Министерство энергетики США, публичная информация

В Интернете остаются тысячи объектов ICS, многие из которых имеют слабую аутентификацию или не имеют ее вовсе. На популярном сайте Shodan есть целая страница, посвященная обнаружению таких систем (<https://clck.ru/3CKXFK>). Как показывает этот ресурс, даже поиск конкретных продуктов ICS дает тысячи результатов, многие из которых являются уязвимыми и используются в критических процессах.

Что это может значить для организаций критической инфраструктуры в свете акцента на сегментации активов, недоступных для Интернета? Многие атаки до сих пор основываются на первичном проникновении в корпоративные компьютерные сети посредством фишинга или иных способов с последующим переключением на систему операционных технологий. Но если эти среды действительно разделены, организациям стоит обратить внимание на то, как ПО попадает в среду через контрагентов, предоставляющих внешние USB-ключи или CD для обновления прошивок. Эти векторы тесно связаны с физической безопасностью, что требует комплексной стратегии контроля за внедрением ПО. То же касается поставщиков продуктов. Они должны предоставлять операторам механизмы для локальной валидации образов, если онлайн это сделать невозможно.

Умные электросети

Умные электросети — это модернизированная, цифровая версия традиционных сетей, задействующая передовые технологии для повышения эффективности, надежности и гибкости. Она включает использование датчиков, коммуникационных сетей и систем контроля для мониторинга и управления потоком электричества, позволяя интегрировать возобновляемые источники электроэнергии и управление спросом. Это дает возможность более рационально использовать распределенные энергоресурсы (distributed energy resources, DER), такие как солнечная и ветряная энергия, и реализовывать программы прогнозирования спроса для регулирования потребления энергии в периоды пиковой нагрузки.

Следуя инициативе Администрации Президента США по спонсированию инноваций в сфере возобновляемой энергии и развитию инфраструктуры, DER быстро становится актуальной темой, так как в течение ближайших лет (с момента публикации книги. — *Примеч. пер.*) на нее планируется выделение миллиардов долларов. Речь идет о небольших системах генерации энергии вроде комплекса солнечных панелей на крыше или солнечных ферм. Примером таких систем являются проекты компании NextEra Energy,

представляющие солнечные панели и ветрогенераторы, накапливающие энергию в аккумуляторах, подключенных к распределенной сети. Умные электросети позволяют интегрировать DER за счет коммуникации в реальном времени и механизмов контроля между сетью и распределяемыми ресурсами. Такая связь сильно отличается от традиционно разрозненной и изолированной инфраструктуры энергосетей. В этом смысле риски прямого доступа к таким устройствам создают для их операторов новые угрозы кибербезопасности.

Применяемые в DER инверторные технологии становятся все более важной частью умных электросетей. Эти технологии преобразуют постоянный ток в переменный, который может использоваться энергосетью и обеспечивать энергией систему самой сети, например устройства контроля напряжения и частоты. Инверторы и прочие управляющие системы DER подключены к Интернету и позволяют проводить удаленный мониторинг и контроль. Естественно, это увеличивает поверхность атаки и создает потенциальные уязвимости для эксплуатации.

Помимо DER, для измерения и передачи подробной информации об энергопотреблении сетям и потребителям используется передовая измерительная инфраструктура, которая позволяет более точно проводить расчеты стоимости и реализовывать программы регуляции потребления. Более того, с помощью новейших средств аналитики и алгоритмов машинного обучения выполняется анализ больших объемов данных, генерируемых умными электросетями, для выявления паттернов, прогнозирования сбоев оборудования и оптимизации работы сети.

Это повышение сложности открывает новые возможности для динамической корректировки на случай обнаружения опасных ситуаций. Однако это также создает условия для создания программным обеспечением катастрофических последствий. Время для раннего предупреждения проблем может сократиться, особенно из-за погодозависимой природы ресурсов DER и появления новых паттернов атак, акцентированных на емкости и разряде батарей, что может привести к дефициту энергии. Таким образом, атаки на основе логики могут стать новой нормой для хакеров, желающих воздействовать на новую и более совершенную энергосистему, которую мы проектируем сегодня.

Вся эта сложность сопровождается и потребностью в стандартизации технологий. Протоколы коммуникации и модели данных, используемые в системах подстанций, определяются международным стандартом IEC 61850. Он играет ключевую роль в умных электросетях, предоставляя единый язык

и схему для взаимодействия различных устройств и систем, составляющих энергосеть. Одно из основных преимуществ IEC 61850 в том, что он позволяет интегрировать различные устройства и системы от разных поставщиков, что повышает эффективность и экономичность развертывания технологий умных электросетей. Стандарт также предоставляет общую модель данных для различных устройств и систем, упрощая обмен информацией между ними и выполнение продвинутой аналитики данных сети. С учетом развития рынка IoT-устройств, формирующих умные электросети, наша структура безопасности начнет больше соответствовать типичным для IoT поверхностям атаки. Гонда Ламберик в своей статье от 2021 года изучила поверхность рисков цепочек поставок в отношении IoT и конкретно умных электросетей (<https://clck.ru/3CKXGa>).

В этой статье с целью снижения рисков для подобных электросетей автор призывает опираться на модель нулевого доверия, что особенно интересно, поскольку в традиционных электросетях зачастую подразумеваются доверительные отношения. В большинстве систем нет ни аутентификации, ни шифрования. Коммуникации требуют нулевой или минимальной задержки и времени конвергенции сети, чего традиционные технологии безопасности обеспечить не могут. Такая повышенная открытость умных электросетей говорит о том, что традиционные подходы к защите подобной инфраструктуры путем предоставления пользователям ограниченного доступа не сработают.

Более того, если взглянуть на системы стандартов для этих сред, то традиционные NERC CIP и IEC 62443 по-прежнему применимы, но многие опираются на более привычные фреймворки IoT от NIST и прочих организаций.

Резюме

В этой главе мы обсудили кинетические эффекты ПО и его способность оказывать физическое влияние на нашу жизнь. Кроме того, в ней были освещены риски, связанные с устаревшим ПО и релейной логикой в промышленных системах управления. Мы более развернуто разобрали тему поверхности атак ICS и угроз, которые она создает для критической инфраструктуры, обороны и общества, а также поговорили об умных электросетях. В следующих главах речь пойдет о практическом руководстве для поставщиков и потребителей ПО.

10

Практическое руководство для поставщиков

Несмотря на то что мы уже обсудили многие руководства, следующие две главы будут посвящены практическим рекомендациям для поставщиков и потребителей. В них будут обобщены инструкции из уже рассмотренных нами источников, а также лучшие практики отрасли и советы, основанные на личном профессиональном опыте авторов книги.

Не все приведенные указания, практики и процессы окажутся подходящими для каждого поставщика. Как и в случае с разнообразной экосистемой потребителей ПО, поставщики тоже отличаются по части ресурсов, опыта и ограничений. Зачастую это будет двойственное разделение между потребителями и поставщиками ПО в зависимости от отрасли, бизнес-отношений, нормативных требований и договорных обязательств, определяющих, что должно или не должно реализовываться, предоставляться или раскрываться и при каких обстоятельствах.

Раскрытие информации об уязвимостях и реагирование на них

В предыдущих главах мы немало говорили о специализированном фреймворке SSDF NIST, так что есть смысл приводить его в качестве примера.

Как уже отмечалось, SSDF призывает организации реализовывать грамотные практики для защиты разработки ПО. Рекомендуемые практики

в нем распределяются по четырем группам: «Подготовка организации», «Защита ПО», «Создание надежно защищенного ПО» и «Реагирование на уязвимости». Одной из ключевых тем во всех этих группах выступает возможность организаций и поставщиков раскрывать уязвимости и реагировать на них. Это подчеркивается в таких областях, как реализация политик, направленных на раскрытие уязвимостей и их устранение. Наличие таких процессов создает несколько ключевых преимуществ для поставщиков, например возможность информировать исследователей о своих практиках управления уязвимостями и получать от них данные об обнаруженных потенциальных угрозах.

Любой, кто работал в сфере кибербезопасности или разработки ПО достаточно долго, знает, что вопрос не в том, появятся ли уязвимости, а в том, когда это произойдет. В связи с этим для организаций крайне важно использовать систематизированные процессы реагирования и инструменты для информирования клиентов об уязвимостях. Это традиционно происходит с помощью статических мер уведомления, таких как сайты, электронная почта и PDF-документы. Тем не менее, как мы уже писали, сейчас отрасль переходит на автоматизированные машиночитаемые средства оповещения. Наиболее заметными примерами являются CSAF и VEX. Последний часто используется вместе со SBOM, показывая фактическую эксплуатируемость уязвимостей, связанных с получаемыми от поставщиков ПО и продуктами. Задача CSAF — позволить стейкхолдерам автоматизировать создание и потребление информации о безопасности. При этом он стремится к поддержке VEX, хотя на момент написания книги данная возможность находится в разработке. Еще один яркий пример — CycloneDX, ведущий отраслевой формат SBOM, который также имеет свой «Список уязвимостей» (<https://clck.ru/3CKXHA>), позволяющий обмениваться сведениями об уязвимостях между системами и проектами по их обнаружению.

Появление CSAF и VEX демонстрирует стремление отрасли перейти от традиционных статичных процессов обмена информацией об уязвимостях к машиночитаемым и автоматизированным. Это повысит их эффективность, масштабируемость и снизит вероятность ошибок.

Наличие программы по раскрытию уязвимостей (Vulnerability Disclosure Program, VDP) является признаком зрелости поставщиков ПО и повышает доверие среди потребителей, позволяя им сохранять осведомленность о возникающих уязвимостях и инцидентах, а также принимать необходимые меры в собственных средах. В некоторых отраслях, например в правительственном

секторе США, уже становится обязательным проведение поставщиками самоаттестаций на предмет соответствия практикам SSDF, включая наличие VDP.

Реализация VDP, которая позволяет реагировать на запросы исследователей и вести открытый диалог с потребителями, укрепляет доверие и снижает риски для поставщиков, которые могут выражаться в виде финансовых, юридических или репутационных последствий. При этом даже если вы являетесь поставщиком, то наверняка будете выступать еще и потребителем или использовать сторонние компоненты в своих цепочках поставок. Это означает, что вам необходимо понимать структуры цепочки поставок и знать, какие меры безопасности применяют ваши контрагенты. Реализовывать это можно в виде аттестаций и проверки программных компонентов, получаемых от третьих сторон. Если это поставщик проприетарного ПО, вы можете использовать контрактные меры для обеспечения требуемого уровня надежности. Если же в получаемых компонентах используется OSS, вам нужно реализовать соответствующий механизм контроля, о чем мы говорили в других разделах книги, рекомендуя обратиться к руководствам таких лидеров отрасли, как NIST. Если этого не сделать, возникнут риски для ваших потребителей и, возможно, бизнес-проблемы, поскольку клиенты могут выражать обеспокоенность по поводу безопасности получаемого ПО, уточнять применяемые вами практики защищенной разработки или запрашивать артефакты вроде SBOM, которые помогут раскрыть потенциальные угрозы в ваших продуктах или приложениях.

PSIRT

Еще одной ключевой рекомендацией, относящейся как к SSDF, так и к другим передовым практикам поставки ПО, является создание команды реагирования на инциденты, связанные с безопасностью продукта (PSIRT). PSIRT подобна группе реагирования на инциденты в области кибербезопасности (CSIRT), но нацелена не на внутреннюю инфраструктуру и системы организации, а на рабочую среду и вопросы, связанные с уязвимостями и угрозами продукта. Эта команда упоминается в SSDF в разделе «Реагирование на уязвимости» в качестве примера, позволяющего организации реагировать на отчеты об уязвимостях и инциденты, а также поддерживать коммуникацию между ее стейкхолдерами, как внутренними, так и внешними. Независимо от того, формируете вы новую PSIRT или же анализируете зрелость существующей для ее улучшения, прекрасным руководством для этого

будет документ PSIRT Maturity Document от FIRST, той же организации, которая занимается CVSS и EPSS (<https://clck.ru/3CKXJU>). В данном документе описываются уровни зрелости команд PSIRT, в отношении которых можно проводить анализ: «базовый», «средний» и «продвинутый» (рис. 10.1). Далее мы кратко поговорим о каждом из них, чтобы понять их особенности.

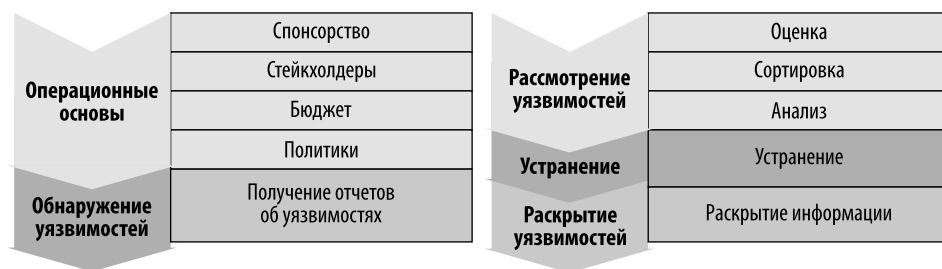


Рис. 10.1

Источник: <https://clck.ru/3CKXN9>

Прежде чем разбирать эти уровни, нужно создать документ PSIRT, в котором будут прописаны задача этой группы, стейкхолдеры, причастная/спонсирующая организация и область деятельности.

На первом, «базовом» уровне PSIRT еще только сформирована и налаживает свою основную деятельность. Для этого ей потребуется заручиться поддержкой руководства, определить внутренних и внешних стейкхолдеров, выделить бюджет на первичное укомплектование кадрами и становление, а также разработать политики и процессы для систематизации своего функционирования.

К примерам политик и процессов, которые приводит руководство FIRST, относятся управление уязвимостями, обработка информации и соглашение об уровне услуг по их устранению. Сюда входит возможность создавать отчеты об уязвимостях, включая публикацию контактной информации и схему передачи этих отчетов в PSIRT.

Однако получение отчетов об уязвимостях — это лишь первый шаг. Далее PSIRT должна рассмотреть и проанализировать эти отчеты. Сюда входит оценка, выяснение валидности уязвимости и решение вопроса о том, принимать или отклонять ее. Согласно рекомендации FIRST, команда PSIRT должна собирать ключевую информацию об уязвимостях и использовать машиночитаемые форматы для повышения эффективности

и расширения масштабов. В качестве доступных ресурсов приводятся Common Vulnerability Reporting Framework (CVRF), CSAF и CVE. Последняя поддерживается базой данных NVD и позволяет передавать информацию об уязвимостях потребителям, зачастую через инструменты сканирования, которые интегрируют и опрашивают NVD на предмет различных данных, таких как CVE.

После оценки PSIRT уязвимости как валидной и ее принятия команде нужно выполнить анализ, а именно понять, как эта уязвимость работает, как может быть эксплуатирована, какие версии продуктов и сервисов ей подвержены и как ее исправить. При этом меры исправления будут отличаться в зависимости от среды и используемых в ней средств противодействия, которых может и не быть. Это обычно уже определяют потребители, так как лучше знают свою внутреннюю рабочую среду.

Во время анализа PSIRT выполняет приоритезацию и присваивает уязвимости показатели. Документация FIRST рекомендует использовать систему CVSS, что вполне логично, учитывая причастность этой организации к ней. Но, как мы уже говорили, в адрес CVSS звучат серьезные критические высказывания со стороны академической сферы и отрасли. Тем не менее данной системой все более активно пользуются всевозможные организации, а если PSIRT решит применить другой метод оценки уязвимостей, то им потребуются объяснить свое решение потребителям, которые зачастую знакомы именно с CVSS.

Теперь, когда уязвимость оценена и проанализирована, поставщику нужно ее устранить. Устранение может включать такие мероприятия, как исправление кода, патч продукта, выдача рекомендаций потребителям, или не производиться вовсе. Решение принимается на основе оправданности в плане затрат со стороны как самой PSIRT, так и их коллег.

Последним аспектом «базового» уровня зрелости механизма раскрытия уязвимостей является уведомление о ней потребителей продукта или услуги. Это сообщение в идеале сопровождается руководством о том, как устранить или ослабить уязвимость, если исправления кода или решения не существует. FIRST также рекомендует в первую очередь поблагодарить исследователя безопасности или субъекта, который уведомил поставщика об уязвимости. Это способствует укреплению доверия внутри сообщества.

Далее PSIRT переходит на второй, или «средний», уровень. Здесь команда начинает предоставлять уже более комплексные услуги, взаимодействовать

с большим числом стейкхолдеров и совершенствовать свои возможности (рис. 10.2).



Рис. 10.2

Как пишет FIRST, на среднем уровне PSIRT уже отчетливо понимает всех своих стейкхолдеров, имеет четко установленные процессы, а также способна оптимизировать схему анализа и реагирования на уязвимости. Такие команды могут использовать инструменты для повышения своих возможностей по получению информации об уязвимостях и ее обработке. FIRST рекомендует командам на текущем уровне начинать вникать в компоненты, которые попадают в итоговый продукт, отражая эти данные в манифесте или ведомости материалов, обычно SBOM, если речь идет о программном продукте.

Обладание BOM не только помогает повысить видимость компонентов ПО и облегчает передачу информации об уязвимостях потребителям, но и позволяет организации понять, с какими третьими сторонами им может потребоваться связаться, например с OSS-проектами.

Руководство FIRST отмечает, что в более зрелых PSIRT четко определены роли и зоны ответственности, что позволяет понять, как именно каждый стейкхолдер вписывается в процесс рассмотрения и анализа уязвимостей,

а также налажен контакт с исследователями безопасности, которые могут сообщать командам об уязвимостях. Этот контакт выстраивается на основе функциональных процессов и инструментов вроде системы тикетов, упрощающей процесс взаимодействия.

Помимо использования CVSS и системы оценки серьезности уязвимостей, PSIRT зачастую прибегают к маркировке уязвимостей с помощью CWE. Как мы обсуждали в главе 3, CWE — это система под управлением MITRE, которая предоставляет разработанный сообществом список слабых мест ПО и аппаратного обеспечения, что позволяет оперативно размечать и категоризировать уязвимости.

Продвинутые PSIRT также могут опираться на прежние операции по устранению уязвимостей, чтобы не начинать каждый раз с нуля, а использовать налаженные процессы, способствующие устранению и раскрытию уязвимостей. Это ведет к повышению качества взаимодействия команды с различными сторонами в контексте раскрытия, и в этом смысле FIRST рекомендует использовать руководство *Guidelines and Practices for Multi-Party Vulnerability Coordination and Disclosure*, чтобы повысить зрелость схемы раскрытия уязвимостей (<https://clck.ru/3CKXNt>).

Повышение зрелости включает стандартизированные процессы для раскрытия, отслеживания метрик и постоянного совершенствования. FIRST также отмечает, что бывают случаи, когда PSIRT решает уведомить потребителя до фактического разглашения уязвимости. И это имеет смысл с учетом того, что после обнародования информация становится доступна не только защитникам, но и злоумышленникам, которые обязательно начнут искать возможность эксплуатировать уязвимость до того, как потребители ее устраним.

Наконец, в идеале PSIRT дорастает до третьего, «продвинутого» уровня (рис. 10.3).

На рис. 10.3 показана PSIRT с обширным портфолио услуг и возможностей, а также развитыми базовыми процессами. Это помогает организациям и ее стейкхолдерам понимать процесс получения отчетов об уязвимостях, анализа и последующих обновлений для продуктов, за которые отвечает PSIRT.

Одно из ключевых преимуществ продвинутых команд состоит в их переходе от реактивной культуры к предиктивной. Это подразумевает динамичный поиск уязвимостей или недочетов в продуктах, активное взаимодействие и сотрудничество со стейкхолдерами и командами инженеров.

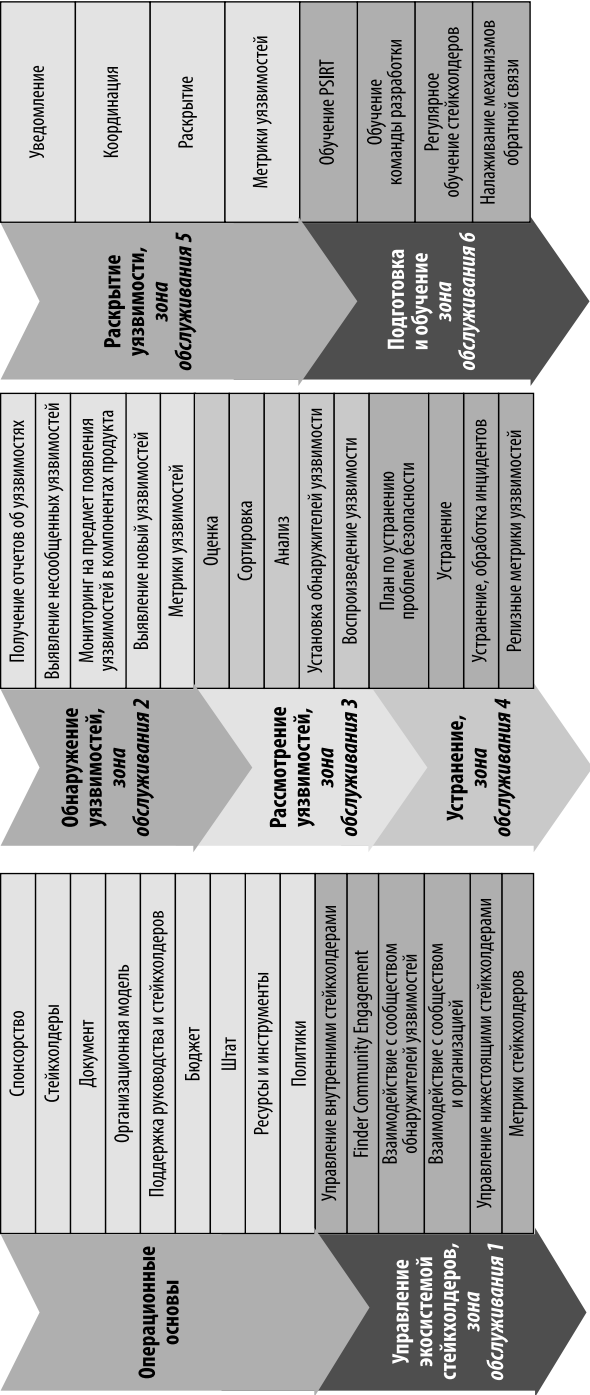


Рис. 10.3

Продвинутость во многом также означает прекрасное выполнение всех основных процессов. Сюда входит тесная связь со стейкхолдерами для предоставления и получения обратной связи, позволяющей повысить эффективность PSIRT. Данный процесс постоянного совершенствования привязан к осязаемым метрикам и KPI, таким как удовлетворенность потребителей, а также выполнение соглашений и задач об уровне услуг (Service-Level Objectives and Agreements, SLO/SLA).

В соответствии с принципами DevSecOps, FIRST отмечает, что еще одной особенностью продвинутой PSIRT является тесная интеграция и взаимодействие с командой разработки для изучения дорожной карты продукта и планируемых версий и функций. Кроме того, в подобных PSIRT уже оптимизированы инструменты для всего цикла обработки уязвимостей. Деятельность продвинутых команд сильно завязана на этапах разработки продуктов и сервисов организации, чтобы анализ и сканирование уязвимостей выполнялись стандартизированным образом.

Учитывая, что PSIRT постоянно взаимодействует с исследователями безопасности, ее участники начинают понимать профессионализм и валидность получаемых от разных представителей сообщества отчетов и способны на основе этого расставлять приоритеты. Продвинутые PSIRT могут также включать в свой рабочий процесс принцип «сдвига влево» за счет раннего предоставления обратной связи командам инженеров в ходе разработки продукта. Сюда входят не только выявление уязвимостей, но также их приоритизация и устранение.

Кроме того, очень важно, чтобы PSIRT хорошо осознавала экосистему, в которой работает, то есть не только потребителей, но и поставщиков. Это знание сопровождается отчетливым и действенным взаимодействием во всех направлениях цепочки поставки ПО.

Наконец, накопив достаточный профессионализм в работе с уязвимостями и недочетами продукта, PSIRT способна взяться за обучение других. Проявляться это может в виде внутренних тренингов на тему безопасности продукта и практик управления уязвимостями, чтобы повысить эффективность не только PSIRT, но и других команд в организации. Помимо улучшения качества работы самой PSIRT, это также повышает безопасность продукта для потребителей, что и является основной целью.

Делиться или не делиться?

В контексте прозрачности ПО у разработчика может возникнуть вопрос, какими данными нужно делиться и стоит ли вообще это делать. Несмотря на то что процесс принятия решения и анализа в каждой организации строится по-своему, одно ясно точно — стремление к прозрачности ПО со стороны потребителей, правительственных агентств и регулирующих органов продолжает расти. Объясняется это постоянным ростом числа атак на цепочки поставок и обеспокоенностью отрасли в отношении рисков, связанных с потреблением неизвестных или потенциально небезопасных программных компонентов.

На то, какие именно данные должен предоставлять поставщик, влияет множество факторов, таких как условия договора, запросы потребителей и нормативные требования. В одних случаях потребители могут просить поставщиков предоставлять список программных компонентов, получаемых от первых лиц, в других — исчерпывающий и практический перечень, в том числе транзитивных зависимостей их продуктов. Возьмем, например, отчет *NTIA SBOM Minimum Elements Report*. В отношении глубины он утверждает, что «SBOM должен содержать все первичные компоненты (верхнего уровня) с перечислением всех их транзитивных зависимостей. Как минимум все зависимости верхнего уровня должны быть указаны достаточно подробно для рекурсивного поиска всех транзитивных зависимостей».

Очевидно, что это условие будет применимо только к субъектам экосистемы США, подпадающим под требования NTIA, но оно может послужить и в качестве полезного руководства. Важно отметить, что для определения транзитивных зависимостей необходимо приложить немало усилий, поэтому поставщики, не предоставляющие такую глубину отчетности, перекладывают эти хлопоты на потребителей, что, естественно, приводит к экспоненциальному увеличению нагрузки на всех пользователей. Кроме того, они также не сообщают клиентам никакой информации о связанных с этими зависимостями уязвимостях и рисках. А согласно отчету *State of Dependency Management* компании Endor Labs, о котором мы уже говорили в других главах, 95 % уязвимых зависимостей оказываются именно транзитивными. Это значит, что если они не будут включены в SBOM, большинство уязвимостей останутся незамеченными для потребителей, если только они сами не проделают всю работу по их выявлению. Это не относится к детальному анализу и оценке того, является ли зависимость, будь то прямая или транзитивная, доступной и эксплуатируемой, но, по крайней мере, предоставляет дополнительный контекст по всем зависимостям и уязвимостям приложения или продукта.

Поставщикам, предоставляющим SBOM, также важно обеспечить, чтобы эти документы были полезны и содержали достаточно метаданных. Пригодится им в данном вопросе проект SBOM Scorecard, реализуемый платформой eBay (<https://clck.ru/3CKXPm>).

Еще одним вариантом является проект SCVS от OWASP, о котором мы говорили в главе 6. И хотя он еще только развивается, его конечная цель — помочь организациям оценить BOM на предмет его соответствия внутренним политикам, форматированию и систематике. По мере того как концепции SBOM набирают популярность в индустрии, подобные инструменты будут необходимы. Как мы уже говорили, уровень глубины, полноценность и точность SBOM на данный момент разнятся, и организации захотят обеспечить соответствие этих документов собственным конкретным требованиям (<https://clck.ru/3CKXQT>).

SBOM Scorecard можно использовать для определения ключевых критериев, таких как соответствие спецификации SBOM, наличие в ней информации о ее создании, а также информации о пакетах, например их ID, данных лицензии и версий.

О минимальных элементах SBOM, определенных NTIA, мы говорили в главе 4. Эти элементы важны для поставщиков, желающих поставлять свои продукты правительственным учреждениям, учитывая, что их предоставление в этом случае наверняка станет обязательным. Полезным инструментом здесь будет `ntia-conformance-checker`, который способен определять, содержат ли документы SBOM минимальный набор информации (<https://github.com/spdx/ntia-conformance-checker>), установленный NTIA.

Используя эти два инструмента, компания Chainguard в рамках своего проекта Chainguard Labs в конце 2022 года проанализировала публично доступные OSS SBOM с целью определения их качества и содержимого (<https://clck.ru/3CKXRM>).

И хотя их исследование показало, что некоторые SBOM действительно содержали качественные данные, во многих этого не наблюдалось. Тем не менее ни один из проанализированных SBOM не соответствовал минимальным требованиям NTIA, а большая часть не содержала сведений о лицензировании и версии пакетов. Это, конечно, не радует, но нужно понимать текущую ситуацию, а именно то, что индустрия еще только начинает внедрять концепцию SBOM, включая оттачивание процесса их генерации, проработку требований и качества. В любом случае хорошо, что есть такие исследования и инструменты, которые позволяют поставщикам понять существующие

пробелы и способы обеспечения высокого качества SBOM, которые они предоставляют своим потребителям. Тенденция к инвентаризации ПО, повышению его прозрачности и созданию SBOM связана с данными. Без качественных данных поставщики и потребители будут обладать недостаточным пониманием для принятия учитывающих риски решений и не получат те результаты, которые помогли бы улучшить сферу цепочек поставок ПО.

Лицензирование и личная ответственность за код

Как говорилось в главе 5, лицензирование тоже является важным вопросом для многих поставщиков, особенно в отношении OSS. Один из методов лицензирования, который может интересовать многих поставщиков ПО, называется *Copyleft*. Под этим понимается метод лицензирования, делающий программное обеспечение бесплатным и требующий, чтобы все последующие его изменения или дополнения тоже были свободно распространяемыми. Проявляется такое лицензирование в различных форматах, но чаще в виде GNU General Public Licensing (GNU GPL; <https://clck.ru/3CKXTr>).

В связи с растущим вниманием к прозрачности ПО некоторые поставщики сталкиваются с вопросами, связанными с нарушением авторских прав, включая нарушения Copyleft-лицензий. Учитывая необходимость принятия мер для предотвращения подобных ситуаций, дополнительная прозрачность, обеспечиваемая такими артефактами, как SBOM, позволит раскрывать OSS-компоненты в их продуктах и ПО. Лицензионная информация, включающая мейнтейнеров и участников открытых проектов (FOSS), станет доступной не только для поставщиков, но и для потребителей. Юридические действия в отношении нарушений авторских прав, таких как Copyleft, могут быть предприняты правообладателями, то есть создателями и мейнтейнерами FOSS-проектов.

Поставщики должны принимать дополнительные меры для инвентаризации используемых ими компонентов и обеспечивать соблюдение любых применимых авторских прав FOSS, чтобы избежать проблем с законом и действовать в соответствии с этическими принципами.

Как говорилось в предыдущем разделе, недостаточное наполнение SBOM, в том числе отсутствие лицензионной информации, не позволит поставщикам в достаточной степени понимать, насколько они соответствуют всем требованиям. Они должны добиться полноценного понимания всех используемых ими в программном обеспечении FOSS-компонентов и условий их лицензий.

Помимо того что это может вести к юридическим последствиям, нюансы лицензирования также могут создавать проблемы безопасности. Учитывая добровольные начала большинства OSS-проектов, многие мейнтейнеры предоставляют доступ к своему коду в исходном виде. Это означает, что у них может не быть желания или возможности исправлять проблемы, обнаруженные пользователями. Следовательно, потребители и организации, использующие OSS, должны быть готовы делать форки проектов и брать на себя устранение уязвимостей, поскольку разработчик не несет таких обязательств. Многие участники отрасли до сих пор этого не осознают, и мы нередко видим нереалистичные ожидания от мейнтейнеров OSS по части устранения проблем или предоставления поддержки.

И хотя использование OSS-проектов и компонентов активно растет, некоторые организации неверно понимают динамику этого сегмента мира ПО. В контексте безопасности цепочек поставок большинство мейнтейнеров OSS не должны рассматриваться как «поставщики». Как правило, у вашей организации не установлено с ними никаких деловых отношений, и вы применяете их код как есть, без каких-либо ожиданий в плане обслуживания, обновлений или реагирования. В связи с этим организации, задействующие OSS в своих продуктах, приложениях и сервисах, должны понимать масштабы этого использования и при необходимости быть готовыми взять ответственность за данные компоненты.

Эту динамику хорошо отражает статья мейнтейнера OSS Томаса Деспьера (Thomas Depierre) под названием «Я не поставщик» (<https://clck.ru/3CKXW2>).

Деспьер говорит о том, что, несмотря на повсеместное применение FOSS в цифровой экономике, мейнтейнеры не являются «поставщиками» в традиционном смысле и потребители не должны предъявлять им какие-либо требования.

Подразделения по управлению OSS

Как мы говорили, к OSS прибегают организации любого размера и из любой сферы. Финансовая отрасль, здравоохранение, промышленный сектор и даже национальная оборона — открытые компоненты используются везде. Сюда входят и поставщики ПО, которые обычно применяют OSS в своих продуктах и разработке приложений.

Тем не менее столь обширное распространение имеет и обратную сторону: по данным отчета *State of the Software Supply Chain Report* компании Sonatype, число атак на цепочки поставок возросло почти на 800 %.

В свете все большего использования OSS организации начали создавать внутренние подразделения по управлению OSS (OSPO) для систематизации стратегий потребления OSS, а также для расширения своего сотрудничества с сообществом разработчиков OSS. Ключевыми обязанностями OSPO обычно выступают выработка стратегии использования OSS, курирование ее реализации и содействие внедрению OSS-продуктов и сервисов внутри корпорации (<https://clck.ru/3CKXYF>). По мере развития OSS организация получает огромную пользу от таких подразделений. OSPO помогает решать ряд вопросов.

И хотя поставщики и корпоративные потребители, зачастую ведущие собственные внутренние разработки, могут извлекать выгоду от формирования OSPO, мы рассмотрим этот процесс с позиции поставщиков, которые часто задействуют OSS в своих продуктах (<https://clck.ru/3CKXZh>).

Уникальная позиция OSPO в управлении OSS делает эти отделы основным игроком в контексте обеспечения безопасности. Исследования показывают, что современные приложения зачастую содержат более 500 OSS-компонентов. При столь широком использовании важно учитывать и негативную статистику.

Согласно исследованию компании Synopsys от 2022 года, 81 % таких компонентов содержит минимум одну уязвимость, 88 % были старше двух лет, а 88 % имели не последнюю версию. Все эти метрики подводят нас к одному тревожному факту: организации активно используют устаревшие и небезопасные компоненты. Это значит, что современные корпоративные среды имеют обширную поверхность атаки, удобную для злоумышленников (<https://clck.ru/3CKXc6>).

Мы уже знакомы с Указом о повышении уровня кибербезопасности, включая его раздел 4, «Повышение безопасности цепочек поставок ПО». На основе этого указа NIST разработал обширное руководство, включающее меры управления компонентами с открытым исходным кодом, о которых мы поговорим далее, а также способы, которыми OSPO могут продвигать их реализацию внутри корпорации (<https://clck.ru/3CKXcf>).

NIST разделяет все эти меры контроля на три уровня зрелости: основные, устойчивые и расширенные возможности. Среди этих мер присутствует применение фреймворка SSDF и отслеживание, чтобы OSS-компоненты поступали по безопасным каналам от доверенных источников. Реалии современных корпоративных сред таковы, что большинство организаций

не имеют полной видимости и механизмов контроля за OSS-компонентами, не говоря уже об отслеживании связанных с ними уязвимостей. Это прекрасный пример ситуации, когда поможет OSPO, которое выступает сторонником эффективного использования коммерческих OSS-продуктов и сервисов, призывает к бдительности в их отношении и следит за тем, чтобы эти практики безопасности включались в политики и процессы организации.

К рекомендуемым NIST мерам защиты также относится создание внутренних репозиториев известных и проверенных OSS-компонентов для снижения рисков организации. Вместо того чтобы допустить формирование среды, в которой разработчики вольны получать и использовать любые OSS-компоненты без учета их уязвимостей, внутренние репозитории позволяют вести их надежный каталог, который не только повысит скорость разработки, но и снизит риски.

Еще одной значимой рекомендацией SSDF является реализация сопутствующих наборов инструментов, включая указание, какие из них должны быть включены в пайплайны CI/CD для минимизации угроз. Примером может быть интеграция в CI/CD инструментов SCA и SBOM для обеспечения максимальной видимости уязвимых OSS-компонентов, перемещающихся через пайплайн в рабочие среды. Данный подход также способствует «смещению влево» из-за проверки безопасности на более ранних этапах цикла разработки и выявления уязвимости до ее попадания в эксплуатацию, где ими могли бы воспользоваться злоумышленники. В качестве политик и процессов в этой области OSPO могут применять не только сканирование и SBOM, но и механизм подписания, позволяющий создавать неизменяемые записи и логи для поддержания целостности и проверяемости процесса разработки. На самом зрелом уровне NIST приводит расширенные методы, такие как приоритизация использования защищенных языков программирования и стремление к автоматизации сбора, сохранения и сканирования OSS-компонентов в вышеупомянутых усиленных внутренних репозиториях. Это, пожалуй, является наиболее важным шагом на пути снижения рисков без вреда для скорости разработки. В противном случае команды наверняка искали бы пути обхода политик организации.

Сотрудники OSPO активно взаимодействуют с разработчиками и продвигают идею следования правильной работы с OSS. Они поддерживают энтузиазм и заинтересованность работников, а также выстраивают с ними прочные и продуктивные отношения. Эти связи позволяют в дальнейшем убеждать

разработчиков следовать лучшим практикам защищенного использования OSS, которые многие организации просто еще не реализовали, несмотря на широкое применение таких компонентов в своих приложениях и продуктах. Все это создает множество преимуществ, включая сокращение потребления небезопасных зависимостей и создание более безопасных приложений как для внутренних, так и для внешних задач.

Исследование Чинмайи Шармы также предполагает, что основными бенефициарами OSS являются поставщики ПО, в связи с чем они наиболее заинтересованы в устранении рисков своих цепочек поставок. Это создает для контрагентов прекрасные условия, при которых они могут активно включиться в жизнь OSS-сообществ, проводить сканирование на уязвимости и помогать OSS-проектам их устранять. Таким образом получится перестроить текущую модель, в которой вся нагрузка ложится на открытое сообщество без равноценного вклада поставщиков. Последние получают огромную пользу от OSS-компонентов и, реализуя OSPO, могут лучше понять все нюансы использования ими этих компонентов и связанные с ними риски, а также оказать поддержку сообществу, на которое опираются.

OSPO играют ключевую роль в перестройке парадигмы и повышении безопасности цепочек поставок OSS. Развитие OSS способствует инновациям и появлению новых возможностей практически во всех отраслях. И теперь OSPO могут вовлечь эти отрасли и организации в OSS-сообщество, снизив тем самым текущий уровень системных рисков, связанных с открытым ПО и вызванных разрозненностью и недостатком финансирования. Это поможет минимизировать риски для потребителей за счет более эффективного управления и повышенной безопасности OSS-компонентов, используемых в продуктах поставщиков.

Согласованность среди команд по разработке продуктов

Никакие усилия по обеспечению прозрачности ПО и предоставлению таких артефактов, как SBOM и аттестации, не окажутся эффективными, если они не будут последовательно реализованы среди всех групп разработки в организации. Очень крупные корпорации, присутствующие на глобальном рынке, зачастую имеют несколько команд разработчиков, часть из которых используют собственные инструменты и процессы SDLC.

Авторам книги известен как минимум один производитель, который создает новую дочернюю компанию для каждой линейки продуктов. В итоге получаются десятки или сотни субъектов, которые создают продукты с очень похожими названиями, что влечет путаницу для конечного потребителя. В таких условиях анализ безопасности одной компании и ее процессов разработки неприменим к другим компаниям и их продуктам. Ситуация становится очень сложной.

Был случай, когда перед одним из авторов книги стояла задача обеспечить безопасность приложения глобальной компании-производителя, где трудилось более 100 команд разработчиков, большинство из которых следовали собственным практикам. Его инициатива по стандартизации встретила серьезное сопротивление, отчасти из-за регионального менталитета: европейские мнения столкнулись с мнением представителя Северной Америки по части того, чьи процессы лучше.

Поскольку отрасль в целом еще только знакомится с CycloneDX, SPDX, VEX и VDR, не удивительно, что в организациях могут возникать подобные разногласия. В связи с этим важно, чтобы клиенты четко формулировали в контрактах свои требования по безопасности в приложениях, описывая практики, которые они ожидают от поставщика, чтобы впоследствии можно было спросить с него их соблюдение.

Поскольку потребители стремятся реализовать стандартизированные процессы, механизмы контроля и предоставить качественный опыт работы с их продуктами для своих клиентов, важно прийти к консенсусу в отношении того, что подо всем этим понимается. К примеру, можно договориться, что отдельные инструменты CI/CD могут отличаться, но при этом утвердить между командами стандартизированные результаты и форматы артефактов.

Ручной труд или автоматизация

Поставщики также должны учитывать роль автоматизации в контексте обеспечения точности, предоставляемой потребителям информации. Обеспечение надежности в крупных компаниях становится одним из факторов масштабируемости, особенно в условиях сжатых сроков разработки сотен или даже тысяч программных продуктов. Однако автоматические процессы не всегда способны успешно выявлять критические программные компоненты. Например, образ Docker, загружающий компонент динамически в среде

выполнения или не содержащий компонента в `Dockerfile`, может потребовать ручной идентификации этого компонента. Но с помощью автоматизации вполне реально определить «достаточно хороший» объем информации для включения в начальный SBOM.

Если поставщик может создать SBOM или другой артефакт, превышающий определенный уровень уверенности, например 80 %, будет ли этого достаточно для потребителя данного артефакта? Вероятность ошибки присутствует всегда, и при автоматизации процесса шанс возрастает. Если не 80 %, то каким будет ваш предполагаемый уровень уверенности? Кто ваши клиенты? Обсуждали ли вы с ними реалистичные ожидания? Есть ли у вас процесс для обновления и исправления артефакта цепочки поставки, ранее определенного как ошибочный? А для уведомления клиента? Для обновления любого анализа на основе этой новой информации? Что, если артефакт был корректен, но в ходе анализа вскрылись новые сведения, например, обнаружена новая CVE? Корректность может касаться не только создаваемого вами артефакта, но и полученных в ходе его анализа данных.

Одной из типичных проблем, особенно в случае реверс-инжиниринга SBOM, является непонимание того, что именно анализируется. Существует множество техник для определения компонента, но некоторые не сложнее простого сопоставления строк в файле. Например, комментарий «`cve_x` была исправлена в `component_version_1.2.3`» можно трактовать как имеющий отношение конкретно к компоненту версии 1.2.3, когда по факту у вас 2.0. Проводить исчерпывающий разбор каждой версии нереалистично, поэтому возникает вопрос: как оценить уровень уверенности в таких случаях? Пожалуй, он будет меньше 80 %. По нашему опыту, уверенность обычно составляет около 60 %. Так что понимание метода создания артефактов тоже может стать одним из факторов его определения.

Имеет ли значение этап создания SBOM? В ходе проектирования, сборки и развертывания некоторых приложений тоже могут создаваться SBOM, и они будут значительно отличаться. Документы, написанные на ранних стадиях, редко бывают полезны для конечного потребителя, и попытка полностью автоматизировать SBOM на стадии сборки может оказаться безуспешной. Возможно, такая информация будет достаточно полезна для внутренних практик организации, направленных на обеспечение надежности, а ее максимально проработанный вариант можно приберечь на будущее. Это переносит значительную долю ручных усилий в конец процесса контроля качества, что дает возможность исправить все обнаруженные отклонения.

Резюме

В этой главе мы обратили внимание потребителей на необходимость более глубокого осмысления проблем безопасности цепочек поставок и роли SBOM. Мы объяснили, почему организациям нужен SBOM и обеспечение прозрачности ПО со стороны поставщиков, а также привели потенциальные сценарии использования SBOM и управления этим процессом в больших масштабах. Помимо этого, было отмечено, что, несмотря на полезность SBOM в решении ряда проблем, этот документ далеко не панацея и может сильно отличаться по таким аспектам, как используемые инструменты, автоматизация, стандартизация и зрелость. В следующей главе мы обсудим прогнозы отрасли в плане повышения прозрачности ПО, международные инициативы в данном направлении, а также порассуждаем, куда наше общество движется в контексте безопасности цепочек поставок программного обеспечения.

11

Практическое руководство для потребителей

Поставщики ПО и потребители стремятся разобраться во множестве руководств, лучших практик и условий. У каждой стороны свое видение, свои интересы и цели, а иногда и собственные нормативные требования. И хотя некоторыми потребителями могут выступать крупные корпоративные среды с квалифицированными специалистами безопасности и богатым опытом, не во всех случаях это так, поскольку многие потребители — это средние и малые бизнес-проекты с ограниченными или отсутствующими ресурсами и опытом по обеспечению внутренней безопасности. Подобные небольшие организации зачастую для обеспечения безопасного потребления ПО вынуждены опираться на внешних поставщиков услуг и партнеров, а также расставлять приоритеты, исходя из собственных ресурсов. В этой главе мы обсудим практическое руководство для потребителей, опираясь на множество рекомендаций из главы 7 и 8.

Мыслить в ширину и глубину

Несмотря на то что эта глава включает вопросы и рекомендации, связанные со SBOM, мы хотим подчеркнуть, что SBOM — это просто один из новых инструментов и ресурс в более обширном контексте безопасности цепочек поставок ПО. Тем не менее, учитывая его сильный акцент на отрасли, мы считаем необходимым предоставить соответствующее руководство.

Как уже говорилось при описании инцидентов с SolarWinds, Log4j и Kaseya, риски в цепочках поставок ПО могут быть связаны с коммерческими

поставщиками, OSS-компонентами, поставщиками управляемых услуг, облачными провайдерами и прочими факторами.

Помимо прочего, по существующим и появляющимся руководствам от NIST, CNCF, NSA и других, лучшие практики цепочек поставок включают ряд ключевых мероприятий. К ним относятся понимание своих поставщиков, разделение ответственности в отношениях с ними, управление потреблением ПО, а также внутренние практики и процессы для обеспечения безопасного потребления, разработки и использования ПО в реализации уникальных потребностей организации.

Более того, как отмечалось в разделе «Критика и недочеты SBOM» главы 4, еще многое предстоит сделать по части инструментов и внедрения SBOM, чтобы организации могли успешно получать, анализировать, наполнять и хранить SBOM для применения в рамках собственных цепочек поставок и более масштабных мероприятий C-SCRM. Помимо этого, необходимо понимание текущего состояния и зрелости SBOM в отношении некоторых сообществ вроде OSS-проектов, опирающееся на исследования различных организаций и оценку с помощью доступных инструментов.

Как мы уже говорили, в зависимости от отрасли и актуальных нормативных факторов скорость внедрения SBOM и глубина отражаемых этим документом данных зачастую отличаются. Дело в том, что поставщики не всегда заинтересованы в предоставлении потребителям этих артефактов, если это не несет для них коммерческой выгоды или не является требованием. По этим причинам мы хотим подчеркнуть, что, хотя SBOM и полезен в контексте стремления к обеспечению прозрачности ПО и безопасности его цепочек поставок, эти документы не стоит рассматривать в качестве универсального спасения. Это комплекс проблем, не имеющий единого решения, откуда и происходит известный принцип *защиты в глубину*. Угрозы, нависающие над ИТ-системами, включают в себя пользователей, конечные точки, данные, сложные взаимозависимые системы и прочие элементы, создавая ситуации, когда невозможно с помощью одного решения или технологии разобраться со всеми вызовами.

Действительно ли мне нужен SBOM?

В свете шумихи вокруг SBOM некоторые потребители ПО могут задаться вопросом, а нужен ли он им вообще. Несмотря на то что отрасль десятилетиями существовала и без этой концепции, это не значит, что отсутствовали проблемы, которые сегодня ввиду все более активного использования OSS и сторонних компонентов только усиливаются.

Среди поставщиков и потребителей в сфере ПО спрос на SBOM быстро растет, и пока ничто не говорит о возможном замедлении данной тенденции. Согласно опросу The State of Software Bill of Materials (SBOM) and Cybersecurity Readiness, проведенному The Linux Foundation, в 2022 году использование SBOM возросло на 66 %. Прогнозируется также, что в 2023 году 88 % организаций уже будут применять этот документ в той или иной степени (<https://clck.ru/3CKXdr>).

Современное ПО и приложения преимущественно состоят из OSS и сторонних компонентов. То есть без SBOM или сложных мероприятий по контролю, например SCA, клиенты не будут иметь твердого понимания потребляемых ими программных компонентов или зависимостей. Это означает, что сейчас пользователи продолжают получать ПО от поставщиков без отчетливого понимания, какие именно компоненты в него включены, какие в них уязвимости и каким рискам они потенциально подвергают организацию и ее партнеров. Как говорилось в главе 2, традиционные процессы получения ПО и анализа рисков поставщиков недостаточно глубоки для включения описи программных компонентов и управления этими рисками.

И хотя зрелые и предусмотрительные поставщики в идеале публикуют патчи или обновления для устранения уязвимостей в компонентах, это происходит не всегда, о чем говорится в исследовании State of Software Security от 2017 года (<https://clck.ru/3CKXgP>) компании Veracode. Его авторы выяснили, что только половина поставщиков выпускает исправления уязвимостей, выявленных в сторонних компонентах их ПО. Это не значит, что все уязвимости обязательно подвержены эксплуатации, но потребители все равно остаются без какого-либо проясняющего контекста или руководства, а также подвергаются сопутствующим рискам.

В итоге получается, что клиенты зачастую продолжают использовать потенциально уязвимое ПО, о недостатках которого они даже не подозревают. Если поставщик не предоставляет им SBOM, в котором перечислены все его элементы, потребители не могут учитывать уязвимости, связанные с этими компонентами, спрашивать поставщиков об их эксплуатируемости или самостоятельно анализировать и оценивать этот аспект. Прозрачность, которую привносит SBOM, помогает восполнить данный пробел в осведомленности и понимании.

Еще одна причина для использования SBOM в том, что базы данных вроде NVD показывают уязвимости на уровне продукта, но обычно не дают подробного представления о его компонентах. Потребители просто не могут

опираться на устаревшие подходы, подразумевающие строгое использование NVD и системы CPE, если они хотят получить представление об уязвимых компонентах продукта.

Как мы уже говорили, развиваются новые базы данных уязвимостей, предоставляющие более широкие возможности в отношении уязвимостей OSS, а также поддержку PURL, которая может стать более эффективным подходом по выявлению уязвимостей на уровне компонентов. Мы уже затрагивали инициативы проекта OWASP под названием SBOM Forum, о которых говорится в работе *A Proposal to Operationalize Component Identification for Vulnerability Management*. В ней участники форума призывают MITRE и NVD внедрить PURL для идентификации OSS и коммерческого ПО. До определенной степени это поможет восполнить текущие пробелы NVD в отношении информации об уязвимостях на уровне компонентов. Как пишут авторы работы, достижение полноценной автоматизации и эффективности SBOM будет сдерживаться, пока проблема с именованием не будет решена или хотя бы значительно проработана.

И хотя поставщики порой могут проявлять инициативу по выявлению уязвимостей в своих компонентах и продуктах, устраняя их и уведомляя потребителей, так происходит не всегда. Видимость компонентов позволяет потребителю сохранять понимание об уязвимостях на уровне компонентов и уведомлять поставщиков о новых угрозах по мере их выявления (если их до этого не уведомит сам поставщик, что было бы идеально). Это также дает пользователю возможность делать запросы в отношении эксплуатируемости обнаруженных уязвимостей, если поставщик не предоставляет эту информацию сам. Как мы вскоре увидим, это также позволяет потребителям реализовывать виртуальные патчи в обстоятельствах, когда существует известная уязвимость, для которой поставщик еще не выпустил исправления, а может, и не выпустит.

Это возлагает ответственность на поставщика, осознающего, что пользователи в курсе и обеспокоены уязвимостями компонентов. Реальность такова, что определение и устранение угроз несут свои затраты и поставщики не всегда замотивированы делать это оперативно в ущерб другим процессам, приносящим прибыль, — например, разработке новой функциональности продукта.

Инвентаризация активов уже многие годы указывается в качестве лучшей практики и критической меры защиты в руководствах SANS и CIS. Тем не менее обсуждение этой темы набирало обороты, и теперь она переросла уже в инвентаризацию компонентов, поскольку представители индустрии

понимают, что даже единичный уязвимый компонент может создать угрозу не только для одной организации, но и для целых отраслей.

Принимая во внимание, что для организаций непрактично реализовывать все меры защиты или во всех случаях уделять им равноценное внимание, CIS приводит список критических мер, связанных с наиболее распространенными атаками. CIS утверждает, что этот список является необходимой стартовой точкой для каждой корпорации, желающей повысить уровень своей кибербезопасности (www.cisecurity.org/controls).

Данные критические меры защиты соотносятся и с другими фреймворками, такими как Payment Card Industry Data Security Standard (PCI DSS), Health Insurance Portability and Accountability Act (HIPAA), North American Electric Reliability Corporation Critical Infrastructure Protection (NERC CIP) и Federal Information Security Modernization Act (FISMA), все из которых в той или иной степени также указывают на необходимость инвентаризации программного обеспечения. И хотя традиционно под этим подразумевались приложения, сейчас в свете увеличения атак на цепочки поставок и появления все большего числа уязвимых компонентов речь идет уже о более детальной инвентаризации программных активов.

Как указывает руководство CIS, инвентаризация и защита программных активов являются критической основой для предотвращения атак. Злоумышленники зачастую выискивают уязвимые версии ПО, чтобы их эксплуатировать. Это может привести к различным последствиям для организаций, включая горизонтальное перемещение на другие корпоративные активы и даже сценарий цепной атаки, когда хакеры с первичной пораженной организации переключаются на ее бизнес-партнеров, клиентов или другие компании.

В отсутствие достаточно надежного и точного процесса инвентаризации активов потребители не смогут определить присутствие уязвимых компонентов в своей среде и выяснить, если ли другие проблемы, например связанные с лицензированием.

Помимо того что SBOM играет решающую роль в контексте инвентаризации программных компонентов, этот документ также помогает и в других случаях, например при реагировании на инциденты. К слову, в отношении истории с Log4j Совет CSRB сообщил, что некоторые федеральные агентства потратили десятки тысяч часов на расследование в попытке выявить присутствие уязвимых компонентов Log4j в их корпоративных средах. В обширных, сложных средах в любом контексте на реагирование неизбежно потребуется время. Но, если инцидент будет связан с чем-то вроде Log4j,

а у вашей организации не будет полноценной описи всех используемых компонентов, окажется крайне сложно определить, где и до какой степени ваша организация является уязвимой.

Ценность SBOM также проявляется при приобретении и получении ПО. Поскольку потребители зачастую проводят проверки новых приложений и поставщиков на предмет пригодности для включения в свои рабочие процессы, им важно понимать потенциальные риски, которые эти поставщики несут.

Нередко бывает, что организации используют такие методы, как запрос результатов сканирования на уязвимости, отчеты пентестов, артефакты соответствия вроде SOC 2 и так далее, чтобы определить, подходит ли поставщик для сотрудничества. Тем не менее эти мероприятия традиционно не включают сбор инвентаризационных данных о программных компонентах, в результате чего потребители остаются не в курсе о составе получаемого ими ПО, как и о связанных с ним уязвимостях.

Если же они запрашивают от поставщиков документы SBOM и VEX, то получают возможность заблаговременно узнать обо всех уязвимостях, связанных с продуктами этих поставщиков, об их эксплуатируемости и потенциальном влиянии, которое данные продукты могут оказать на безопасность.

Потребителям также необходимо убедиться, что они будут получать SBOM в формате, совместимом с их потребностями и возможностями (например, SPDX или CycloneDX). При этом они могут заложить это требование в условия контрактов. Причем в контракте можно прописать не только необходимый формат SBOM, но также периодичность и способ его предоставления, обязательные поля и прочие нюансы. В качестве примера можно привести федеральный сектор США, который опирается на установленный NTIA минимальный набор требований, определяющий обязательные поля и содержимое.

Что мне с ним делать?

Отлично — поставщик предоставил вам SBOM, или вы начали создавать его сами, исходя из внутренних потребностей разработки. Что делать дальше для получения от этого документа максимальной пользы?

Как говорилось ранее, SBOM может оказаться очень полезен для различных случаев, таких как управление уязвимостями, гигиена зависимостей, реагирование на инциденты, лицензирование и получение ПО. Для того же, чтобы

сделать их действенными, нужно интегрировать SBOM в политики и процессы организации, а также подключить стейкхолдеров, которые сделают эти артефакты ценными в контексте их собственных ролей и ответственностей.

Несмотря на то что в индустрии достаточно быстро начали появляться инструменты для создания SBOM, в случае их масштабного потребления, анализа, дополнения, хранения и создания отчетов еще многое предстоит проделать.

Это взросление также касается вопросов устаревшего ПО и облачных сред. Причем облачной инфраструктуре необходимо прийти к общему консенсусу по части того, как SBOM должны выглядеть в этой среде, исходя из сложности и огромного числа взаимозависимостей между сервисами и провайдерами.

При отсутствии согласованного плана использования SBOM после его создания или получения организации окажутся в неловкой ситуации. С одной стороны, у них будут артефакты, способные прояснить информацию об уязвимостях, рисках и зависимостях, но с другой — вся эта информация принесет ценность, только когда организация раскроет ее с помощью сопутствующих политик и процессов. SBOM предполагает огромное число возможных случаев применения, но у организаций должен быть четкий план, чтобы использовать их подобающим образом без внесения дополнительной когнитивной нагрузки.

Получение и управление SBOM в больших масштабах

Если рассматривать концепцию SBOM отдельно, то ее использование для определения компонентов потребляемого ПО выглядит логичным. Если же делать это в больших масштабах корпоративных сред среди сотен или тысяч команд разработчиков и многих сторонних поставщиков ПО, то такая затея окажется очень трудоемкой. Как организации справиться с обработкой данных в таких масштабах? Как обеспечить использование надежного и всеобъемлющего подхода для непрерывной обработки SBOM при принятии решений в отношении рисков, при получении ПО и в ряде других рассмотренных нами случаев?

Потребитель может получать SBOM разными способами. Одним из ресурсов, расширяющих эти возможности, является работа NTIA под названием

Sharing and Exchanging SBOMs, которая была выпущена при поддержке множества заинтересованных сторон и о которой мы уже говорили в других частях книги (<https://clck.ru/3CKXjJ>).

В контексте этого руководства обмен SBOM включает размещение информации или ее нахождение и доступ. Под размещением и нахождением подразумевается публикация SBOM и его обнаружение либо регистрация конечной точки для получения обновлений этого артефакта.

Доступ — это другая сторона процесса, связанная с получением или передачей самого SBOM. К примерам можно отнести сайты, электронную почту, протокол передачи файлов (File Transfer Protocol, FTP), API или системы управления исходным кодом. Руководство приводит и другие варианты, такие как включение URL-адреса SBOM в документацию продукта или предоставление пакетов от поставщиков, что позволяет потребителям ознакомиться с его информацией. И хотя такое понимание изначальных компонентов продукта кажется полезным, могут возникнуть трудности при дальнейшем обновлении продукта или добавлении в него новых компонентов. Здесь потребуются поддерживать страницу SBOM в актуальном состоянии с указанием всех последних компонентов, добавленных в те или иные версии.

Инструменты управления пакетами также помогают в распространении и поиске SBOM, предоставляя его в каталоге верхнего уровня репозитория или в виде SBOM для файла манифеста контейнера.

В качестве еще одного примера NTIA приводит возможность распространять SBOM с помощью системы «издатель — подписчик». Эта система создает ситуацию, когда вышестоящий поставщик может регулярно публиковать обновленные SBOM для своих потребителей, которые будут получать эту информацию вручную или автоматизированным способом. Такой механизм позволяет практически в реальном времени предоставлять клиентам информацию о содержащихся в ПО компонентах.

SBOM опираются на два формата обмена данными — JSON и XML, — которые делают их машиночитаемыми и позволяют автоматизировать обработку этих документов, что очень важно в масштабных инфраструктурах. Эти артефакты понятны людям, но для их получения и обработки намного эффективнее будет использовать компьютеры. Это также позволяет дополнять их с помощью анализа уязвимостей или интеграции с базами данных и другими системами хранения. Автоматизация также поможет интегрировать данные SBOM в существующие инструменты и рабочие потоки организации, что

позволит ее сотрудникам принимать более взвешенные решения на основе информации о компонентах продукта и их уязвимостях.

Современные корпоративные среды включают несколько внутренних команд разработки и множество внешних поставщиков ПО. Принимая во внимание рекомендации создавать и получать новые SBOM с каждым релизом программы, становится ясно, что статичный подход к генерации и распространению этих артефактов не является масштабируемым и рациональным для большинства современных технологических сред.

Крупные корпорации развивали и продолжают развивать удобные инструменты на основе API и автоматизации для упрощения создания, получения, обновления, анализа и сохранения SBOM в больших объемах. Впереди нас ждет дальнейший рост числа MSP и поставщиков, которые будут предлагать платформы, упрощающие использование и управление SBOM для организаций, которые либо не хотят делать это внутренне, либо не имеют достаточно ресурсов и опыта. Часть провайдеров примут форму эскроу-агентов и будут выступать в качестве доверенного посредника между поставщиками и потребителями.

Помимо получения SBOM, потребители будут проверять их целостность и происхождение, чтобы убедиться в валидности. Сюда входит использование для SBOM указанных в них компонентов и аттестаций цифровых подписей и хешей с целью прояснения среды сборки и процессов, использованных для разработки и распространения ПО.

По мере того как SBOM становится для организаций все более востребованным артефактом в рамках их практик защиты цепочек поставок, злоумышленники неизбежно постараются эксплуатировать эти практики для обхода мер защиты и компрометации.

Как отмечает NTIA в своем руководстве *Software Consumers Playbook: SBOM Acquisition, Management and Use* (<https://clck.ru/3CKXkf>), компании способны повысить эффективность SBOM путем передачи данных из этих документов в свои рабочие потоки. Это превращает SBOM из отдельного артефакта в коллекцию данных, которые можно парсить, извлекать и загружать в другие автоматизированные процессы. Реализовать это можно как с помощью внутренне разработанных OSS-инструментов организации, упрощающих извлечение данных из SBOM, так и с помощью коммерческих решений.

В руководстве отмечается, что на данный момент к внедрению SBOM стремятся различные коммерческие проекты, включая медицинское оборудование,

системы управления флотами и прочие. Это стремление создает ситуацию, в которой отдельные части данных SBOM, такие как определенные компоненты, поставщики, версии и прочее, могут предоставить компании нужную информацию в отношении конкретных случаев применения и целей. Это позволит организациям объединять данные SBOM с другими актуальными данными для повышения эффективности различных бизнес-решений.

Когда речь идет о получении и хранении SBOM, потребителям необходимо обеспечить безопасное хранилище и контроль доступа, если только иное не предусмотрено поставщиком. Многие контрагенты, понимая ситуацию, обеспокоены обширным разглашением своих SBOM, поскольку это раскрывает уязвимости в их приложениях и продуктах, чем непременно воспользуются злоумышленники.

В идеале поставщики должны активно устранять такие уязвимости, но, как это бывает в любой организации, сталкиваются с ограничениями в плане кадровых ресурсов, времени, бюджета и опыта. То есть существование продуктов, полностью лишенных уязвимостей, очень маловероятно. По этой причине потребители, которым поставщики предоставили SBOM, должны предпринимать необходимые шаги для его защиты во избежание неавторизованного доступа и разглашения. В качестве примера можно привести памятку OMB 22-18, которая призывает конкретные федеральные агентства США к реализации подходящих механизмов для защиты информации и обмена артефактами вроде SBOM, получаемыми ими от сторонних поставщиков ПО.

Уменьшение шума

Наличие SBOM — это прекрасное начало, но без контекста, отражающего фактическую эксплуатируемость уязвимостей, он создаст много дополнительного шума, вызывающего лишнее беспокойство среди потребителей и заинтересованных сторон. В связи с этим очень важно получать точную информацию о программных компонентах и их эксплуатируемости.

Подробно рассмотренный в главе 4 формат VEX быстро становится незаменимым дополнением к SBOM. Причина в том, что без учета эксплуатируемости пользователи ПО не смогут точно расставлять приоритеты в устранении рисков. Если же потребители обладают информацией об этом, они более отчетливо понимают, какие компоненты приложения требуют дополнительного внимания и представляют реальную угрозу.

Причем при наличии этого контекста мы получаем возможность дополнять изначальный SBOM другими VEX и обмениваться информацией даже без изменения указанного в SBOM списка компонентов. Объясняется это тем, что постоянно обнаруживаются новые уязвимости, но в ПО, связанное с VEX, соответствующие изменения не вносятся. В существующем компоненте может быть найдена еще одна уязвимость, а значит, потребителям необходимо передать дополнительный VEX, информирующий о ее эксплуатируемости. В отрасли начали набирать популярность открытые инициативы вроде спецификации OpenVEX. Их цель — реализовать все требования и сценарии использования VEX, о которых говорят CISA и другие организации, а также предоставить минимальный метод создания, слияния и аттестации документов VEX для передачи информации об эксплуатируемости уязвимостей со SBOM любого формата и программным обеспечением (<https://github.com/openvex/spec>).

Одним из форматов BOM является CycloneDX. Он предоставляет контекст в отношении уязвимых компонентов, чтобы потребителям было проще расставлять акценты при устранении рисков. CycloneDX предлагает и дополнительные возможности, включая поддержку VEX для передачи информации об эксплуатируемости используемых в продуктах уязвимостей, отчеты VDR, посредством которых передается информация об известных и неизвестных уязвимостях компонентов и сервисов, и даже список уязвимостей (*bill of vulnerabilities, BoV*), позволяющий обмениваться данными между системами и организациями, занятыми поиском угроз.

Наконец, осознавая всю сложность нашей экосистемы ПО, сервисов и компонентов, CycloneDX предоставляет поддержку так называемого механизма *BOM-Link*, который позволяет ссылаться на компоненты, сервисы или уязвимости в BOM из других систем.

Потребители должны требовать от поставщиков предоставления сопроводительных артефактов вроде VEX для получения ясности относительно эксплуатируемости уязвимых компонентов в их продуктах и приложениях. Иначе пользователи столкнутся с большими трудностями при устранении уязвимостей.

Помимо VEX и других рассмотренных методов, потребители могут обращаться к базам данных уязвимостей, включая NVD, OSV, OSS Index и др., а также дополнять эту информацию с помощью EPSS и анализа угроз. Все это поможет максимально повысить точность собранных данных и правильно расставить приоритеты по устранению наиболее опасных для организации уязвимостей.

Когда применить патч нельзя

В предыдущем разделе мы уже упоминали о том, что, несмотря на присутствие уязвимостей в ПО, доступные исправления или патчи могут отсутствовать. Объясняется это обычно тем, что у поставщика ограниченные ресурсы, установлен график применения патчей или исправлений, расставлены приоритеты доработки срочной функциональности или у продукта просто закончились дальнейшая поддержка и гарантия.

К примеру, исследование компании Aberdeen Strategy & Research показало, что поставщики не выпустили патчи почти для 20 % из более чем 20 000 уязвимостей, обнаруженных в 2020 году. Причем среди них также были тысячи угроз, для которых есть публично доступные эксплойты. Реальность такова, что аналогично потребителям ПО, которые традиционно пренебрегали применением доступных патчей, поставщики зачастую не успевают латать дыры в своих продуктах и ПО.

В результате создается ситуация, когда пользователи вынуждены принимать собственные меры для устранения рисков, представляемых этими уязвимостями. Обычно это проявляется в виде так называемого *виртуального патчинга*, который, по определению OWASP, является «слоем политики безопасности, предотвращающим эксплуатацию известной уязвимости» (<https://clck.ru/3CKXne>).

В случае веб-приложений это обычно подразумевает перехват потока транзакций и данных, чтобы вредоносный трафик, нацеленный на эксплуатацию уязвимости, не проник внутрь. Зачастую эти меры реализуются в виде WAF, развертываемого главным образом для защиты приложений от таких атак, как XSS или внедрение SQL-кода, нацеленных на компрометацию слабых мест приложения.

Многие исследования показывают, что почти половина уязвимостей на момент своего раскрытия не имеют доступных патчей от поставщиков. Это говорит о нарастающей необходимости оперативно взаимодействовать с сообществом потребителей в отношении уязвимостей и рисков, но также неизбежно создает ситуацию раскрытия уязвимости без патча.

Вдобавок к этим проблемам с отсутствием патчей от поставщиков также нередко наблюдаются бизнес-ограничения, которые не позволяют применить эти патчи, даже когда они доступны. Например, если это требует отключения критических для бизнеса систем или прерывания механизмов генерации дохода. Кроме того, организации могут откладывать решение до

очередного окна обслуживания, прибегая к временному решению в виде виртуального патча.

Именно здесь на помощь приходит технология виртуального патчинга. Вдобавок к методам вроде WAF у потребителей есть возможность вносить изменения в среды, системы и конфигурации для снижения эксплуатационности или влияния уязвимости, исходя из ее особенностей.

Однако не во всех организациях существуют зрелые подходы к виртуальному патчингу. В качестве прекрасного руководства рекомендуем использовать *Virtual Patching Cheat Sheet om OWASP* (<https://clck.ru/3CKXpn>). В этой памятке предлагается методика виртуального патчинга из таких шагов: подготовка, выявление, анализ, создание патча, реализация, тестирование, восстановление и дальнейшее сопровождение. Разберем эти этапы.

Подготовка

На текущем этапе организация разрабатывает основу для процесса виртуального патчинга, не дожидаясь возникновения необходимости устранять уязвимость или противодействовать проникновению. Как пишет OWASP, инциденты могут быть напряженными и вносить хаос, а попытка наладить процесс в подобных условиях приведет к неудаче. Организациям нужно обеспечить надежный мониторинг оповещений от поставщиков с помощью списков рассылки и отслеживания публично раскрываемых уязвимостей. Если помните, в главе 9 мы упоминали это как ключевую меру. Потребителям также следует получить предварительную авторизацию для обхода традиционных процессов патчинга и управления, чтобы иметь возможность применить виртуальный патч как можно быстрее. Им потребуется заранее развернуть соответствующие инструменты, чтобы быть готовыми к оперативному реагированию. Наконец, им необходимо наладить ведение журнала HTTP-аудита для мониторинга полей веб-трафика вроде URI запросов, а также заголовков и содержимого запросов и ответов. Эти данные позволят потребителям выявлять соответствующий трафик и реагировать на него на последующих этапах.

Идентификация

После подготовки к реализации виртуального патчинга потребитель переходит к выявлению уязвимостей в своей среде. По определению OWASP, этот процесс может быть предиктивным или реактивным. В предиктивном случае организация активно анализирует свою систему безопасности,

используя такие методы, как пентесты и автоматизированный веб-анализ, или просматривая исходный код своих приложений в поисках недочетов. Реактивное выявление, которое является нормой для всех организаций, за исключением наиболее зрелых, подразумевает обнаружение уязвимостей путем получения информации от поставщика, из публичных источников или, в худшем случае, в результате реального инцидента.

Анализ

На этапе анализа реализуется несколько мероприятий, нацеленных на то, чтобы организация могла адекватно отреагировать на обнаруженную уязвимость, в том числе определить возможность применения виртуального патча. В чем фактически заключается уязвимость? Есть ли у организации инструменты или возможности для ее устранения? Уязвимость также должна быть отражена в системе отслеживания или тикетов организации и иметь присвоенный идентификатор. Это могут быть имя и номер CVE, идентификатор из уведомления от поставщика или даже присвоенный инструментами сканирования самой организации. Главное здесь — использовать единое имя для любой уязвимости, чтобы обеспечить согласованный процесс реагирования и отслеживания.

Помимо правильной идентификации и отслеживания, организация также должна осознавать степень влияния уязвимости. Не все они одинаковы и имеют один и тот же уровень серьезности. Организации нужно будет понять, какие именно версии ПО подвержены уязвимости, и определить, где именно в их средах эти версии присутствуют. И хотя определение версии и расположения является ключом, организации также необходимо задокументировать конкретную конфигурацию, делающую уязвимость эксплуатируемой. В случае некоторых уязвимостей для этого может быть достаточно одного их присутствия, но в других случаях эксплуатация окажется возможна лишь при определенной конфигурации или факторах среды. Наконец, OWASP рекомендует фиксировать PoC-код эксплойта, если он представлен в уведомлении об уязвимости. Его организация сможет использовать при создании виртуального патча.

Создание виртуального патча

После подготовки, выявления уязвимости и ее анализа можно приступить к созданию виртуального патча. Как это происходит? OWASP определяет два общих принципа, на которых строится виртуальный патчинг: отсутствие ложных срабатываний или пропусков. Это значит, что вы не должны

блокировать легитимный трафик, что может негативно повлиять на миссию организации, бизнес-процессы, доход, клиентов и так далее. Но при этом вы также не должны допускать пропуски, которые позволят вредоносному трафику проникнуть внутрь из-за того, что хакер избежал обнаружения. Очевидно, что достичь обе эти цели в 100 % случаев невозможно, но следует стремиться к тому, чтобы ваши попытки применить виртуальный патч не повлияли отрицательным образом на организацию.

Виртуальные патчи можно создавать вручную или автоматически. В первом случае это могут быть белые списки для сверки с ними входящего трафика и отклонения всего, что не вписывается в определенные критерии. Еще один ручной метод подразумевает использование списка блокировки, основанного на наборе правил и критериев атаки и уязвимости. На основе этого списка система будет блокировать весь трафик, подходящий под указанные в нем параметры. OWASP отмечает, что данный метод неидеален. В этом случае возрастают вероятность ложных срабатываний и шанс проскальзывания вредоносного трафика. Например, мы уже говорили об PoC-коде эксплойта, который может быть предоставлен во время разглашения уязвимости. Этот код организации могут использовать для создания списков блокировки.

В памятке OWASP по виртуальному патчингу отмечается, что оба подхода имеют как плюсы, так и минусы. Списки блокировки проще и быстрее реализовать, но с ними выше шанс пропустить атаку. Белые списки более строгие, но требуют повышенной точности, учитывая, что блокироваться будет все, что не соответствует их критериям, включая легитимный трафик, прерывание которого способно негативно сказаться на бизнесе. Подобные ситуации могут подорвать доверие руководства к безопасности и поставить под угрозу быстрое применение виртуальных патчей в будущем.

Помимо ручного создания патчей, в некоторых случаях их также допускается генерировать автоматически. OWASP приводит несколько примеров, таких как их же инструмент ModSecurity Core Rule Set (CRS) Scripts, ThreadFix Virtual Patching и прямой импорт на устройство под защитой WAF. Эти варианты основываются на получении от инструментов импортированных XML-данных об уязвимости и их преобразовании в виртуальные патчи. И хотя подобные патчи являются отличным решением и очень важны в контексте масштабирования процессов виртуального патчинга, для обеспечения их эффективности зачастую требуются ручное вмешательство, настройка и надзор.

Реализация и тестирование

После создания виртуального патча организации необходимо приступить к его тестированию и реализации для защиты от злоумышленников, ищущих возможность эксплуатировать соответствующие уязвимости. На текущем этапе можно применять различные средства, такие как браузеры, интерфейс командной строки (CLI), прокси-серверы и другие. OWASP рекомендует сначала тестировать виртуальные патчи в режиме «только логирования», чтобы убедиться в эффективности патча, не блокируя при этом никакой легитимный трафик. Затем, когда первичная проверка будет пройдена, разрешается использовать виртуальный патч для блокировки любого потенциально вредоносного трафика. Если он согласуется с ранее определенными белыми списками или списками блокировок, его можно реализовать полноценно с уверенностью, что патч не навредит бизнес-операциям.

Восстановление и дальнейшее сопровождение

На последнем этапе реализуются такие мероприятия, как обновление данных в системе тикетов организации, периодический повторный анализ и ведение отчетности об оповещениях виртуального патча. Правильное обновление системы тикетов позволит собирать метрики, связанные с управлением уязвимостями, для создания будущих виртуальных патчей, а также получать прочую информацию, которая пригодится при последующих инцидентах. Регулярный повторный анализ позволяет определить, когда виртуальный патч уже можно удалить (например, если исходный код приложения был исправлен непосредственно). Для получения сведений о срабатывании виртуального патча можно использовать систему отчетов, проясняющих потенциально вредоносную активность.

Перспективное мышление

Однако важно помнить, что виртуальный патч подобен бандажу, то есть является временным решением проблемы. И хотя это неидеальный метод для защиты уязвимых приложений, которые нельзя пропатчить непосредственно по ряду описанных выше причин, его вполне можно рассматривать как промежуточную меру. Организации должны понимать, что злоумышленники также способны обойти WAF различными хитроумными способами. Например, в 2022 году специалисты компании по кибербезопасности Claroty

продемонстрировали, что могут обойти решения пяти самых популярных поставщиков в сфере WAF, используя JSON для маскировки команд БД и уклонения от обнаружения средствами WAF (<https://clck.ru/3CKXrA>).

Это говорит о том, что организациям на случай обхода злоумышленниками виртуального патча следует заранее разработать планы по устранению уязвимостей приложения, а также иметь проработанную архитектуру безопасности и применять практики вроде модели нулевого доверия для снижения угрозы горизонтального перемещения или ограничения радиуса поражения.

Резюме

В этой главе мы привели практическое руководство для пользователей, которое подразумевает тщательную и глубокую оценку потребления программного обеспечения, а также различных его источников и связанных с ним потенциальных векторов атаки. Мы также обсудили появление SBOM и актуальность этих артефактов с позиции потребителей ПО, попутно подчеркнув ряд потенциальных проблем и областей, которые требуют проработки для эффективного использования SBOM в рамках стратегий управления рисками цепочек поставок. Кроме того, мы отметили, что пользователи должны быть готовы к самостоятельному устранению уязвимостей с помощью виртуальных патчей, особенно когда речь идет об OSS, но также и при работе с поставщиками, которые не исправляют уязвимости своевременно. В следующей главе мы поговорим о тенденциях развития нашей отрасли и общества, а также сделаем некоторые прогнозы в отношении перспектив движения за повышение прозрачности ПО.

12

Перспективы повышения прозрачности ПО

К настоящему моменту уже должно быть ясно, что тема прозрачности ПО и усилия для укрепления уровня защиты цепочек поставок далеко не мимо-летны. По всему миру наблюдается появление огромного числа инициатив по разработке нормативов, инструментов, технологий и стандартов. Экспоненциальный рост числа атак на цепочки поставок, включая описанные нами, появление новых фреймворков, а также постепенное развитие базы данных уязвимостей и экосистемы их оценки — все это привлекает огромное внимание к вопросам безопасности.

В текущей главе мы разберем актуальные законы, требования и потенциаль-ные решения, которые могут сыграть значительную роль в развитии иннова-ций, а также попробуем представить дальнейшие перспективы в этой области.

Современные инициативы, законы и требования

В плане появляющихся законов и требований уже должно быть ясно, что правительства по всему миру постепенно осознают критическую важность ПО для своих структур, агентств и общества в целом. Программное обеспе-чение стало нераздельно связано практически со всеми аспектами социума, начиная с повседневных досуговых активностей и заканчивая наиболее критической инфраструктурой и государственной обороной. В книге мы

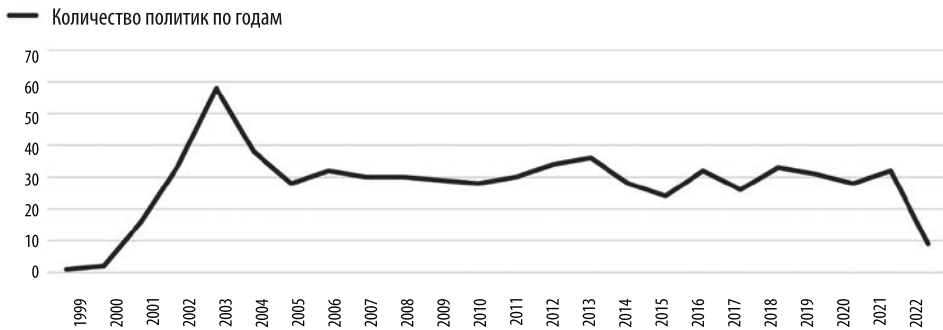
приводили заявления различных правительственных учреждений США, в том числе Министерства обороны, и технологических гигантов, подчеркивающие важность ПО для современного общества.

Правительства разных стран, наконец, все больше признают зависимость людей от программного обеспечения и начинают делать шаги к тому, чтобы сфера открытого ПО была признана общественным благом из-за ее повсеместного присутствия в нашей жизни. К примеру, Германия недавно запустила государственный технологический фонд для поддержки OSS (<https://clck.ru/3CKXs3>). Руководство страны подчеркивает значимость сферы OSS в современной цифровой инфраструктуре, несмотря на хрупкость ее экосистемы.

В Великобритании группа под названием OpenUK в 2022 году запустила программу *Summer of Open Source Software Security*. Участники инициативы отметили, что современная государственная критическая инфраструктура строится на основе OSS. Этот проект был направлен на потребность в защите и обслуживании открытых проектов ввиду зависимости от них национальной критической инфраструктуры (<https://clck.ru/3CKXtP>).

Еще одним ярким законодательным примером стал Закон о защите программного обеспечения с открытым исходным кодом, представленный Сенату США в 2022 году. Данный закон возлагает на CISA ответственность за обеспечение безопасности OSS. В обязанности агентства входят распространение информации в отрасли и участие в ее процессах, поддержка федеральных инициатив по защите OSS, а также сотрудничество с негосударственными субъектами для обеспечения долгосрочной безопасности OSS. Перед CISA поставлена задача разработать фреймворк для правительства, отрасли и сообщества OSS с акцентом на защите OSS с помощью передовых практик. Кроме того, в закон включена формулировка, предусматривающая проведение исследований с целью определения роли OSS в критической инфраструктуре США (<https://clck.ru/3CKXuR>).

Исходя из этих материалов вместе с памяткой Министерства обороны США в отношении OSS, о которой мы ранее говорили, можно решить, что интерес и стремление к использованию и управлению OSS растут. Тем не менее работа Центра стратегических и международных исследований, опубликованная в начале 2023 года, показывает, что Правительство США уже не одно десятилетие разрабатывает политики в отношении OSS. Данный отчет показал, что в период с 1999 по 2022 год по всему миру появилось 669 правительственных инициатив по этому вопросу (рис. 12.1).

**Рис. 12.1**

Источник: <https://clck.ru/3CKZ74>, Центр стратегических и международных исследований

Тем не менее увеличение потребления OSS, которое встречается в 97 % кодовых баз, вместе с ростом числа атак на цепочки поставок ПО привлекают интерес со стороны правительства и отрасли в плане обеспечения их безопасности.

В США наблюдается повышенная активность правительства в отношении защиты цепочек поставок ПО. И хотя определенные инициативы уже существовали, публикация Указа 14 028, посвященного повышению уровня национальной кибербезопасности, бесспорно, тому способствовала. Раздел 4 упомянутого указа, нацеленный на повышение уровня безопасности цепочек поставок ПО, конкретно фокусируется на этой проблеме и ставит задачи различным правительственным субъектам США по разработке руководств и требований для устранения рисков, связанных с цепочками поставок ПО.

Одна из таких задач была поставлена институту NIST, где требовалось разработать «руководства для определения практик, повышающих уровень безопасности цепочки поставок ПО». В итоге NIST обновил свой фреймворк SSDF (<https://clck.ru/3CKXw5>), попутно предоставив отдельное руководство по защите цепочек поставок (<https://clck.ru/3CKXxT>). На его основе OMB выпустило памятку *Enhancing the Security of the Software Supply Chain through Secure Software Development Practices (M-22-18)*. В ней отмечается, что Правительство США опирается на технологии и цифровые продукты, а также говорится о постоянных угрозах, с которыми оно сталкивается и которые могут помешать ему предоставлять необходимые обществу услуги.

Памятка требует от федеральных агентств США соответствия ранее упомянутому руководству при использовании в собственных системах стороннего ПО. Это касается всего программного обеспечения с момента публикации памятки, включая как вновь приобретенное, так и существующее.

Стоит отметить, что документ специально исключает ПО, разработанное самими агентствами, что, естественно, вызывает беспокойство в отношении его безопасности и требует применения ими внутренних мер защиты. Федеральные агентства также уязвимы к инцидентам безопасности, примером чего служит взлом Управления кадровой службы США в 2015 году, в результате которого были раскрыты данные более 21 миллиона граждан США.

В памятке утверждается, что агентства должны использовать *только то* ПО, которое предоставлено производителями, способными провести самоаттестацию на соответствие ранее упомянутому руководству NIST. В ней содержится призыв к начальникам отделов информационной безопасности и руководителям отделов снабжения обеспечить выполнение этих требований. Производители ПО должны иметь возможность предоставлять самоаттестации в виде *деклараций соответствия*, и эти аттестации должны быть получены агентством для *всех* сторонних программ, подходящих под требования памятки. Подчеркивается также важность включения портфолио продуктов в аттестации для повторного их использования всеми заинтересованными сторонами. Это должно повысить эффективность и облегчить процесс обмена данными внутри федеральной экосистемы США.

Понимая, что у производителей ПО не всегда получается соответствовать всем аспектам SSDF и руководства NIST (временно или, возможно, постоянно), памятка предусматривает использование так называемых программ реализации мероприятий (plans of action and milestones, POAM). С помощью POAM поставщики могут документировать недостатки в своих аттестациях, поясняя предпринимаемые меры по их устранению и указывая запланированные даты. Подобный механизм существует и в других программах соответствия, таких как FedRAMP и RMF (Risk Management Framework) NIST.

Поставщикам нужно предоставить детали в отношении их организации и продуктов, подвергаемых аттестации, после чего задокументировать их соответствие руководству по защищенной разработке и отразить это в стандартизированной форме самоаттестации. Агентства смогут запросить сторонний анализ, если сервис или программное обеспечение считаются критическими, и это обусловлено рисками. Привлечение к анализу третьих сторон может включать группы вроде FedRAMP и прочие независимые организации.

Важно отметить, что в случае самоаттестаций и иногда при привлечении сертификационных фирм есть возможность уменьшить количество доступных поставщиков ПО, с которыми федеральное правительство может работать. Как говорилось ранее при рассмотрении FedRAMP, платформа FedRAMP Marketplace (<https://clck.ru/3CKXyY>), где представлены

авторизованные предложения облачных провайдеров, на момент написания книги содержит примерно 300 таких сервисов. И это несмотря на то, что она существует уже около десяти лет и на коммерческом рынке облачных услуг присутствуют десятки тысяч провайдеров. Поэтому, хотя привлечение независимых организаций для анализа направлено на поиск более безопасных поставщиков и предложений, это также ограничивает их количество из-за дополнительных затрат и требований.

Многие утверждают, что риск утратить доступ к передовым поставщикам и решениям должен рассматриваться в более широком контексте угроз, не ограничиваясь лишь сферой кибербезопасности. То есть возникающий для бизнеса и не только риск упустить возможную модернизацию тоже должен учитываться. Тем не менее некоторые лидеры, например Джошуа Корман, также выступают с идеей «ограничения числа поставщиков наиболее качественными» с целью повышения безопасности цепочек поставок за счет использования преимущественно надежных и защищенных предложений и компонентов. Оба аргумента, естественно, имеют свой вес, который будет отличаться в зависимости от уровня допустимых рисков организации.

Помимо самостоятельной или сторонней аттестации на соответствие руководству NIST, агентства также могут в своих требованиях указывать SBOM, особенно если по определению NIST их ПО считается критическим. Это определение мы ранее рассматривали, и оно отражается в другой памятке ОМВ под названием *Protecting Critical Software Through Enhanced Security Measures* (<https://clck.ru/3CKXzC>).

Предоставляемые поставщиками SBOM должны соответствовать установленным NTIA форматам и включать минимально необходимые элементы, о которых речь шла в главе 4. Как и в случае самоаттестаций, памятка ОМВ подчеркивает потребность во взаимном обмене и повторном использовании SBOM среди агентств во избежание дублирования работ как с их стороны, так и со стороны поставщиков. Основываясь на возможности применения SBOM, агентства могут потребовать и другие артефакты, такие как результаты проверки на уязвимости и целостность в отношении кода и подтверждение участия в программах по раскрытию уязвимостей.

Руководители Федеральной комиссии по регулированию энергоресурсов (Federal Energy Regulatory Commission, FERC) также выразили желание обновить стандарты защиты критической инфраструктуры, ориентированные на обеспечение кибербезопасности предприятий электросетей и прочих субъектов сектора энергетики. Звучат также призыв к повышению прозрачности ПО и критика самоаттестаций в сравнении с использованием SBOM,

которые предоставляют машиночитаемые свидетельства о присутствии или отсутствии уязвимых программных компонентов (<https://clck.ru/3CKXzu>).

Исходя из работы *Module Procurement Contract Language Addressing Cyber-security Supply Chain Risk 3.0* Института Edison Electric, опубликованной в октябре 2022 года, становится ясно, каков вектор мышления FERC и NERC. Документ содержит требование R.1.2.5 «Предполагаемые условия контрактов EEI», которое касается сохранения целостности и достоверности аппаратного обеспечения, прошивок, ПО и патчей. Конкретно в разделе (e) сказано, что «исполнитель должен предоставлять SBOM для создаваемых (включая лицензированные) продуктов, состоящий из списка компонентов и связанных с ними метаданных» (<https://clck.ru/3CKY2b>).

Помимо этого, SBOM упоминался в первоначальной версии Закона об оборонном бюджете США на 2023 год. В разделе 6722 «Управление рисками цепочек поставок Министерства внутренней безопасности» содержалось требование о включении в заявку на участие в тендере спецификации BOM и сертификата, подтверждающего, что каждый элемент в этом BOM «не содержит известных уязвимостей или дефектов, влияющих на безопасность конечного продукта либо сервиса». Это предполагает использование таких ресурсов, как NVD NIST, в которой отслеживаются уязвимости и угрозы безопасности OSS и стороннего ПО. Многие представители отрасли сразу же раскритиковали данное предложение, ссылаясь на невозможность создания ПО, полностью лишённого уязвимостей. Тем не менее стоит отметить, что допускалось указывать планы по устранению и разрешению каждой отдельной уязвимости или недочёта, присутствующего в представленном BOM.

Ярким примером критики со стороны индустрии стало письмо, опубликованное Альянсом цифровых инноваций — группой, представляющей таких технологических гигантов, как AWS, Google Cloud и VMWare. В нем авторы призвали Конгресс «удалить из NDAA требование предоставления SBOM и дать отрасли и агентствам больше времени для выработки решений, которые будут лучше защищать цепочки поставок страны». Ознакомиться с этим письмом можно здесь: <https://clck.ru/3CKY3X>.

И хотя первоначальный вариант NDAA включал пункты в отношении SBOM и повышения прозрачности цепочек поставок ПО, давление со стороны индустрии взяло верх. В результате из итоговой версии этого законопроекта все связанные с этими инициативами формулировки были удалены (см. <https://clck.ru/3CKY4K>). Тем не менее, как мы уже говорили, отдельные субъекты в среде национальной обороны, такие как Армия США, предпринимают

собственные усилия по внедрению SBOM, используют механизм RFI и прочие договорные методы, подавая индустрии сигналы, что они нацелены на повышение прозрачности ПО.

Помимо требований на федеральном уровне, отдельные подразделения Министерства обороны США заявили о своих намерениях широко внедрить SBOM для управления уязвимостями и потребления ПО. В конце 2022 года Армия США выпустила RFI с целью сбора обратной связи от отрасли в отношении «приобретения, валидации, потребления и использования SBOM и связанных с этим вопросов». В RFI признается, что в инфраструктуре Армии США используются сотни тысяч программных компонентов, что отражает заявления руководителей из федеральной, военной и промышленной сфер о роли ПО в будущих военных конфликтах.

Госдепартамент США намерен сделать SBOM основным компонентом при взаимодействии с поставщиками. В июле 2022 года Госдеп опубликовал проект запроса предложений в рамках своей программы госзакупок. Бюджет этого проекта, получившего название *Evolve*, по приблизительным оценкам, составит от 8 до 10 миллиардов долларов. Согласно требованиям раздела H.14.7, исполнители обязаны предоставлять SBOM в утвержденном отраслью формате SPDX или CycloneDX, который отражает компоненты поставляемого ПО. Предоставленный SBOM должен содержать установленный NTIA минимум элементов. При этом требования распространяются на все заказы по контракту, а частота предоставления SBOM будет определяться выходом любых обновлений ПО и его компонентов (<https://clck.ru/3CKY5h>).

Причем подобная активность вокруг прозрачности ПО и SBOM не ограничивается США. В Евросоюзе в 2022 году появилась инициатива под названием *EU Cyber Resilience Act* (Закон о киберустойчивости ЕС). Этот закон устанавливает требования к кибербезопасности для продуктов с цифровыми элементами, которые продаются на рынке ЕС (<https://clck.ru/3CKY6R>).

Согласно данному документу, по примерным оценкам, за 2021 год киберпреступления нанесли ущерб 5,5 триллиона евро. В нем также отмечается, что основной проблемой низкого уровня кибербезопасности является непонимание того, какие продукты являются надежными и до какой степени. Основная задача закона — это создание условий для разработки защищенных программных и аппаратных продуктов за счет сокращения их уязвимостей, а также требование к производителям устранять проблемы безопасности в течение всего срока жизни их продуктов. Все это даст пользователям возможность принимать взвешенные решения с учетом реального уровня безопасности продуктов с цифровыми элементами.

В акте также указан призыв к поставщикам определять и документировать компоненты продукта с помощью SBOM: «Для упрощения анализа уязвимостей производители должны идентифицировать и документировать компоненты, содержащиеся в их продуктах с цифровыми элементами, включая составление SBOM». Отдельно в нем указывается необходимость выявления сторонних уязвимых компонентов.

Влияние правительственных цепочек поставок на рынки

Данные в разных источниках немного отличаются, но мы отчетливо видим, что Правительство США тратит значительные средства на рынке IT и программного обеспечения. По некоторым сведениям, федеральное правительство в 2023 году выделило на эти нужды 65 миллиардов долларов (<https://clck.ru/3CKY73>). Это больше, чем в 2022 году, когда бюджет составил 58 миллиардов долларов, и говорит о непрерывном увеличении расходов правительства на сферу IT среди гражданских ведомств (рис. 12.2).

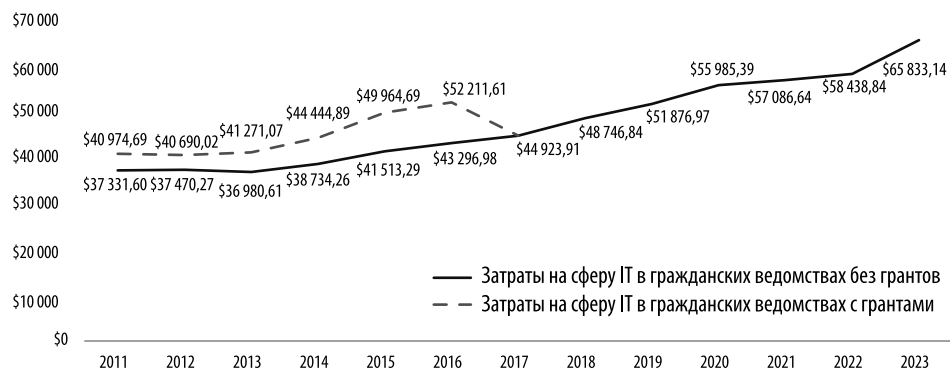


Рис. 12.2

На 2023 год Министерство обороны США запросило 773 миллиарда долларов. И хотя значительная часть этого бюджета предназначена не для технологий и программного обеспечения, определенная сумма все же выделялась именно на эти нужды. К тому же, как мы неоднократно упоминали, все больше современных систем вооружений и платформ основываются на ПО. Разумеется, эти расходы касаются только Министерства обороны США на федеральном уровне и не включают другие правительственные рынки, а именно региональных и локальных субъектов, каждый из которых имеет

собственные статьи расходов на ИТ. При этом это не значит, что все затраты на ИТ связаны именно с ПО. Но, как мы говорили в книге, почти все современные технологии в той или иной форме основаны на ПО. Это говорит о том, что при появлении новых политик, законодательных или контрактных требований в правительственной сфере эти изменения влияют на значительную часть всей индустрии ПО и технологий.

Многие утверждали, что, поскольку Правительство США часто приобретает ПО у тех же поставщиков, что и остальное сообщество, разрабатываемые им требования повлияют на всю отрасль, даже вне государственного сектора. Такие аспекты, как использование практик защищенной разработки, обеспечение видимости компонентов ПО и реализация механизмов вроде программ по раскрытию уязвимостей, будут влиять не только на сотни и тысячи поставщиков ПО, сотрудничающих с правительством, но и на тех, кто работает с коммерческими потребителями, а также на миллионы людей по всему миру.

Предполагается также, что стандарты NIST, применяемые правительством, например фреймворк SSDF и прочие руководства по обеспечению безопасности цепочек поставок, наверняка начнут добровольно применяться и субъектами коммерческого сектора.

Одним из ярких примеров является Cybersecurity Framework (CSF), опубликованный NIST в 2014 году как следствие предыдущего Указа об укреплении кибербезопасности федеральной и критической инфраструктуры. Этот указ сделал обязательным использование CSF федеральными агентствами, но, как пишет NIST в своем CSF FAQ, сейчас данный фреймворк применяется многими организациями частного сектора, несмотря на то что для них он необязателен (<https://clck.ru/3CKY8B>).

К другим примерам относятся руководства и фреймворки вроде FedRAMP, который необходим только для CSP, сотрудничающих с федеральным правительством. Он тоже послужил ориентиром для частного сектора и стал упоминаться в коммерческом руководстве Cloud Controls Matrix (CCM) от CSA.

Помимо этого, мы обсуждали фреймворк СММС, который затронул более чем 300 000 поставщиков ВПК, сотрудничающих с Министерством обороны США. В том числе он повлиял на общую дискуссию вокруг управления рисками цепочек поставок в отношении контрагентов за пределами сферы ПО. Здесь также будет уместно добавить, что, по данным опроса 2022 года, большинство поставщиков ВПК были не готовы к выполнению новых требований, а 87 % исполнителей не набрали достаточного балла по меркам

разработанного Министерством обороны США приложения к Федеральному регламенту закупок, в рамках которого они должны указывать собственные показатели оценки в Системе определения рисков поставщиков. Также были отчеты о результатах самоаттестаций, расходящихся с результатами анализа сторонними организациями. И это вызывает опасения в свете появления новых требований к безопасности цепочек поставок в памятке ОМВ М-22-18, обязывающей сторонних поставщиков ПО проходить самоаттестацию на предмет надлежащего использования практик защищенной разработки. Механизм самоаттестации подвержен таким слабостям, как субъективность и отсутствие прозрачности, когда на кону стоят заключение контракта и получение прибыли (<https://clck.ru/3CKY8a>).

Многие полагают, что из-за размеров и масштабов правительства коммерческий сектор вынужден следовать его примеру в отношении внедряемых требований и подходов. В итоге, несмотря на то что такие механизмы, как SBOM и аттестация в области разработки защищенного ПО, могут быть обязательными только для государственного сектора, они наверняка станут таковыми и для коммерческой сферы, особенно в других строго регулируемых и предусмотрительных отраслях вроде финансов и здравоохранения.

Рост числа атак на цепочки поставок

В завершение книги стоит напомнить, что, по нашим прогнозам, количество атак на цепочки поставок будет только расти. Это связано с несколькими факторами, такими как усложнение цифровой среды и широкое использование OSS и стороннего ПО. Кроме того, злоумышленники осознают высокую эффективность подобных атак.

Среди наиболее тревожных показателей можно выделить отчет *State of the Software Supply Chain* компании Sonatype от 2022 года. В ходе своего исследования специалисты Sonatype выяснили, что в течение последних трех лет число атак на цепочки поставок ежегодно возрастало в среднем на 742 %. В публикации также сказано, что шесть из семи уязвимостей возникают в транзитивных зависимостях. И эти данные подкрепляются другими источниками, такими как отчет *State of Dependency Management om Endor Labs*, о котором мы говорили в главе 4 при обсуждении проблем с обеспечением прозрачности ПО и OSS.

Один уязвимый компонент или кусок кода может повторно использоваться множеством проектов, что ведет к его экспоненциальному распространению

в экосистеме и в итоге к повышению уровня рисков. Например, уязвимый код Log4j был задействован в сотнях проектах и попал в десятки тысяч других компонентов, которые были скачаны более 150 миллионов раз. Причем, по данным Sonatype на момент написания книги, он до сих пор ежедневно скачивается десятки тысяч раз (<https://clck.ru/3CKYBC>). В наше время эксперты выявляют сотни тысяч зараженных пакетов, распространяющихся по экосистеме. Это говорит о возросшем внимании злоумышленников к цепочкам поставок.

В исследовании Sonatype также прослеживается один общий фактор — массивный рост разработки и потребления OSS. Согласно рис. 12.3, взятому из отчета, в ведущих экосистемах сейчас насчитывается более 3 миллионов проектов, 47 миллионов версий, а ежегодное число запросов превышает 3 триллиона. Все это говорит о более чем 30%-ном приросте в год.

При этом увеличение масштабов разработки и потребления OSS ведет к повышению скорости, эффективности и развития почти во всех отраслях за счет повторного применения кода. Однако это также создает значительные системные риски ввиду повсеместного использования стороннего ПО, которое также является и самым уязвимым. В определенной степени широкий успех и внедрение OSS неизбежно способствуют повышению рисков, поскольку это увеличивает поверхность атаки в отраслях, задействующих такое ПО.

Экосистема	Всего проектов	Всего версий проектов	Примерный объем запросов за 2022 год	Ежегодный прирост числа проектов	Ежегодный прирост скачиваний	Среднее число версий проекта
Java (Maven)	492 k	9,5 M	675 B	14 %	36 %	19
JavaScript (npm)	2,06 M	29 M	2,1 T	9 %	32 %	14
Python (PyPI)	396 K	3,7 M	179 B	18 %	41 %	9
.NET (NuGet)	321 K	4,7 M	96 B	–5 %	23 %	15
Всего/ в среднем	3,3 M	47 M	3,1 T	9 %	33 %	14

Рис. 12.3

Источник: <https://clck.ru/3CKYCV>, Sonatype

Дополнительно эту проблему усугубляют транзитивные зависимости. Одна средняя библиотека имеет 5,7 таких зависимостей, 62 % которых содержат уязвимости в собственных сторонних зависимостях. Поэтому очень важно обеспечить тщательный отбор зависимостей ввиду потенциальных связанных с ними рисков. Некоторые поставщики интегрируют функциональность, которая помогает разработчикам принимать решения с учетом рисков при выборе компонентов для своего ПО.

Продолжая тему управления зависимостями, отчет Sonatype ставит перед разработчиком ряд вопросов. К ним относятся отслеживание и управление 150 изначальными зависимостями, годовое увеличение числа этих зависимостей до 1500 на приложение, а также наличие достаточного опыта в области безопасности и юриспруденции для выбора наиболее безвредных версий. В среде, где многие разработчики оцениваются по скорости поставки, этот уровень анализа и тонкостей зачастую не выступает ключевым критерием, не говоря уже о его практичности и реалистичности.

И хотя доклад Sonatype является одним из наиболее достоверных, он далеко не единственный, который подтверждает экспоненциальный рост числа атак на цепочки поставок. Все более обширное использование OSS вкупе с возросшим вниманием хакеров к этому вектору атак и когнитивной перегрузкой многих субъектов, отвечающих за принятие решений по безопасности, создает сложную ситуацию.

К сожалению, мы не видим признаков ослабления данной тенденции, поскольку злоумышленники осознали эффективность этого направления атак. Лично мы считаем, что хакеры продолжают разрабатывать все более изощренные методы нападения на OSS-компоненты, сервисы и поставщиков ПО, о чем свидетельствуют некоторые из приведенных известных случаев.

Дополнительно подтверждая эту тенденцию, в 2022 году Европейское агентство по кибербезопасности (European Union Agency for Cybersecurity, ENISA) назвало компрометацию программных зависимостей угрозой номер 1 в перспективе развития до 2030 года (рис. 12.4). Агентство оценило ее выше других угроз, таких как дезинформация, дефицит специалистов и злоупотребление технологиями ИИ. Это показывает, что опасность для цепочек поставок ПО, будучи критической уже сейчас, также сохранит данный статус в отношении организаций и государств еще длительное время, о чем свидетельствуют рассмотренные нами тенденции, исторический контекст и каталоги атак на цепочки поставок.

Топ-10 угроз кибербезопасности в перспективе на 2030 год



Рис. 12.4

Источник: <https://clck.ru/3CKYDa>, Европейское агентство по кибербезопасности

Растущая взаимосвязанность IoT-устройств

Одна из наиболее заметных тенденций в современном обществе — быстрый и непрерывный рост числа подключенных к Интернету устройств. Как мы неоднократно отмечали, практически каждая сфера нашей жизни привязана к цифровому устройству и ПО, начиная с обычных бытовых предметов вроде умных часов и медицинских приборов и заканчивая критической инфраструктурой и оборонным комплексом.

И хотя значительная часть этой экосистемы управляется серверами, конечными точками и мобильными устройствами, данная тенденция экспоненциально ускоряется в свете невероятно быстрого развития сферы IoT.

IoT-устройства зачастую определяются как объекты, подключенные к Интернету и способные передавать данные. При этом они могут представлять собой такие изделия, как беспроводные датчики, предметы домашнего обихода, промышленные инструменты и производственное оборудование.

В современном обществе существует невероятное множество подключаемых устройств. Росту их числа способствует большое число факторов, включая технологии создания недорогих датчиков, облачные вычисления и инновационные сетевые протоколы. Кроме того, мы наблюдаем быстрый подъем так называемой сферы промышленного IoT (Industrial IoT, IIoT). IIoT использует облачные технологии, аналитику и машинное обучение для обеспечения работы таких сфер, как умное производство, умные города, умные электросети и цифровые цепочки поставки.

Подобная связь открывает огромный потенциал для инноваций и возможностей, которые ранее были невозможны. Эта всепроникающая связность дает такие преимущества, как новые бизнес-модели и потоки доходов, повышение эффективности и операционной деятельности и даже улучшение качества личной жизни. Вместе с тем она несет в себе и грандиозные системные риски, подобные которым мы еще не видели.

К типичным проблемам безопасности в области IoT-устройств относятся жестко закодированные и слабозащищенные учетные данные, отсутствие регулярных и своевременных обновлений аппаратных прошивок, а также ограниченная видимость масштаба охвата IoT-устройствами, которые использует организация.

Как мы видели из отчета Data Breach компании Verizon, компрометация учетных данных является популярным вектором атаки. И это не сулит ничего хорошего с учетом того, что многие устройства IoT и OT (операционных технологий) зачастую используют предустановленную учетную информацию от поставщиков, становясь легкой целью для злоумышленников. Еще одной типичной проблемой IoT-устройств является отсутствие регулярных обновлений ПО. Если принять в расчет охват и масштабность увеличивающейся сферы IoT, становится очевидно, что этот рост может усугубить проблемы, связанные с цепочками поставок ПО, когда один скомпрометированный патч способен оказать огромное влияние.

Некоторые источники прогнозировали, что к концу 2022 года в мире будет более 13 миллиардов подключаемых устройств, а к 2025 это число увеличится до 75 миллиардов. И правительства с регулирующими органами просто не успевают за таким быстрым ростом. Тем не менее уже реализуются некоторые инициативы, нацеленные на регулирование рынка IoT с акцентом на конфиденциальность и особенно кибербезопасность.

Британское общество по защите прав потребителей под названием Which? выпустило свой отчет от 2023 года, подчеркнув, что многие лидирующие

производители умных устройств могут отказываться от поддержки ПО для своих продуктов спустя буквально пару лет. В документе говорится, что это вызывает беспокойство в отношении не только операционной поддержки и долговечности «умных» устройств, но и создаваемых ими рисков ввиду повсеместного использования.

Взгляд в сторону Евросоюза и США показывает, что в этих регионах прорабатываются меры регулирования в отношении конфиденциальности и кибербезопасности, часть из которых является специфичной для IoT, а некоторые затрагивают также и более широкую сферу. На потребительском фронте Евросоюза самым ярким примером таких мер выступает регламент *General Data Protection Regulation (GDPR)*, который вступил в силу в 2018 году на территории ЕС и Великобритании. GDPR акцентируется на правилах обработки личных данных и предоставляет схему, выстроенную на основе ключевых принципов справедливости, законности и прозрачности.

И хотя GDPR разработан в ЕС, он может действовать в отношении организаций за его пределами, если в распоряжении подобных организаций есть данные граждан ЕС. До этого мы уже упоминали закон о киберустойчивости Евросоюза. Он требует от поставщиков реализации безопасности в рамках процессов проектирования и производства, а также обеспечения прозрачности компонентов ПО, присутствующих в их устройствах, включая IoT.

В США также реализуется программа классификации IoT-устройств на основе их уровня кибербезопасности (<https://clck.ru/3CKYFW>). Эта программа нацелена на граждан США и, аналогично принятому в Евросоюзе Закону о киберустойчивости, на повышение прозрачности и информирования потребителей в отношении безопасности покупаемых и используемых ими продуктов. Кроме того, данная инициатива призвана стимулировать производителей IoT-устройств уделять больше внимания безопасности при производстве своей продукции. Белый дом, воспользовавшись вызванной Указом 14 028 активностью в сообществе, провел серию встреч с представителями крупнейших интернет-компаний для продвижения этой инициативы (<https://clck.ru/3CKYGb>).

Параллельно NIST запустил программу кибербезопасности для IoT, которая предоставляет стандарты, руководство и необходимые инструменты. В нее входит информация для производителей IoT-устройств, а также для федеральных агентств и рядовых потребителей, желающих использовать подобные устройства. В конце 2022 года Министерство экономики США сформировало Консультативный совет по вопросам IoT, который должен помочь в защите Интернета вещей, а также предоставить всю основную информацию

и рекомендации, исходя из опыта индустрии (<https://clck.ru/3CKYHj>). Еще одним ярким примером стал законопроект об ассигнованиях на 2023 год, в частности его раздел 3305, который, помимо прочего, требует от производителей медицинских IoT-устройств обеспечения адекватного уровня киберзащиты своей продукции и связанных с ней систем, включая мониторинг, выявление и устранение уязвимостей, а также предоставление послепродажных обновлений и патчей. Контроль за соблюдением этих требований возложен на FDA (<https://clck.ru/3CKYKV>).

Очевидно, что расширение сферы IoT-устройств, используемых в профессиональных и личных нуждах практически во всех отраслях, ведет к значительному увеличению поверхности атаки и возможностей для злоумышленников. Эти IoT-устройства работают на программном обеспечении и активно распространяются по всем сферам жизни общества. При этом организации, как поставщики, так и потребители, редко уделяют достаточно внимания их безопасности. Мы предполагаем дальнейшее появление все новых требований, аналогичных тем, что прописаны в законопроекте об ассигнованиях от 2023 года и в прочих нормативных актах.

Что дальше?

Несмотря на то что предсказать будущее в лучшем случае сложно, а чаще всего невозможно, растущее стремление к повышению прозрачности ПО отчетливо ощутимо. Риски, которые уязвимое программное обеспечение представляет для простых граждан, правительственных субъектов и общества в целом, больше нельзя игнорировать.

Сегодня ПО уже укоренилось повсюду, начиная с базовых досуговых активностей до наиболее критической инфраструктуры, гражданских служб и систем государственной обороны. Как мы говорили на протяжении книги, программное обеспечение представляет практически безграничный потенциал для инноваций и изобретений, но оно также создает определенные системные риски для нашей жизни, которые нельзя недооценивать. Помимо рассмотренных нами многочисленных инициатив со стороны правительств и индустрии, мы прогнозируем увеличение вовлеченности со стороны государственного сектора, академической сферы и всей отрасли, нацеленное на повышение прозрачности и безопасности ПО.

В начале 2023 года в своем интервью директор CISA Джен Истерли подчеркнула необходимость сотрудничества правительства и IT-индустрии для повышения уровня кибербезопасности. Истерли отметила, что атаки

постоянно совершаются не только на технологические компании, но и на больницы, учебные заведения и критическую инфраструктуру, а текущий подход к кибербезопасности назвала неэффективным. Она заявила, что компаниям нужно предпринять шаги по созданию безопасных продуктов и программного обеспечения. Обозначив цифровую сферу как общественное благо, Истерли сказала, что инициативы по повышению безопасности ПО должны быть напрямую связаны с социальной устойчивостью. Ее позиция совпадает с мнениями других исследователей, таких как Чинмайя Шарма, который в своей работе *A Tragedy of the Digital Commons* говорит о том, что ответственность за обеспечение кибербезопасности больше не должна лежать на плечах потребителей и простых граждан, — вместо этого ее должны реализовывать компании и поставщики ПО, которые находятся в гораздо более удобном положении (<https://clck.ru/3CKYKv>).

Подтверждая происходящие изменения, газета *Washington Post* в январе 2023 года написала, что новая стратегия общенациональной безопасности не похожа на все предыдущие и что «впервые регулирование включено в стратегию по обеспечению государственной кибербезопасности» (<https://clck.ru/3CKYLS>).

Джим Демпси, эксперт по вопросам конфиденциальности и интернет-политике, который также читает лекции в Калифорнийском университете, раскрывает тему усиления законодательного контроля в сфере кибербезопасности в своей статье *One Small Legislative Step for Cybersecurity*, опубликованной в январе 2023 года (<https://clck.ru/3CKYR9>). В ней Демпси отмечает, что упомянутые ранее требования к медицинским IoT-устройствам, которые были заявлены в законопроекте об ассигнованиях 2023 года, являются первым случаем с 2005 года, когда Конгресс безоговорочно уполномочил все агентства регулировать кибербезопасность частных и управляемых систем любого типа. До этого момента большинство мер кибербезопасности в отношении критической инфраструктуры были добровольными. Критики утверждали, что кибербезопасность является проблемой, которую рынок не в состоянии решить самостоятельно, и данная точка зрения остается актуальной. Если это предположение верно, то возникает обеспокоенность относительно того, что наиболее критическая инфраструктура США находится в руках частного сектора, где зачастую используются уязвимые программные компоненты.

Многие полагают, что Национальная стратегия США в области кибербезопасности и связанные с ней инициативы, которые традиционно основывались на добровольном участии, начинают смещаться в сторону законодательного регулирования. На фоне новых требований возрастет ответственность поставщиков и субъектов, которые пренебрегают своими обязанностями

в отношении киберзащиты ПО, продуктов и цифровых систем. Как мы только что сказали, некоторые считают сферу кибербезопасности провалом и проблемой, которую путем одних только добровольных усилий решить не удастся. Этот растущий запрос на законодательное регулирование должен скорректировать текущую ситуацию, восполнив пробелы в сферах национальной и общественной безопасности, обе из которых в значительной степени опираются на программное обеспечение.

В Соединенных Штатах существуют 16 секторов инфраструктуры, которые охватывают такие сферы, как коммуникации, энергетика, экстренные службы, транспорт, ИТ, ВПК и др. Из-за увеличения числа кибератак на субъекты критической инфраструктуры ведущие специалисты сферы безопасности стали уделять этим секторам больше внимания, в том числе их регулированию. В качестве одного из примеров можно привести заместителя советника по национальной безопасности Энн Нойбергер и ее команду. И хотя эта группа определила, что во многих секторах критической инфраструктуры меры регулирования кибербезопасности реализованы, это касается не всех областей. В нескольких сферах США также были выявлены недоработки, включая продовольственный, сельскохозяйственный и образовательный секторы.

Как пишет Федеральное агентство по управлению в чрезвычайных ситуациях (Federal Emergency Management Agency, FEMA), частному сектору принадлежит 85 % критической инфраструктуры и важнейших ресурсов страны. В их отчете также говорится, что в связи с увеличением среднего возраста инфраструктуры она становится все более уязвимой для сбоев (<https://clck.ru/3CKYXD>).

Правдивость этого отчета подтвердилась в начале 2023 года, когда Федеральное управление гражданской авиации США (Federal Aviation Administration, FAA) было вынуждено задержать все полеты 11 января в связи с выходом из строя системы оповещения летного состава NOTAM. Ранее в своем докладе от 2022 года FAA проговаривала имеющиеся сложности с устаревшим оборудованием и программным обеспечением (<https://clck.ru/3CKYY3>). Как отмечается в исследовании *Open Source Security and Risk Analysis Report* компании Synopsis от 2022 года, США подвергались 10 миллионам попыток внедрения эксплойта в час. Причем атаки были нацелены конкретно на секторы критической инфраструктуры. Было выяснено, что половина кодовых баз этой инфраструктуры содержит OSS. Как сообщает тот же отчет, практически все базы включали в себя OSS-компоненты, которые не обновлялись уже около двух лет. Программное обеспечение, как и материальная инфраструктура, с возрастом становится все более хрупким, особенно в результате появления новых уязвимостей.

Если рассмотреть несовременную материальную инфраструктуру, зачастую работающую на устаревшем и потенциально уязвимом ПО, которое регулярно подвергается атакам, то можно легко заметить, когда начинают возникать серьезные поводы для беспокойства. Это подтверждает отчет ВЭФ от 2023 года на тему глобальных рисков. В нем атаки на критическую инфраструктуру приводятся как представляющие один из наиболее значительных рисков. В ходе опроса практически все лидеры в сфере кибербезопасности высказали серьезную озабоченность сокрушительными последствиями глобальных кибератак за последние два года (<https://clck.ru/3CKYYz>).

Помимо того что в отношении ряда сфер начали появляться законодательные инициативы, такие как обязательная отчетность об инцидентах взлома данных и классификация устройств по степени защищенности, также растет стремление к повышению прозрачности ПО, предоставляемого поставщиками. Сюда входит обеспечение прозрачности компонентов ПО, в том числе его прямых и транзитивных зависимостей. И хотя эти инициативы пока несовершенны, они помогают выровнять информационный баланс между поставщиками и потребителями.

Глупо ожидать, что рядовые граждане и представители бизнеса, чья основная специализация и деятельность не связаны с кибербезопасностью, окажутся экспертами в вопросах защиты ПО, защищенной разработки и управления уязвимостями. Мы не требуем подобного и в других областях вроде автомобилестроения, авиастроения, фармацевтики и продовольствия. В этих отраслях мы приняли соответствующие меры для установления равновесия между поставщиками и потребителями с целью обеспечения безопасности, ответственности и подотчетности. При этом мы также предоставили потребителям возможность принимать взвешенные решения с учетом рисков. Теперь же мы наблюдаем аналогичные инициативы, направленные на сферу программного обеспечения и технологий.

Появление подобных движений и инициатив в других странах говорит о том, что данный вопрос не ограничивается одними только Соединенными Штатами. Правительства и государства по всему миру начинают осознавать системные риски, создаваемые небезопасным ПО, отсутствием прозрачности и ответственности по созданию защищенных продуктов и технологий. Многие страны реагируют на эту ситуацию аналогичными мерами с целью решить обозначенные проблемы.

При этом мы не сомневаемся, что в противовес стремлению к прозрачности и защите ПО со стороны правительств и регулирующих органов будет

и обратная реакция со стороны отраслевых групп, лоббистов и коммерческих субъектов. Например, Торговая палата США (U.S. Chamber of Commerce) заявила, что планирует ознакомиться с новой стратегией кибербезопасности и определить ее влияние на соответствующих участников. Предпринимаемые лоббистами и некоторыми отраслевыми группами усилия в некоторых случаях достигали успеха по сдерживанию ряда исторических инициатив, направленных на повышение прозрачности и подотчетности поставщиков ПО, в том числе недавней попытки включить SBOM в закон о полномочиях в сфере национальной обороны США на 2023 год.

Разумеется, в возражениях этих групп есть объективность, например, некоторые требуемые артефакты еще недостаточно развиты, а законодательные требования, регулирующие их деятельность, нуждаются в согласовании. Несвязные законопроекты и требования вызывают путаницу в различных отраслях, ведут к потере времени и инвестиций, а также сдерживают инновации. Возможность правительственных органов и граждан использовать передовые технологии и инструменты является ключом не только к повышению качества жизни, но также к экономическому процветанию и национальной безопасности. Инициативы по регулированию сферы ПО должны соизмеряться с ограничениями и влиянием, которые оказывают. Например, в Министерстве обороны США оперируют устаревшими требованиями и нормативными актами, мешающими им выполнять свои обязанности с должной скоростью.

Это подчеркивается в последней стратегии по модернизации ПО Министерства обороны США, которая объясняет ключевую роль модернизации ПО в обеспечении государственной безопасности. Вот выдержка из этого документа.

Адаптируемость министерства все больше зависит от программного обеспечения, а возможность быстро поставлять отказоустойчивое ПО — это конкурентное преимущество, которое будет определять исход будущих конфликтов. Для сокращения времени поставки от нескольких лет до минут потребуются значительные изменения в наших процессах, политиках, кадрах и технологиях (<https://clck.ru/3CKYbt>).

Поэтому, несмотря на то что стремление к повышению прозрачности ПО и безопасности цепочек его поставок продиктовано историческим отсутствием такой прозрачности, экспоненциальным ростом потребления OSS и возрастающим числом атак, эти инициативы необходимо соотносить с нагрузкой и проблемами, которые они создадут для команд, организаций и отрасли в целом.

Мы также предвидим, что инструменты, необходимые для достижения этих задач, требуют еще более значительных доработок. Это касается таких областей, как IoT, OT, устаревшее ПО и даже облачные среды, когда речь идет об инвентаризации активов, высокоточных данных об уязвимостях и автоматизации обмена информацией между поставщиками ПО, потребителями и другими субъектами индустрии. Процесс получения информации из инструментов и артефактов должен быть автоматизируемым и полезным для команд разработки, например, чтобы не замедлять темпы поставки кода в рабочую среду, позволяя организациям своевременно выполнять поставленные бизнес-задачи.

Кроме того, продолжится переход от традиционной статичной системы анализа рисков поставщиков, основанной на заполнении форм, к системе на API, подкрепляемой облачными технологиями. Этот переход позволит использовать автоматизацию и выполнять анализ практически в реальном времени. Тенденция к автоматизации также наверняка станет ключом к успешному масштабированию и достижению желаемых результатов в отношении SBOM и передачи информации об эксплуатируемости уязвимостей. Особенно это окажется актуальным для крупных и комплексных сред разработки, которые используют модели Agile и DevSecOps и делают частые релизы, что требует повышенной скорости генерации SBOM и сопутствующей информации об эксплуатируемости.

Долгие годы вопросы соответствия требованиям и безопасности отставали от основных тенденций индустрии, связанных с внедрением DevOps, повышением скорости вывода новых продуктов на рынок и быстрой разработкой ПО. Сейчас же мы наблюдаем в этом отношении динамику и зрелость, когда подходы вроде DevSecOps и инновационных инструментов безопасности в облачных средах и цепочках CI/CD предоставляют необходимые средства, в то же время уменьшая негативное влияние на скорость разработки. Учитывая тенденцию к кодификации (например, представления чего-либо «как кода»), декларативным инфраструктурам и микросервисам, безопасность и требования должны продолжать развиваться, чтобы идти в ногу со временем, способствуя реализации этой тенденции и предоставляя необходимый уровень прозрачности в отношении цепочек поставок ПО.

Как говорилось в главе 11, мы также ожидаем дальнейших инициатив от поставщиков и потребителей, направленных на улучшение практик управления цепочками поставок и безопасностью. С позиции поставщиков это будет проявляться в таких направлениях, как более эффективное управление OSS, прозрачность программных компонентов продуктов и пересмотр реальных

ожиданий в отношении мейнтейнеров и сообщества OSS, чьими трудами они активно пользуются.

Что же касается потребителей, то они продолжают требовать повышенной прозрачности ПО от поставщиков. Сюда входит получение гарантии соблюдения практик защищенной разработки ПО, а также запрос предоставления SBOM и сопутствующих артефактов, которые будут показывать не только уязвимости, но и их эксплуатируемость. Между поставщиками и потребителями должен развиваться диалог для прояснения не только присутствия уязвимостей и их эксплуатируемости, но также сроков для их устранения и мер противодействия, которые пользователи могут принять для снижения рисков на время, пока уязвимость не будет закрыта.

Организации всевозможных форм и размеров также продолжают уделять пристальное внимание своим цепочкам поставок ПО. И это внимание будет распространяться не только на OSS и сторонних поставщиков, чье ПО установлено и работает в их средах, но и на MSP, CSP и SaaS, которые все чаще становятся мишенью злоумышленников, выискивающих способы эксплуатации запутанной и сложной экосистемы поставок ПО.

Все эти инициативы будут способствовать большей согласованности в экосистеме программного обеспечения, так как более зрелые и бдительные потребители станут сотрудничать с не менее надежными поставщиками и сервисами, снижая риски для своих организаций, стейкхолдеров и клиентов. Мы предполагаем, что эта тенденция будет наиболее актуальной для сферы государственной безопасности, финансов и авиационно-космического комплекса.

Помимо изменений в командах, разрабатывающих, поставляющих и потребляющих ПО, мы ожидаем дальнейшего увеличения участия и контроля вопросов кибербезопасности на самых высоких уровнях организаций. Отчасти этому будет способствовать осознание того факта, что практически любой бизнес находится в мире технологий и ПО, даже если это не является его основной деятельностью. В любом случае именно технологии и программное обеспечение помогают предприятиям реализовывать свои процессы и получать прибыль. Кроме того, к этому подтолкнут изменения со стороны SEC, которая рекомендует советам директоров компаний обратить внимание на киберугрозы в рамках контроля за деятельностью.

Например, недавно предложенные SEC изменения в виде правила Cybersecurity Risk Management, Strategy, Governance and Incident Disclosure

(Управление рисками, стратегия, контроль и обнаружение инцидентов в сфере кибербезопасности) требуют от организаций предоставлять информацию в отношении опыта их совета директоров в области кибербезопасности. Это поспособствует перестройке парадигмы, при которой руководству организаций придется активнее вникать в вопросы, связанные с киберугрозами в своих компаниях, включая защиту цепочек поставок ПО и инциденты (<https://clck.ru/3CKYfq>). Эти преобразования говорят об изменении сферы регулирования, которое приводит к тому, что теперь руководители организаций при формировании совета директоров должны будут учитывать опыт ее членов в вопросах кибербезопасности. Поскольку атак на цепочки поставок становится все больше и в течение последних лет они постоянно обсуждаются и вызывают беспокойство организаций, можно смело предположить, что эта тема привлечет внимание руководства, а в некоторых организациях уже привлекла.

В этой книге мы разобрали множество актуальных тем в области безопасности цепочек поставок ПО. Они включали известные случаи атак, традиционные методики анализа, новые руководства и лучшие практики, историю появления SBOM и связанные с этим аспекты, а также практическое руководство для потребителей и поставщиков. Мы также прояснили роль передовых технологий вроде облачных услуг, Kubernetes и контейнеров в экосистеме цепочек поставок ПО. И хотя с учетом сложности и масштабыности вопроса невозможно охватить все связанные с ним темы, данный материал должен помочь специалистам лучше понять принципы создания безопасного программного обеспечения и риски потребляемого ими ПО, будь то из экосистемы OSS или напрямую от сторонних поставщиков.

Как мы уже говорили в книге, ПО играет важнейшую роль практически во всех аспектах нашей жизни. Поэтому, учитывая активное развитие цифрового мира, крайне важно стремиться создавать и потреблять именно защищенные продукты. Программное обеспечение открывает широкие возможности для инноваций, развития и роста, но также создает беспрецедентные системные риски, которые могут серьезно повлиять на общество, если их не устранить.

Как сказал судья Верховного суда США Луис Брандис более века назад: «Считается, что солнечный свет — это лучшая дезинфекция». Современное общество стремится к прозрачности цифровой экосистемы, которая окружает каждый аспект нашей жизни, и связанных с ней рисков.

Мы — паладины этого солнечного света.

Крис Хьюз, Тони Тернер

**Прозрачное программное обеспечение:
безопасность цепочек поставок ПО**

Перевел с английского Д. Брайт

Руководитель дивизиона	<i>Ю. Сергиенко</i>
Ведущий редактор	<i>Н. Гринчик</i>
Литературный редактор	<i>А. Аверьянов</i>
Художественный редактор	<i>В. Мостипан</i>
Корректоры	<i>О. Андриевич, Т. Никифорова</i>
Верстка	<i>Г. Блинов</i>

Изготовлено в России. Изготовитель: ООО «Прогресс книга».
Место нахождения и фактический адрес: 194044, Россия, г. Санкт-Петербург,
Б. Сампсониевский пр., д. 29А, пом. 52. Тел.: +78127037373.

Дата изготовления: 10.2024. Наименование: книжная продукция. Срок годности: не ограничен.

Налоговая льгота — общероссийский классификатор продукции ОК 034-2014, 58.11.12 — Книги печатные
профессиональные, технические и научные.

Импортер в Беларусь: ООО «ПИТЕР М», 220020, РБ, г. Минск, ул. Тимирязева, д. 121/3, к. 214, тел./факс: 208 80 01.

Подписано в печать 29.08.24. Формат 70×100/16. Бумага офсетная. Усл. п. л. 28,380. Тираж 700. Заказ 0000.